

以下逐文件列出问题四默认策略、上一版联合策略及原格网策略的本地依赖源码和运行入口。为避免字体缺字，仅在本排版副本中将数学符号转换为等义的 LaTeX 命令，告警符号显示为 [!], 省略无可见字形的表情呈现选择符；可执行源码未作修改。逐文件原始字节 SHA256、依赖关系及转换数量保存在支撑材料的源码清单中。

0.1 Q4/src/p3_arena.py

```
"""B 题问题 3：机器狗与模拟器之间的统一接口层。

两个后端对上层暴露**完全相同的接口与返回字段**（字段名与模拟器 JSON 一致），
因此 :mod:`p3_robot` 里的策略代码在"离线演练"和"真实模拟器"上完全一致：

* :class:`MockArena` — 完全离线复刻附件 1/附件 2 的物理与计时规则。
  用于自检、调参、批量演练（可以跑几百局，正式测试只有 3 次机会）。
* :class:`HttpArena` — 真实模拟器的 HTTP+JSON 客户端，负责协议里最容易踩的坑：
  串行发送、``request_id`` 幂等与重试、``accepted`` 与 HTTP 状态双重校验、
  ``remaining_real_duration_s`` 动态预算、自身行为日志（论文支撑材料需要）。

计时规则（附件 1 表 1 / 附件 2 第 4 节）
-----
* ``/measure``：移动距离/5 + （频道变化 ? 1 : 0） + 5 秒；只有 ``/measure`` 会切换频道；
* ``/clear``：移动距离/5 + （成功 5 / 未发现 3）秒；**不切换频道**；
* ``/enter``、``/exit`` 不推进虚拟时钟；
* 示向度误差在  $[-1^\circ, 1^\circ]$  内，**同一地点固定**（本模块用"平滑随机场 + 局地量化项"建模）。
"""

from __future__ import annotations

import json
import math
import os
import socket
import time
import urllib.error
import urllib.request
from dataclasses import dataclass, field as dc_field

import numpy as np

# -----
# 物理常数（题目附录 1/2、模拟器接口说明）
# -----
R_ARENA = 1800.0      # 靶区半径 (m)
V_ROBOT = 5.0         # 移动速度 (m/s)
T_MEASURE = 5.0       # 一次检测耗时 (s)
T_SWITCH = 1.0        # 频道切换耗时 (s)
T_CLEAR_OK = 5.0       # 成功清除耗时 (s)：光学 3 + 激光 2
T_CLEAR_FAIL = 3.0     # 未发现目标时的光学精确定位耗时 (s)
T_CLEAR = T_CLEAR_OK
NEAR_R = 5.0          # 近场阈值 (m)：更近时返回 near，无示向度
R_CLEAR = 20.0        # 清除半径 (m)
R_RECV_MIN, R_RECV_MAX = 1000.0, 1500.0
```

```

BEARING_ERR = 1.0      # 示向度误差界（度）
CHANNELS = tuple(range(1, 21))
# 程序运行时间（现实时间）上限（s）：题目附录 3，/enter 响应里由 remaining_real_duration_s
    给出
DEF_BUDGET_S = 1200.0
# 虚拟世界限时（s）：接口说明 1.4/4.5，默认 360000（100 小时）。
# **注意**：这是虚拟时钟的上限，与上面 1200 s 的现实时间预算是两回事。
# 离线 mock 必须用同一个值，否则等于给离线演练加了一条真实测试里不存在的约束。
DEF_VIRTUAL_LIMIT_S = 360000.0
COORD_LIMIT = 2_000_000.0 # 接口允许的坐标绝对值上限（m）

class ArenaError(RuntimeError):
    """接口层不可恢复错误（网络重试用尽、响应无法解析等）。"""

# -----
# 离线模拟器
# -----
@dataclass
class MockSource:
    channel: int
    x: float
    y: float
    r_recv: float
    cone_half: float = 180.0 # 覆盖半角：180 = 全向（问题 4 会用到 <180）
    dir_deg: float | None = None
    cleared: bool = False

    @property
    def xy(self):
        return (self.x, self.y)

class MockArena:
    """离线复刻的模拟器（问题 3：全向源）。"""

    def __init__(self, seed=0, n_sources=None, budget_s=DEF_VIRTUAL_LIMIT_S,
                  r_arena=R_ARENA, sources=None, error_corr_m=250.0,
                  error_local_w=0.25, verbose=False):
        """`budget_s` = **虚拟时钟上限**（默认 360000 s，与真实模拟器一致）。"""
        self.seed = int(seed)
        self.rng = np.random.default_rng(self.seed)
        self.r_arena = float(r_arena)
        self.budget = float(budget_s)
        self.sources = list(sources) if sources is not None else
            self._gen_sources(n_sources)
        self._init_error_field(error_corr_m, error_local_w)
        self.pos = (0.0, 0.0)
        self.channel = 1
        self.vtime = 0.0

```

```

self.entered = False
self.finished = False
self.verbose = verbose
self.actions: list[dict] = []
self.stats = {"n_measure": 0, "n_clear": 0, "n_clear_fail": 0,
              "travel_m": 0.0, "n_direction": 0, "n_near": 0, "n_no_signal": 0}

# ----- 场景生成 -----
def _gen_sources(self, n_sources=None):
    rng = self.rng
    n = int(n_sources) if n_sources else int(rng.integers(10, 17))
    n = max(1, min(n, len(CHANNELS)))
    chans = rng.choice(np.asarray(CHANNELS), size=n, replace=False)
    out = []
    for ch in chans:
        r = self.r_arena * math.sqrt(float(rng.random()))
        a = float(rng.uniform(0, 2 * math.pi))
        out.append(MockSource(channel=int(ch), x=r * math.cos(a), y=r *
                               math.sin(a),
                               r_recv=float(rng.uniform(R_RECV_MIN, R_RECV_MAX))))
    return out

def _init_error_field(self, corr_m, local_w):
    """示向度误差场: 平滑随机场(模拟电磁环境) + 局地量化项; 量程严格  $\pm 1^\circ$ ."""
    rng = self.rng
    self._waves = []
    for _ in range(12):
        lam = float(rng.uniform(max(corr_m * 1.5, 120.0), 3.0 * self.r_arena))
        ang = float(rng.uniform(0, 2 * math.pi))
        self._waves.append((2 * math.pi * math.cos(ang) / lam,
                           2 * math.pi * math.sin(ang) / lam,
                           float(rng.uniform(0, 2 * math.pi)),
                           1.0 / lam))
    # 归一化: 用靶区内抽样点的最大绝对值做标定, 再乘 1.05 并截断到  $\pm 1$ 
    pts = self._sample_disc(4000, rng)
    raw = self._raw_field(pts[:, 0], pts[:, 1])
    self._scale = max(float(np.max(np.abs(raw)))) * 1.05, 1e-9)
    self._local_w = float(local_w)
    self._local_seed = int(rng.integers(1, 10 ** 6))

def _sample_disc(self, n, rng):
    r = self.r_arena * np.sqrt(rng.random(n))
    a = rng.uniform(0, 2 * math.pi, n)
    return np.stack([r * np.cos(a), r * np.sin(a)], axis=1)

def _raw_field(self, x, y):
    x = np.asarray(x, float)
    y = np.asarray(y, float)
    v = np.zeros_like(x)
    for (kx, ky, ph, a) in self._waves:
        v = v + a * np.sin(kx * x + ky * y + ph)

```

```

        return v

def _local_field(self, x, y):
    i = int(math.floor(x / 25.0))
    j = int(math.floor(y / 25.0))
    h = (i * 73856093) ^ (j * 19349663) ^ (self._local_seed * 83492791)
    h ^= 0x7FFFFFFF
    return (h % 200001) / 100000.0 - 1.0

def env_error(self, x, y):
    """该地点的示向度误差（度，[-1,1]，同一地点固定）。"""
    v = (float(self._raw_field(x, y)) / self._scale
          + self._local_w * self._local_field(x, y))
    return float(max(-1.0, min(1.0, v)))

# ----- 状态查询 -----
@property
def virtual_time_s(self):
    return self.vtime

def remaining_budget_s(self):
    return max(0.0, self.budget - self.vtime)

def budget_kind(self):
    return "virtual"

def truth(self):
    return {"n_sources": len(self.sources),
            "n_cleared": sum(1 for s in self.sources if s.cleared),
            "sources": [{"channel": s.channel, "x_m": round(s.x, 3),
                          "y_m": round(s.y, 3), "r_recv_m": round(s.r_recv, 3),
                          "cleared": bool(s.cleared)} for s in self.sources]}

# ----- 内部工具 -----
def _src_of(self, channel):
    for s in self.sources:
        if s.channel == int(channel):
            return s
    return None

def _reject(self, reason, **kw):
    r = {"accepted": False, "real_timestamp_ms": int(time.time() * 1000),
         "virtual_time_s": 0.0, "reason": reason}
    r.update(kw)
    return r

def _advance(self, x, y, dt, kind, payload):
    d = math.hypot(x - self.pos[0], y - self.pos[1])
    self.stats["travel_m"] += d
    self.vtime += dt
    self.pos = (float(x), float(y))

```

```

rec = {"kind": kind, "x": float(x), "y": float(y), "dt": float(dt),
       "dist_m": float(d), "virtual_time_s": float(self.vtime)}
rec.update(payload)
self.actions.append(rec)
if self.verbose:
    print(f" [mock] {kind} ({x:.0f},{y:.0f}) dt={dt:.1f}s "
          f"t={self.vtime:.1f}s {payload}")
return rec

# ----- 四条指令 -----
def enter(self):
    if self.entered:
        return self._reject("already_entered")
    self.entered = True
    self.actions.append({"kind": "enter", "virtual_time_s": 0.0})
    return {"accepted": True, "real_timestamp_ms": int(time.time() * 1000),
           "virtual_time_s": 0.0, "max_virtual_duration_s": self.budget,
           "max_real_duration_s": self.budget,
           "remaining_real_duration_s": int(self.budget)}

def measure(self, x, y, channel):
    if not self.entered:
        return self._reject("not_entered")
    if self.finished:
        return self._reject("finished")
    x, y, ch = float(x), float(y), int(channel)
    if not (math.isfinite(x) and math.isfinite(y)) \
        or abs(x) > COORD_LIMIT or abs(y) > COORD_LIMIT:
        return self._reject("bad_position")
    if ch not in CHANNELS:
        return self._reject("bad_channel")
    switch = T_SWITCH if ch != self.channel else 0.0
    d = math.hypot(x - self.pos[0], y - self.pos[1])
    dt = d / V_ROBOT + switch + T_MEASURE
    if self.vtime + dt > self.budget + 1e-9:
        self.finished = True
        return self._reject("timeout")
    src = self._src_of(ch)
    svd = None
    if src is None or src.cleared:
        result = "no_signal"
    else:
        dist = math.hypot(x - src.x, y - src.y)
        in_cone = True
        if src.cone_half < 180.0:
            a = math.degrees(math.atan2(src.y - y, src.x - x))
            ref = src.dir_deg if src.dir_deg is not None else 0.0
            in_cone = abs((a - ref + 180.0) % 360.0 - 180.0) <= src.cone_half +
                1e-12
        if dist > src.r_recv or not in_cone:
            result = "no_signal"

```

```

        elif dist <= NEAR_R:
            result = "near"
        else:
            result = "direction"
            a = math.degrees(math.atan2(src.y - y, src.x - x))
            svd = round((a + self.env_error(x, y)) % 360.0, 2)
self.channel = ch
self.stats["n_measure"] += 1
self.stats["n_" + result] += 1
self._advance(x, y, dt, "measure",
              {"channel": ch, "measure_result": result, "svd_deg": svd,
               "switch_s": switch})
resp = {"accepted": True, "real_timestamp_ms": int(time.time() * 1000),
        "virtual_time_s": self.vtime, "measure_result": result}
if svd is not None:
    resp["svd_deg"] = svd
return resp

def clear(self, x, y, channel):
    if not self.entered:
        return self._reject("not_entered")
    if self.finished:
        return self._reject("finished")
    x, y, ch = float(x), float(y), int(channel)
    if not (math.isfinite(x) and math.isfinite(y)) \
        or abs(x) > COORD_LIMIT or abs(y) > COORD_LIMIT:
        return self._reject("bad_position")
    if ch not in CHANNELS:
        return self._reject("bad_channel")
    d = math.hypot(x - self.pos[0], y - self.pos[1])
    src = self._src_of(ch)
    ok = bool(src is not None and not src.cleared
              and math.hypot(x - src.x, y - src.y) <= R_CLEAR + 1e-9)
    dt = d / V_ROBOT + (T_CLEAR_OK if ok else T_CLEAR_FAIL)
    if self.vtime + dt > self.budget + 1e-9:
        self.finished = True
        return self._reject("timeout")
    if ok:
        src.cleared = True
    self.stats["n_clear"] += 1
    if not ok:
        self.stats["n_clear_fail"] += 1
    self._advance(x, y, dt, "clear",
                  {"channel": ch, "clear_result": "success" if ok else
                   "no_target_in_range"})
    return {"accepted": True, "real_timestamp_ms": int(time.time() * 1000),
            "virtual_time_s": self.vtime,
            "clear_result": "success" if ok else "no_target_in_range"}

def exit(self):
    self.finished = True

```

```

        self.actions.append({"kind": "exit", "virtual_time_s": self.vtime})
        return {"accepted": True, "real_timestamp_ms": int(time.time() * 1000),
                "virtual_time_s": self.vtime, "exit_reason": "user_exit"}

# -----
# 真实模拟器客户端
# -----

class HttpArena:
    """模拟器 HTTP+JSON 客户端（默认 http://127.0.0.1:2026）。"""

    def __init__(self, base_url="http://127.0.0.1:2026", robot_id="",
                  arena_id="default", timeout=5.0, max_retries=4, retry_delay=0.4,
                  log_path=None, budget_s=DEF_BUDGET_S, safety_margin_s=3.0,
                  enter_wait_s=40.0):
        self.base_url = base_url.rstrip("/")
        self.robot_id = robot_id
        self.arena_id = arena_id
        self.timeout = float(timeout)
        self.max_retries = int(max_retries)
        self.retry_delay = float(retry_delay)
        self.log_path = log_path
        self.budget = float(budget_s)
        self.safety_margin_s = float(safety_margin_s)
        self.enter_wait_s = float(enter_wait_s)
        self._n = 0
        self.entered = False
        self.pos = (0.0, 0.0)
        self.channel = 1
        self.vtime = 0.0
        self.remaining_real0 = float(budget_s)
        self._t0 = None
        self.actions: list[dict] = []
        # 与 MockArena 同名的统计（接口不返回行程，客户端自己按动作位置累加，
        # 只统计 accepted=true 的动作，与虚拟时间的推进口径一致）
        self.stats = {"n_measure": 0, "n_clear": 0, "n_clear_fail": 0,
                      "travel_m": 0.0, "n_direction": 0, "n_near": 0, "n_no_signal": 0}
        if self.log_path:
            os.makedirs(os.path.dirname(os.path.abspath(self.log_path)),
                        exist_ok=True)
            with open(self.log_path, "w", encoding="utf-8") as f:
                f.write(json.dumps({"kind": "session", "base_url": self.base_url,
                                   "robot_id": self.robot_id}, ensure_ascii=False) + "\n")

# ----- HTTP -----

    def _rid(self, tag):
        self._n += 1
        return f"{tag}-{self._n}"

    def _base(self, rid):
        return {"arena_id": self.arena_id, "robot_id": self.robot_id, "request_id":

```

```

        rid)

def _post(self, path, payload, tag):
    data = json.dumps(payload, ensure_ascii=False).encode("utf-8")
    last_err = None
    for attempt in range(self.max_retries):
        req = urllib.request.Request(
            self.base_url + path, data=data,
            headers={"Content-Type": "application/json"}, method="POST")
        try:
            with urllib.request.urlopen(req, timeout=self.timeout) as r:
                body = r.read().decode("utf-8")
            resp = json.loads(body)
            self._log({"kind": "resp", "path": path, "attempt": attempt, "resp":
                resp})
            return resp
        except urllib.error.HTTPError as e:
            body = ""
            try:
                body = e.read().decode("utf-8", "replace")
                resp = json.loads(body)
                self._log({"kind": "http_error", "path": path, "code": e.code,
                    "resp": resp})
                return resp
            except Exception:
                self._log({"kind": "http_error", "path": path, "code": e.code,
                    "body": body[:500]})
                raise ArenaError(f"{path} HTTP {e.code}: {body[:200]}") from e
        except (urllib.error.URLError, socket.timeout, TimeoutError,
            ConnectionError) as e:
            # 网络超时/中断: 按协议重试完全相同的动作(复用 request_id 与内容)
            last_err = e
            self._log({"kind": "net_error", "path": path, "attempt": attempt,
                "error": repr(e)})
            if attempt + 1 < self.max_retries:
                time.sleep(self.retry_delay * (attempt + 1))
            raise ArenaError(f"{path} 请求失败(已重试 {self.max_retries}
                次): {last_err!r}")

def _log(self, rec):
    rec["wall_s"] = round(time.time() - (self._t0 or time.time()), 3)
    self.actions.append(rec)
    if self.log_path:
        with open(self.log_path, "a", encoding="utf-8") as f:
            f.write(json.dumps(rec, ensure_ascii=False) + "\n")

def _new(self, path, extra, tag):
    rid = self._rid(tag)
    payload = self._base(rid)
    payload.update(extra)
    self._log({"kind": "req", "path": path, "payload": payload})

```

```

        return self._post(path, payload, tag)

# ----- 状态 -----
@property
def virtual_time_s(self):
    return self.vtime

def remaining_budget_s(self):
    if self._t0 is None:
        return self.remaining_real0
    used = time.monotonic() - self._t0
    return max(0.0, self.remaining_real0 - used - self.safety_margin_s)

def budget_kind(self):
    return "real"

def truth(self):
    return None

# ----- 四条指令 -----
def enter(self):
    """进入靶区。接口未开放时（倒计时未结束 / 测试未开始）会重试同一 request_id。

    协议规定"同一 request_id + 同一内容返回第一次的完整响应"，所以用固定的
    request_id 重试 /enter 是安全的（不会重复进入、也不会推进时钟）。
    """
    rid = self._rid("enter")
    payload = self._base(rid)
    self._log({"kind": "req", "path": "/enter", "payload": payload})
    t0 = time.monotonic()
    last = None
    keep_retries = self.max_retries
    self.max_retries = min(keep_retries, 2) # 接口未开放时缩短单次重试，尽快进入下一轮
    try:
        while True:
            try:
                resp = self._post("/enter", payload, "enter")
            except ArenaError as e:
                resp, last = None, e
            if resp is not None and resp.get("accepted") is True:
                self.entered = True
                self._t0 = time.monotonic()
                self.remaining_real0 = float(resp.get("remaining_real_duration_s",
                                                       self.budget))
                self.vtime = float(resp.get("virtual_time_s", 0.0))
                return resp
            if resp is not None and resp.get("accepted") is False \
                and resp.get("reason") == "already_entered":
                return resp
            if time.monotonic() - t0 > self.enter_wait_s:
                if resp is not None:

```

```

        return resp
        raise ArenaError(
            f"/enter 在 {self.enter_wait_s:.0f} s 内一直失败: {last!r}")
    time.sleep(2.0)
finally:
    self.max_retries = keep_retries

def measure(self, x, y, channel):
    resp = self._new("/measure", {"position": {"x": float(x), "y": float(y)},
                                   "channel": int(channel)}, "measure")
    if resp.get("accepted") is True:
        self.stats["travel_m"] += math.hypot(float(x) - self.pos[0],
                                              float(y) - self.pos[1])
        self.pos = (float(x), float(y))
        self.channel = int(channel)
        self.vtime = float(resp.get("virtual_time_s", self.vtime))
        res = resp.get("measure_result")
        self.stats["n_measure"] += 1
        if res in ("direction", "near", "no_signal"):
            self.stats["n_" + res] += 1
    return resp

def clear(self, x, y, channel):
    resp = self._new("/clear", {"position": {"x": float(x), "y": float(y)},
                                   "channel": int(channel)}, "clear")
    if resp.get("accepted") is True:
        self.stats["travel_m"] += math.hypot(float(x) - self.pos[0],
                                              float(y) - self.pos[1])
        self.pos = (float(x), float(y))
        self.vtime = float(resp.get("virtual_time_s", self.vtime))
        self.stats["n_clear"] += 1
        if resp.get("clear_result") != "success":
            self.stats["n_clear_fail"] += 1
    return resp

def exit(self):
    return self._new("/exit", {}, "exit")

# -----
# 自检
# -----
def selfcheck(verbose=True, seed=20260913):
    """核对 MockArena 是否忠实复刻附件 1/2 的规则。"""
    out = []
    ok_all = True

    def rec(name, ok, detail=""):
        nonlocal ok_all
        ok_all = ok_all and bool(ok)
        out.append({"check": name, "ok": bool(ok), "detail": detail})

```

```

    if verbose:
        print(f"[{'OK ' if ok else 'FAIL'}] {name} {detail}")

# A1 附件 1 表 2 的计时算例 ((300,400) 频道1 -> 频道2 -> /clear 未发现 -> 频道2)
a = MockArena(seed=1, sources=[MockSource(3, 100000.0, 0.0, 1000.0)])
a.enter()
r1 = a.measure(300.0, 400.0, 1)
r2 = a.measure(300.0, 400.0, 2)
r3 = a.clear(300.0, 0.0, 3)
r4 = a.measure(300.0, 0.0, 2)
ok1 = (abs(r1["virtual_time_s"] - 105.0) < 1e-9
        and abs(r2["virtual_time_s"] - 111.0) < 1e-9
        and abs(r3["virtual_time_s"] - 194.0) < 1e-9
        and abs(r4["virtual_time_s"] - 199.0) < 1e-9)
rec("A1 计时算例与附件1表2逐条一致 (105/111/194/199 s)", ok1,
    f"{r1['virtual_time_s']}, {r2['virtual_time_s']}, "
    f"{r3['virtual_time_s']}, {r4['virtual_time_s']}")

# A2 /clear 不切换频道, 也不产生切换耗时
b = MockArena(seed=2, sources=[MockSource(1, 0.0, 0.0, 1000.0)])
b.enter()
b.measure(0.0, 100.0, 1)
t0 = b.virtual_time_s
b.clear(0.0, 50.0, 7)
rec("A2 /clear 不切换频道、不产生切换耗时",
    abs(b.virtual_time_s - t0 - (50.0 / V_ROBOT + T_CLEAR_FAIL)) < 1e-9
    and b.channel == 1,
    f"Δt={b.virtual_time_s - t0:.1f}s, 当前频道={b.channel}")

# A3 三种检测结果与清除半径
src = MockSource(5, 0.0, 0.0, 1000.0)
c = MockArena(seed=3, sources=[src])
c.enter()
m_far = c.measure(0.0, 0.0, 5) # 距离 0 <= 5 m -> near
m_dir = c.measure(0.0, 300.0, 5) # direction (真方位 270°)
m_no = c.measure(0.0, 1200.0, 5) # 超出 1000 m -> no_signal
ok3 = (m_far["measure_result"] == "near" and m_dir["measure_result"] ==
        "direction"
        and abs(m_dir["svd_deg"] - 270.0) <= BEARING_ERR + 1e-9
        and m_no["measure_result"] == "no_signal")
rec("A3 near / direction / no_signal 三种结果与真方位一致", ok3,
    f"near={m_far['measure_result']}, dir={m_dir.get('svd_deg')}°, "
    f"no={m_no['measure_result']}")
cl_far = c.clear(0.0, 1000.0, 5)
cl_ok = c.clear(0.0, 20.0, 5)
rec("A4 清除半径 20 m: 远处未发现、20 m 内成功",
    cl_far["clear_result"] == "no_target_in_range"
    and cl_ok["clear_result"] == "success",
    f"{cl_far['clear_result']} / {cl_ok['clear_result']}")
rec("A5 同一干扰源只能清除一次",
    c.clear(0.0, 0.0, 5)["clear_result"] == "no_target_in_range",

```

```

    "二次清除返回 no_target_in_range")

# A6 误差界与"同一地点固定"
d = MockArena(seed=4)
e1 = d.env_error(123.0, -456.0)
e2 = d.env_error(123.0, -456.0)
vals = [d.env_error(float(x), float(y)) for x, y in d._sample_disc(500, d.rng)]
rec("A6 示向度误差:  $|e| \leq 1^\circ$  且同一地点固定",
    abs(e1 - e2) < 1e-12 and max(abs(v) for v in vals) <= 1.0 + 1e-12,
    f"e={e1:.4f}, 重复一致; 500 点最大  $|e|={max(abs(v) for v in vals):.4f}")$ ")

# A7 真源一定落在"测得方位  $\pm 1^\circ$ "的楔内(区间算法成立的前提)
bad = 0
nchk = 0
d2 = MockArena(seed=4, budget_s=1e9)
d2.enter()
for s in d2.sources:
    for _ in range(3):
        aa = float(d2.rng.uniform(0, 2 * math.pi))
        dd = float(d2.rng.uniform(50.0, 0.9 * s.r_recv))
        x, y = s.x + dd * math.cos(aa), s.y + dd * math.sin(aa)
        r = d2.measure(x, y, s.channel)
        if r.get("accepted") is not True or r.get("measure_result") !=
            "direction":
            continue
        nchk += 1
        true_a = math.degrees(math.atan2(s.y - y, s.x - x)) % 360.0
        if abs((true_a - r["svd_deg"] + 180.0) % 360.0 - 180.0) > BEARING_ERR +
            1e-9:
            bad += 1
rec("A7 真方位与示向度之差恒  $\leq 1^\circ$ ", bad == 0 and nchk > 0,
    f"{nchk} 次检测, 越界 {bad} 次")

# A8 超时拒动
e = MockArena(seed=5, budget_s=50.0, sources=[MockSource(1, 0.0, 0.0, 1000.0)])
e.enter()
e.measure(0.0, 1000.0, 1) # 200 + 5 = 205 s > 50 s
rec("A8 超出时间预算的请求 accepted=false 且不推进时钟",
    e.virtual_time_s == 0.0 and e.finished,
    f"vtime={e.virtual_time_s}")

if verbose:
    print(f"\n自检总结: {'全部通过' if ok_all else '存在失败项'}")
return {"ok": ok_all, "checks": out}

if __name__ == "__main__":
    import argparse
    ap = argparse.ArgumentParser(description="问题3 接口层自检")
    ap.add_argument("--selfcheck", action="store_true")
    args = ap.parse_args()

```

```
res = selfcheck()
raise SystemExit(0 if res["ok"] else 1)
```

0.2 Q4/src/pl_intersection.py

"""B 题问题 1: 示向度交会定位区域、区域直径与直径圆覆盖判定。

模型的严格表述

 检测点 :math: S_i=(x_i, y_i), 该点测得示向度 :math: \theta_i (度, x 轴正向逆时针, $[0, 360)$),
 示向度误差有界 :math: |e_i| \leq \epsilon (题目附录 2 图 2: 实线与相邻虚线夹角均为 1°)。
 真源必在

$$R = \bigcap_i W_i, \quad W_i = \{ X : |\arg(X - S_i) - \theta_i| \leq \epsilon \}$$

W_i 是以 S_i 为顶点、张角 2ϵ 的**角楔**, 等价于两个半平面之交:

$$W_i = \{ X : a_i^- X + b_i^- \leq 0 \} \cap \{ X : a_i^+ X + b_i^+ \leq 0 \}$$

因此 R 是 $2k$ 个半平面的交 — **凸多边形**, 顶点数 $\leq 2k$ 。

算法要点 (含两处易错点的正确处理)

-
1. **可行性 / 有界性** (先判空、再判无界, 顺序不能反)
 - * 空集: 两两示向度可以相差 $> 2\epsilon$, 但两个"错开"的角楔也可能没有公共点;
 - * 无界: 存在方向 d 使 $|\arg(d) - \theta_i| \leq \epsilon$ 对所有 i 成立
 $\iff$ 所有示向度**两两夹角 $\leq 2\epsilon$ ** ; 此时直径 $= +\infty$, 不能报有限值。
 2. **区域顶点**: 凸集的顶点必是两条起作用约束边界直线的交点, 故枚举 $2k$ 条边界直线的 $O(k^2)$ 个交点、用**半平面判据** $a \cdot X + b \leq 0$ 筛选即可 (精确)。
 注意: **检测点本身也可能是顶点** (当它落在其它所有楔内时)。旧实现在这里用 "候选点到检测点的距离 $< 1e-9$ 则丢弃" 处理角度奇异, 会把这种顶点误删 — 见 ``region_by_vertices`` 的半平面判据。
 3. 提供两套**相互独立**的区域算法, 供自检交叉验证:
 - * ``region_by_vertices``: 顶点枚举 + 凸包 (主算法, 精确);
 - * ``region_by_clipping``: Sutherland-Hodgman 逐个半平面裁剪大框 (参考实现, 可判无界)。
 4. **直径**: 顶点数 $\leq \text{DIAM_BRUTE_LIMIT}$ 时暴力枚举; 否则旋转卡壳, 两者在自检中互相校验。
 5. **覆盖判定**: 以直径 AB 为直径的圆 C ,
 $V \in C \iff \angle AVB \geq 90^\circ$ (Thales) $\iff |V - c| \leq D/2$ (c 为 AB 中点)。
 一般**不能覆盖**: 正确做法是用**最小包围圆** (半径 $r^* \in [D/2, D/\sqrt{3}]$, Jung 定理)。

```
"""
from __future__ import annotations
import csv
import json
import math
import os
import sys

import numpy as np
```

```

BEARING_ERR = 1.0 # 示向度误差界 (度)
ANG_TOL = 1e-9    # 角度判据容差 (度)
DIST_TOL = 1e-6   # 半平面判据容差 (米)
CLIP = 1e7        # 裁剪参考实现的外框半边长 (米)
DIAM_BRUTE_LIMIT = 128 # 顶点数不超过该值时直径用暴力枚举

# -----
# 基本几何
# -----
def bearing(p, q):
    """p 处指向 q 的方位角 (度, [0,360)) """
    return math.degrees(math.atan2(q[1] - p[1], q[0] - p[0])) % 360.0

def ang_diff(a, b):
    """a-b 归一化到 (-180,180]"""
    return (a - b + 180.0) % 360.0 - 180.0

def wedge_halfplanes(p, theta, err=BEARING_ERR):
    """角楔 W = {X : |ang(X-p) - theta| <= err} 的两个半平面, 形式 a·X + b <= 0。

    err < 90°, 故 W 是两条边界直线所夹的尖楔。
    """
    p = np.asarray(p, float)
    tm = math.radians((theta - err) % 360.0)
    tp = math.radians((theta + err) % 360.0)
    dm = np.array([math.cos(tm), math.sin(tm)])
    dp = np.array([math.cos(tp), math.sin(tp)])
    # cross(dm, X-p) >= 0 <=> a1·X + b1 <= 0
    a1 = np.array([dm[1], -dm[0]])
    b1 = float(-dm[1] * p[0] + dm[0] * p[1])
    # cross(dp, X-p) <= 0 <=> a2·X + b2 <= 0
    a2 = np.array([-dp[1], dp[0]])
    b2 = float(dp[1] * p[0] - dp[0] * p[1])
    return [(a1, b1), (a2, b2)]

def region_is_unbounded(svds, err=BEARING_ERR):
    """区域无界判据: 存在方向 d 与所有示向度夹角 <= err。

    返回一个这样的方向 (度), 有界时返回 None。
    仅在区域非空时有意义 (先判空、再判无界)。
    """
    for th in svds:
        for cand in ((th - err) % 360.0, (th + err) % 360.0):
            if all(abs(ang_diff(cand, t)) <= err + ANG_TOL for t in svds):
                return cand
    return None

```

```

def _line_intersection(P, u, Q, v):
    """直线  $P + t \cdot u$  与  $Q + s \cdot v$  的交点, 平行时返回 None"""
    det = u[0] * (-v[1]) - (-v[0]) * u[1]
    if abs(det) < 1e-14:
        return None
    r = Q - P
    t = (r[0] * (-v[1]) - (-v[0]) * r[1]) / det
    return P + t * u

def convex_hull(P):
    """Andrew monotone chain, 逆时针, 去掉共线点"""
    def half(pts):
        st = []
        for p in pts:
            while len(st) >= 2 and _cross2(st[-1] - st[-2], p - st[-2]) <= 0:
                st.pop()
            st.append(p)
        return st
    P = P[np.lexsort((P[:, 1], P[:, 0]))]
    lo = half(P)
    hi = half(P[::-1])
    return np.array(lo[:-1] + hi[:-1])

def _cross2(u, v):
    return float(u[0] * v[1] - u[1] * v[0])

def _dedup_hull(cand):
    """候选顶点去重 + 凸包, 返回 (P, "empty"|"degenerate"|"polygon")"""
    if len(cand) == 0:
        return np.zeros((0, 2)), "empty"
    P = np.array(cand, float)
    P = P[np.lexsort((P[:, 1], P[:, 0]))]
    ded = [P[0]]
    for v in P[1:]:
        if np.linalg.norm(v - ded[-1]) > 1e-7:
            ded.append(v)
    P = np.array(ded)
    if len(P) < 3:
        return P, "degenerate"
    H = convex_hull(P)
    if len(H) < 3:
        return H, "degenerate"
    return H, "polygon"

# -----
# 区域算法 1: 顶点枚举 + 凸包 (主算法, 精确)

```

```

# -----
def region_by_vertices(pts, svds, err=BEARING_ERR, tol=DIST_TOL):
    """R =  $\cap W_i$  的顶点集合。

    R 是凸集，其任一极点必是两条起作用约束边界直线的交点；反之，落在所有半平面内的
    边界直线交点必为极点（若  $X = (Y+Z)/2$  且两个非平行约束在 X 处取等，则 Y、Z 也
    必须取等，故  $Y = Z = X$ ）。因此枚举 + 筛选是精确的。
    """
    if len(pts) == 0:
        return np.zeros((0, 2)), "empty"
    H, lines = [], []
    for p, th in zip(pts, svds):
        H += wedge_halfplanes(p, th, err)
        for s in (-1.0, 1.0):
            a = math.radians((th + s * err) % 360.0)
            lines.append((np.asarray(p, float), np.array([math.cos(a), math.sin(a)])))
    cand = []
    n = len(lines)
    for i in range(n):
        for j in range(i + 1, n):
            X = _line_intersection(lines[i][0], lines[i][1], lines[j][0], lines[j][1])
            if X is None or not np.all(np.isfinite(X)):
                continue
            # 半平面判据：在楔顶点处也良定义（无角度奇异），故检测点可作顶点保留
            if all(float(a @ X + b) <= tol for (a, b) in H):
                cand.append(X)
    return _dedup_hull(cand)

# -----
# 区域算法 2：半平面裁剪（独立参考实现，可直接判无界）
# -----
def clip_halfplane(poly, a, b, tol=DIST_TOL):
    """Sutherland-Hodgman: 保留  $\{X : a \cdot X + b \leq 0\}$  的部分"""
    if not poly:
        return []
    out = []
    n = len(poly)
    for i in range(n):
        P = poly[i]
        Q = poly[(i + 1) % n]
        fp = float(a @ P + b)
        fq = float(a @ Q + b)
        if fp <= tol:
            out.append(P)
        if (fp < -tol and fq > tol) or (fp > tol and fq < -tol):
            t = fp / (fp - fq)
            out.append(P + t * (Q - P))
    ded = []
    for v in out:
        if not ded or np.linalg.norm(v - ded[-1]) > 1e-9:

```

```

        ded.append(v)
    if len(ded) > 1 and np.linalg.norm(ded[0] - ded[-1]) < 1e-9:
        ded.pop()
    return ded

def region_by_clipping(pts, svds, err=BEARING_ERR, box=CLIP, tol=DIST_TOL):
    """从大框出发逐个半平面裁剪。

    返回 (P, status): status ∈ {"empty", "degenerate", "polygon", "unbounded"};
    多边形触到外框 (|坐标| > box/2) 即判定区域无界, P 是被截断的多边形。
    """
    poly = [np.array([-box, -box], float), np.array([box, -box], float),
            np.array([box, box], float), np.array([-box, box], float)]
    for p, th in zip(pts, svds):
        for (a, b) in wedge_halfplanes(p, th, err):
            poly = clip_halfplane(poly, a, b, tol)
            if not poly:
                return np.zeros((0, 2)), "empty"
    P = np.array(poly, float)
    if len(P) < 3:
        return P, "degenerate"
    if float(np.max(np.abs(P))) > box * 0.5:
        return P, "unbounded"
    H = convex_hull(P)
    return (H if len(H) >= 3 else P), "polygon"

def region_from_bearings(pts, svds, err=BEARING_ERR):
    """主入口 (兼容旧接口): 返回 (顶点数组, status)。

    status ∈ {"empty", "unbounded", "degenerate", "bounded"}。
    """
    P, st = region_by_vertices(pts, svds, err)
    if st == "empty":
        return P, "empty"
    rec = region_is_unbounded(list(svds), err) if len(svds) else None
    if rec is not None:
        return P, "unbounded"
    if st == "degenerate":
        return P, "degenerate"
    return P, "bounded"

# -----
# 直径与最小包围圆
# -----
def _diameter_brute(P):
    n = len(P)
    best, pair = -1.0, (P[0], P[1])
    for i in range(n - 1):

```

```

    d = np.linalg.norm(P[i + 1:] - P[i], axis=1)
    k = int(np.argmax(d))
    if float(d[k]) > best:
        best, pair = float(d[k]), (P[i], P[i + 1 + k])
    return best, pair

def rotating_calipers(P):
    """凸多边形直径（旋转卡壳， $O(n)$ ）。P 需为逆时针凸序（允许共线点）。"""
    n = len(P)
    if n < 3:
        return diameter(P, brute_limit=2)
    k = 1
    best, pair = -1.0, (P[0], P[1])
    for i in range(n):
        i2 = (i + 1) % n
        e = P[i2] - P[i]
        while True:
            k2 = (k + 1) % n
            if abs(_cross2(e, P[k2] - P[i])) > abs(_cross2(e, P[k] - P[i])):
                k = k2
            else:
                break
        for j in (k, (k + 1) % n):
            d = float(np.linalg.norm(P[i] - P[j]))
            if d > best:
                best, pair = d, (P[i], P[j])
            d = float(np.linalg.norm(P[i2] - P[j]))
            if d > best:
                best, pair = d, (P[i2], P[j])
    return best, pair

def diameter(P, brute_limit=DIAM_BRUTE_LIMIT):
    """凸多边形直径，返回 (D, (A, B))"""
    n = len(P)
    if n == 0:
        return 0.0, None
    if n == 1:
        return 0.0, (P[0], P[0])
    if n == 2:
        return float(np.linalg.norm(P[0] - P[1])), (P[0], P[1])
    if n <= brute_limit:
        return _diameter_brute(P)
    return rotating_calipers(P)

def min_enclosing_circle(P):
    """最小包围圆 (center, radius)。顶点数很少，直接枚举 1/2/3 点候选圆。

    理论:  $r^* \in [D/2, D/\sqrt{3}]$  (Jung 定理, 平面)，等号右端仅当区域为等边三角形时取到。

```

```

"""
P = np.asarray(P, float)
n = len(P)
if n == 0:
    return None, 0.0
if n == 1:
    return P[0].copy(), 0.0
cands = [(P[i].copy(), 0.0) for i in range(n)]
for i in range(n):
    for j in range(i + 1, n):
        c = 0.5 * (P[i] + P[j])
        cands.append((c, float(np.linalg.norm(P[i] - c))))
for i in range(n):
    for j in range(i + 1, n):
        for k in range(j + 1, n):
            A, B, C = P[i], P[j], P[k]
            d = 2.0 * (A[0] * (B[1] - C[1]) + B[0] * (C[1] - A[1])
                        + C[0] * (A[1] - B[1]))
            if abs(d) < 1e-12:
                continue
            aa, bb, cc = float(A @ A), float(B @ B), float(C @ C)
            ux = (aa * (B[1] - C[1]) + bb * (C[1] - A[1]) + cc * (A[1] - B[1])) / d
            uy = (aa * (C[0] - B[0]) + bb * (A[0] - C[0]) + cc * (B[0] - A[0])) / d
            c = np.array([ux, uy])
            cands.append((c, float(np.linalg.norm(A - c))))
best = None
for c, r in cands:
    if all(float(np.linalg.norm(P[i] - c)) <= r + 1e-9 for i in range(n)):
        if best is None or r < best[1]:
            best = (c, r)
return best

def region_area(P):
    n = len(P)
    if n < 3:
        return 0.0
    return abs(0.5 * float(np.sum(P[:, 0] * np.roll(P[:, 1], -1)
                                         - np.roll(P[:, 0], -1) * P[:, 1])))

# -----
# 汇总
# -----

def analyse(pts, svds, err=BEARING_ERR):
    """给定检测点与示向度，输出定位区域的全套几何量。"""
    P, status = region_from_bearings(pts, svds, err)
    base = {"n_bearings": len(pts), "status": status}
    if status == "empty":
        base["reason"] = ("所有示向度锥无公共交集（数据不一致：两两夹角可以 > 2°，"
                          "但错开的两个楔也可能不相交）")

```

```

    return base
if status == "unbounded":
    rec = region_is_unbounded(list(svds), err)
    base["reason"] = ("示向度锥近似平行（两两夹角  $\leq 2^\circ$ ），定位区域无界，"
                     "直径 = +inf; 必须换检测点重测")
    base["recession_direction_deg"] = None if rec is None else round(float(rec),
4)
    base["n_vertices"] = int(len(P))
    base["vertices"] = [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in
P]
    return base
if status == "degenerate" or len(P) < 3:
    base["status"] = "degenerate"
    base["reason"] = "公共交集退化为线段或点"
    base["vertices"] = [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in
P]
    return base

D, (A, B) = diameter(P)
c = 0.5 * (A + B)
maxr = float(max(np.linalg.norm(v - c) for v in P))
circle = min_enclosing_circle(P)
if circle is None:
    # Very thin polygons far from the origin can lose circumscribed
    # precision. Translation preserves the geometric problem.
    origin = np.mean(P, axis=0)
    local = min_enclosing_circle(P - origin)
    mc = origin + local[0] if local is not None else origin
    mr = float(np.max(np.linalg.norm(P - mc, axis=1)))
else:
    mc, mr = circle
base.update({
    "n_vertices": int(len(P)),
    "vertices": [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P],
    "diameter_m": round(D, 4),
    "diameter_endpoints": [[round(float(A[0]), 4), round(float(A[1]), 4)],
                           [round(float(B[0]), 4), round(float(B[1]), 4)]],
    "circle_center": [round(float(c[0]), 4), round(float(c[1]), 4)],
    "circle_radius_m": round(D / 2.0, 4),
    "max_vertex_dist_m": round(maxr, 4),
    "diameter_circle_covers": bool(maxr <= D / 2.0 + 1e-9),
    "min_enclosing_center": [round(float(mc[0]), 4), round(float(mc[1]), 4)],
    "min_enclosing_radius_m": round(float(mr), 4),
    "min_circle_over_half_diameter": round(float(mr) / (D / 2.0), 6),
    "area_m2": round(region_area(P), 4),
})
return base

def run_case_csv(path, truth_path=None, err=BEARING_ERR):
    with open(path, encoding="utf-8") as f:

```

```

        rows = list(csv.DictReader(f))
    pts = [(float(r["x_m"]), float(r["y_m"])) for r in rows]
    svds = [float(r["svd_deg"]) for r in rows]
    res = analyse(pts, svds, err)
    res["n_bearings"] = len(rows)
    if truth_path and os.path.exists(truth_path):
        with open(truth_path, encoding="utf-8") as f:
            truth = json.load(f)
            G = truth["source_xy"]
            res["truth_xy"] = G
            if "circle_center" in res:
                res["center_error_vs_truth_m"] = round(math.dist(res["circle_center"],
                                                                    G), 3)
            if "min_enclosing_center" in res:
                res["mincircle_error_vs_truth_m"] = round(
                    math.dist(res["min_enclosing_center"], G), 3)
            # 真值是否满足全部约束（等价于在半平面内）
            H = []
            for p, th in zip(pts, svds):
                H += wedge_halfplanes(p, th, err)
            res["truth_inside_region"] = bool(all(a @ np.asarray(G, float) + b <= 1e-6
                                                  for (a, b) in H))

    return res

def main():
    d = sys.argv[1] if len(sys.argv) > 1 else os.path.join(
        os.path.dirname(os.path.abspath(__file__)), "..", "data")
    out = []
    for fn in sorted(os.listdir(d)):
        if not (fn.startswith("p1_case") and fn.endswith(".csv")):
            continue
        res = run_case_csv(os.path.join(d, fn),
                           os.path.join(d, fn.replace(".csv", ".truth.json")))
        res["case"] = fn
        out.append(res)
    jl = os.path.join(d, "p1_results.jsonl")
    with open(jl, "w", encoding="utf-8") as f:
        for r in out:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")

    ok = [r for r in out if r["status"] == "bounded"]
    unlimited = [r for r in out if r["status"] == "unbounded"]
    summary = {
        "n_cases": len(out),
        "n_bounded": len(ok),
        "n_unbounded": len(unlimited),
        "n_degenerate": sum(1 for r in out if r["status"] == "degenerate"),
        "n_empty": sum(1 for r in out if r["status"] == "empty"),
        "diameter_m_range": [min((r["diameter_m"] for r in ok), default=None),
                             max((r["diameter_m"] for r in ok), default=None)],
    }

```

```

    "n_diameter_circle_misses": sum(1 for r in ok if not
        r["diameter_circle_covers"]),
    "min_circle_over_half_diameter_max": max(
        (r["min_circle_over_half_diameter"] for r in ok), default=None),
    "n_truth_inside_region": sum(1 for r in ok if r.get("truth_inside_region")),
}
with open(os.path.join(d, "p1_summary.json"), "w", encoding="utf-8") as f:
    json.dump(summary, f, ensure_ascii=False, indent=2)
for r in out:
    print("%-16s %-10s D=%-9s cover=%-5s n=%-2s r*/D*2=%-8s center_err=%s"
          % (r["case"], r["status"], r.get("diameter_m", "-"),
             r.get("diameter_circle_covers", "-"), r.get("n_bearings", "-"),
             r.get("min_circle_over_half_diameter", "-"),
             r.get("center_error_vs_truth_m", "-")))
print(json.dumps(summary, ensure_ascii=False))

if __name__ == "__main__":
    main()

```

0.3 Q4/src/p2_grid_expectation.py

"""B 题问题 2 (模拟): 固定首次探测扇形后, 第二检测点网格上的"交会区域直径期望"热力图。

问题设定 (按需求方给定的简化模型)

1. **固定当前探测得到的扇形区域**：探测器 (机器狗) 位于原点 $S_1 = (0, 0)$ ，探测方向为 x 轴正方向，示向度误差界 $\epsilon = 1 \text{ deg}$ 。于是第一次探测给出的信息是角楔

$$W_1 = \{ X : |\text{ang}(X - S_1) - \theta_1| \leq \epsilon \}$$

θ_1 取自当前探测数据 (默认 0 deg , 即"探测方向为 x 轴正方向")。

2. **干扰源均匀分布在扇形内, 近似认为都在 x 轴上**：源位 $G = (t, 0)$ ，权重由"扇形内面积均匀"退化得到：面积元 $dA = r dr d\theta \rightarrow w(t) \propto t$ ($--weight \text{ area}$ ，默认)；另提供等权 ($--weight \text{ length}$) 作对照。
3. **网格**：把每个网格点当作第二个检测点 S_2 。默认网格框在右侧多留 $0.2R$ ($x_{\text{range}} = (-1800, 2160)$)，因为最优点在右半区，这样同样点数下右半区更密。注意**可行域仍是靶区圆域** $R \leq 1800 \text{ m}$ ， $x > 1800$ 的部分是空框。
4. **交会区域与直径**：给定源位 G ，两个检测点各得一条带 $\pm\epsilon$ 的示向度锥，两者之交是凸四边形 $R = W_1 \cap W_2$ ，其**直径** $D =$ 区域内任意两点距离的最大值。
5. **期望**： $E[D](S_2) = \sum_i w_i D(S_2, t_i)$ ，只在"可检测且有界"的样本上取平均，同时记录"可检测概率"，避免把不可用点伪装成好点。
6. **热力图**： $E[D](S_2)$ 作为网格点 (x, y) 的函数，对数色标。

两套口径 (主 / 辅)

- * $--measured$ (主)：两个楔都用**测得值** $\pm 1^\circ$ ，即 $R = W_1(\theta_1) \cap W_2(\theta_2)$ ，期望只对源位 t 取 —— 完全对应"固定当前探测数据"的字面含义。
- * $--error_avg$ (辅)：同一个区域，但期望**同时对 $\pm 1^\circ$ 测量误差取**，即 $E_{\{t, e_1, e_2\}}[D]$ —— 更贴近"定位效果"的真实统计含义。

为什么网格必须是真正二维的（重要结论）

若把 ``S2`` 放在 x 轴上（``y = 0``），则两条示向度近似平行，夹角 ``|dtheta| <= 2\*eps``，
交会区域**无界**、直径为 $+\infty$ （这正是 ``p1\_intersection.region\_by\_vertices`` 会报
``unbounded`` 的退化几何，等价于“沿示向度方向逼近”）。所以 ``y = 0`` 必须在网格上被
剔除。更进一步，``|y|`` 很小但未退化的点，交会区域会被拉成极长的细条
（直径可达 10^4 – 10^5 m），数学上没错但工程上无用，程序用
``usable\_threshold = 10 \* 中位数`` 标注并掩码。

可检测性门控

第二个检测点要能收到该频道信号，必须 ``|S2 G| <= 有效接收半径上界 1500 m``。
不加这个门控会选出几何有效但物理收不到信号的假最优点，所以它是模型的一部分
（默认开启，``--no-reach`` 可关闭对照）。

正确性验证

直径由三条**互相独立**的实现给出，随机算例最大相对差 $< 1e-13$ ：
``p1\_intersection.region\_by\_vertices``（半平面枚举）、``independent\_diameter``
（世界坐标直接解边界交点）、``fast\_diameters\_vec``（角点枚举 + 楔定义筛选）。
``--selfcheck`` 另含蒙特卡洛与退化算例（T0–T10）。

用法

python src/p2_grid_expectation.py --selfcheck
python src/p2_grid_expectation.py --step 20 --n-t 500 --out out/p2_grid
python src/p2_grid_expectation.py --theta1 30 --out out/p2_grid_t30
"""
from __future__ import annotations

import argparse
import csv
import json
import math
import os
import sys
import time

import numpy as np

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
 sys.path.insert(0, _HERE)

from p1_intersection import BEARING_ERR, ang_diff, bearing, region_by_vertices #
 noqa: E402

物理常数（题目附录 1/2）

EPS_DEG = BEARING_ERR # 示向度误差界 1°

```

R_ARENA = 1800.0      # 目标区域半径 (m)
R_RECV_MIN = 1000.0   # 有效接收半径下界 (m, 接口不返回)
R_RECV_MAX = 1500.0   # 有效接收半径上界 (m, 取信号的必要条件)
NEAR_R = 5.0          # 近场阈值 (m): 更近则没有示向度

# 默认计算参数
DEF_STEP = 20.0
DEF_X_RANGE = (-R_ARENA, 1.2 * R_ARENA) # 右侧多留 0.2R (框内留白, 便于看全)
DEF_Y_RANGE = (-R_ARENA, R_ARENA)
DEF_N_T = 500          # x 轴上源位采样数
DEF_N_E = 400          # 误差 MC 次数 (error_avg 口径)
DEFAULT_SEED = 20260913

# -----
# 基本几何
# -----
def point_in_wedge(X, p, theta, err=EPS_DEG, tol=1e-9):
    """X 是否在角楔内 (与楔顶点重合视为在楔内)"""
    X = np.asarray(X, float)
    p = np.asarray(p, float)
    if float(np.linalg.norm(X - p)) < tol:
        return True
    return abs(ang_diff(bearing(p, X), theta)) <= err + 1e-12

def _unbounded_bearings(theta1, theta2, err=EPS_DEG):
    """两楔交无界 <=> 存在方向与两条示向度夹角都 <= err。

    对两个示向度, 等价于 ``min(|dth|, 180-|dth|) <= 2*err``。
    """
    d = abs(ang_diff(theta2, theta1))
    return min(d, 180.0 - d) <= 2.0 * err + 1e-12

def _corner_points(Pw, thP, Qw, thQ, eps):
    """两个角楔交的 6 个候选角点, **全部在世界坐标下求解**。

    每个角楔 ``|ang(X - P) - θ| <= eps`` 的两条边界线方向为 ``θ ± eps``,
    交点由四线两两相交得到 (6 个候选); 顶点判定直接用楔的定义
    (候选点 X 是顶点 <=> 对每个楔都有 ``|ang(X-P) - θ| <= eps`` 或 X 恰在顶点 P)。
    这个判据与坐标系定向、2x2 解法的行列式符号都无关, 因此最稳健。

    返回 ``(corner_world, valid)`` 形状 ``(..., 6, 2)`` 与 ``(..., 6)``。
    """
    Pw = np.asarray(Pw, float)
    Qw = np.asarray(Qw, float)
    thP = np.asarray(thP, float)
    thQ = np.asarray(thQ, float)

    def two_lines(P, th):

```

```

    out = []
    for s in (-eps, eps):
        a = np.radians(th + s)
        n = np.stack([np.sin(a), -np.cos(a)], axis=-1)
        out.append((n, np.einsum("...i,...i->...", n, P)))
    return out

lines = two_lines(Pw, thP) + two_lines(Qw, thQ)

def in_wedge(X, P, th):
    d = X - P
    dist = np.hypot(d[..., 0], d[..., 1])
    ang = np.degrees(np.arctan2(d[..., 1], d[..., 0]))
    diff = (ang - th + 180.0) % 360.0 - 180.0
    return (np.abs(diff) <= eps + 1e-9) | (dist <= 1e-9)

corners, valid = [], []
for ii in range(4):
    for jj in range(ii + 1, 4):
        (n1, c1), (n2, c2) = lines[ii], lines[jj]
        det = n1[..., 0] * n2[..., 1] - n1[..., 1] * n2[..., 0]
        safe = np.where(np.abs(det) < 1e-15, 1e-15, det)
        X = np.stack([(c1 * n2[..., 1] - c2 * n1[..., 1]) / safe,
                      (n1[..., 0] * c2 - n2[..., 0] * c1) / safe], axis=-1)
        ok = in_wedge(X, Pw, thP) & in_wedge(X, Qw, thQ)
        corners.append(X)
        valid.append(ok)
    return np.stack(corners, axis=-2), np.stack(valid, axis=-1)

def _diam_from_corners(cw, v, shp):
    """从候选角点与其可用性掩码求凸四边形直径（世界坐标下，距离即真实距离）"""
    nP = cw.shape[-2]
    best = np.zeros(shp)
    for ii in range(nP):
        for jj in range(ii + 1, nP):
            d2 = ((cw[..., ii, 0] - cw[..., jj, 0]) ** 2
                  + (cw[..., ii, 1] - cw[..., jj, 1]) ** 2)
            best = np.where(v[..., ii] & v[..., jj], np.maximum(best, d2), best)
    return best, v.sum(axis=-1)

def independent_diameter(t, S2, eps=EPS_DEG):
    """交会区域直径的**独立参考实现**（标量、世界坐标、直接解边界交点）。

    与 ``_corner_points`` 走的是不同代码路径（这里用射线参数式解交点）。
    无界（``min(|dth|, 180-|dth|) <= 2*eps``）返回 ``inf``。
    """
    S2 = np.asarray(S2, float)
    t = float(t)
    th1 = 0.0 # S1 -> G

```

```

th2 = math.degrees(math.atan2(-S2[1], t - S2[0])) % 360.0 # S2 -> G
if _unbounded_bearings(th1, th2, eps):
    return math.inf

lines = []
for apex, th in (((0.0, 0.0), th1), (tuple(S2), th2)):
    for s in (-eps, eps):
        a = math.radians(th + s)
        lines.append((apex, (math.cos(a), math.sin(a))))

def in_wedge(X, apex, th):
    dx, dy = X[0] - apex[0], X[1] - apex[1]
    if math.hypot(dx, dy) < 1e-9:
        return True
    ad = math.degrees(math.atan2(dy, dx))
    return abs((ad - th + 180.0) % 360.0 - 180.0) <= eps + 1e-9

verts = []
for i in range(4):
    for j2 in range(i + 1, 4):
        (P1, u), (P2, v) = lines[i], lines[j2]
        det = u[0] * (-v[1]) - (-v[0]) * u[1]
        if abs(det) < 1e-14:
            continue
        rv = (P2[0] - P1[0], P2[1] - P1[1])
        s = (rv[0] * (-v[1]) - (-v[0]) * rv[1]) / det
        X = (P1[0] + s * u[0], P1[1] + s * u[1])
        if in_wedge(X, (0.0, 0.0), th1) and in_wedge(X, tuple(S2), th2):
            verts.append(X)
if len(verts) < 2:
    return math.inf
return max(math.dist(a, b) for i, a in enumerate(verts) for b in verts[i + 1:])

def fast_diameters_vec(S2s, ts, eps=EPS_DEG):
    """批量快路径: S2s (m,2) × ts (n,) -> D (m,n), status (m,n)。

    **源必须固定在 x 轴正半轴 (s1 在 原点) **，这是本题设定。

    status 编码: 0 = 有界多边形, 1 = 空, 2 = 退化, 3 = 无界。
    """
    S2s = np.asarray(S2s, float)
    ts = np.asarray(ts, float).ravel()
    m, n = S2s.shape[0], ts.size
    D = np.full((m, n), np.nan)
    status = np.ones((m, n), dtype=np.int8)
    if m == 0 or n == 0:
        return D, status
    r = np.hypot(S2s[:, 0], S2s[:, 1])
    ok_row = r >= 1e-9
    status[~ok_row, :] = 2

```

```

th2 = np.degrees(np.arctan2(-S2s[:, 1][:, None],
                           ts[None, :] - S2s[:, 0][:, None])) % 360.0
dth = np.abs((th2 - 0.0 + 180.0) % 360.0 - 180.0)
sep = np.minimum(dth, 180.0 - dth)
unb = (sep <= 2.0 * eps + 1e-12) | (~ok_row[:, None])
status[unb] = np.where(np.broadcast_to(ok_row[:, None], unb.shape), 3, 2)[unb]
ok = ~unb
if not np.any(ok):
    return D, status

S1w = np.zeros((m, n, 2))
S2w = np.broadcast_to(S2s[:, None, :], (m, n, 2))
th1g = np.zeros((m, n))
corner, valid = _corner_points(S1w, th1g, S2w, th2, eps) # 世界坐标
cw = corner.reshape(m, n, corner.shape[-2], 2)
v = valid.reshape(m, n, valid.shape[-1])
best, n_valid = _diam_from_corners(cw, v, (m, n))
status = np.where(ok & (n_valid < 2), 1, status).astype(np.int8)
finite = ok & (n_valid >= 2) & np.isfinite(best)
D = np.where(finite, np.sqrt(np.maximum(best, 0.0)), np.nan)
status = np.where(finite, 0, status).astype(np.int8)
return D, status

def fast_diameters(S2, ts, eps=EPS_DEG):
    """`fast_diameters_vec` 的单点版"""
    D, st = fast_diameters_vec(np.asarray(S2, float)[None, :],
                               np.asarray(ts, float).ravel(), eps)
    return D[0], st[0]

def _fast_pair_grid(S1, th1, S2, th2, eps):
    """通用批量快路径: `(...,)` 形状的 S1/th1/S2/th2 -> D 与 status。

    与 `fast_diameters_vec` 同一套做法, 但两个角楔都任意
    (不要求 S1 在原点、源在 x 轴上)。用于 error_avg 口径。
    """
    P1 = np.asarray(S1, float)
    P2 = np.asarray(S2, float)
    th1 = np.asarray(th1, float)
    th2 = np.asarray(th2, float)
    if th2.ndim > th1.ndim:
        th1 = th1.reshape(th1.shape + (1,) * (th2.ndim - th1.ndim))
    shp = np.broadcast_shapes(th1.shape, th2.shape)
    th1 = np.broadcast_to(th1, shp)
    th2 = np.broadcast_to(th2, shp)
    P1 = np.broadcast_to(np.asarray(P1, float)[..., None, :], shp + (2,))
    P2 = np.broadcast_to(np.asarray(P2, float)[..., None, :], shp + (2,))

    rho = np.linalg.norm(P1 - P2, axis=-1)
    sep = np.abs((th2 - th1 + 180.0) % 360.0 - 180.0)

```

```

sep = np.minimum(sep, 180.0 - sep)
D = np.full(shp, np.nan)
status = np.ones(shp, dtype=np.int8)
status[sep <= 2.0 * eps + 1e-12] = 3
status[rho < 1e-9] = 2
ok = ~((sep <= 2.0 * eps + 1e-12) | (rho < 1e-9))
if not np.any(ok):
    return D, status
corner, valid = _corner_points(P1, th1, P2, th2, eps)
cw = corner.reshape(shp + (corner.shape[-2], 2))
v = valid.reshape(shp + (valid.shape[-1],))
best, n_valid = _diam_from_corners(cw, v, shp)
status = np.where(ok & (n_valid < 2), 1, status).astype(np.int8)
finite = ok & (n_valid >= 2) & np.isfinite(best)
D = np.where(finite, np.sqrt(np.maximum(best, 0.0)), np.nan)
status = np.where(finite, 0, status).astype(np.int8)
return D, status

# -----
# 源位采样与权重
# -----
def sector_ray_exit(S1, a_deg, R=R_ARENA):
    """从 S1 沿方位角 a 到半径 R 圆域边界的距离"""
    S1 = np.asarray(S1, float)
    u = np.array([math.cos(math.radians(a_deg)), math.sin(math.radians(a_deg))])
    b = float(S1 @ u)
    c = float(S1 @ S1) - R * R
    disc = b * b - c
    return 0.0 if disc <= 0 else float(-b + math.sqrt(disc))

def source_samples(S1, theta1, err=EPS_DEG, n_t=DEF_N_T, r_arena=R_ARENA,
                  r_recv_max=R_RECV_MAX, near=NEAR_R):
    """x 轴上的源位采样（面积均匀权重）。

    来源：干扰源在扇形内面积均匀 ->  $dA = r dr d\theta$ ，
    对角向积分后  $w(r) \propto r$ ，故  $w_i \propto t_i$ （归一化后即离散先验）。
     $t_{hi} = \min(r_{recv\_max}, \text{沿 } \theta_{t1} \text{ 到靶区边界的距离})$ 。
    """
    S1 = np.asarray(S1, float)
    t_hi = min(r_recv_max, sector_ray_exit(S1, theta1, r_arena))
    t_lo = near
    if t_hi <= t_lo:
        return np.zeros(0), np.zeros(0), {"t_lo": t_lo, "t_hi": t_hi, "n_t": 0}
    ts = np.linspace(t_lo, t_hi, int(n_t))
    w_area = ts / float(ts.sum())
    info = {"t_lo": float(t_lo), "t_hi": float(t_hi), "n_t": int(n_t),
            "weight": "area",
            "t_peak_note": "w(t)  $\propto t$ （面积均匀退化的严格结果）",
            "sector_half_angle_deg": float(err),
    }

```

```

        "sector_full_width_at_t_hi_m": float(2.0 * t_hi *
            math.tan(math.radians(err)))})
    return ts, w_area, info

def length_weights(ts):
    w = np.ones_like(np.asarray(ts, float))
    return w / float(w.sum())

# -----
# 网格与期望
# -----

def make_grid(step=DEF_STEP, x_range=DEF_X_RANGE, y_range=DEF_Y_RANGE,
    arena_r=R_ARENA, domain="arena"):
    """生成网格点（仅保留靶区圆域内、且不贴 x 轴退化带的点）。

    默认网格框右侧多留 0.2R（`x_range=(-1800, 2160)`）：最优点在右半区
    （约  $x \approx +1080$  m），把点堆在那里比均匀铺满  $\pm 1800$  更划算。
    **注意可行域仍是靶区圆域**  $R \leq 1800$  m， $x > 1800$  的部分不会进入结果。
    """
    xs = np.arange(x_range[0], x_range[1] + 1e-9, step)
    ys = np.arange(y_range[0], y_range[1] + 1e-9, step)
    DX, DY = np.meshgrid(xs, ys) # shape (ny, nx)
    X, Y = DX.ravel(), DY.ravel()
    keep = (X * X + Y * Y <= arena_r * arena_r) & (np.abs(Y) > 1e-9)
    if domain == "reach":
        d = np.sqrt(np.minimum(X ** 2, (np.abs(X) - R_RECV_MAX) ** 2) + Y ** 2)
        keep &= (d <= R_RECV_MAX)
    return X[keep], Y[keep], xs, ys, DX.shape

def reach_ok(X, Y, ts, r_recv_max=R_RECV_MAX):
    """可检测性门控矩阵：`|S2 - G_i| <= r_recv_max`。

    源在 x 轴上，对固定 (x,y) 关于 t 的最小值在 `t = clip(x, t_lo, t_hi)` 处取到。
    """
    X = np.asarray(X, float)[: , None]
    Y = np.asarray(Y, float)[: , None]
    ts = np.asarray(ts, float)[None, :]
    t_clip = np.clip(X, float(np.min(ts)), float(np.max(ts)))
    return np.sqrt((X - t_clip) ** 2 + Y ** 2) <= r_recv_max + 1e-9

def expected_diameter_measured(X, Y, ts, w, eps=EPS_DEG, reach=True, chunk=4096):
    """主口径：区域用测得值  $\pm \text{eps}$ ，期望只对源位取。

    返回 `E`（`E[D*1{可检测}]`）、`E_det`（`E[D|可检测]`，主目标）、
    `cov`、`p_det`、`Dmin`、`Dmax`、`n_ok`。
    """
    S2 = np.column_stack([X, Y])

```

```

n = X.size
E = np.zeros(n)
E_det = np.full(n, np.nan)
cov = np.zeros(n)
p_det = np.zeros(n)
Dmin = np.full(n, np.nan)
Dmax = np.full(n, np.nan)
n_ok = np.zeros(n, dtype=int)
wsum = float(np.sum(w))
for s in range(0, n, chunk):
    e = min(n, s + chunk)
    D, _ = fast_diameters_vec(S2[s:e], ts, eps)
    m = np.isfinite(D)
    if reach:
        m &= reach_ok(S2[s:e, 0], S2[s:e, 1], ts)
    sw = np.where(m, w[None, :], 0.0).sum(axis=1)
    contrib = (np.where(m, D, 0.0) * w[None, :]).sum(axis=1)
    with np.errstate(invalid="ignore"):
        E[s:e] = contrib
        E_det[s:e] = np.where(sw > 0, contrib / np.maximum(sw, 1e-300), np.nan)
        Dmin[s:e] = np.where(m.any(axis=1),
                               np.min(np.where(m, D, np.inf), axis=1), np.nan)
        Dmax[s:e] = np.where(m.any(axis=1),
                               np.max(np.where(m, D, -np.inf), axis=1), np.nan)
    cov[s:e] = sw
    p_det[s:e] = sw / wsum
    n_ok[s:e] = m.sum(axis=1)
return {"E": E, "E_det": E_det, "cov": cov, "p_det": p_det,
        "Dmin": Dmin, "Dmax": Dmax, "n_ok": n_ok}

```

```

def expected_diameter_error_avg(X, Y, ts, w, eps=EPS_DEG, n_e=DEF_N_E,
                                chunk=1024, seed=DEFAULT_SEED, reach=True):
    """辅口径：区域仍取  $W1(\theta_1) \cap W2(\theta_2)$ ，但期望**同时对  $\pm\epsilon$  误差取**。
```

定义为 $E_{err}(X) = E_{\{e1, e2\}}[E_{\{t|e\}}[D * 1_{\{\text{可检测}\}}]]$ 。

实现用**分块蒙特卡洛**：对每批网格点抽 n_e 组独立误差 $(e1, e2)$ ，第 k 组只作用到该批第 k 个点上 — 每个网格点得到 n_e 个独立无偏估计，取均值。代价是 $n_e \times n_t \times \text{网格点数}$ ，而不是误差笛卡尔积（后者在 1 万网格点上要十几 GB，实测会被拖死）。

```

"""
S2 = np.column_stack([X, Y])
n = X.size
if n == 0:
    return {"E_err": np.zeros(0), "E_err_det": np.zeros(0),
            "cov_err": np.zeros(0), "n_pairs": 0}
rng = np.random.default_rng(seed)
E_err = np.zeros(n)
E_err_det = np.full(n, np.nan)
cov_err = np.zeros(n)

```

```

for s in range(0, n, chunk):
    e = min(n, s + chunk)
    blk = e - s
    E1 = rng.uniform(-eps, eps, blk)
    E2 = rng.uniform(-eps, eps, blk)
    S2b = S2[s:e]
    S1b = np.zeros((blk, 2))
    base2 = np.degrees(np.arctan2(-S2b[:, 1][:, None],
                                   ts[None, :] - S2b[:, 0][:, None])) % 360.0
    D, _ = _fast_pair_grid(S1b, E1[:, None], S2b, base2 + E2[:, None], eps)
    m = np.isfinite(D)
    if reach:
        m &= reach_ok(S2b[:, 0], S2b[:, 1], ts)
    contrib = (np.where(m, D, 0.0) * w[None, :]).sum(axis=1)
    wsum = np.where(m, w[None, :], 0.0).sum(axis=1)
    with np.errstate(invalid="ignore"):
        E_err_det[s:e] = np.where(wsum > 0, contrib / np.maximum(wsum, 1e-300),
                                   np.nan)
    cov_err[s:e] = wsum
    E_err[s:e] = contrib
return {"E_err": E_err, "E_err_det": E_err_det, "cov_err": cov_err,
        "n_pairs": int(n_e), "estimator": "逐点蒙特卡洛（无偏）"}

# -----
# 自检
# -----
def selfcheck(verbose=True, seed=7):
    """独立实现交叉校验 + 退化算例 + 期望一致性 (T0-T10) """
    out = []
    ok_all = True

    def rec(name, ok, detail=""):
        nonlocal ok_all
        ok_all = ok_all and ok
        out.append({"check": name, "ok": bool(ok), "detail": detail})
        if verbose:
            print(f"[{'OK ' if ok else 'FAIL'}] {name} {detail}")

    def th2_of(S2, t):
        return float(np.degrees(np.arctan2(-S2[1], t - S2[0])) % 360.0)

    def mc_points(S2, t, n_pt, rng_):
        """在两个楔各自锥内采样，保留落在交会区域内的点（向量化）"""
        th2 = th2_of(S2, t)
        L = rng_.uniform(5.0, 1900.0, n_pt)
        a1 = np.radians(rng_.uniform(-EPS_DEG, EPS_DEG, n_pt))
        P1 = np.stack([L * np.cos(a1), L * np.sin(a1)], axis=1)
        a2 = math.radians(th2) + np.radians(rng_.uniform(-EPS_DEG, EPS_DEG, n_pt))
        P2 = np.asarray(S2)[None, :] + np.stack([L * np.cos(a2), L * np.sin(a2)],
                                                  axis=1)

```

```

K = np.vstack([P1, P2])

def inw(P, apex, th):
    d = P - np.asarray(apex)[None, :]
    dist = np.hypot(d[:, 0], d[:, 1])
    angd = np.degrees(np.arctan2(d[:, 1], d[:, 0]))
    return ((np.abs((angd - th + 180.0) % 360.0 - 180.0) <= EPS_DEG + 1e-9)
            & (dist > 1e-9))

return K[inw(K, (0.0, 0.0), 0.0) & inw(K, S2, th2)]

def max_pair_dist(K, blk=512):
    dmax = 0.0
    for s0 in range(0, len(K), blk):
        dmax = max(dmax, float(np.max(
            np.linalg.norm(K[s0:s0 + blk, None, :] - K[None, :, :], axis=2))))
    return dmax

rng = np.random.default_rng(seed)

# ---- T0: 顶点个数=4 且每个顶点真在两个楔内 ----
n0, bad0 = 0, 0
for _ in range(2500):
    S2 = np.array([rng.uniform(-1500, 1500), rng.uniform(-1400, 1400)])
    if np.hypot(*S2) < 200.0:
        continue
    t = float(rng.uniform(200.0, 1450.0))
    if np.hypot(S2[0] - t, S2[1]) > R_RECV_MAX:
        continue
    th2 = th2_of(S2, t)
    if min(abs(ang_diff(th2, 0.0)), 180 - abs(ang_diff(th2, 0.0))) <= 10.0:
        continue
    D, _ = fast_diameters(S2, np.array([t]))
    if not np.isfinite(D[0]):
        continue
    n0 += 1
    c, v = _corner_points(np.zeros((1, 1, 2)), np.zeros((1, 1)),
                          S2.reshape(1, 1, 2), np.array([[th2]]), EPS_DEG)
    c, v = c[0, 0], v[0, 0]
    if int(v.sum()) != 4:
        bad0 += 1
        continue
    for k in np.nonzero(v)[0]:
        if not (point_in_wedge(c[k], (0.0, 0.0), 0.0)
                and point_in_wedge(c[k], S2, th2)):
            bad0 += 1
            break
rec("T0 顶点个数=4 且每个顶点真在两个楔内", bad0 <= 2,
    f"检查 {n0} 组, 异常 {bad0} (允许 ≤2 组数值退化)")

# ---- T1: 快路径 vs p1 参考实现 (良态) ----

```

```

n_bad, n_cmp, n_skip, max_dd = 0, 0, 0, 0.0
for _ in range(6000):
    S2 = np.array([rng.uniform(-1500, 1500), rng.uniform(-1500, 1500)])
    if np.hypot(*S2) < 1.0:
        continue
    t = float(rng.uniform(30.0, 1500.0))
    th2 = th2_of(S2, t)
    if min(abs(ang_diff(th2, 0.0)), 180 - abs(ang_diff(th2, 0.0))) <= 2 *
        EPS_DEG + 0.3:
        n_skip += 1
        continue
    D, _ = fast_diameters(S2, np.array([t]))
    P, pst = region_by_vertices([(0.0, 0.0), tuple(S2)], [0.0, th2])
    if pst != "polygon" or len(P) < 3:
        continue
    n_cmp += 1
    Pa = np.asarray(P, float)
    d_ref = max(math.dist(a, b) for i, a in enumerate(Pa) for b in Pa[i + 1:])
    if not np.isfinite(D[0]):
        n_bad += 1
        continue
    rel = abs(float(D[0]) - d_ref) / max(d_ref, 1e-12)
    max_dd = max(max_dd, rel)
    if rel > 1e-7:
        n_bad += 1
rec("T1 快路径 vs p1 参考实现 (直径, 良态)", n_bad == 0,
    f"比较 {n_cmp} 组, 跳过近临界 {n_skip} 组, 最大相对差 {max_dd:.3e}")

# ---- T2: 快路径 vs 独立实现 ----
worst, worst_cfg, n2 = 0.0, None, 0
for _ in range(8000):
    S2 = np.array([rng.uniform(-1500, 1500), rng.uniform(-1400, 1400)])
    if np.hypot(*S2) < 5.0:
        continue
    t = float(rng.uniform(20.0, 1500.0))
    D, _ = fast_diameters(S2, np.array([t]))
    if not np.isfinite(D[0]):
        continue
    dc = independent_diameter(t, S2)
    if not math.isfinite(dc):
        continue
    n2 += 1
    rel = abs(float(D[0]) - dc) / max(dc, 1e-12)
    if rel > worst:
        worst, worst_cfg = rel, (S2.copy(), t, float(D[0]), dc)
rec("T2 快路径 vs 独立实现 (直接解边界交点)", worst < 1e-9,
    f"{n2} 组, 最大相对差 {worst:.3e}"
    + (" if worst_cfg is None else "
    f"(最差 S2={np.round(worst_cfg[0],3)}, t={worst_cfg[1]:.1f}, "
    f"数值={worst_cfg[2]:.4f}, 独立={worst_cfg[3]:.4f})")

```

```

# ---- T3: 蒙特卡洛真值—直径 >= 区域内采样点对最大距离 ----
worst3, worst3_cfg, n3 = 0.0, None, 0
for _ in range(40):
    S2 = np.array([rng.uniform(-1200, 1200), rng.uniform(-1200, 1200)])
    if np.hypot(*S2) < 50.0:
        continue
    t = float(rng.uniform(100.0, 1400.0))
    if np.hypot(S2[0] - t, S2[1]) > R_RECV_MAX:
        continue
    th2 = th2_of(S2, t)
    if _unbounded_bearings(0.0, th2):
        continue
    D, _ = fast_diameters(S2, np.array([t]))
    if not np.isfinite(D[0]):
        continue
    K = mc_points(S2, t, 20000, rng)
    if len(K) < 2:
        continue
    dmax = max_pair_dist(K)
    n3 += 1
    over = dmax / float(D[0]) - 1.0
    if over > worst3:
        worst3, worst3_cfg = over, (S2.copy(), t, float(D[0]), dmax)
rec("T3 直径 >= 区域内 MC 采样点对最大距离", worst3 <= 1e-9,
    f"{n3} 组, 最大超出 {worst3:.3e}"
    + (" if worst3_cfg is None else "
        f"(S2={np.round(worst3_cfg[0],2)}, t={worst3_cfg[1]:.1f}, "
        f"数值={worst3_cfg[2]:.4f}, MC={worst3_cfg[3]:.4f}) "))

# ---- T4: MC 逼近性 ----
n4, ratios = 0, []
for _ in range(40):
    S2 = np.array([rng.uniform(-800, 800), rng.uniform(-800, 800)])
    if np.hypot(*S2) < 50.0:
        continue
    t = float(rng.uniform(100.0, 1400.0))
    if np.hypot(S2[0] - t, S2[1]) > R_RECV_MAX:
        continue
    th2 = th2_of(S2, t)
    if _unbounded_bearings(0.0, th2):
        continue
    D, _ = fast_diameters(S2, np.array([t]))
    if not np.isfinite(D[0]):
        continue
    K = mc_points(S2, t, 40000, rng)
    if len(K) < 2:
        continue
    n4 += 1
    ratios.append(max_pair_dist(K) / float(D[0]))
ratios = np.array(ratios)
ok4 = bool(ratios.size and np.all(ratios >= 0.6) and np.all(ratios <= 1.0 +

```

```

1e-9))
rec("T4 MC 最大距离 / 数值直径  $\in [0.6, 1]$ ", ok4,
    f"{n4} 组, 最小比值 {ratios.min() if ratios.size else float('nan'):.4f}, "
    f"最大比值 {ratios.max() if ratios.size else float('nan'):.4f}")

# ---- T5: 贴 x 轴退化必须被识别 ----
bad5, det5 = 0, []
for y in (0.0, 1e-6, 1.0):
    S2 = np.array([600.0, y])
    D, _ = fast_diameters(S2, np.array([300.0, 900.0]))
    det5.append(f"y={y}: D={np.round(D, 3).tolist()}")
    if not np.all(np.isnan(D)):
        bad5 += 1
rec("T5 贴 x 轴退化被识别 (无界/退化)", bad5 == 0, " | ".join(det5))

# ---- T6: y=0 且源比 S2 远 => 无界 ----
D, st = fast_diameters(np.array([600.0, 0.0]), np.array([900.0]))
rec("T6 y=0 且源比 S2 远 -> 无界 (必须剔除)",
    np.isnan(D[0]) and int(st[0]) == 3, f"D={D[0]}, status={int(st[0])}")

# ---- T7: 交会角  $90^\circ$  ( $S_2$  在源正上方) 时  $D \approx 2 \tan(\epsilon) |S_1 S_2|$  ----
t = 1000.0
t7_bad, t7_worst = [], 0.0
for y in (30.0, 60.0, 120.0, 1000.0):
    S2 = np.array([t, y])
    Dv, _ = fast_diameters(S2, np.array([t]))
    dc = independent_diameter(t, S2)
    target = 2 * math.tan(math.radians(EPS_DEG)) * float(np.hypot(*S2))
    rel = abs(float(Dv[0]) - target) / target
    t7_worst = max(t7_worst, rel)
    if abs(float(Dv[0]) - dc) / dc > 1e-9:
        t7_bad.append(f"y={y}: 数值={float(Dv[0]):.6f} 独立={dc:.6f} 不一致")
    if rel > 2e-3:
        t7_bad.append(f"y={y}: 相对差={rel:.2e}")
if _unbounded_bearings(0.0, 270.0):
    t7_bad.append("gamma*=90° 应判定为良态")
rec("T7 交会角=90° 时  $D \approx 2 \tan(\epsilon) |S_1 S_2|$  (实测误差 <  $2e-3$ )", not t7_bad,
    "; ".join(t7_bad) if t7_bad else f"四组, 最大相对差 {t7_worst:.2e}")

# ---- T8: 源位采样权重归一 + 上界 ----
ts, w, info = source_samples((0.0, 0.0), 0.0, n_t=200)
rec("T8 源位采样权重归一 + 上界正确",
    abs(w.sum() - 1.0) < 1e-12 and abs(info["t_hi"] - min(R_RECV_MAX, R_arena))
    < 1e-9,
    f"t $\in$  [{info['t_lo']:.1f}, {info['t_hi']:.1f}],  $\Sigma w - 1 = \{w.sum() - 1\} \leq .2e$ ")

# ---- T9: E[D|可检测] 与逐点加权平均一致 ----
X = np.array([300.0, 447.2, -200.0, 900.0])
Y = np.array([600.0, 800.0, 300.0, 0.0])
tt, ww, _ = source_samples((0.0, 0.0), 0.0, n_t=60)
res9 = expected_diameter_measured(X, Y, tt, ww)

```

```

manual = []
for i in range(X.size):
    num = den = 0.0
    for j in range(tt.size):
        D, _ = fast_diameters(np.array([X[i], Y[i]]), np.array([tt[j]]))
        if np.isfinite(D[0]):
            num += ww[j] * float(D[0])
            den += ww[j]
    manual.append(num / den if den > 0 else np.nan)
ok9 = np.allclose(np.array(manual), res9["E_det"], rtol=1e-10, equal_nan=True)
rec("T9 E[D|可检测] 与逐点加权平均一致", ok9,
    f"manual={np.round(manual,4).tolist()},
    batch={np.round(res9['E_det'],4).tolist()}")

# ---- T10: 网格框扩展的语义 (诚实版) ----
# 靶区是半径 1800 m 的圆域, 格点是"圆域 ∩ 网格框"; 因此**平移/放大框边界
# 不会改变圆域内的点数** (只改变框内空白的多少)。要真正加密只能缩小 step。
Xa, Ya, xsa, _, _ = make_grid(step=50.0, x_range=(-R_ARENA, R_ARENA))
Xb, Yb, xsb, _, _ = make_grid(step=50.0)
inside = (Xb ** 2 + Yb ** 2) <= R_ARENA ** 2 + 1e-9
same_count = (Xa.size == Xb.size)
rec("T10 网格框扩展不改变圆域内点数 (加密只能靠缩小 step)",
    bool(np.all(inside)) and same_count and xsb[-1] > xsa[-1],
    f"x_max {xsa[-1]:.0f}->{xsb[-1]:.0f} m: 圆域内点数均为 {Xa.size} (不变), "
    f"全部满足 R<=1800; 右半区点数与左半区相同 (格点本已对称)")

if verbose:
    print(f"\n自检总结: {'全部通过' if ok_all else '存在失败项'})"
return {"ok": ok_all, "checks": out}

# -----
# 输出: CSV / JSON
# -----
def _json_default(o):
    """把 numpy 标量/布尔/数组转成可序列化对象 (自检结果里会混进 np.bool_) """
    if isinstance(o, (np.bool_,)):
        return bool(o)
    if isinstance(o, np.integer):
        return int(o)
    if isinstance(o, np.floating):
        return float(o)
    if isinstance(o, np.ndarray):
        return o.tolist()
    raise TypeError(f"Object of type {type(o).__name__} is not JSON serializable")

def _fmt(v, nd=6):
    if v is None:
        return ""
    if isinstance(v, float) and (math.isnan(v) or math.isinf(v)):

```

```

        return "nan" if math.isnan(v) else "inf"
    return round(float(v), nd)

def save_outputs(res, outdir, tag=""):
    os.makedirs(outdir, exist_ok=True)
    X, Y = res["X"], res["Y"]
    path = os.path.join(outdir, f"p2_grid_map{tag}.csv")
    thr = res.get("usable_threshold_m", float("inf"))
    with open(path, "w", newline="", encoding="utf-8-sig") as f:
        wr = csv.writer(f)
        wr.writerow(["x_m", "y_m", "r_m", "phi_deg",
                     "E_diam_given_detectable_m", "E_diam_mixed_mm",
                     "p_detectable", "cov_weight", "Dmin_m", "Dmax_m",
                     "E_diam_err_given_detectable_m", "E_diam_err_mixed_m",
                     "cov_err", "n_ok", "usable", "near_axis_unusable_band"])
        for i in range(X.size):
            r = math.hypot(X[i], Y[i])
            phi = math.degrees(math.atan2(Y[i], X[i])) % 360.0
            ed = res["E_det"][i]
            usable = int(np.isfinite(ed) and ed <= thr and res["p_det"][i] > 0)
            wr.writerow([_fmt(X[i], 4), _fmt(Y[i], 4), _fmt(r, 4), _fmt(phi, 4),
                             _fmt(ed), _fmt(res["E"][i]),
                             _fmt(res["p_det"][i]), _fmt(res["cov"][i]),
                             _fmt(res["Dmin"][i]), _fmt(res["Dmax"][i]),
                             _fmt(res["E_err_det"][i]), _fmt(res["E_err"][i]),
                             _fmt(res["cov_err"][i]), int(res["n_ok"][i]), usable,
                             int(np.isfinite(ed) and ed > thr)])
        rep = {k: v for k, v in res.items()
               if k not in ("X", "Y", "E", "E_det", "cov", "p_det", "Dmin", "Dmax",
                           "n_ok",
                           "E_err", "E_err_det", "cov_err", "usable")}
        rep["csv"] = os.path.basename(path)
        with open(os.path.join(outdir, f"p2_grid_report{tag}.json"), "w",
                  encoding="utf-8") as f:
            json.dump(rep, f, ensure_ascii=False, indent=2, default=_json_default)
        return path

# -----
# 热力图
# -----

def _savefig(fig, path, dpi=150, tries=6):
    """保存图片并重试（Windows 上杀毒/索引偶尔短暂锁住刚写出的 PNG）。"""
    import time as _t
    last = None
    for k in range(tries):
        try:
            fig.savefig(path, dpi=dpi)
            return path
        except OSError as ex:

```

```

        last = ex
        _t.sleep(0.5 * (k + 1))
    raise last

def make_figures(res, outdir, tag=""):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    from matplotlib.colors import LogNorm
    from matplotlib.ticker import FixedLocator, FuncFormatter

    os.makedirs(outdir, exist_ok=True)
    X, Y, xs, ys, shape = res["X"], res["Y"], res["xs"], res["ys"], res["shape"]
    ny, nx = shape
    figs = []
    t_hi = float(res["source_prior"]["t_hi"]) # 扇形径向上界 = 有效接收半径上界

    def to_grid(vals):
        g = np.full((ny, nx), np.nan)
        g[np.searchsorted(ys, Y), np.searchsorted(xs, X)] = vals
        return g

    XLIM = (-1500.0, 2200.0) # x 负方向小一点; 正方向盖住右侧扩展后的网格框
    YLIM = (-2000.0, 2000.0)

    # 关注区 (E ~ 50-1500 m) 用对数色标, 低值段的差异才看得出来
    ZMIN, ZMAX = 50.0, 2000.0
    lnorm = LogNorm(vmin=ZMIN, vmax=ZMAX)
    MAJOR = [50, 70, 100, 150, 200, 300, 500, 700, 1000, 1500, 2000]
    MINOR = [60, 80, 120, 250, 400, 600, 800, 1200]

    ok_use = res.get("usable", np.ones(X.size, bool))
    GX = to_grid(X)
    GY = to_grid(Y)
    Gm = to_grid(np.where(ok_use, res["E_det"], np.nan))
    Ge = to_grid(np.where(ok_use, res["E_err_det"], np.nan))
    Gc = to_grid(res["p_det"])
    th = np.linspace(0, 2 * math.pi, 2000)

    def best_xy(G):
        r, c = np.unravel_index(int(np.nanargmin(G)), G.shape)
        return float(GX[r, c]), float(GY[r, c]), float(G[r, c])

    def show(ax, G, cmap, vmin=None, vmax=None, norm=None, ttl=None):
        kw = {"norm": norm} if norm is not None else {"vmin": vmin, "vmax": vmax}
        im = ax.imshow(G, origin="lower", extent=[xs[0], xs[-1], ys[0], ys[-1]],
                        cmap=cmap, interpolation="nearest", aspect="equal", **kw)
        ax.set_xlim(XLIM[0], XLIM[1])
        ax.set_ylim(YLIM[0], YLIM[1])
        ax.set_xlabel("x (m)"); ax.set_ylabel("y (m)")

```

```

if ttl:
    ax.set_title(ttl, fontsize=9)
return im

def ring(ax, style="w--", lw=1.0, alpha=0.85):
    """只画落在显示范围内的靶区圆弧 (R=1800 m) """
    cx, cy = R_ARENA * np.cos(th), R_ARENA * np.sin(th)
    m = ((cx >= XLIM[0]) & (cx <= XLIM[1]) & (cy >= YLIM[0]) & (cy <= YLIM[1]))
    ax.plot(np.where(m, cx, np.nan), np.where(m, cy, np.nan),
            style, lw=lw, alpha=alpha)

def draw_sector(ax, zorder=6, label=True):
    """画出**首次探测的扇形** (源位先验所在区域), 半透明。

    角楔半张角 ``eps = 1°``, 径向范围 ``[near, t_hi]``。``eps`` 只有 1°,
    所以这个扇形在整张图上很窄 (1500 m 处全宽仅约 52 m) — 为让它看得见,
    填充用半透明 + 亮色描边。**图内不放任何说明文字/框** (会遮挡内容),
    相关信息一律写进图例、标题或 `report.json`。

    注意 ``ax.fill`` 的 label 会按**网格点数**产生成百上千条图例项,
    所以这里不传 label, 图例项改由下面的 ``sector_patch`` 提供。
    """
    t1 = t_hi
    ang_f = np.linspace(-EPS_DEG, EPS_DEG, 400)
    ax.plot(t1 * np.cos(np.radians(ang_f)), t1 * np.sin(np.radians(ang_f)),
            "-", color="darkorange", lw=1.6, alpha=0.95, zorder=zorder)
    n = 240
    rr = np.linspace(NEAR_R, t1, n)
    aa = np.linspace(-EPS_DEG, EPS_DEG, n)
    A, R_ = np.meshgrid(np.radians(aa), rr)
    ax.fill(R_ * np.cos(A), R_ * np.sin(A), color="darkorange", alpha=0.6,
            lw=0, zorder=zorder)
    for s in (-EPS_DEG, EPS_DEG):
        a = math.radians(s)
        ax.plot([NEAR_R * math.cos(a), t1 * math.cos(a)],
                [NEAR_R * math.sin(a), t1 * math.sin(a)],
                "-", color="darkorange", lw=1.6, alpha=0.95, zorder=zorder)

def sector_patch():
    from matplotlib.patches import Patch
    return Patch(fc="darkorange", alpha=0.6, ec="darkorange",
                label=f"1st-detection sector ( $\pm{EPS\_DEG:g}^\circ$ ,  $r \leq {t\_hi:.0f}$  m)")

def set_ticks(cb, size=9):
    """色标刻度: 主/次刻度都带数字, 且**字号统一** (不再大小不一) """
    cb.set_ticks(MAJOR)
    cb.ax.set_yticklabels([f"{t:g}" for t in MAJOR])
    cb.ax.yaxis.set_minor_locator(FixedLocator(MINOR))
    cb.ax.yaxis.set_minor_formatter(FuncFormatter(lambda v, pos: f"{v:g}"))
    cb.ax.tick_params(which="both", labelsz=size, length=3.5)

```

```

def origin_marker(ax, ms=13, label=True):
    """第一检测点（加粗圆点）。返回 artist，便于显式组织图例。"""
    ln, = ax.plot([0], [0], marker="o", ms=ms, mfc="red", mec="k", mew=1.5,
                  ls="none", zorder=9,
                  label=("S1 = origin (detector)" if label else None))
    return ln

def best_marker(ax, G, ms=10, label=True):
    """最优第二检测点及关于 x 轴对称的那一点（图内不加文字，信息进图例）"""
    bx, by, vb = best_xy(G)
    pts = [by, -by] if abs(by) > 1e-9 else [by]
    first = None
    for n_, yy in enumerate(pts):
        ln, = ax.plot([bx], [yy], "wo", ms=ms, mfc="none", mew=2, ls="none",
                      zorder=7,
                      label=("best S2 (symmetric pair)" if (label and n_ == 0) else
                              None))
    if first is None:
        first = ln
    return bx, by, vb, first

# ----- 图 1: 主口径热力图 -----
fig, ax = plt.subplots(figsize=(10.6, 10.6))
im = show(ax, Gm, "viridis_r", norm=lnorm)
fig.subplots_adjust(top=0.88, bottom=0.07, left=0.09, right=0.98)
ax.set_title("B-Problem 2: expected intersection-region diameter vs 2nd
             detection point",
             fontsize=10, pad=10)
fig.text(0.5, 0.965,
        "log colour scale; measured bearings; source prior: area-uniform inside
        the"
        " 1st-detection sector;\n"
        f"${\varepsilon}={res['eps_deg']}^\circ$, $\theta_1={res['theta1']}^\circ$, "
        f"reach gate $|S_{2G}|\leq\{\text{int}(R_{\text{RCV\_MAX}})\} \text{ m}$, arena
        $\leq\{\text{int}(R_{\text{ARENA}})\} \text{ m}$, "
        f"grid step $\{\text{res['step']}:g\} \text{ m}$ ($\{\text{res['grid\_points']}\} \text{ points}$)",
        ha="center", va="bottom", fontsize=9)
cb = fig.colorbar(im, ax=ax, shrink=0.8, extend="both")
cb.set_label(f"E[intersection diameter | detectable] (m, log scale), "
            f"colourbar range {ZMIN:g}-{ZMAX:g} m")
set_ticks(cb)
ring(ax)
draw_sector(ax)
h_origin = origin_marker(ax)
bx, by, vb, h_best = best_marker(ax, Gm)
ax.legend(handles=[h_origin, h_best, sector_patch()],
          loc="lower left", fontsize=8, framealpha=0.9)
f1 = os.path.join(outdir, f"p2_grid_E_heatmap{tag}.png")
savefig(fig, f1); plt.close(fig); figs.append(f1)

# ----- 图 2: 两口径并排 -----

```

```

fig, axes = plt.subplots(1, 2, figsize=(16.4, 9.4))
for ax2, G, ttl in (
    (axes[0], Gm, "(a) measured bearings fixed\n"
     "expectation over source position only"),
    (axes[1], Ge, "(b) error-averaged\n"
     "expectation over source position AND  $\pm 1^\circ$  reading errors")):
    im = show(ax2, G, "viridis_r", norm=lnorm)
    ring(ax2)
    draw_sector(ax2, label=False)
    origin_marker(ax2, ms=11, label=False)
    ibx, iby, ibv, _ = best_marker(ax2, G, ms=8, label=False)
    ax2.set_title(f"{ttl}\nmin E = {ibv:.1f} m at ({ibx:.0f}, {iby:.0f}) m",
                  fontsize=9)
    cb2 = fig.colorbar(im, ax=ax2, shrink=0.75, extend="both")
    cb2.set_label("E[D | detectable] (m, log scale)")
    set_ticks(cb2, size=8)
f2 = os.path.join(outdir, f"p2_grid_E_heatmap_two{tag}.png")
fig.tight_layout(); _savefig(fig, f2); plt.close(fig); figs.append(f2)

# ----- 图 3: 可检测概率 -----
fig, ax = plt.subplots(figsize=(10.6, 10.0))
im = show(ax, Gc, "magma", 0.0, 1.0,
          ttl="Detectability: probability the 2nd detector can even hear the
            channel")
cbp = fig.colorbar(im, ax=ax, shrink=0.8)
cbp.set_label("P(|S2G| ≤ 1500 m) under source prior")
cbp.ax.tick_params(which="both", labelsize=9)
ring(ax, "c--", 1.0, 0.9)
draw_sector(ax)
h_origin = origin_marker(ax)
ax.legend(handles=[h_origin, sector_patch()],
          loc="lower left", fontsize=8, framealpha=0.9)
f3 = os.path.join(outdir, f"p2_grid_coverage{tag}.png")
_savefig(fig, f3); plt.close(fig); figs.append(f3)

# ----- 图 4: E(r) 与 E(phi) 剖面 -----
fig, axes = plt.subplots(1, 2, figsize=(14.0, 5.4))
r_all = np.hypot(X, Y)
phi_all = np.degrees(np.arctan2(Y, X)) % 360.0
for phi0, style in ((90.0, "-"), (80.0, "--"), (70.0, "-."), (45.0, ":")):
    m = np.abs((phi_all - phi0 + 180) % 360 - 180) < \
        res["step"] / (2 * np.maximum(r_all, 1)) * 1.2
    if m.sum() < 3:
        continue
    o = np.argsort(r_all[m])
    axes[0].plot(r_all[m][o], res["E_det"][m][o], style, lw=1.4,
                label=f" $\phi={\phi0:.0f}^\circ$ ")
axes[0].set_xlabel("baseline r = |S1S2| (m)"); axes[0].set_ylabel("E[D |
    detectable] (m)")
axes[0].set_title("radial profile (measured bearings)", fontsize=10)
axes[0].legend(fontsize=8); axes[0].grid(alpha=0.3)

```

```

for r0, style in ((200.0, "-"), (400.0, "--"), (800.0, "-."), (1200.0, ":")):
    m = np.abs(r_all - r0) < res["step"] * 1.2
    if m.sum() < 3:
        continue
    o = np.argsort(phi_all[m])
    axes[1].plot(phi_all[m][o], res["E_det"][m][o], style, lw=1.4,
        label=f"r={r0:.0f} m")
axes[1].set_xlabel(" $\phi = \arg(S_2)$  (deg)"); axes[1].set_ylabel("E[D | detectable] (m)")
axes[1].set_title("angular profile: minima near the 90°-crossing geometry",
    fontsize=10)
axes[1].legend(fontsize=8); axes[1].grid(alpha=0.3)
f4 = os.path.join(outdir, f"p2_grid_profiles{tag}.png")
fig.tight_layout(); _savefig(fig, f4); plt.close(fig); figs.append(f4)

# ----- 图 5: 交会区域随 S2 变化的示意 -----
from p1_intersection import convex_hull
fig, axes = plt.subplots(1, 3, figsize=(15.0, 5.2))
t_demo = 900.0
for ax2, (rr_, pp_) in zip(axes, ((900.0, 90.0), (180.0, 80.0), (900.0, 20.0))):
    S2 = np.array([rr_ * math.cos(math.radians(pp_)), rr_ *
        math.sin(math.radians(pp_))])
    th2 = float(np.degrees(np.arctan2(-S2[1], t_demo - S2[0])) % 360.0)
    P, stt = region_by_vertices([(0.0, 0.0), tuple(S2)], [0.0, th2])
    D, stf = fast_diameters(S2, np.array([t_demo]))
    for base, colr, anch in ((0.0, "tab:blue", np.zeros(2)), (th2, "tab:orange",
        S2)):
        for s in (-1, 1):
            a = math.radians(base + s * EPS_DEG)
            L = 2200.0
            ax2.plot([anch[0], anch[0] + L * math.cos(a)],
                [anch[1], anch[1] + L * math.sin(a)],
                color=colr, lw=1.0, alpha=0.75)
    if len(P) >= 3 and not _unbounded_bearings(0.0, th2):
        H = convex_hull(np.asarray(P, float))
        H = np.vstack([H, H[:1]])
        ax2.fill(H[:, 0], H[:, 1], color="crimson", alpha=0.55, zorder=5)
        ax2.plot([0], [0], marker="o", ms=11, mfc="red", mec="k", mew=1.4, zorder=6)
        ax2.plot([S2[0]], [S2[1]], "o", color="tab:orange", ms=7, zorder=6)
        ax2.plot([t_demo], [0], "k^", ms=8, zorder=6)
        ax2.set_xlim(-1500, 2400); ax2.set_ylim(-1400, 2100)
        dv = float(D[0]) if np.isfinite(D[0]) else float("nan")
        ax2.set_title(f"source t=900 m, r={rr_:.0f} m,  $\phi$ ={pp_:.0f}°\n"
            f"intersection D = {dv:.1f} m", fontsize=9)
        ax2.set_xlabel("x (m)"); ax2.set_ylabel("y (m)"); ax2.grid(alpha=0.25)
    f5 = os.path.join(outdir, f"p2_grid_region_demo{tag}.png")
    fig.tight_layout(); _savefig(fig, f5); plt.close(fig); figs.append(f5)
return figs

```

```

# 主流程
# -----
def run(thetal=0.0, step=DEF_STEP, n_t=DEF_N_T, n_e=DEF_N_E, weight="area",
        domain="arena", eps=EPS_DEG, outdir=None, figures=True,
        seed=DEFAULT_SEED, reach=True, checks=True, verbose=True,
        x_range=DEF_X_RANGE, y_range=DEF_Y_RANGE):
    t0 = time.time()
    S1 = (0.0, 0.0)
    ts, w_area, sinfo = source_samples(S1, thetal, eps, n_t)
    w = w_area if weight == "area" else length_weights(ts)
    if ts.size == 0:
        raise SystemExit("源位采样为空: 检查 thetal 与靶区/接收半径设置")
    X, Y, xs, ys, shape = make_grid(step=step, x_range=x_range, y_range=y_range,
                                     arena_r=R_ARENA, domain=domain)

    if verbose:
        print(f"网格点 {X.size} 个 (step={step} m, domain={domain}, "
              f"x∈[{xs[0]:.0f},{xs[-1]:.0f}], y∈[{ys[0]:.0f},{ys[-1]:.0f}]); "
              f"源位 {ts.size} 个 t∈[{ts[0]:.1f},{ts[-1]:.1f}] m, 权重={weight}")
    meas = expected_diameter_measured(X, Y, ts, w, eps, reach=reach)
    err = expected_diameter_error_avg(X, Y, ts, w, eps, n_e=n_e, seed=seed,
                                     reach=reach)

    t_nom = float(np.sum(w * ts))
    med = float(np.nanmedian(meas["E_det"]))
    usable_thr = 10.0 * med
    usable = np.isfinite(meas["E_det"]) & (meas["E_det"] <= usable_thr) &
            (meas["p_det"] > 0)
    E_use = np.where(usable, meas["E_det"], np.nan) if np.any(usable) else
            meas["E_det"]

    def _pick(a):
        a = np.asarray(a, float)
        if not np.any(np.isfinite(a)):
            return None
        i = int(np.nanargmin(a))
        return {"S2": [float(X[i]), float(Y[i])],
                      "r_m": float(math.hypot(X[i], Y[i])),
                      "phi_deg": float(math.degrees(math.atan2(Y[i], X[i])) % 360.0),
                      "value_m": float(a[i]), "index": i,
                      "p_det": float(meas["p_det"][i]),
                      "E_mixed_m": float(meas["E"][i]),
                      "E_err_det_m": (float(err["E_err_det"][i])
                                      if np.isfinite(err["E_err_det"][i]) else None),
                      "E_err_mixed_m": float(err["E_err"][i])}

    best_det, best_mix = _pick(E_use), _pick(meas["E"])
    n_band = int((np.isfinite(meas["E_det"]) & (meas["E_det"] > usable_thr)).sum())
    res = {
        "model": "P2 grid simulation: E[intersection-region diameter] over 2nd
                 detection point",
        "thetal": float(thetal), "eps_deg": float(eps), "weight": weight, "domain":

```

```

    domain,
    "step": float(step), "reach_gate": bool(reach),
    "grid_x_range": [float(xs[0]), float(xs[-1])],
    "grid_y_range": [float(ys[0]), float(ys[-1])],
    "source_prior": sinfo, "arena_r": R_ARENA,
    "r_recv_range": [R_RECV_MIN, R_RECV_MAX], "near_r": NEAR_R,
    "S1": list(S1), "grid_points": int(X.size), "shape": list(shape),
    "xs": [float(v) for v in xs], "ys": [float(v) for v in ys],
    "usable_threshold_m": float(usable_thr),
    "X": X, "Y": Y,
    "E": meas["E"], "E_det": meas["E_det"], "cov": meas["cov"], "p_det":
        meas["p_det"],
    "Dmin": meas["Dmin"], "Dmax": meas["Dmax"], "n_ok": meas["n_ok"], "usable":
        usable,
    "E_err": err["E_err"], "E_err_det": err["E_err_det"],
    "cov_err": err["cov_err"], "n_err_pairs": err["n_pairs"],
    "summary": {
        "objective": "E[D | detectable] ( $|S2G| \leq 1500$  m
            门控, 按扇形内面积均匀先验加权)",
        "E_det_min_m": float(np.nanmin(E_use)),
        "E_det_median_m": med,
        "E_det_max_m": float(np.nanmax(meas["E_det"])),
        "E_mixed_min_m": float(np.nanmin(meas["E"])),
        "usable_points": int(usable.sum()),
        "usable_fraction": float(usable.mean()),
        "near_axis_band_points": n_band,
        "best_det": best_det, "best_mixed": best_mixed,
        "best_S2": best_det["S2"] if best_det else None,
        "best_r_m": best_det["r_m"] if best_det else None,
        "best_phi_deg": best_det["phi_deg"] if best_det else None,
        "best_r_over_t": (best_det["r_m"] / t_nom) if best_det else None,
        "mean_source_range_m": t_nom,
        "theory_note": ("交会角  $90^\circ$  时 D 取该基线下的最小值  $\approx 2 \tan(\epsilon) |S1S2|$ ; "
            "一般情形 D 随基线与近筒并程度增大"),
        "x_axis_degenerate": ("y $\approx 0$  时两示向度近平行 ( $|\Delta\theta| \leq 2\epsilon$ )  $\rightarrow$ 
            交会区域无界、直径 $=+\infty$ , "
            "被剔除: 其邻域 ( $|y|$  小) 直径可达  $10^4$ - $10^5$  m, 已用 "
            f"usable_threshold={usable_thr:.1f} m 标注为不可用带"),
        "reach_gate": (" $|S2G| >$  有效接收半径上界 1500 m 的源位样本不可检测, "
            "不计入 E[D]可检测; 混口径 E[D $\cdot 1$ {可检测}] 另列"),
        "grid_note": ("网格框右侧多留  $0.2R$  (x 到 2160 m) 以便右半区更密; "
            "可行域仍为靶区圆域  $R \leq 1800$  m"),
        "runtime_s": None,
    },
},
}
if checks:
    res["checks"] = selfcheck(verbose=verbose)["checks"]
res["summary"]["runtime_s"] = round(time.time() - t0, 3)
if outdir:
    save_outputs(res, outdir)
if figures:

```

```

    try:
        figs = make_figures(res, outdir)
        res["figures"] = [os.path.basename(f) for f in figs]
    except Exception as ex:
        # 图失败不影响数值结果落盘
        res["figures_error"] = f"{type(ex).__name__}: {ex}"
    if verbose:
        print(f"[warn] 出图失败 (数值结果已保存): {ex}")
    save_outputs(res, outdir)
if verbose:
    s = res["summary"]
    b = s["best_det"]
    print(f"\n最优第二检测点 S2* = ({b['S2'][0]:.1f}, {b['S2'][1]:.1f}) m "
          f" (r={b['r_m']:.1f} m,  $\phi$ = {b['phi_deg']:.1f}°,
          r/(t)={s['best_r_over_t']:.3f}) ")
    print(f"E[D|可检测] 最小 = {s['E_det_min_m']:.3f} m; 中位数 =
          {s['E_det_median_m']:.3f} m; "
          f"单条基线 2tan(eps)r = {2*math.tan(math.radians(eps))*b['r_m']:.3f} m")
    print(f"该点 P(可检测) = {b['p_det']:.4f}; 混口径 E[D-1] = {b['E_mixed_m']:.3f}
          m; "
          f"误差口径 E[D|可检测] = {b['E_err_det_m']}")
    print(f"可用点 {s['usable_points']}/{res['grid_points']}")
    f" (近轴不可用带 {s['near_axis_band_points']} 个, 阈值 {usable_thr:.1f} m) "
    print(f"耗时 {s['runtime_s']} s")
return res

```

```

def main(argv=None):
    ap = argparse.ArgumentParser(
        description="B 题问题 2: 第二检测点网格上的交会区域直径期望热力图")
    ap.add_argument("--theta1", type=float, default=0.0,
                    help="首次探测的示向度 (度), 默认 0 (x 轴正向)")
    ap.add_argument("--step", type=float, default=DEF_STEP, help="网格步长 (m)")
    ap.add_argument("--n-t", type=int, default=DEF_N_T, help="x 轴源位采样数")
    ap.add_argument("--n-e", type=int, default=DEF_N_E, help="误差 MC 采样数")
    ap.add_argument("--weight", choices=["area", "length"], default="area",
                    help="源位先验权重: area  $\propto$  t (面积均匀, 默认) / length 等权")
    ap.add_argument("--domain", choices=["arena", "reach"], default="arena",
                    help="S2 取值范围: 整个靶区域圆 / 还要满足 |S2G|<=1500")
    ap.add_argument("--x-range", type=float, nargs=2, default=list(DEF_X_RANGE),
                    metavar=("XMIN", "XMAX"), help="网格 x 范围 (m), 默认右侧多留 0.2R")
    ap.add_argument("--y-range", type=float, nargs=2, default=list(DEF_Y_RANGE),
                    metavar=("YMIN", "YMAX"), help="网格 y 范围 (m)")
    ap.add_argument("--out", default=None, help="输出目录")
    ap.add_argument("--no-reach", action="store_true", help="关闭可检测性门控")
    ap.add_argument("--no-figures", action="store_true")
    ap.add_argument("--no-checks", action="store_true",
                    help="跳过自检 (只出结果, 快很多)")
    ap.add_argument("--selfcheck", action="store_true")
    a = ap.parse_args(argv)
    if a.selfcheck:
        r = selfcheck()

```

```

    if a.out:
        os.makedirs(a.out, exist_ok=True)
        with open(os.path.join(a.out, "p2_grid_selfcheck.json"), "w",
                    encoding="utf-8") as f:
            json.dump(r, f, ensure_ascii=False, indent=2, default=_json_default)
        return 0 if r["ok"] else 1
    run(theta1=a.theta1, step=a.step, n_t=a.n_t, n_e=a.n_e, weight=a.weight,
        domain=a.domain, outdir=a.out, figures=not a.no_figures,
        checks=not a.no_checks, reach=not a.no_reach,
        x_range=tuple(a.x_range), y_range=tuple(a.y_range))
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

0.4 Q4/src/p3_expect_field.py

"""B 题问题 3: 机器狗"基于期望的行动"决策场。

本模块只负责问题 3 的**决策层**：把"下一步该去哪里检测"变成一个可以在全局网络上逐格比较的标量场。它严格按下面的思路实现：

1. **全局网格**：在半径 1800 m 的靶区圆域内铺一张规则网格（`grid\_step`，默认 100 m），每个格子记录"如果机器狗去那里做下一次检测，这一步有多好"。
2. **第二问的网格模拟结果**：直接复用 :mod:`p2\_grid\_expectation` 产出的热力图 `out/p2\_grid/p2\_grid\_map.csv`。该图是在"第一检测点 :math:S_1=(0,0)`、示向度 :math:\theta\_1=0^\circ`"的基准几何下算出的 :math:E[D|\text{可检测}]`（ D = 交会定位区域直径）。
3. **变换 + 插值**： $E[D]$ 在**刚体变换下不变**（旋转 + 平移保持距离与夹角）。因此真实一次检测 `(S, θ)` 对应的场，就是把基准图**绕原点旋转 θ、再平移到 S**：

$$b = R(-\theta) \cdot (g - S) \quad (g \text{ 是全局网格点, } b \text{ 是基准图坐标})$$

$$E_g = E_{\text{base}}(b) \quad (\text{双线性插值})$$

4. **逐次叠加**：把每个"待测频道"的每一次检测都按上式投到全局网络上叠加。
5. **未覆盖的网格取最大值**：基准图里没有数据（靶区外 / 该点不可检测 / 近轴退化带）或数值超过上限的格子一律记 :math:E_{\max}=1500` m — 也就是"最差的位置"。这样它们只有在附近没有更好的格子时才会被选中，天然形成探索行为。
6. **归一化**：`E\_norm = E\_mean / E\_max ∈ [0,1]`（0 = 交会几何最好，1 = 最差）。
7. **距离惩罚层**：行动要花时间，所以在归一化场之上再叠加一层**与探测器距离线性增长**的代价：

$$\text{score}(g) = E_{\text{norm}}(g) + \lambda \cdot |g - p_{\text{robot}}| / (2 \cdot R_{\text{arena}})$$

λ 即"距离项系数"，可调（`dist\_weight`，默认 0.35；λ=1 表示"跑到靶区对面"等价于把定位质量从最好拉到最差）。已访问过的扫描点邻域、以及离当前位置过近的格子会被直接排除（重复检测不产生新信息）。

8. **选点**：取 `argmin score`。若整张图都被排除（例如一条方位都还没有），退化为

"最大最小距离"探索点（离所有已访问点最远的格子）。

用法

```
python src/p3_expect_field.py --selfcheck
python src/p3_expect_field.py --demo --dist-weight 0.35 --out out/p3
"""
from __future__ import annotations

import argparse
import csv
import json
import math
import os
import sys
from dataclasses import dataclass, field as dc_field

import numpy as np

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)

from pl_intersection import BEARING_ERR, ang_diff # noqa: E402

# -----
# 物理常数（题目附录 1/2 与模拟器接口说明）
# -----
R_ARENA = 1800.0      # 靶区半径（m）
E_MAX = 1500.0        # 未覆盖网格的默认最大值（m）——即"最差位置"
R_RECV_MIN = 1000.0   # 有效接收半径下界（m）
R_RECV_MAX = 1500.0   # 有效接收半径上界（m）
NEAR_R = 5.0          # 近场阈值（m）
R_CLEAR = 20.0        # 清除半径（m）
DIST_REF = 2.0 * R_ARENA # 距离惩罚的归一化尺度（靶区直径，m）

DEFAULT_BASE_CSV = os.path.normpath(
    os.path.join(_HERE, "..", "out", "p2_grid", "p2_grid_map.csv"))
DEFAULT_FAMILY_DIR = os.path.normpath(os.path.join(_HERE, "..", "out",
    "p3_family"))
E_COL_CANDIDATES = ("E_diam_given_detectable_m", "E_diam_mixed_mm",
    "E_diam_mixed_m")

# -----
# 基准图（第二问的网格模拟结果）
# -----
@dataclass
class BaseMap:
    """问题 2 的期望热力图：`E[j, i]` 对应 `(xs[i], ys[j])`，nan = 无数据。"""
```

```

xs: np.ndarray
ys: np.ndarray
E: np.ndarray
source: str = "p2_grid_expectation"
e_col: str = "E_diam_given_detectable_m"

# ----- 构造 -----
@property
def step(self) -> float:
    return float(self.xs[1] - self.xs[0])

@property
def extent(self) -> tuple:
    return (float(self.xs[0]), float(self.xs[-1]), float(self.ys[0]),
            float(self.ys[-1]))

@classmethod
def from_points(cls, X, Y, E, source="synthetic") -> "BaseMap":
    """由散点（规则网格掩码后）构造。"""
    X = np.asarray(X, float)
    Y = np.asarray(Y, float)
    E = np.asarray(E, float)
    xs = np.unique(np.round(X, 6))
    ys = np.unique(np.round(Y, 6))
    # 步长取最小正间隔（避免浮点噪声把网格拆散）
    dx = _grid_step(xs)
    dy = _grid_step(ys)
    xs = np.round(np.arange(xs[0], xs[-1] + 1e-9, dx), 6)
    ys = np.round(np.arange(ys[0], ys[-1] + 1e-9, dy), 6)
    G = np.full((ys.size, xs.size), np.nan)
    ix = np.searchsorted(xs, np.round(X, 6))
    iy = np.searchsorted(ys, np.round(Y, 6))
    ok = (ix < xs.size) & (iy < ys.size)
    G[iy[ok], ix[ok]] = E[ok]
    return cls(xs=xs, ys=ys, E=G, source=source)

@classmethod
def load_csv(cls, path=DEFAULT_BASE_CSV, e_col=None) -> "BaseMap":
    if not os.path.exists(path):
        raise FileNotFoundError(
            f"找不到第二问的网格模拟结果 {path}。请先运行\n"
            f"python src/p2_grid_expectation.py --step 40 --out out/p2_grid")
    with open(path, encoding="utf-8-sig") as f:
        rows = list(csv.DictReader(f))
    if not rows:
        raise ValueError(f"{path} 为空")
    col = e_col
    if col is None:
        for c in E_COL_CANDIDATES:
            if c in rows[0]:
                col = c

```

```

        break
    if col is None:
        raise ValueError(f"{path} 缺少期望值列, 现有列: {list(rows[0])}")
    X = np.array([float(r["x_m"]) for r in rows])
    Y = np.array([float(r["y_m"]) for r in rows])
    E = np.array([_to_float(r.get(col)) for r in rows])
    m = cls.from_points(X, Y, E, source=os.path.basename(path))
    m.e_col = col
    return m

@classmethod
def synthetic(cls, step=100.0, R=R_ARENA) -> "BaseMap":
    """自检用解析基准图（形状与真实图同类：近轴退化带 + 斜前方最优翼）。"""
    xs = np.arange(-R, R + 1e-9, step)
    X, Y = np.meshgrid(xs, xs)
    r = np.hypot(X, Y)
    phi = np.abs(np.degrees(np.arctan2(Y, X)))
    E = 60.0 + 900.0 * (1.0 - np.exp(-r / 700.0)) + 700.0 * np.exp(-phi / 12.0)
    E[r > R] = np.nan
    return cls(xs=xs, ys=xs.copy(), E=E, source="synthetic")

# ----- 查询 -----
def lookup(self, X, Y, e_max=E_MAX):
    """双线性插值查询。返回 `(E, covered)`；四点任一无数据即视为未覆盖。

    ``E`` 已按“未覆盖/超过上限一律取 e_max”的规则裁剪，可直接用于叠加。
    """
    X = np.asarray(X, float)
    Y = np.asarray(Y, float)
    shp = np.broadcast(X, Y).shape
    Xf = np.broadcast_to(X, shp).astype(float).ravel()
    Yf = np.broadcast_to(Y, shp).astype(float).ravel()
    x0, y0 = float(self.xs[0]), float(self.ys[0])
    dx, dy = self.step, float(self.ys[1] - self.ys[0])
    nx, ny = self.xs.size, self.ys.size
    fx = (Xf - x0) / dx
    fy = (Yf - y0) / dy
    i = np.floor(fx).astype(int)
    j = np.floor(fy).astype(int)
    inside = (i >= 0) & (i <= nx - 2) & (j >= 0) & (j <= ny - 2)
    ic = np.clip(i, 0, nx - 2)
    jc = np.clip(j, 0, ny - 2)
    tx = np.clip(fx - ic, 0.0, 1.0)
    ty = np.clip(fy - jc, 0.0, 1.0)
    G = self.E
    e00 = G[jc, ic]
    e10 = G[jc, ic + 1]
    e01 = G[jc + 1, ic]
    e11 = G[jc + 1, ic + 1]
    ok = inside & np.isfinite(e00) & np.isfinite(e10) & np.isfinite(e01) &
        np.isfinite(e11)

```

```

        val = (e00 * (1 - tx) * (1 - ty) + e10 * tx * (1 - ty)
              + e01 * (1 - tx) * ty + e11 * tx * ty)
        val = np.where(ok, val, np.nan)
        E_out = np.where(ok, np.minimum(val, e_max), e_max)
        return E_out.reshape(shp), ok.reshape(shp)

# ----- 变换 -----
def transformed_lookup(self, S, theta_deg, X, Y, e_max=E_MAX):
    """把基准图按"检测点 S + 示向度  $\theta$ "旋转平移后查询全局网格点 (X, Y)。

    ``b = R(- $\theta$ )·(g - S)``: 把全局点转回基准图坐标系（基准图的源在 +x 轴方向）。
    E[D] 在刚体变换下不变, 故 ``E(g) = E_base(b)``。
    """
    S = np.asarray(S, float)
    c = math.cos(math.radians(theta_deg))
    s = math.sin(math.radians(theta_deg))
    Xf = np.asarray(X, float) - S[0]
    Yf = np.asarray(Y, float) - S[1]
    bx = Xf * c + Yf * s
    by = -Xf * s + Yf * c
    return self.lookup(bx, by, e_max=e_max)

def _grid_step(v):
    d = np.diff(np.asarray(v, float))
    d = d[d > 1e-9]
    return float(np.min(d)) if d.size else 1.0

def _to_float(v):
    if v is None or v == "" or v == "nan":
        return float("nan")
    try:
        return float(v)
    except (TypeError, ValueError):
        return float("nan")

def save_map_csv(path, X, Y, E, e_col="E_diam_given_detectable_m"):
    os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
    with open(path, "w", newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f)
        w.writerow(["x_m", "y_m", e_col])
        for i in range(len(X)):
            v = E[i]
            w.writerow([round(float(X[i]), 4), round(float(Y[i]), 4),
                        "" if not np.isfinite(v) else round(float(v), 6)])]
    return path

# -----

```

```

# 基准图族: 按"先验外径 t_hi"参数化
# -----
class BaseMapFamily:
    """一族基准图 ``{t_hi: BaseMap}``。

    **为什么需要它（这一步不做对，整个"变换 + 插值"就不成立）**

    基准图里源位先验是"从检测点沿指向度射线、``t ∈ [5, t_hi]``、权重  $\propto t$ "。
    真实场景中源还必须落在靶区内，所以射线上先验的**外径** ``t_hi`` 随 ``(s,  $\theta$ )``
    变化（射线出靶区的距离可以只有几百米）。而  $\mathbb{E}[D]$  只依赖
    (相对位置 b, t_hi) 这一对量，因此把 t_hi 也当作族参数，旋转 + 平移 + 插值才严格成立。

    查询时按 ``t_hi`` 在族内做线性插值（先把未覆盖格子填成 e_max，再混合）。
    """

    def __init__(self, maps, t_ref=R_RECV_MAX, e_max=E_MAX):
        self.maps = {float(k): v for k, v in sorted(maps.items())}
        self.t_list = np.array(sorted(self.maps.keys()))
        self.t_ref = float(t_ref)
        self.e_max = float(e_max)
        self._cache = {}

# ----- 构造 -----
@classmethod
def build(cls, outdir, t_his=(150.0, 250.0, 350.0, 450.0, 600.0, 800.0, 1000.0,
                             1250.0, 1500.0), step=40.0, n_t=200, verbose=True):
    """按 p2 的口径重算一族基准图并落盘。"""
    from p2_grid_expectation import ( # 局部导入，避免循环依赖
        expected_diameter_measured, make_grid, source_samples)

    X, Y, xs, ys, shape = make_grid(step=step, arena_r=R_arena, domain="arena")
    os.makedirs(outdir, exist_ok=True)
    maps = {}
    files = []
    for T in t_his:
        ts, w, info = source_samples((0.0, 0.0), 0.0, n_t=n_t, r_arena=float(T))
        res = expected_diameter_measured(X, Y, ts, w)
        E = np.asarray(res["E_det"], float)
        m = BaseMap.from_points(X, Y, E, source=f"p3_family_thi{T:g}")
        fn = os.path.join(outdir, f"p2_grid_map_thi{int(round(T)):04d}.csv")
        save_map_csv(fn, X, Y, E)
        maps[float(T)] = m
        files.append(os.path.basename(fn))
    if verbose:
        fin = E[np.isfinite(E)]
        print(f" t_hi={T:6.0f} m -> {os.path.basename(fn)} "
              f"有效格 {fin.size}/{E.size}, 中位数 "
              f"{np.median(fin) if fin.size else float('nan'):.1f} m")
    man = {"kind": "p3_base_map_family", "step": float(step), "n_t": int(n_t),
           "arena_r": R_arena, "t_his": [float(t) for t in t_his], "files": files,
           "note": "每个文件是 s1=(0,0)、 $\theta_1=0$  下，源位先验  $t \in [5, t_{hi}]$  的  $E[D]$  可检测"}

```

```

        with open(os.path.join(outdir, "p2_grid_family.json"), "w",
                    encoding="utf-8") as f:
            json.dump(man, f, ensure_ascii=False, indent=2)
        return cls(maps)

    @classmethod
    def load_dir(cls, outdir):
        man_path = os.path.join(outdir, "p2_grid_family.json")
        if not os.path.exists(man_path):
            return None
        with open(man_path, encoding="utf-8") as f:
            man = json.load(f)
        maps = {}
        for t, fn in zip(man["t_hi"], man["files"]):
            maps[float(t)] = BaseMap.load_csv(os.path.join(outdir, fn))
        return cls(maps)

    @classmethod
    def load_or_build(cls, outdir, **kw):
        fam = cls.load_dir(outdir)
        if fam is not None:
            return fam
        return cls.build(outdir, **kw)

# ----- 查询 -----
def select(self, t_hi):
    """返回该 t_hi 对应的（可缓存的）混合基准图。"""
    t = float(min(max(t_hi, self.t_list[0]), self.t_list[-1]))
    key = round(t / 25.0)
    if key in self._cache:
        return self._cache[key]
    tt = key * 25.0
    if tt <= self.t_list[0]:
        m = self.maps[float(self.t_list[0])]
    elif tt >= self.t_list[-1]:
        m = self.maps[float(self.t_list[-1])]
    else:
        i = int(np.searchsorted(self.t_list, tt) - 1)
        T1, T2 = float(self.t_list[i]), float(self.t_list[i + 1])
        a = (tt - T1) / max(T2 - T1, 1e-9)
        m = blend_maps(self.maps[T1], self.maps[T2], a, self.e_max)
    if len(self._cache) < 64:
        self._cache[key] = m
    return m

def rebase(self, S, theta_deg):
    """给出 (S,  $\theta$ ) 检测对应的基准图：先按射线出靶区距离取 t_hi，再在族内插值。"""
    S = np.asarray(S, float)
    u = np.array([math.cos(math.radians(theta_deg)),
                  math.sin(math.radians(theta_deg))])
    b = float(S @ u)

```

```

        c = float(S @ S) - R_ARENA * R_ARENA
        disc = b * b - c
        exit_d = float(-b + math.sqrt(disc)) if disc > 0 else 0.0
        t_hi = min(R_RECV_MAX, exit_d)
        return self.select(t_hi), t_hi

def blend_maps(m1: BaseMap, m2: BaseMap, a, e_max=E_MAX) -> BaseMap:
    """把两张同网格基准图按权重 a 线性混合（未覆盖格子先填 e_max，偏保守）。"""
    a = float(min(max(a, 0.0), 1.0))
    E1 = np.where(np.isfinite(m1.E), m1.E, e_max)
    E2 = np.where(np.isfinite(m2.E), m2.E, e_max)
    covered = np.isfinite(m1.E) | np.isfinite(m2.E)
    E = np.where(covered, (1.0 - a) * E1 + a * E2, np.nan)
    return BaseMap(xs=m1.xs, ys=m1.ys, E=E,
        source=f"blend({m1.source},{m2.source},{a:.2f})")

# -----
# 决策场
# -----

@dataclass
class FieldConfig:
    grid_step: float = 100.0 # 全局网格步长 (m)
    e_max: float = E_MAX # 未覆盖网格的默认最大值 (m)
    dist_weight: float = 0.35 # 距离项系数  $\lambda$  (可调)
    visited_radius: float = 250.0 # 已访问扫描点的排除半径 (m)
    min_move: float = 60.0 # 离当前位置太近的格子直接排除 (m)
    arena_r: float = R_ARENA
    normalize: str = "mean" # mean (期望均值归一化) | max

class ExpectField:
    """全局网格上的"期望定位质量 + 距离"标量场。"""

    def __init__(self, base, cfg: FieldConfig | None = None):
        self.base = base
        self.family = base if isinstance(base, BaseMapFamily) else None
        self.cfg = cfg or FieldConfig()
        step = float(self.cfg.grid_step)
        R = float(self.cfg.arena_r)
        xs = np.arange(-R, R + 1e-9, step)
        GX, GY = np.meshgrid(xs, xs)
        m = np.hypot(GX, GY) <= R + 1e-9
        self.gx = GX[m]
        self.gy = GY[m]
        self.terms: list[tuple[np.ndarray, float, tuple]] = []

    # ----- 叠加 -----

    def add_bearing(self, S, theta_deg, weight=1.0, tag=None):
        """叠加一次检测 ``S,  $\theta$ `` 给出的期望场（已裁剪：未覆盖 = e_max）。"""

```

有基准图族时先用 (s, θ) 的"射线出靶区距离"选 t_{hi} , 再在族内插值, 这样"旋转 + 平移 + 插值"才是严格的刚体不变。

```

"""
S = np.asarray(S, float)
bm = self.base
t_hi = R_RECV_MAX
if self.family is not None:
    bm, t_hi = self.family.rebase(S, theta_deg)
E, cov = bm.transformed_lookup(S, theta_deg, self.gx, self.gy,
                                e_max=self.cfg.e_max)
self.terms.append((E, float(weight),
                    (tuple(float(v) for v in S), float(theta_deg), tag,
                     float(t_hi))))
return cov

def add_channel_bearings(self, bearings):
    """bearings = [(x, y, svd_deg), ...]"""
    for (x, y, th) in bearings:
        self.add_bearing((x, y), th)

def add_raw(self, values, weight=1.0, tag=None):
    """叠加一层已经算好的、直接定义在全局网格上的值（用于"探索项"）。"""
    v = np.asarray(values, float)
    if v.shape != self.gx.shape:
        v = np.broadcast_to(v, self.gx.shape)
    self.terms.append((v, float(weight), ("raw", tag)))
    return v

def discovery_term(self, meas_pts, min_dist_m, e_max=None):
    """探索项: 某频道"在哪些格点上重新检测才有新信息"。

    对"还没听到过"的频道, 若新检测点距它所有历史检测点都 < ``min_dist_m``,
    那么能新覆盖到的区域很小（有效接收半径至少 1000 m）, 几乎不可能听到 —— 记 e_max;
    反之记 0（无惩罚）。这样全局网格就会自动把扫描点推到"能发现新源"的地方。
    """
    e_max = float(self.cfg.e_max if e_max is None else e_max)
    if not meas_pts:
        return np.zeros(self.gx.shape)
    P = np.asarray(meas_pts, float)
    dmin = np.min(np.hypot(self.gx[:, None] - P[None, :, 0],
                           self.gy[:, None] - P[None, :, 1]), axis=1)
    return np.where(dmin >= float(min_dist_m), 0.0, e_max)

# ----- 聚合 / 打分 -----
def aggregate(self):
    """返回 ``(E_mean_m, coverage_count, n_terms)``。

    * 每一项先把未覆盖格子记成 ``e_max`` ("最差位置");
    * ``normalize='mean'``: 对各项取加权平均（即"总和归一化"）;
    * ``coverage_count``: 至少有一项覆盖到的格子数。
    """

```

```

"""
n = self.gx.size
if not self.terms:
    return (np.full(n, float(self.cfg.e_max)), np.zeros(n, dtype=int), 0)
W = 0.0
acc = np.zeros(n)
cov = np.zeros(n, dtype=int)
covered = np.zeros(n, dtype=bool)
for (E, w, _tag) in self.terms:
    acc += w * E
    W += w
    covered |= (E < self.cfg.e_max - 1e-9)
    cov += (E < self.cfg.e_max - 1e-9).astype(int)
E_mean = acc / max(W, 1e-12)
if self.cfg.normalize == "max" and np.any(covered):
    ref = float(np.max(E_mean))
    E_mean = E_mean * (float(self.cfg.e_max) / max(ref, 1e-12))
    E_mean = np.minimum(E_mean, float(self.cfg.e_max))
return E_mean, cov, len(self.terms)

def score(self, cur_xy, visited=(), extra_penalty=None, max_dist=None):
    """计算整张网格的 score = 归一化期望 + λ·距离/靶区直径 (排除项 = +inf)。"""
    E_mean, cov, n_terms = self.aggregate()
    e_norm = E_mean / max(float(self.cfg.e_max), 1e-12)
    if extra_penalty is not None:
        e_norm = e_norm + np.asarray(extra_penalty, float)
    p = np.asarray(cur_xy, float)
    d = np.hypot(self.gx - p[0], self.gy - p[1])
    score = e_norm + float(self.cfg.dist_weight) * d / DIST_REF
    mask = d < float(self.cfg.min_move)
    if max_dist is not None:
        mask |= d > float(max_dist)
    for q in visited:
        q = np.asarray(q, float)
        mask |= np.hypot(self.gx - q[0], self.gy - q[1]) <
            float(self.cfg.visited_radius)
    score = np.where(mask, np.inf, score)
    return score, {"E_mean_m": E_mean, "coverage": cov, "n_terms": n_terms,
        "e_norm": e_norm, "dist_m": d, "excluded": mask}

# ----- 选点 -----
def argmin(self, cur_xy, visited=(), extra_penalty=None, max_dist=None):
    if not self.terms:
        return None # 一条方位都没有: 交给探索兜底
    score, info = self.score(cur_xy, visited, extra_penalty, max_dist=max_dist)
    if not np.any(np.isfinite(score)):
        return None
    i = int(np.argmin(np.where(np.isfinite(score), score, np.inf)))
    return self._describe(i, score[i], info)

def fallback_point(self, cur_xy, visited=(), prefer_near=True, max_dist=None):

```

```

"""探索兜底：离所有已访问点"最远"的格子（最大最小距离准则）。

一条方位都没有、或整张图都被排除时使用；也用于避免反复在同一片区域打转。
``max_dist`` 给定时只在"还走得到"的格子里挑（用于时间预算收尾阶段）。
"""

p = np.asarray(cur_xy, float)
if visited:
    V = np.asarray(visited, float)
    dmin = np.min(np.hypot(self.gx[:, None] - V[None, :, 0],
                          self.gy[:, None] - V[None, :, 1]), axis=1)
else:
    dmin = np.hypot(self.gx - p[0], self.gy - p[1])
dc = np.hypot(self.gx - p[0], self.gy - p[1])
key = dmin - (1e-6 * dc if prefer_near else 0.0)
key = np.where(dc < float(self.cfg.min_move), -np.inf, key)
if max_dist is not None:
    key = np.where(dc > float(max_dist), -np.inf, key)
if not np.any(np.isfinite(key)):
    key = dmin - 1e-6 * dc # 放宽：忽略 max_dist/min_move 再挑一次
i = int(np.argmax(key))
E_mean, cov, n_terms = self.aggregate()
info = {"E_mean_m": E_mean, "coverage": cov, "n_terms": n_terms,
        "e_norm": E_mean / max(float(self.cfg.e_max), 1e-12),
        "dist_m": dc, "excluded": np.zeros(self.gx.size, bool)}
return self._describe(i, float("nan"), info, kind="explore")

def _describe(self, i, score, info, kind="min"):
    return {
        "kind": kind,
        "index": int(i),
        "xy": (float(self.gx[i]), float(self.gy[i])),
        "score": float(score),
        "E_mean_m": float(info["E_mean_m"][i]),
        "E_norm": float(info["e_norm"][i]),
        "dist_m": float(info["dist_m"][i]),
        "dist_penalty": float(self.cfg.dist_weight * info["dist_m"][i] /
                               DIST_REF),
        "n_terms": int(info["n_terms"]),
        "n_cover": int(info["coverage"][i]),
        "covered": bool(info["coverage"][i] > 0),
    }

# ----- 导出 -----
def save_csv(self, path):
    E_mean, cov, n_terms = self.aggregate()
    score, info = self.score((0.0, 0.0))
    os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
    with open(path, "w", newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f)
        w.writerow(["x_m", "y_m", "E_mean_m", "E_norm", "n_cover",
                    "score_at_origin"])

```

```

        for k in range(self.gx.size):
            w.writerow([round(float(self.gx[k]), 3), round(float(self.gy[k]), 3),
                        round(float(E_mean[k]), 4), round(float(E_mean[k]) /
                        self.cfg.e_max, 6),
                        int(cov[k]),
                        "" if not np.isfinite(score[k]) else round(float(score[k]),
                        6)])

    return path

# -----
# 精确期望（独立实现）：任意  $S_1/\theta_1$  下，候选第二检测点的  $E[D]$ 
# -----
def ray_source_samples(S, theta_deg, err=BEARING_ERR, n_t=200, r_arena=R_ARENA,
                      r_recv_max=R_RECV_MAX, near=NEAR_R):
    """源位先验：在  $S$  沿  $\theta$  的射线上、靶区内、可被有效接收的范围内按面积均匀采样。

    与 :func:`p2_grid_expectation.source_samples` 同一先验，但支持任意  $S$ （不只是原点）。
    返回 `` (ts, w, t_hi) ``: ``ts`` 为源到  $S$  的距离采样。
    """
    S = np.asarray(S, float)
    u = np.array([math.cos(math.radians(theta_deg)),
                  math.sin(math.radians(theta_deg))])
    b = float(S @ u)
    c = float(S @ S) - r_arena * r_arena
    disc = b * b - c
    exit_d = float(-b + math.sqrt(disc)) if disc > 0 else 0.0
    t_hi = min(r_recv_max, exit_d)
    if t_hi <= near:
        return np.zeros(0), np.zeros(0), 0.0
    ts = np.linspace(near, t_hi, int(n_t))
    w = ts.copy()
    return ts, w / float(w.sum()), t_hi

def exact_expected_diameter(S, theta_deg, S2, ts, w, err=BEARING_ERR,
                           r_recv_max=R_RECV_MAX, per_sample_gate=False):
    """独立实现（用于自检）：直接枚举源位样本，用精确的两楔交会算  $E[D]$ 。

    与基准图完全同构（源位先验 = 射线上的面积均匀），但**不做任何插值/变换**，
    因此可以用来校验 "旋转平移 + 双线性插值" 的正确性。

    ``per_sample_gate=False``（默认，与 :mod:`p2_grid_expectation` 一致）：
    "可检测"门控按**候选点**判定 —— 只要该点能收到先验中某个源位的信号就保留全部样本；
    置 True 则逐样本判定 ``|S2 - G| <= 有效接收半径上界``。
    """
    from p2_grid_expectation import _fast_pair_grid # 局部导入，避免循环依赖

    S = np.asarray(S, float)
    S2 = np.atleast_2d(np.asarray(S2, float))
    ts = np.asarray(ts, float)

```

```

w = np.asarray(w, float)
m, n = S2.shape[0], ts.size
if n == 0:
    return np.full(m, np.nan)
u = np.array([math.cos(math.radians(theta_deg)),
              math.sin(math.radians(theta_deg))])
G = S[None, :] + ts[:, None] * u[None, :]      # (n,2) 源位样本
th2 = np.degrees(np.arctan2(G[None, :, 1] - S2[:, None, 1],
                             G[None, :, 0] - S2[:, None, 0])) % 360.0
# 注意 _fast_pair_grid 的形状约定: 坐标数组要比 th 少一个尾维 ((m,2) 对 (m,n))
S1b = np.broadcast_to(S.reshape(1, 2), (m, 2))
th1b = np.full((m,), float(theta_deg))
S2b = S2
D, _st = _fast_pair_grid(S1b, th1b, S2b, th2, err)
d2 = np.hypot(S2[:, None, 0] - G[None, :, 0], S2[:, None, 1] - G[None, :, 1])
if per_sample_gate:
    reach = d2 <= r_recv_max + 1e-9
else:
    reach = np.broadcast_to((d2 <= r_recv_max + 1e-9).any(axis=1)[:, None],
                             D.shape)
ok = np.isfinite(D) & reach
wc = np.where(ok, w[None, :], 0.0)
sw = wc.sum(axis=1)
num = (np.where(ok, D, 0.0) * w[None, :]).sum(axis=1)
with np.errstate(invalid="ignore"):
    return np.where(sw > 0, num / np.maximum(sw, 1e-12), np.nan)

def best_refine_point(rec_pts, rec_svds, cur_xy, region, cfg: FieldConfig | None =
    None,
                      n_t=120, n_ang=16, radii=(100.0, 200.0, 300.0, 450.0, 600.0,
                      800.0),
                      travel_cap_m=900.0, err=BEARING_ERR):
    """在"已有一条/多条方位"的基础上, 选一个**局部**最优的下一检测点。

    与 :class:`ExpectField` (全局、用预计算基准图) 互补: 这里对任意  $S1/\theta1$  直接算
    精确的  $E[D]$ , 用于机器狗已经逼近目标、或刚发现一个新频道需要"再测一条方位把区域
    定下来"的场合。候选点取以当前位置为中心、半径 ``radii`` 的极角网格,
    代价 = 走到候选点 + 检测 + 走到目标区域 + 预期覆盖式清扫时间。

    ``region`` 为 ``{'center': (x, y)}``; 为 ``None`` (只有一条方位) 时用源位先验的
    中位点作为"目标区域", 从而把"测完还要往回走多远"算进代价。

    返回 ``(xy, info)``; 无可候选时返回 ``(None, None)``。
    """
    pts = [np.asarray(p, float) for p in rec_pts]
    if not pts:
        return None, None
    S1 = pts[-1]
    th1 = float(rec_svds[-1])
    ts, w, t_hi = ray_source_samples(S1, th1, n_t=n_t)

```

```

if ts.size == 0:
    return None, None
cur = np.asarray(cur_xy, float)
ang0 = math.degrees(math.atan2(cur[1] - S1[1], cur[0] - S1[0]))
cand = [cur]
for r in radii:
    for k in range(n_ang):
        a = math.radians(ang0 + 360.0 * k / n_ang)
        p = cur + r * np.array([math.cos(a), math.sin(a)])
        if np.hypot(*p) <= R_ARENA - 1.0:
            cand.append(p)
C = np.array(cand, float)
keep = np.hypot(C[:, 0] - cur[0], C[:, 1] - cur[1]) <= travel_cap_m
C = C[keep]
if C.size == 0:
    return None, None
E = exact_expected_diameter(S1, th1, C, ts, w, err=err)
dcur = np.hypot(C[:, 0] - cur[0], C[:, 1] - cur[1])
if region is not None:
    c = np.asarray(region["center"], float)
    back_target = "region"
else:
    u = np.array([math.cos(math.radians(th1)), math.sin(math.radians(th1))])
    c = S1 + float(ts[ts.size // 2]) * u # 先验中位源位
    back_target = "prior_median"
dreg = np.hypot(C[:, 0] - c[0], C[:, 1] - c[1])
# 代价 (秒) = 走到候选点 + 检测 + 走到目标 + 按 E[D] 估计的覆盖式清扫
# 清扫路程 ≈ 圆域面积 / 覆盖行距 =  $\pi \cdot (r)^2 / (2 \cdot 20)$ , r 取 E[D]/2 + 20 m
v = 5.0
sweep_m = np.pi * np.maximum(E / 2.0 + 20.0, 20.0) ** 2 / (2.0 * R_CLEAR)
cost = dcur / v + 6.0 + dreg / v + np.where(np.isfinite(E), sweep_m / v, 1e6)
i = int(np.argmin(cost))
info = {"S1": tuple(float(v) for v in S1), "th1": th1,
        "xy": (float(C[i, 0]), float(C[i, 1])),
        "E_diam_m": float(E[i]), "cost_s": float(cost[i]),
        "move_s": float(dcur[i] / v), "back_s": float(dreg[i] / v),
        "n_cand": int(C.shape[0]), "t_hi": float(t_hi), "back_target":
            back_target}
return (float(C[i, 0]), float(C[i, 1])), info

def _ring_candidates(S, theta_deg, radii=(400.0, 800.0, 1200.0), n_ang=8,
                    dphi0=30.0, arena_r=R_ARENA):
    """绕检测点 S 的环形候选点 (相对  $\theta$  偏开 dphi0, 避免正落在退化射线上)。"""
    S = np.asarray(S, float)
    C = []
    for r in radii:
        for k in range(n_ang):
            a = math.radians(theta_deg + dphi0 + (360.0 - 2 * dphi0) * k / max(n_ang - 1, 1))
            p = S + r * np.array([math.cos(a), math.sin(a)])

```

```

        if np.hypot(*p) <= arena_r - 1.0:
            C.append(p)
    return np.array(C) if C else np.zeros((0, 2))

# -----
# 自检
# -----
def selfcheck(base_csv=DEFAULT_BASE_CSV, verbose=True, seed=20260913):
    out = []
    ok_all = True

    def rec(name, ok, detail=""):
        nonlocal ok_all
        ok_all = ok_all and bool(ok)
        out.append({"check": name, "ok": bool(ok), "detail": detail})
        if verbose:
            print(f"[{'OK ' if ok else 'FAIL'}] {name} {detail}")

    rng = np.random.default_rng(seed)
    base = BaseMap.synthetic(step=50.0)
    E, cov = base.lookup(0.0, 0.0)
    rec("T1 基准图基本可查（原点有值）", bool(cov) and np.isfinite(float(E)),
        f"E(0,0)={float(E):.3f}, covered={bool(cov)}")

    # T2 旋转平移不变性：把基准网格点做刚体变换后查回来，应完全一致
    S = np.array([137.0, -421.0])
    th = 37.0
    c, s = math.cos(math.radians(th)), math.sin(math.radians(th))
    worst = 0.0
    for (x, y) in ((-500.0, 300.0), (0.0, 0.0), (1000.0, -800.0), (-1200.0,
        -1200.0)):
        e0, c0 = base.lookup(x, y)
        if not bool(c0):
            continue
        g = S + np.array([x * c - y * s, x * s + y * c]) # 正向变换到全局
        e1, c1 = base.transformed_lookup(S, th, g[0], g[1])
        worst = max(worst, abs(float(e1) - float(e0)))
    rec("T2 旋转+平移不变性（刚体变换下 E 不变）", worst < 1e-9,
        f"最大绝对差 {worst:.3e}")

    # T3 双线性插值精度：中心值应落在四邻点区间内，且与真值相差小
    bad = 0
    worst_rel = 0.0
    for _ in range(400):
        i = int(rng.integers(1, base.xs.size - 2))
        j = int(rng.integers(1, base.ys.size - 2))
        if not np.all(np.isfinite(base.E[j - 1:j + 2, i - 1:i + 2])):
            continue
        x = float(base.xs[i]) + float(rng.uniform(-0.49, 0.49)) * base.step
        y = float(base.ys[j]) + float(rng.uniform(-0.49, 0.49)) * base.step

```

```

v, cv = base.lookup(x, y)
if not bool(cv):
    continue
nb = base.E[j - 1:j + 2, i - 1:i + 2]
if not (nb.min() - 1e-9 <= float(v) <= nb.max() + 1e-9):
    bad += 1
# 与最近邻的差应不超过 1 个格距内的局部变化
worst_rel = max(worst_rel, abs(float(v) - base.E[j, i]) / max(abs(base.E[j,
    i]), 1e-9))
rec("T3 双线性插值落在邻域值域内", bad == 0,
    f"越界 {bad} 次; 与中心格最大相对差 {worst_rel:.3f}")

# T4 未覆盖 / 超上限 -> e_max
v_out, c_out = base.lookup(5000.0, 0.0)
rec("T4 靶区外/无数据 -> 默认最大值 e_max",
    (not bool(c_out)) and abs(float(v_out) - E_MAX) < 1e-9,
    f"lookup(5000,0)={float(v_out)}, covered={bool(c_out)}")

# T5 叠加 + 归一化 + 距离惩罚的结构性质
f = ExpectField(base, FieldConfig(grid_step=100.0, dist_weight=0.35))
d0 = f.argmin((0.0, 0.0))
fb = f.fallback_point((0.0, 0.0), visited=[(0.0, 0.0)])
rec("T5a 无任何方位时退化为探索点 (离已访问点最远)",
    d0 is None and fb["kind"] == "explore"
    and math.hypot(fb["xy"][0], fb["xy"][1]) > 1200.0,
    f"argmin=None, 探索点 {np.round(fb['xy'], 1).tolist()} (半径 "
    f"{math.hypot(*fb['xy']):.0f} m)")
for (x, y, t) in ((0.0, 0.0, 0.0), (0.0, 0.0, 90.0)):
    f.add_bearing((x, y), t)
sel = f.argmin((0.0, 0.0), visited=[(0.0, 0.0)])
sc, info = f.score((0.0, 0.0), visited=[(0.0, 0.0)])
i = sel["index"]
rec("T5b argmin 与全网格暴力最小一致",
    abs(float(sc[i]) - float(np.min(sc))) < 1e-9,
    f"argmin score={sel['score']:.6f}")
rec("T5c 归一化性质: E_norm∈[0,1] 且未覆盖格子=1",
    bool(np.all(info["e_norm"] <= 1.0 + 1e-9)) and
    float(np.max(info["e_norm"][info["coverage"] == 0])) == 1.0,
    f"max E_norm={float(np.max(info['e_norm'])):.4f}")
# 距离项的定量性质: score ≡ E_norm + λ·d/(2R_arena), 且 λ=0 时与距离无关
c0 = FieldConfig(grid_step=100.0, dist_weight=0.35, min_move=0.0)
f1 = ExpectField(base, c0)
for (x, y, t) in ((0.0, 0.0, 0.0), (0.0, 0.0, 90.0)):
    f1.add_bearing((x, y), t)
sc1, inf1 = f1.score((120.0, -340.0))
manual = inf1["e_norm"] + 0.35 * inf1["dist_m"] / DIST_REF
f0 = ExpectField(base, FieldConfig(grid_step=100.0, dist_weight=0.0,
    min_move=0.0))
for (x, y, t) in ((0.0, 0.0, 0.0), (0.0, 0.0, 90.0)):
    f0.add_bearing((x, y), t)
sc0, _ = f0.score((120.0, -340.0))

```

```

rec("T5d 距离层: score = E_norm +  $\lambda \cdot d / (2R_{\text{arena}})$  ( $\lambda=0$  时与距离无关)",
    bool(np.all(np.isfinite(sc1)))
    and float(np.max(np.abs(sc1 - manual))) < 1e-12
    and float(np.max(np.abs(sc0 - inf1["e_norm"]))) < 1e-12,
    f"最大偏差 {float(np.max(np.abs(sc1 - manual))):.3e}")

# T6 与精确实现交叉校验 (用基准图族; 含"射线被靶区截断"的病态情形)
fam = BaseMapFamily.load_dir(DEFAULT_FAMILY_DIR) if
    os.path.isdir(DEFAULT_FAMILY_DIR) else None
rec("T6a 基准图族可用 (按先验外径 t_hi 分族)", fam is not None,
    "族目录 " + DEFAULT_FAMILY_DIR if fam is not None
    else f"未找到 {DEFAULT_FAMILY_DIR} (可运行 --build-family 生成); "
    f"此时只能对 t_hi=1500 的几何严格成立")

def _cmp(rb_or_fam, cases, label):
    rels, n_local = [], 0
    worst = None
    for (Sd, thd, C) in cases:
        ts, w, _ = ray_source_samples(Sd, thd, n_t=200)
        if ts.size == 0:
            continue
        E_ex = exact_expected_diameter(Sd, thd, C, ts, w)
        if rb_or_fam is None:
            continue
        if isinstance(rb_or_fam, BaseMapFamily):
            rb, _t = rb_or_fam.rebase(Sd, thd)
        else:
            rb = rb_or_fam
        E_bm, _cov = rb.transformed_lookup(Sd, thd, C[:, 0], C[:, 1])
        u = np.array([math.cos(math.radians(thd)), math.sin(math.radians(thd))])
        G = np.asarray(Sd)[None, :] + ts[:, None] * u[None, :]
        for k in range(C.shape[0]):
            if not np.isfinite(E_ex[k]) or not np.isfinite(E_bm[k]):
                continue
            u1 = G - np.asarray(Sd)[None, :]
            u2 = G - C[k][None, :]
            cosang = (u1 * u2).sum(1) / np.maximum(
                np.linalg.norm(u1, axis=1) * np.linalg.norm(u2, axis=1), 1e-9)
            ang = np.degrees(np.arccos(np.clip(cosang, -1.0, 1.0)))
            gmin = min(float(np.min(ang)), float(np.min(180.0 - ang)))
            if gmin < 15.0: # 近简并: E 是尖峰, 插值无意义 (决策场里会截断成 e_max)
                continue
            rel = abs(float(E_bm[k]) - float(E_ex[k])) / max(float(E_ex[k]), 1e-9)
            rels.append(rel)
            n_local += 1
            if worst is None or rel > worst[0]:
                worst = (rel, float(E_bm[k]), float(E_ex[k]),
                    float(np.hypot(*(C[k] - Sd))))
    if not rels:
        return None
    return {"n": n_local, "med": float(np.median(rels)),

```

```

        "p90": float(np.percentile(rels, 90)), "max": max(rels),
        "worst": worst, "label": label}

# 生成两组几何: s 在靶区中部 (先验基本不被截断) 与 s 贴近靶区边界且朝外测 (被截断)
cases_mid, cases_edge = [], []
for _ in range(6):
    Sd = np.round(rng.uniform(-700, 700, 2), 1)
    thd = float(rng.uniform(0, 360))
    cases_mid.append((Sd, thd, _ring_candidates(Sd, thd)))
for _ in range(6):
    a = float(rng.uniform(0, 360))
    Sd = 1560.0 * np.array([math.cos(math.radians(a)),
                           math.sin(math.radians(a))])
    thd = float((a + rng.uniform(-25, 25)) % 360.0) # 朝靶区外测
    cases_edge.append((Sd, thd, _ring_candidates(Sd, thd)))
cases = [(S, t, C) for (S, t, C) in cases_mid + cases_edge if len(C)]

r_fam = _cmp(fam, cases, "family") if fam is not None else None
r_plain = _cmp(BaseMap.load_csv(base_csv) if os.path.exists(base_csv) else None,
               cases, "plain1500") if os.path.exists(base_csv) else None
if r_fam is not None:
    ok6 = (r_fam["med"] < 0.01) and (r_fam["p90"] < 0.05)
    detail6 = (f"{r_fam['n']} 个良态候选点 (含边界截断几何): 相对差 中位数 "
               f"{r_fam['med']:.4f}、P90 {r_fam['p90']:.4f}、最大 {r_fam['max']:.4f}")
    if r_plain is not None:
        detail6 += (f"; 不用族 (只按 t_hi=1500 的图) 时中位数 {r_plain['med']:.4f}、"
                    f"最大 {r_plain['max']:.4f}")
    elif r_plain is not None:
        ok6 = (r_plain["med"] < 0.01) and (r_plain["p90"] < 0.05)
        detail6 = (f"{r_plain['n']} 个良态候选点: 中位数 {r_plain['med']:.4f}、"
                    f"P90 {r_plain['p90']:.4f}")
    else:
        ok6, detail6 = True, "跳过 (既无基准图族也无单张基准图)"
rec("T6b 变换+插值 ≈ 精确期望 (良态几何)", ok6, detail6)

# T7 反例: 忽略 t_hi (只用 t_hi=1500 的图) 在边界朝外几何上会明显失真
if fam is not None and r_plain is not None and r_fam is not None:
    edge_rel = [x for x in
                 [_cmp(fam, [(S, t, C)], "f") for (S, t, C) in cases_edge
                  if len(C)] if x]
    edge_plain = [x for x in
                  [_cmp(BaseMap.load_csv(base_csv), [(S, t, C)], "p")
                   for (S, t, C) in cases_edge if len(C)] if x]
    if edge_rel and edge_plain:
        med_f = float(np.median([e["med"] for e in edge_rel]))
        med_p = float(np.median([e["med"] for e in edge_plain]))
        rec("T7 边界朝外几何: 按 t_hi 分族显著优于固定 t_hi=1500",
            med_f < 0.05 and med_f < med_p / 5.0,
            f"族: 中位相对差 {med_f:.4f}; 固定 1500: {med_p:.4f} ("
            f"差 {med_p / max(med_f, 1e-6):.0f} 倍)")

```

```

if verbose:
    print(f"\n自检总结: {'全部通过' if ok_all else '存在失败项'}")
return {"ok": ok_all, "checks": out}

# -----
# 演示 / 命令行
# -----

def field_figure(f, cur_xy, sel, path, title=None):
    """把决策场画出来: 叠加后的 E 场、距离惩罚、合成 score、选点与探测器位置。

    图注用英文 (本机 matplotlib 无中文字体, 避免缺字告警)。
    """
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    E_mean, cov, n_terms = f.aggregate()
    score, info = f.score(cur_xy)
    fig, axes = plt.subplots(1, 3, figsize=(16.5, 5.6))
    panels = [(E_mean, "E[intersection diameter] (m)\n(uncovered cells = 1500)",
               "viridis_r"),
              (np.where(np.isfinite(info["dist_m"]),
                       info["dist_m"] / DIST_REF, np.nan),
               f"distance term  $\lambda \cdot d / (2R)$ ,  $\lambda=\{f.cfg.dist\_weight\}$ ", "magma"),
              (np.where(np.isfinite(score), score, np.nan),
               "score = normalised E + distance term\n( $\Delta = \text{argmin}$ )", "cividis_r")]
    for ax, (V, ttl, cmap) in zip(axes, panels):
        sc = ax.scatter(f.gx, f.gy, c=V, s=26, cmap=cmap, marker="s")
        fig.colorbar(sc, ax=ax, shrink=0.8)
        ax.set_aspect("equal")
        ax.set_title(ttl, fontsize=10)
        ax.set_xlim(-1950, 1950)
        ax.set_ylim(-1950, 1950)
        ax.set_xlabel("x (m)")
        ax.set_ylabel("y (m)")
        th = np.linspace(0, 2 * math.pi, 400)
        ax.plot(f.cfg.arena_r * np.cos(th), f.cfg.arena_r * np.sin(th), "k--",
                lw=0.8)
        ax.plot([cur_xy[0]], [cur_xy[1]], "r*", ms=14)
        for (S, thd, _tag, _thi) in [t[2] for t in f.terms]:
            a = math.radians(thd)
            ax.plot([S[0], S[0] + 300 * math.cos(a)],
                    [S[1], S[1] + 300 * math.sin(a)], "w-", lw=1.2, alpha=0.8)
        if sel is not None:
            ax.plot([sel["xy"][0]], [sel["xy"][1]], "w^", ms=11, mec="k")
        ax.grid(alpha=0.2)
    fig.suptitle(title or (f"Q3 decision field: {n_terms} bearing/discovery terms "
                          f"superimposed on the global grid"), fontsize=11)
    fig.tight_layout()
    os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)

```

```

fig.savefig(path, dpi=150)
plt.close(fig)
return path

def demo(base_csv=DEFAULT_BASE_CSV, outdir=None, dist_weight=1.5, grid_step=100.0,
        bearings=((0.0, 0.0, 0.0),), verbose=True, family_dir=DEFAULT_FAMILY_DIR):
    fam = BaseMapFamily.load_dir(family_dir) if os.path.isdir(family_dir) else None
    base = fam if fam is not None else BaseMap.load_csv(base_csv)
    cfg = FieldConfig(grid_step=grid_step, dist_weight=dist_weight)
    f = ExpectField(base, cfg)
    for (x, y, t) in bearings:
        f.add_bearing((x, y), t)
    sel = f.argmin((0.0, 0.0), visited=[(0.0, 0.0)])
    if verbose:
        kind = "基准图族" if fam is not None else f"单张基准图 {base.source}"
        print("基准: ", kind)
        print(f"叠加 {len(f.terms)} 条方位; 选点 {np.round(sel['xy'], 1).tolist()}, "
              f"E_mean={sel['E_mean_m']:.1f} m, E_norm={sel['E_norm']:.3f}, "
              f"距离 {sel['dist_m']:.0f} m, score={sel['score']:.4f}")
    if outdir:
        os.makedirs(outdir, exist_ok=True)
        p = f.save_csv(os.path.join(outdir, "p3_field.csv"))
        with open(os.path.join(outdir, "p3_field_report.json"), "w",
                  encoding="utf-8") as fh:
            json.dump({"cfg": cfg.__dict__, "terms": len(f.terms), "pick": sel},
                      fh, ensure_ascii=False, indent=2)
        try:
            fig = field_figure(f, (0.0, 0.0), sel,
                              os.path.join(outdir, "p3_field.png"))
        except Exception as e: # 画图失败不影响主流程
            fig = f"figure_error: {e!r}"
        if verbose:
            print("写出", p, "与", fig)
        return f, sel
    return f, sel

def main():
    ap = argparse.ArgumentParser(description="问题3 期望决策场")
    ap.add_argument("--selfcheck", action="store_true")
    ap.add_argument("--demo", action="store_true")
    ap.add_argument("--build-family", action="store_true",
                    help="按第二问口径重算一族基准图 (不同先验外径 t_hi)")
    ap.add_argument("--base-map", default=DEFAULT_BASE_CSV)
    ap.add_argument("--family-dir", default=DEFAULT_FAMILY_DIR)
    ap.add_argument("--family-step", type=float, default=40.0)
    ap.add_argument("--family-nt", type=int, default=200)
    ap.add_argument("--out", default=None)
    ap.add_argument("--dist-weight", type=float, default=1.5,
                    help="距离项系数  $\lambda$  (问题 3 策略默认 1.5)")

```

```

ap.add_argument("--grid-step", type=float, default=100.0)
args = ap.parse_args()
if args.selfcheck:
    res = selfcheck(args.base_map)
    raise SystemExit(0 if res["ok"] else 1)
if args.build_family:
    print(f"计算基准图族 -> {args.family_dir}")
    BaseMapFamily.build(args.family_dir, step=args.family_step,
                        n_t=args.family_nt)
if args.demo or not args.build_family:
    demo(args.base_map, args.out, args.dist_weight, args.grid_step,
        family_dir=args.family_dir)

if __name__ == "__main__":
    main()

```

0.5 Q4/src/p3_robot.py

"""B 题问题 3: 机器狗"基于期望的行动"在线搜索-定位-清除策略。

策略流程（与设计说明一一对应）

1. **第一轮扫描**：机器狗从原点出发，测向机初始频道 1，按频道 1..20 依次 ``/measure``，得到若干条"检测点 + 示向度"（部分频道无信号属正常）。
2. **期望场选点**：对每个"只测到一次方位"的频道，用第二问的网格模拟结果（``out/p3\_family`` 或 ``out/p2\_grid``）做旋转 + 平移 + 双线性插值，把 $E[D]$ 叠加到全局网格上；没有数据/超阈值的格子取最大值 1500 m；归一化后再叠加一层**与探测器距离线性增长**的代价（系数 λ 可调），取 ``argmin`` 作为下一个检测点，随后开始新一轮 20 频道扫描。
3. **清理**：某些频道已经测到 ≥ 2 条方位 — 用问题 1 的交会算法算出定位区域（最小包围圆 = 中心 + 半径 r ）。对所有可清理的频道按最近邻顺序规划航路，依次前往；到达区域后继续检测把区域压小，直到进入清除范围，再用 **覆盖式试探**（六边形格点，间距 $\leq 20\sqrt{3}$ m，保证圆域内任一点到某个试探点 ≤ 20 m）逐个 ``/clear``，必然有一点成功。
4. **清除后继续扫描**：已经清除的频道、已经定位够的频道、以及"在同一个有效接收半径内重复测过的频道"都会被跳过，从而节省时间。
5. 反复执行 2-4，直到时间预算耗尽或没有新的可用信息。

关键设计

- * **为什么"未覆盖 = 1500 m"是对的**： $E[D]$ 越小定位越好；把没有信息的格子记成最差，只有在附近确实没有更好格子时才会选它 — 这天然形成了"顺路探索"的行为。
- * **为什么不能沿示向度方向逼近**：那会退化成问题 1 的"无界区域"几何。所以清理时的侧移精定位点取在**与上一条视线近似正交**的方向上。
- * **清理阶段的覆盖式试探是"保证成功"的**：真源必落在 $\pm 1^\circ$ 锥的交集（定位区域）内，区域内任意一点到某个试探点的距离 ≤ 20 m = 清除半径。

```

"""
from __future__ import annotations

import argparse

```

```

import json
import math
import os
import sys
import time
from dataclasses import dataclass, field as dc_field

import numpy as np

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)

from p1_intersection import analyse # noqa: E402
from p3_arena import ( # noqa: E402
    ArenaError, BEARING_ERR, CHANNELS, MockArena, NEAR_R, R_ARENA, R_CLEAR,
    T_CLEAR_FAIL, T_CLEAR_OK, T_MEASURE, T_SWITCH, V_ROBOT,
)
from p3_expect_field import ( # noqa: E402
    DEFAULT_BASE_CSV, DEFAULT_FAMILY_DIR, BaseMap, BaseMapFamily, ExpectField,
    FieldConfig, best_refine_point,
)

# -----
# 覆盖式试探的三种结局
# -----
# 必须区分"预算不够、一个试探点都没来得及发"与"试探了但没打中":
# 前者不是策略失败（不该累加 clear_fail、不该写进论文的失败归因），
# 后者才是真正的"区域内没找到目标"。
SWEEP_OK = "success" # 某一点 /clear 成功
SWEEP_MISS = "miss" # 试探了全部候选点（或中途超时）都没打中
SWEEP_BUDGET = "no_budget" # 剩余预算不足，**一个 /clear 都没发出**，主动放弃

# -----
# 配置
# -----
@dataclass
class P3Config:
    # ---- 期望场（全局网格 + 距离惩罚） ----
    grid_step: float = 100.0 # 全局网格步长 (m)
    e_max: float = 1500.0 # 未覆盖网格的默认最大值 (m)
    dist_weight: float = 1.5 # 距离项系数  $\lambda$  (离线演练调参得到, 见 review/P3-design.md)
    visited_radius: float = 250.0 # 已扫描点邻域排除半径 (m)
    min_move: float = 60.0 # 距当前位置过近的格子排除 (m)
    field_terms: str = "pending" # pending: 只叠加"还缺方位"的频道
    field_discovery: bool = False # 始终叠加"探索项"（把扫描点推向能发现新源的地方）
    auto_discovery_when_empty: bool = True # 场上一条可用方位都没有时自动启用探索项

    # ---- 扫描 ----

```

```

channels: tuple = CHANNELS
scan_lbearing_first: bool = True
repeat_gap_m: float = 60.0 # 同一频道在附近重复测量的最小间隔 (m)
nosignal_gap_m: float = 1000.0 # "无信号"频道要换足够远的地方重测
                                # (= 有效接收半径下界: 在 1000 m 外重测才有
                                # 真正的新信息; 实测 450→1000 使检测次数
                                # 289→186、平均定位清除时间 398→315 s,
                                # 清除率 0.992→0.995, 见 review/P3-design.md
                                # §5.1) (m)
enroute_scan: bool = True # 清理途中顺路补测
enroute_scan_max: int = 2
second_bearing_refine: bool = False # 新发现频道立刻补第二条方位 (实测收益不稳, 默认关)
second_bearing_max: int = 2 # 每个停点最多补几条
second_bearing_budget_s: float = 320.0 # 剩余时间低于它就不再补测
max_rounds: int = 30
tour: str = "nn2opt" # 清理航路: nn | nn2opt
dynamic_replan: bool = True # 清理阶段每步重规划 (滚动时域); False=阶段内固定顺序
clear_policy: str = "tour" # tour (NN+2-opt 全局结构) | rollout (一步前瞻+SPT)
clear_budget_truncation: bool = False # 是否按剩余预算裁剪航路 (实测关掉更好)
optimistic_probe_steps: int = 3 # 打开裁剪时的试探点数封顶
min_scan_rounds_before_clear: int = 1 # 先扫描几轮再开始清理

# ---- 定位与清除 ----
clear_min_bearings: int = 2
clear_max_radius_m: float = 600.0 # 定位区域半径超过它就先不清理
                                # (400 → 600: 扫圈在  $r \approx 1100$  m 处交会、区域半径
                                # 常在 400-650 m, 400 会让这些源被"跳过";
                                # 600 实测把扫圈的清除率从 0.958 提到 0.989-1.000)
arrive_radius_m: float = 30.0
direct_probe_radius_m: float = 60.0 #  $r^*$  不超过它就无需侧移精定位, 直接试探
probe_spacing_m: float = 34.0 #  $\leq 20\sqrt{3} \approx 34.6$  m 才能保证覆盖
probe_max_points: int = 60 # 候选点数上限 (仅"可覆盖"档位, 见下)
probe_unlimited: bool = True # True=不按点数截断 (保证覆盖; 截断会破坏 20 m 保证)
probe_max_radius_m: float = 200.0
refine_leg_m: float = 320.0
refine_travel_cap_m: float = 700.0
refine_mode: str = "perp" # perp (正交侧移) | exact (精确期望最优)
max_approach_steps: int = 3

# ---- 时间预算 ----
time_reserve_s: float = 3.0
min_action_budget_s: float = 10.0
min_scan_channels: int = 2 # 单轮扫描至少要能测这么多频道才值得跑一趟

# ---- 预算感知的扫描闸门 ("剩余时间不够就不要再扫") ----
# 诊断依据: 30 局里扫描阶段吃掉约 1016 s / 1200 s, 而末尾有 23% 的未清除源
# **已经定位好却没时间走过去**。所以关键不是"怎么选扫描点", 而是"还要不要扫"。
scan_budget_gate: bool = False # 开: 本轮扫描后必须还够清最近的一个目标, 否则不扫
scan_gate_margin: float = 0.0 # 额外预留秒数 (越大越倾向"别扫了, 去清除")
scan_gate_factor: float = 1.0 # 对"清最近目标"耗时估计的放大系数
scan_only_toward_targets: bool = False # 扫描点若背着已知目标走, 则该轮不扫

```

```

# 扫描时间的硬性上限（占总预算比例）。诊断显示扫描占掉约 1016 s / 1200 s，
# 而 29/49 个"已定位未清除"的目标在变成可清目标时**剩余时间完全够用**却没去清。
scan_budget_frac: float = 1.0 # 1.0 = 不限制；0.5 = 扫描累计不超过一半预算

# ---- 清理航路：可达近点子集（"优先清路程近的点"）----
clear_shortlist: bool = False # 开：只保留"预算内可达"的近点子集，再排序
clear_shortlist_reserve_s: float = 0.0 # 为后续扫描预留的秒数

# -----
# 频道台账
# -----
@dataclass
class ChannelRec:
    channel: int
    pts: list = dc_field(default_factory=list) # 测到方位的检测点
    svds: list = dc_field(default_factory=list) # 对应的示向度
    meas_pts: list = dc_field(default_factory=list) # 所有检测过的位置（含无信号）
    meas_log: list = dc_field(default_factory=list) # (x, y, virtual_time, result)
    meas_stations: list = dc_field(default_factory=list) # 测过该频道的**站位编号**
                                                # （问题 4 的覆盖记账用：只有"这一站
                                                # 本来能听到它"才抵扣覆盖）

    cleared: bool = False
    cleared_at: float | None = None
    near_hits: int = 0
    no_signal: int = 0
    clear_fail: int = 0 # 自最近一次"新增方位"以来清理失败的次数
    last_bounded: tuple | None = None # (center, radius, n_bearings) 最近一次有界区域
    tried_probes: list = dc_field(default_factory=list)
    history: list = dc_field(default_factory=list)

    @property
    def n_bearings(self) -> int:
        return len(self.pts)

# -----
# 机器狗
# -----
def load_base(family_dir=DEFAULT_FAMILY_DIR, base_csv=DEFAULT_BASE_CSV):
    """优先用"基准图族"（按先验外径 t_hi 分族，变换才严格成立），否则退回单张基准图。"""
    if family_dir and os.path.isdir(family_dir):
        fam = BaseMapFamily.load_dir(family_dir)
        if fam is not None:
            return fam
    if os.path.exists(base_csv):
        return BaseMap.load_csv(base_csv)
    return None

def _dist_point_segment(p, a, b):

```

```

ab = b - a
L2 = float(ab @ ab)
if L2 < 1e-12:
    return float(np.linalg.norm(p - a))
t = float(np.clip((p - a) @ ab / L2, 0.0, 1.0))
return float(np.linalg.norm(p - (a + t * ab)))

def _point_in_polygon(p, P):
    """射线法 (P 为凸多边形顶点, 逆时针或顺时针均可)。"""
    n = len(P)
    inside = False
    for i in range(n):
        a, b = P[i], P[(i + 1) % n]
        if (a[1] > p[1]) != (b[1] > p[1]):
            x = a[0] + (p[1] - a[1]) * (b[0] - a[0]) / (b[1] - a[1])
            if x > p[0]:
                inside = not inside
    return inside

def _dist_point_polygon(p, P):
    p = np.asarray(p, float)
    P = np.asarray(P, float)
    if _point_in_polygon(p, P):
        return 0.0
    n = len(P)
    return min(_dist_point_segment(p, P[i], P[(i + 1) % n]) for i in range(n))

class P3Robot:
    def __init__(self, arena, cfg: P3Config | None = None, base=None, log=None,
                 log_prefix=""):
        self.arena = arena
        self.cfg = cfg or P3Config()
        self.base = base if base is not None else load_base()
        self._log_fn = log
        self.log_prefix = log_prefix
        self.recs = {int(ch): ChannelRec(int(ch)) for ch in self.cfg.channels}
        self.scan_points: list[tuple] = []
        self.meas_pts: list[tuple] = [] # **真实做过的检测**位置 (覆盖度评估口径:
                                         # "到过的点"≠"测过的点", 覆盖度只能按后者算)
        self.events: list[dict] = []
        self.aborted = False
        # 覆盖式试探因"预算不足"放弃时置位: 让 clear_phase 立刻收敛,
        # 不要对同一个目标反复重规划 (实测会把重规划次数抬高一到两个数量级)。
        self.abort_clear_phase = False
        self.stats = {"n_measure": 0, "n_clear": 0, "n_clear_fail": 0,
                      "n_scan_rounds": 0, "n_bearings": 0, "n_rejected": 0,
                      "n_refine": 0, "n_second_bearing": 0, "n_replans": 0,
                      "n_midphase_targets": 0, "n_sweep_aborted": 0,

```

```

        "n_sweep_miss": 0, "n_probe_issued": 0,
        "n_scan_gated": 0, "n_shortlist_kept": 0, "n_shortlist_in": 0,
        "t_scan_s": 0.0, "t_clear_s": 0.0, "t_clear_ends_s": 0.0,
        "vtime_at_last_clear": 0.0}

# ----- 基础设施 -----
def log(self, msg):
    self.events.append({"t": float(self.arena.virtual_time_s), "msg": msg})
    if self._log_fn:
        # 日志里避免使用 GBK 控制台无法编码的符号 (⇒ ★ √ × 等),
        # 万一还有, 退化为替换字符也不要让整局崩掉。
        try:
            self._log_fn(f"{self.log_prefix}{msg}")
        except UnicodeEncodeError:
            self._log_fn(f"{self.log_prefix}{msg}".encode(
                "gbk", "replace").decode("gbk"))

def remaining(self):
    return float(self.arena.remaining_budget_s())

def can_afford(self, seconds):
    return self.remaining() > float(seconds) + self.cfg.time_reserve_s

@property
def pos(self):
    return tuple(self.arena.pos)

# ----- 检测 / 清除 -----
def measure(self, x, y, ch):
    resp = self.arena.measure(x, y, ch)
    self.stats["n_measure"] += 1
    if resp.get("accepted") is not True:
        self.stats["n_rejected"] += 1
        self.aborted = True
        self.log(f"! /measure 频道{ch} 未执行 ({resp.get('reason', '?')})")
    return resp
    res = resp.get("measure_result")
    rec = self.recs[int(ch)]
    rec.meas_pts.append((float(x), float(y)))
    rec.meas_log.append((float(x), float(y), float(self.arena.virtual_time_s),
        res))
    if res == "direction":
        self.add_bearing(int(ch), x, y, float(resp["svd_deg"]))
    elif res == "near":
        rec.near_hits += 1
    else:
        rec.no_signal += 1
    return resp

def add_bearing(self, ch, x, y, svd):
    rec = self.recs[int(ch)]

```

```

rec.pts.append((float(x), float(y)))
rec.svds.append(float(svd))
rec.clear_fail = 0
rec.tried_probes = []
self.stats["n_bearings"] += 1
reg = self.region(rec)
if reg.get("status") == "bounded":
    rec.last_bounded = (tuple(reg["min_enclosing_center"]),
                        float(reg["min_enclosing_radius_m"]), len(rec.pts))
rec.history.append({"n": len(rec.pts), "x": float(x), "y": float(y),
                    "svd": float(svd), "status": reg.get("status"),
                    "r_star_m": (rec.last_bounded[1] if rec.last_bounded else
                                None)})

return reg

def do_clear(self, x, y, ch):
    resp = self.arena.clear(x, y, ch)
    self.stats["n_probe_issued"] = self.stats.get("n_probe_issued", 0) + 1
    if resp.get("accepted") is not True:
        self.stats["n_rejected"] += 1
        self.aborted = True
        self.log(f" ! /clear 频道{ch} 未执行 ({resp.get('reason', '?')}) ")
        return False
    if resp.get("clear_result") == "success":
        rec = self.recs[int(ch)]
        rec.cleared = True
        rec.cleared_at = float(self.arena.virtual_time_s)
        self.stats["n_clear"] += 1
        self.stats["vtime_at_last_clear"] = float(self.arena.virtual_time_s)
        self.log(f" [OK] 频道{ch} 已清除 (t={self.arena.virtual_time_s:.1f}s) ")
        return True
    self.stats["n_clear_fail"] += 1
    return False

# ----- 定位区域 -----
def region(self, rec: ChannelRec):
    if not rec.pts:
        return {"status": "none", "n_bearings": 0}
    return analyse(rec.pts, rec.svds, BEARING_ERR)

def bounded_region(self, rec: ChannelRec):
    """当前（或最近一次）有界区域；返回 ``{'center', 'radius', 'poly', 'status'}``。"""
    if len(rec.pts) >= 2:
        reg = self.region(rec)
        if reg.get("status") == "bounded":
            return {"center": tuple(reg["min_enclosing_center"]),
                    "radius": float(reg["min_enclosing_radius_m"]),
                    "poly": [tuple(v) for v in reg.get("vertices", [])],
                    "status": "bounded", "n_bearings": len(rec.pts)}
    if rec.last_bounded is not None:
        c, r, n = rec.last_bounded

```

```

        return {"center": tuple(c), "radius": float(r), "poly": None,
                "status": "fallback", "n_bearings": n}
    return None

# ----- 扫描 -----
def needed_channels(self):
    """还需要（且值得）拿到新方位的频道：已清除、已够定位两次的跳过。"""
    need, done = [], []
    for ch in self.cfg.channels:
        rec = self.recs[int(ch)]
        if rec.cleared or rec.n_bearings >= self.cfg.clear_min_bearings:
            continue
        (done if rec.pts else need).append(int(ch))
    order = (done + need) if self.cfg.scan_lbearing_first else sorted(done +
        need)
    return order

def channels_to_measure(self, limit=None, pos=None, subset=None,
    apply_gap=True):
    """在 ``pos`` 处值得检测的频道。

    **跳过规则**（"节省时间"的关键）：已清除 / 已定位两次的跳过；
    对"无信号"频道，新检测点必须距它所有历史检测点  $\geq$  ``nosignal_gap_m``
    （有效接收半径至少 1000 m，在同一个圈里重复扫几乎拿不到新信息）；
    对已有一条方位的频道，新检测点距历史点  $\geq$  ``repeat_gap_m``（太小交会角没意义）。
    """
    p = np.asarray(pos if pos is not None else self.arena.pos, float)
    out = []
    for ch in (subset if subset is not None else self.needed_channels()):
        rec = self.recs[int(ch)]
        if apply_gap and rec.meas_pts:
            gap = self.cfg.repeat_gap_m if rec.pts else self.cfg.nosignal_gap_m
            d = min(math.hypot(p[0] - q[0], p[1] - q[1]) for q in rec.meas_pts)
            if d < gap:
                continue
        out.append(int(ch))
    return out[:limit] if limit else out

def build_field(self, grid_step=None, dist_weight=None):
    cfg = FieldConfig(grid_step=grid_step or self.cfg.grid_step,
        e_max=self.cfg.e_max,
        dist_weight=(self.cfg.dist_weight if dist_weight is None
            else dist_weight),
        visited_radius=self.cfg.visited_radius,
        min_move=self.cfg.min_move)
    f = ExpectField(self.base, cfg)
    if self.base is None:
        return f, 0
    n = 0
    for ch, rec in self.recs.items():
        if rec.cleared:

```

```

        continue
    if self.cfg.field_terms == "pending" and rec.n_bearings >=
        self.cfg.clear_min_bearings:
        continue
    if rec.n_bearings == 0:
        continue
    for ((x, y), th) in zip(rec.pts, rec.svds):
        f.add_bearing((x, y), th, tag=ch)
        n += 1
if n == 0:
    # 一条"缺第二方位"的频道都没有（例如后半程只剩没听到过的频道）：
    # 自动退化为**探索项**，否则场上没有任何信息、选点无从谈起。
    for ch, rec in self.recs.items():
        if rec.cleared or rec.n_bearings > 0 or not rec.meas_pts:
            continue
        if not self.cfg.field_discovery and not
            self.cfg.auto_discovery_when_empty:
            continue
        f.add_raw(f.discovery_term(rec.meas_pts, self.cfg.nosignal_gap_m),
            tag=("disc", ch))
        n += 1
return f, n

def choose_scan_point(self, dist_weight=None, max_dist=None):
    f, n = self.build_field(dist_weight=dist_weight)
    visited = list(self.scan_points)
    sel = f.argmin(self.pos, visited=visited, max_dist=max_dist)
    if sel is None:
        sel = f.fallback_point(self.pos, visited=visited, max_dist=max_dist)
    sel["n_bearing_terms"] = n
    return sel, f

def plan_scan(self, dist_weight=None):
    """选下一个检测点，并确定该点上值得测哪些频道。

    顺序很重要：**先**用期望场选点，**再**判断该点上哪些频道还没被"测透"，
    否则会出现"因为要换地方才有信息、但没地方可去"的死锁。
    """
    base = self.needed_channels()
    if not base:
        return None
    # 时间不够时把可选点限制在"还能走到"的范围内（给检测与安全余量留出空间）
    slack = 2.0 * (T_MEASURE + T_SWITCH) + 4.0
    dmax = max(0.0, (self.remaining() - self.cfg.time_reserve_s - slack) *
        V_ROBOT)
    sel, f = self.choose_scan_point(dist_weight=dist_weight, max_dist=dmax)
    chans = self.channels_to_measure(pos=sel["xy"], subset=base)
    if not chans:
        sel2 = f.fallback_point(self.pos, visited=self.scan_points)
        chans2 = self.channels_to_measure(pos=sel2["xy"], subset=base)
        if chans2:

```

```

        sel, chans = sel2, chans2
    if not chans:                # 兜底：宁可多测几次也不原地空转
        chans = base[:3]
    # 时间不够时自动缩减本轮的频道数（至少测 min_scan_channels 个），把边角时间也用上
    travel = math.dist(self.pos, sel["xy"]) / V_ROBOT
    avail = self.remaining() - self.cfg.time_reserve_s - travel - 4.0
    kmax = max(1, int(avail // (T_MEASURE + T_SWITCH)))
    if len(chans) > kmax:
        if kmax < self.cfg.min_scan_channels:
            return None        # 只剩时间测一两个频道：不值得为它专门跑一趟
        self.log(f" 时间只剩 {self.remaining():.0f} s -> 本轮只测
            {kmax}/{len(chans)} 个频道")
        chans = chans[:kmax]
    cost = travel + len(chans) * (T_MEASURE + T_SWITCH) + 2.0
    plan = {"chans": chans, "sel": sel, "field": f, "cost_s": cost}
    if self.cfg.scan_budget_gate and not self._scan_worth_it(plan):
        return None
    return plan

# ----- 预算感知的扫描闸门 -----
def nearest_clear_cost_after(self, at_xy):
    """估计"到达 ``at_xy`` 之后，再清掉最近一个已知目标"所需秒数；没有已知目标则 None。"""
    cand = self.clear_targets()
    if not cand:
        return None
    nxt = min(cand, key=lambda t: math.dist(at_xy, t[2]["center"]))
    return self.estimate_clear_cost(nxt[2], from_pos=at_xy,
                                    probe_cap=self.cfg.optimistic_probe_steps)

def _scan_budget_left(self):
    """允许继续扫描的剩余秒数（= frac × 总预算 - 已花在扫描上的时间）。"""
    if self.cfg.scan_budget_frac >= 1.0:
        return float("inf")
    return self.cfg.scan_budget_frac * float(self.arena.budget) \
        - self.stats.get("t_scan_s", 0.0)

def _scan_worth_it(self, plan):
    """本轮扫描值不值得做：**扫完还必须够清最近的一个已知目标**，否则不扫。

    动机（诊断数据）：30 局里扫描阶段平均吃掉约 1016 s / 1200 s，末尾有 23% 的
    未清除源**已经定位好却走不到**。也就是说"再多发现一个源"的边际价值，在
    剩余时间已经不足以走过去清掉它时等于 0 —— 这种扫描纯属消耗预算。

    注意这里只做"取舍"不做"折中"：要么整轮扫，要么整轮不扫；
    因为扫描点本身的移动无法中途变成对目标的逼近（两者方向不同）。
    """
    rem = self.remaining()
    left = self._scan_budget_left()
    if plan["cost_s"] > left:
        self.stats["n_scan_gated"] = self.stats.get("n_scan_gated", 0) + 1
        self.log(f" ■ 放弃本轮扫描：扫描时间额度已用尽（本轮需 {plan['cost_s']:.0f} s, "

```

```

        f"额度剩 {left:.0f} s = {self.cfg.scan_budget_frac:.2f}×总预算) "
        f"-> 预算留给已定位目标")

    return False

cand = self.clear_targets()
need = None
if cand:
    nxt = min(cand, key=lambda t: math.dist(plan["sel"]["xy"],
        t[2]["center"]))
    need = self.estimate_clear_cost(nxt[2], from_pos=plan["sel"]["xy"],
        probe_cap=self.cfg.optimistic_probe_steps)
    # 可选: 扫描点不能"背着"已知目标走 (否则这轮的移动无法回收为进度)
    if self.cfg.scan_only_toward_targets:
        u = np.asarray(plan["sel"]["xy"], float) - np.asarray(self.pos, float)
        w = np.asarray(nxt[2]["center"], float) - np.asarray(self.pos, float)
        nu, nw = float(np.hypot(*u)), float(np.hypot(*w))
        if nu > 1e-9 and nw > 1e-9 and float(u @ w) / (nu * nw) < 0.0:
            self.stats["n_scan_gated"] = self.stats.get("n_scan_gated", 0) + 1
            self.log(f" ■ 放弃本轮扫描: 扫描点 ({plan['sel']['xy'][0]:.0f}, "
                f"{plan['sel']['xy'][1]:.0f}) 与已知目标方向相反 -> 预算留给清除")
            return False
    if need is None:
        return True # 手上没有可清目标: 扫描是唯一的进展方式
    required = (plan["cost_s"] + self.cfg.scan_gate_margin
        + self.cfg.scan_gate_factor * need)
    if rem >= required:
        return True
    self.stats["n_scan_gated"] = self.stats.get("n_scan_gated", 0) + 1
    self.log(f" ■ 放弃本轮扫描: 剩余 {rem:.0f} s < 本轮扫描 {plan['cost_s']:.0f} s "
        f"+ 清最近目标 {need:.0f} s (阈值 {required:.0f} s) "
        f"-> 预算留给已定位目标")
    return False

def execute_scan(self, plan):
    sel, chans = plan["sel"], plan["chans"]
    x, y = sel["xy"]
    if sel["kind"] == "explore":
        self.log(f"扫描轮: 期望场无可用方位 ({sel.get('n_bearing_terms', 0)} 项) "
            f"-> 探索点 ({x:.0f},{y:.0f}), 距 {sel['dist_m']:.0f} m")
    else:
        self.log(f"扫描轮: 选点 ({x:.0f},{y:.0f}) | E={sel['E_mean_m']:.1f} m "
            f"E_norm={sel['E_norm']:.3f} 距离惩罚={sel['dist_penalty']:.3f} "
            f"score={sel['score']:.4f} 移动 {sel['dist_m']:.0f} m | "
            f"待测 {len(chans)} 个频道 {chans}")
    self.scan_points.append((float(x), float(y)))
    n = self.scan_at(x, y, chans)
    self.stats["n_scan_rounds"] += 1
    return n

def scan_round(self, dist_weight=None):
    plan = self.plan_scan(dist_weight=dist_weight)
    if plan is None:

```

```

        return 0
    if not self.can_afford(plan["cost_s"]):
        return 0
    return self.execute_scan(plan)

def scan_at(self, x, y, chans):
    """在 (x, y) 依次检测这些频道（第一次调用会带来移动耗时）。"""
    n = 0
    for i, ch in enumerate(chans):
        if self.aborted or not self.can_afford(T_MEASURE + T_SWITCH + 3.0):
            break
        resp = self.measure(x, y, ch)
        n += 1
        if resp.get("measure_result") == "near":
            # 已在 5 m 内: 直接清除
            self.do_clear(x, y, ch)
    return n

# ----- 清理 -----
def clear_targets(self):
    out = []
    for ch, rec in self.recs.items():
        if rec.cleared or rec.n_bearings < self.cfg.clear_min_bearings:
            continue
        if rec.clear_fail > 0:
            continue # 上次清理失败, 等下一次新方位再试
        br = self.bounded_region(rec)
        if br is None or br["radius"] > self.cfg.clear_max_radius_m:
            continue
        out.append((ch, rec, br))
    out.sort(key=lambda t: math.dist(self.pos, t[2]["center"]))
    return out

# ----- 清理: 滚动时域动态规划 -----
def _probe_count(self, br):
    """该定位区域需要多少个覆盖式试探点（带缓存: 区域不变就不重算）。

    这里**故意**对点数封顶: 它只用于"值不值得跑一趟"的代价预估,
    真实覆盖由 :meth:`probe_points` 保证。不封顶会让大  $r^*$  上的枚举变得很贵
    ( $r^*=200$  m 时 151 个点  $\times$  每步重规划), 实测把程序运行时间抬高两个数量级。
    """
    key = (round(float(br["center"][0]), 1), round(float(br["center"][1]), 1),
           round(float(br["radius"]), 1))
    cache = getattr(self, "_probe_cache", None)
    if cache is None:
        cache = self._probe_cache = {}
    if key not in cache:
        r = min(float(br["radius"]), self.cfg.probe_max_radius_m)
        n = len(self.probe_points(np.asarray(br["center"], float), r,
                                             poly=br.get("poly")))
        cache[key] = min(int(n), int(self.cfg.probe_max_points))

```

```

return cache[key]

def estimate_clear_cost(self, br, from_pos=None, n_probe=None, probe_cap=None):
    """估计"从 ``from_pos`` 出发清除某个目标"的耗时（秒）。

    组成：走到区域中心 + 一次逼近检测 + 覆盖式试探（试探点数 × 平均步长/速度 + 3 s/次）
    + 成功清除 5 s。

    ``probe_cap`` 用于**乐观估计**：实际执行时试探是逐点进行的、随时可以中止
    （多数目标前几个试探点就成功），所以判断"值不值得跑一趟"时把试探数封顶，
    否则会过于保守、把还有 400 s 的尾巴白白浪费掉。
    """
    p = np.asarray(from_pos if from_pos is not None else self.pos, float)
    c = np.asarray(br["center"], float)
    if n_probe is None:
        n_probe = self._probe_count(br)
    if probe_cap is not None:
        n_probe = min(int(n_probe), int(probe_cap))
    t_probe = n_probe * (self.cfg.probe_spacing_m / V_ROBOT + T_CLEAR_FAIL)
    return (math.dist(p, c) / V_ROBOT + T_MEASURE + T_SWITCH
            + t_probe + T_CLEAR_OK)

def budget_feasible_shortlist(self, targets, start=None, budget=None):
    """从 ``targets`` 里挑出"剩余预算内走得完"的**近点子集**（贪心插入）。

    对"近点优先"的回答：滚动时域下每步只执行第一个目标，所以**只保留一个近点**
    在语义上已经等价于"优先清最近的点"；本函数真正的作用是判断
    "这一轮清理是否还排得下任何一个目标"，以及给排序一个**可达性**约束。

    排序键是"距当前位置"（近点优先），逐个累计
    ``走到区域中心 + 一次逼近检测 + 覆盖式试探 + 清除`` 的乐观代价，
    超预算就停止插入（后面更远的点更进不来）。
    """
    if not targets:
        return []
    p = np.asarray(start if start is not None else self.pos, float)
    rem = self.remaining() if budget is None else float(budget)
    room = rem - self.cfg.time_reserve_s - self.cfg.min_action_budget_s \
        - self.cfg.clear_shortlist_reserve_s
    ordered = sorted(targets, key=lambda t: math.dist(p, t[2]["center"]))
    out, used = [], 0.0
    for t in ordered:
        src = p if not out else np.asarray(out[-1][2]["center"], float)
        # estimate_clear_cost 已含"从 from_pos 走到区域中心"的行程
        cost = self.estimate_clear_cost(
            t[2], from_pos=src, probe_cap=self.cfg.optimistic_probe_steps)
        if used + cost > room:
            continue # 这个点排不进，试下一个更远的（更近的已进来）
        out.append(t)
        used += cost
    return out

```

```
def plan_clear_tour(self, targets, remaining_s=None):
    """**动态规划（滚动时域）**清理航路：每执行一步就重新规划一次。

    与旧版（清理阶段开始时规划一次、阶段内顺序冻结）的区别：

    * 每次调用都重读台账 ⇒ **途中新定位出来的频道立刻进入航路**；
    * 各频道用**最新**的定位区域中心（逼近过程中区域会变小、中心会移动）；
    * 以**当前位置**为起点重新做最近邻 + 2-opt（可用 ``tour`` 配置切换）；
    * 本步只执行规划结果的第一个，执行完立刻重规划（rolling horizon）。

    **为什么默认不按剩余预算裁剪目标表**：目标是"边走边判"执行的——
    逼近、一次检测、逐点试探各自都检查剩余时间、随时可以中止，多数目标在
    前几个试探点就成功。用统一的预估代价去裁剪，会把还剩 300-400 s 的尾巴
    整段浪费掉（实测：裁剪 39.8% vs 不裁剪 41.2%）。故默认只排序、不裁剪，
    由 :meth:`clear_target` / :meth:`sweep_clear` 在动作粒度上兜底。
    ``clear_budget_truncation=True`` 可打开裁剪做对照。

    ``clear_policy`` 两种策略：
    * ``tour``（默认）：NN + 2-opt 的**全局航路结构**，按顺序执行；
    * ``rollout``：一步前瞻 + SPT 推演，直接最大化"预算内可清除个数"
      （理论更贴合目标函数，但实测更差 36.3% vs 41.2%：贪心总挑最近的，
      最后把最远的目标拖成"永远清不掉"）。
    """
    if not targets:
        return []
    self.stats["n_replans"] = self.stats.get("n_replans", 0) + 1
    remaining = self.remaining() if remaining_s is None else float(remaining_s)
    budget = remaining - self.cfg.time_reserve_s - self.cfg.min_action_budget_s
    if self.cfg.clear_policy == "rollout":
        return self._plan_rollout(targets, budget)
    if self.cfg.clear_policy == "tour_greedy_cost":
        # 贪心"每次挑预计剩余预算花得完、且最近的"目标，滚动推进。
        # 与 'tour' 的差别：这里显式用**剩余预算**逐目标判定可达性，
        # 而不是先排一条全局航路、再边走边判。
        return self._plan_greedy_cost(targets, remaining)
    # ---- 可选：先用"可达近点子集"过滤，再做航路排序 ----
    pool = list(targets)
    if self.cfg.clear_shortlist:
        pool = self.budget_feasible_shortlist(pool, start=self.pos,
            budget=remaining)
        self.stats["n_shortlist_kept"] = self.stats.get("n_shortlist_kept", 0) \
            + len(pool)
        self.stats["n_shortlist_in"] = self.stats.get("n_shortlist_in", 0) \
            + len(targets)
    if not pool:
        return []
    # ---- tour 策略：全局航路结构（可选预算截断） ----
    order = self.order_targets(list(pool), from_pos=self.pos)
    by_ch = {t[0]: t for t in pool}
    ordered = [by_ch[ch] for ch in order if ch in by_ch]
```

```

if not self.cfg.clear_budget_truncation:
    return ordered
out, used, pos = [], 0.0, tuple(self.pos)
for t in ordered:
    c = self.estimate_clear_cost(t[2], from_pos=pos,
                                probe_cap=self.cfg.optimistic_probe_steps)
    if used + c > budget:
        continue
    out.append(t)
    used += c
    pos = tuple(t[2]["center"])
return out

def _plan_greedy_cost(self, targets, remaining):
    """贪心 + 剩余预算可达性 (`clear_policy='tour_greedy_cost'`)。

    诊断动机：末尾 26/49 个"已定位未清除"的目标在变成可清目标时剩余时间完全够用，
    却因为全局航路把它们排在后面而没轮到（清完前面的目标后预算就没了）。
    本策略每一步只在"当前剩余预算真的能走完"的目标里挑最近的，
    从而避免"先承诺一个远点、结果后面全部落空"。

    与 `clear_budget_truncation` 的区别：那个是对已排好的全局航路做事后裁剪，
    本策略是**在挑选时就**把可达性作为约束。
    """
    pos = np.asarray(self.pos, float)
    left = list(targets)
    out, used = [], 0.0
    room = remaining - self.cfg.time_reserve_s - self.cfg.min_action_budget_s
    while left:
        best, best_cost = None, None
        for t in left:
            c = self.estimate_clear_cost(t[2], from_pos=pos,
                                        probe_cap=self.cfg.optimistic_probe_steps)
            if best_cost is None or c < best_cost:
                best, best_cost = t, c
        if best is None or used + best_cost > room:
            break
        out.append(best)
        used += best_cost
        pos = np.asarray(best[2]["center"], float)
        left.remove(best)
    return out

def _plan_rollout(self, targets, budget):
    """一步前瞻 + SPT 推演 (`clear_policy='rollout'`)，见
    :meth:`plan_clear_tour`。"""
    k = len(targets)
    pos0 = tuple(self.pos)
    centers = [tuple(t[2]["center"]) for t in targets]
    nprobe = []
    for t in targets:

```

```

        nprobe.append(self._probe_count(t[2]))
    t_probe = [n * (self.cfg.probe_spacing_m / V_ROBOT + T_CLEAR_FAIL) for n in
               nprobe]

    def leg(a, b):
        return (math.dist(a, centers[b]) / V_ROBOT + T_MEASURE + T_SWITCH
                + t_probe[b] + T_CLEAR_OK)

    cost0 = [leg(pos0, j) for j in range(k)]

    def rollout(cur, left, tb):
        left = set(left)
        n = 0
        while True:
            best_c, best_j = None, None
            for j in left:
                c = leg(cur, j)
                if c <= tb and (best_c is None or c < best_c):
                    best_c, best_j = c, j
            if best_j is None:
                return n
            tb -= best_c
            n += 1
            cur = centers[best_j]
            left.discard(best_j)

    best_j, best_key = None, None
    for j in range(k):
        if cost0[j] > budget:
            continue
        n = 1 + rollout(centers[j], [i for i in range(k) if i != j], budget -
                       cost0[j])
        key = (n, -cost0[j])
        if best_key is None or key > best_key:
            best_key, best_j = key, j
    if best_j is None:
        return []
    out = [targets[best_j]]
    used, pos = cost0[best_j], centers[best_j]
    for j in sorted((i for i in range(k) if i != best_j), key=lambda i: leg(pos,
        i)):
        c = leg(pos, j)
        if used + c > budget:
            continue
        out.append(targets[j])
        used += c
        pos = centers[j]
    return out

def order_targets(self, targets, from_pos=None):
    """航路排序: 最近邻 + 2-opt (以 ``from_pos`` 为固定起点)。
```

目标点用各自的（最新）定位区域中心； $k \leq 20$ ，暴力 2-opt 代价可忽略。

``tour`` 三种模式（对照用）：

- * ``nn`` ：纯最近邻 — 第一步必然最近，但总路程偏长；
- * ``nn2opt`` ：最近邻 + 2-opt (**默认**) — 总路程最短，但 2-opt 的区间反转会改写**第一步**（60 局实测有 14.1% 的排序其第一步不是最近的）；
- * ``nn2opt\_keepfirst`` ：先钉住最近的点作第一步，只对**其后**的子序列做 2-opt。
动机：滚动时域下**只执行第一步**，后面重排一次；所以"总路程最短"并不是正确的目标函数，"第一步最近的 + 剩余结构合理"才是。

```
"""
start = tuple(self.pos if from_pos is None else from_pos)
if len(targets) <= 1 or self.cfg.tour == "nn":
    targets.sort(key=lambda t: math.dist(start, t[2]["center"]))
    return [t[0] for t in targets]
pts = [tuple(t[2]["center"]) for t in targets]
k = len(pts)
cur = start
# 最近邻构造
left = list(range(k))
seq = []
p = cur
while left:
    j = min(left, key=lambda i: math.dist(p, pts[i]))
    seq.append(j)
    left.remove(j)
    p = pts[j]

def total(order):
    d = math.dist(cur, pts[order[0]])
    for a, b in zip(order, order[1:]):
        d += math.dist(pts[a], pts[b])
    return d

# 钉住第一步时，只允许反转 index >= 1 的区间
lo = 1 if (self.cfg.tour == "nn2opt_keepfirst" and k >= 2) else 0
best = total(seq)
improved = True
while improved:
    improved = False
    for i in range(lo, k - 1):
        for j in range(i + 1, k):
            cand = seq[:i] + seq[i:j + 1][::-1] + seq[j + 1:]
            d = total(cand)
            if d < best - 1e-9:
                seq, best, improved = cand, d, True
return [targets[i][0] for i in seq]

def clear_phase(self):
    """清理阶段：**每清一个目标就重新规划一次**（滚动时域）。"""
```

循环体：读最新台账 → 动态规划 → 只执行第一步 → 顺路补测（可能新增可清理频道）
→ 回到循环顶部重新规划。这样“清除之后扫描过程中新定位的位置”会立刻进入航路，
而且航路始终以当前位置为起点、用最新的区域中心优化。

```

"""
first = self.clear_targets()
if not first:
    return 0
self.abort_clear_phase = False
tour0 = self.plan_clear_tour(first) if self.cfg.dynamic_replan else None
if self.cfg.dynamic_replan and not tour0:
    return 0      # 预算内一个都排不下：静默返回，让主循环把时间用于扫描
if not self.can_afford(self.cfg.min_action_budget_s):
    return 0
self.log(f"清理阶段（动态重规划）：{len(first)} 个频道已定位两次 -> "
        f"[[t[0], round(t[2]['radius'])] for t in first]}")
n = 0
guard = 0
static_order = None
tour = tour0
initial_chs = {t[0] for t in (tour0 or first)}
midphase = 0
while not self.aborted and guard < 4 * len(self.recs) + 8:
    guard += 1
    if self.cfg.dynamic_replan:
        tour = self.plan_clear_tour(self.clear_targets()) # ① 每步重新读台账并规划
        if not tour:
            self.log("剩余时间不足以完成任何一条清理航段 -> 结束清理阶段")
            break
        ch, rec, _br = tour[0]      # ② 只执行第一步
        if ch not in initial_chs:
            midphase += 1           # 本阶段中途才定位出来的目标
            self.log(f" 动态航路：频道{ch} 是本阶段中途新定位的，立即插队执行")
        elif len(tour) > 1:
            self.log(f" 动态航路：本步 频道{ch}，随后 "
                    f"[[t[0] for t in tour[1:4]]]{'...' if len(tour) > 4 else ''}")
    else:
        # 静态对照（消融用）
        targets = self.clear_targets()
        if not targets:
            break
        if not self.can_afford(self.cfg.min_action_budget_s):
            break
        if static_order is None:
            static_order = self.order_targets(list(targets))
        if not static_order:
            break
        ch = static_order.pop(0)
        t = next((x for x in targets if x[0] == ch), None)
        if t is None:
            continue
        rec = t[1]
        if self.clear_target(ch, rec):

```

```

        n += 1
    if self.abort_clear_phase:
        self.log("预算不足 -> 结束清理阶段（剩余时间留给扫描）")
        break
    self.enroute_scan() # ③ 顺路补测（可能新增可清理频道）
    self.stats["n_midphase_targets"] = self.stats.get("n_midphase_targets", 0) +
        midphase
    return n

def enroute_scan(self):
    """清理途中在当前位置顺手补测：既补充方位，也给新发现的频道补第二条方位。

    这一步很关键：发现一个新频道只要 6 s，但只有拿到**第二条**方位才能进入清理队列。
    所以一旦在本停点发现新频道，就立刻用第二问的"局部最优第二检测点"再测一条，
    让它本轮就能被清除，而不是等到下一次扫描（往往等不到）。
    """
    if not self.cfg.enroute_scan or self.aborted:
        return 0
    chans = self.channels_to_measure(limit=self.cfg.enroute_scan_max)
    if not chans:
        return 0
    cost = len(chans) * (T_MEASURE + T_SWITCH) + 2.0
    if not self.can_afford(cost + self.cfg.min_action_budget_s * 0.5):
        return 0
    self.log(f" 顺路补测 {len(chans)} 个频道 {chans}")
    n = self.scan_at(self.pos[0], self.pos[1], chans)
    if self.cfg.second_bearing_refine:
        fresh = [ch for ch in chans
                  if not self.recs[ch].cleared and self.recs[ch].n_bearings == 1]
        fresh.sort(key=lambda ch: -self.recs[ch].near_hits)
        for ch in fresh[:self.cfg.second_bearing_max]:
            if self.remaining() < self.cfg.second_bearing_budget_s +
                self.cfg.time_reserve_s:
                break
            self.acquire_second_bearing(ch)
    return n

def acquire_second_bearing(self, ch):
    """给"只有一条方位"的频道补第二检测点（局部最优，含行驶代价）。"""
    rec = self.recs[int(ch)]
    if rec.n_bearings != 1 or rec.cleared:
        return False
    p, info = best_refine_point(rec.pts, rec.svds, self.pos, None, n_t=100,
                                travel_cap_m=self.cfg.refine_travel_cap_m * 1.4)
    if p is None:
        return False
    trav = math.dist(self.pos, (float(p[0]), float(p[1])))
    if not self.can_afford(trav / V_ROBOT + T_MEASURE + T_SWITCH + 20.0):
        return False
    self.log(f" 频道{ch} 新方位 -> 第二检测点 ({p[0]:.0f},{p[1]:.0f}), "
            f"移动 {trav:.0f} m ( $E[D] \approx \{info['E\_diam\_m']:.0f\}$  m, "

```

```

        f"预计代价 {info['cost_s']:.0f} s) ")
    resp = self.measure(float(p[0]), float(p[1]), ch)
    if resp.get("measure_result") == "near":
        return self.do_clear(float(p[0]), float(p[1]), ch)
    self.stats["n_second_bearing"] = self.stats.get("n_second_bearing", 0) + 1
    return rec.n_bearings >= 2

def choose_refine_point(self, ch, rec, br):
    """侧移精定位点: 与上一条视线近似正交, 把交会角拉到 60-90°."""
    c = np.asarray(br["center"], float)
    if self.cfg.refine_mode == "exact":
        p, info = best_refine_point(rec.pts, rec.svds, self.pos,
                                    {"center": c}, n_t=120,
                                    travel_cap_m=self.cfg.refine_travel_cap_m)

        if p is not None:
            return np.asarray(p, float), info
    if not rec.pts:
        return None, None
    S_last = np.asarray(rec.pts[-1], float)
    u = c - S_last
    nu = float(np.linalg.norm(u))
    if nu < 1e-6:
        return None, None
    u = u / nu
    perp = np.array([-u[1], u[0]])
    rho = min(max(1.5 * br["radius"], 80.0), self.cfg.refine_leg_m)
    best = None
    for sgn in (1.0, -1.0):
        p = c + sgn * rho * perp
        if np.hypot(*p) > R_arena - 1.0:
            continue
        trav = float(np.hypot(*(p - np.asarray(self.pos, float))))
        if trav > self.cfg.refine_travel_cap_m:
            continue
        if best is None or trav < best[0]:
            best = (trav, p)
    if best is None:
        return None, None
    return best[1], {"mode": "perp", "rho": rho, "travel_m": best[0]}

def probe_points(self, c, r, poly=None, tried=()):
    """覆盖式试探点。

    * 有定位区域多边形 ``poly`` 时: 只在"到多边形距离  $\leq 20$  m"的格点上试
      — 交会区域常常是细长四边形, 用外接圆 (面积  $\pi r^2$ ) 试探会浪费一倍以上的点;
    * 没有多边形时退化为覆盖半径  $r$  的圆域。
    格点间距  $\leq 20\sqrt{3}$  m, 保证区域内任一点到某个试探点  $\leq 20$  m = 清除半径, **必然成功**。

    注意"必然成功"只在**候选点不被截断**时成立: ``probe_max_points`` 会按"离中心
    由近及远"截断, 等于丢掉覆盖区域的**外圈**, 保证随之失效 (实测  $r^* \geq 120$  m 即开始
    截断, 60 个点对应  $r^* \approx 120$  m, 而截断前需要 61 个)。因此默认

```

```

``probe_unlimited=True`` 取全部候选点; ``probe_max_points`` 只在显式关掉
``probe_unlimited`` (消融对照)时才起作用。
"""
c = np.asarray(c, float)
s = float(self.cfg.probe_spacing_m)
rr = min(float(r), self.cfg.probe_max_radius_m) + R_CLEAR
u = np.array([s, 0.0])
v = np.array([s / 2.0, s * math.sqrt(3.0) / 2.0])
N = int(math.ceil(rr / s)) + 1
cand = []
for i in range(-N, N + 1):
    for j in range(-N, N + 1):
        p = c + i * u + j * v
        if float(np.hypot(*(p - c))) <= rr + 1e-9:
            cand.append(p)
if poly is not None and len(poly) >= 3:
    P = np.asarray(poly, float)
    keep = []
    for p in cand:
        if _dist_point_polygon(p, P) <= R_CLEAR + 1e-9:
            keep.append(p)
    cand = keep if keep else cand
cand.sort(key=lambda p: float(np.hypot(*(p - c))))
out = []
for p in cand:
    if any(float(np.hypot(*(p - np.asarray(q, float)))) < 2.0 for q in tried):
        continue
    out.append(p)
if self.cfg.probe_unlimited:
    return out
return out[:self.cfg.probe_max_points]

def sweep_clear(self, ch, rec, c, r, poly=None):
    """在区域 ``(c, r, poly)`` 上做覆盖式试探。

    返回 :data:`SWEEP_OK` / :data:`SWEEP_MISS` / :data:`SWEEP_BUDGET` 三者之一。
    **预算不足时立刻返回 ``SWEEP_BUDGET``，一个 ``/clear`` 都不发** —
    否则会污染 ``clear_fail``、并在论文里被误读成"区域覆盖了却没找到目标"。
    """
    probes = self.probe_points(c, r, poly=poly, tried=rec.tried_probes)
    full = self.probe_points(c, r, poly=poly)
    n_try = 0
    for p in probes:
        if self.aborted:
            self.log(f" 覆盖式试探中断: 程序已中止 (已试 {n_try} 点)")
            return SWEEP_MISS if n_try else SWEEP_BUDGET
        if not self.can_afford(T_CLEAR_FAIL + 5.0):
            self.log(f" 覆盖式试探放弃: 剩余 {self.remaining():.0f} s 不足以支付一次 "
                    f"/clear (候选 {len(full)} 个, 实际尝试 {n_try} 个)")
            self.stats["n_sweep_aborted"] += 1
            return SWEEP_BUDGET if n_try == 0 else SWEEP_MISS

```

```

trav = math.dist(self.pos, (float(p[0]), float(p[1])))
if not self.can_afford(trav / V_ROBOT + T_CLEAR_FAIL + 3.0):
    self.log(f" 覆盖式试探放弃: 剩余 {self.remaining():.0f} s 不足以走到下一个"
            f"试探点 ({trav:.0f} m; 候选 {len(full)} 个, 实际尝试 {n_try} 个)")
    self.stats["n_sweep_aborted"] += 1
    return SWEEP_BUDGET if n_try == 0 else SWEEP_MISS
n_try += 1
rec.tried_probes.append((float(p[0]), float(p[1])))
if self.do_clear(float(p[0]), float(p[1]), ch):
    self.log(f" 覆盖式试探命中: 第 {n_try}/{len(full)} 个候选点"
            f" (间距 {self.cfg.probe_spacing_m:.0f} m)")
    return SWEEP_OK
# 候选点全部试完仍未命中: 这才是真正的"区域内没找到目标"
self.stats["n_sweep_miss"] += 1
self.log(f" 覆盖式试探未命中: {n_try}/{len(full)} 个候选点全部试过"
        f" ({'多边形区域' if poly else '外接圆'}, r*={r:.0f} m)")
return SWEEP_MISS

def clear_target(self, ch, rec):
    """定位→逼近→(必要时)侧移精定位→覆盖式试探清除。

    失败归因**必须三分** (见 :data:`SWEEP_BUDGET`):

    * ``SWEEP_OK`` — 清除成功;
    * ``SWEEP_MISS`` — 候选点全试过仍没打中 ⇒ 累加 ``clear_fail`` (真的失败了);
    * ``SWEEP_BUDGET`` — 预算不足、**一个 /clear 都没发** ⇒ **不**累加
      ``clear_fail``,
    也不报"未清除成功", 而是收敛回清理阶段由 ``plan_clear_tour`` 决定收工。
    这正是"预算耗尽后还调用 sweep_clear 并记一笔失败"那个统计/日志缺陷的修复点。
    """
    self.log(f"清理频道{ch}: 当前 {rec.n_bearings} 条方位, "
            f"起点 ({self.pos[0]:.0f}, {self.pos[1]:.0f})")
    budget_stop = False
    # (1) 逼近定位区域中心 (每次逼近顺带测一条方位, 移动本来就要花)
    for _ in range(self.cfg.max_approach_steps):
        br = self.bounded_region(rec)
        if br is None:
            rec.clear_fail += 1
            return False
        c = np.asarray(br["center"], float)
        d = math.dist(self.pos, tuple(c))
        if d <= max(br["radius"], self.cfg.arrive_radius_m):
            break
    cost = d / V_ROBOT + T_MEASURE + T_SWITCH + 4.0
    if not self.can_afford(cost):
        # 走不过去了: 进入第 (3) 步, 由覆盖式试探统一判定"预算不足"
        budget_stop = True
        break
    resp = self.measure(float(c[0]), float(c[1]), ch)
    if resp.get("measure_result") == "near":
        return self.do_clear(float(c[0]), float(c[1]), ch)

```

```

        if self.aborted:
            return False
# (2) 区域还大：侧移一条正交视线，把 r* 压小
if not budget_stop:
    br = self.bounded_region(rec)
    if br is not None and br["radius"] > self.cfg.direct_probe_radius_m:
        p, info = self.choose_refine_point(ch, rec, br)
        if p is not None:
            trav = math.dist(self.pos, (float(p[0]), float(p[1])))
            cost = trav / V_ROBOT + T_MEASURE + T_SWITCH + 4.0
            if self.can_afford(cost):
                self.stats["n_refine"] += 1
                self.log(f" 侧移精定位 -> ({p[0]:.0f},{p[1]:.0f}), "
                        f"移动 {trav:.0f} m ({info}) ")
                resp = self.measure(float(p[0]), float(p[1]), ch)
                if resp.get("measure_result") == "near":
                    return self.do_clear(float(p[0]), float(p[1]), ch)
# (3) 覆盖式试探清除
br = self.bounded_region(rec)
if br is None:
    rec.clear_fail += 1
    return False
c = np.asarray(br["center"], float)
r = min(br["radius"], self.cfg.probe_max_radius_m)
poly = br.get("poly")
st = self.sweep_clear(ch, rec, c, r, poly=poly)
if st == SWEEP_OK:
    return True
if st == SWEEP_BUDGET:
    # 预算不足、“一个 /clear 都没发出”：不是清除失败，不计 clear_fail、
    # 也不上升为空/随机搜索；交由调用方收敛本阶段。
    self.abort_clear_phase = True
    self.log(f" ↪ 频道{ch} 未尝试清除（预算不足），本阶段收敛")
    return False
# (4) 区域内没找到：说明区域被算小了（或误差超界），扩大半径再试一轮
if br["radius"] <= self.cfg.probe_max_radius_m:
    r2 = min(br["radius"] * 1.6 + 20.0, self.cfg.probe_max_radius_m)
    self.log(f" 区域内未发现目标 -> 扩大到 r={r2:.0f} m 再试")
    st2 = self.sweep_clear(ch, rec, c, r2, poly=None)
    if st2 == SWEEP_OK:
        return True
    if st2 == SWEEP_BUDGET:
        self.abort_clear_phase = True
        self.log(f" ↪ 频道{ch} 扩大半径后未尝试（预算不足），本阶段收敛")
        return False
    rec.clear_fail += 1
    self.log(f" [x] 频道{ch} 本轮未清除成功（候选点已试遍仍无目标，留待后续扫描后再清）")
    return False

# ----- 主循环 -----
def run(self):

```

```

t0 = time.time()
resp = self.arena.enter()
if resp.get("accepted") is not True:
    raise ArenaError(f"/enter 未被接受: {resp}")
self.log(f"进入靶区: 可用预算 {self.remaining():.0f} s"
        f" ({self.arena.budget_kind()}) ")
try:
    # 第 0 轮: 机器狗本来就在当前位置 (原点), 直接扫描全部频道
    chans0 = self.channels_to_measure()
    self.log(f"第 0 轮扫描 (原地): 待测 {len(chans0)} 个频道 {chans0}")
    self.scan_points.append(self.pos)
    self.scan_at(self.pos[0], self.pos[1], chans0)
    rounds = 1
    stall = 0
    while (not self.aborted and rounds < self.cfg.max_rounds
           and self.remaining() > self.cfg.time_reserve_s):
        did = 0
        t_before = float(self.arena.virtual_time_s)
        if self.clear_targets() and (rounds >=
                                     self.cfg.min_scan_rounds_before_clear
                                     or not self.plan_scan()):
            did += self.clear_phase()
            self.stats["t_clear_s"] += (float(self.arena.virtual_time_s)
                                       - t_before)
            self.stats["t_clear_ends_s"] = float(self.arena.virtual_time_s)
        if self.aborted or self.remaining() <= self.cfg.time_reserve_s:
            break
        plan = self.plan_scan()
        if plan is None:
            # 扫描被预算闸门否掉时, plan_scan() 也可能只是"暂无值得测的频道";
            # 此时若还有可清目标, 就再走一次清理阶段, 而不是直接判为 stall。
            if self.clear_targets():
                did += self.clear_phase()
                self.stats["t_clear_s"] += (float(self.arena.virtual_time_s)
                                           - t_before)
                self.stats["t_clear_ends_s"] = float(self.arena.virtual_time_s)
            if self.aborted:
                break
        elif self.can_afford(plan["cost_s"]) \
             and (did == 0 or not self.clear_targets()):
            did += self.execute_scan(plan)
            self.stats["t_scan_s"] += (float(self.arena.virtual_time_s)
                                       - t_before)

            rounds += 1
        if did == 0:
            stall += 1
            if stall >= 2:
                self.log("没有可清理的频道、也没有值得检测的频道 -> 提前结束")
                break
    else:
        stall = 0

```

```

finally:
    try:
        self.arena.exit()
    except ArenaError as e: # 接口已关闭等
        self.log(f"/exit 异常: {e}")
self.stats["program_runtime_s"] = time.time() - t0
return self.report()

# ----- 汇报 -----
def report(self):
    truth = self.arena.truth()
    n_cleared = sum(1 for r in self.recs.values() if r.cleared)
    total = (truth or {}).get("n_sources")
    vt = float(self.arena.virtual_time_s)
    last = float(self.stats.get("vtime_at_last_clear", vt))
    st = getattr(self.arena, "stats", {})
    rep = {
        "cleared": int(n_cleared),
        "n_sources": total,
        "clear_ratio": (n_cleared / total) if total else None,
        "virtual_time_s": vt,
        "avg_clear_time_s": (last / n_cleared) if n_cleared else None,
        "program_runtime_s": float(self.stats.get("program_runtime_s", 0.0)),
        "budget_kind": self.arena.budget_kind(),
        "remaining_budget_s": self.remaining(),
        "n_measure": self.stats["n_measure"],
        "n_bearings": self.stats["n_bearings"],
        "n_clear": self.stats["n_clear"],
        "n_clear_fail": self.stats["n_clear_fail"],
        "n_scan_rounds": self.stats["n_scan_rounds"],
        "n_refine": self.stats["n_refine"],
        "n_second_bearing": self.stats.get("n_second_bearing", 0),
        "n_replans": self.stats.get("n_replans", 0),
        "n_midphase_targets": self.stats.get("n_midphase_targets", 0),
        "n_rejected": self.stats["n_rejected"],
        "travel_m": float(st.get("travel_m", 0.0)),
        "mock_stats": dict(st),
        "n_sweep_aborted": self.stats.get("n_sweep_aborted", 0),
        "n_sweep_miss": self.stats.get("n_sweep_miss", 0),
        "n_probe_issued": self.stats.get("n_probe_issued", 0),
        "n_scan_gated": self.stats.get("n_scan_gated", 0),
        "t_scan_s": self.stats.get("t_scan_s", 0.0),
        "t_clear_s": self.stats.get("t_clear_s", 0.0),
        "t_clear_ends_s": self.stats.get("t_clear_ends_s", 0.0),
        "n_leftover_located": sum(
            1 for r in self.recs.values()
            if not r.cleared and r.n_bearings >= self.cfg.clear_min_bearings),
        "scan_points": [[round(float(x), 2), round(float(y), 2)]
                        for (x, y) in self.scan_points],
        "channels": {str(ch): {"n_bearings": rec.n_bearings,
                               "cleared": bool(rec.cleared),

```

```

        "no_signal": rec.no_signal,
        "near": rec.near_hits,
        "status": self.region(rec).get("status"),
        "r_star_m": (self.bounded_region(rec) or
                      {}).get("radius")}
        for ch, rec in self.recs.items() if rec.n_bearings or
            rec.cleared},
        "truth": truth,
    }
    return rep

# -----
# 自检: 策略层的结构性断言 (离线 mock 上跑)
# -----
def selfcheck_mock(verbose=True, seed=20260913, cases=3):
    out = []
    ok_all = True

    def rec(name, ok, detail=""):
        nonlocal ok_all
        ok_all = ok_all and bool(ok)
        out.append({"check": name, "ok": bool(ok), "detail": detail})
        if verbose:
            print(f"[{'OK ' if ok else 'FAIL'}] {name} {detail}")

    # S1 覆盖式试探点确实能覆盖圆域 (任意点 20 m 内有试探点)
    rb = P3Robot(MockArena(seed=1), P3Config())
    rng = np.random.default_rng(7)
    worst = 0.0
    for (cx, cy, r) in ((0.0, 0.0, 40.0), (500.0, -300.0, 80.0), (-900.0, 900.0,
        25.0)):
        pts = np.array(rb.probe_points((cx, cy), r))
        for _ in range(400):
            a = rng.uniform(0, 2 * math.pi)
            rad = r * math.sqrt(rng.random())
            p = np.array([cx + rad * math.cos(a), cy + rad * math.sin(a)])
            d = float(np.min(np.hypot(pts[:, 0] - p[0], pts[:, 1] - p[1])))
            worst = max(worst, d)
    rec("S1 覆盖式试探: 区域内任意点到最近试探点 <= 20 m (清除半径)", worst <= R_CLEAR +
        1e-6,
        f"最大覆盖距离 {worst:.2f} m (试探点间距 {rb.cfg.probe_spacing_m} m)")

    # S2 端到端: mock 上跑通、清除率、时间预算
    ratios, times, cleared = [], [], []
    for k in range(cases):
        arena = MockArena(seed=seed + 100 * k)
        rb = P3Robot(arena, P3Config(), log=None)
        rep = rb.run()
        ratios.append(rep["clear_ratio"])
        times.append(rep["avg_clear_time_s"])

```

```

cleared.append(rep["cleared"])
if verbose:
    print(f" 案例{seed + 100 * k}: 清除 {rep['cleared']}/{rep['n_sources']}, "
          f"虚拟时间 {rep['virtual_time_s']:.0f}s, "
          f"平均 {rep['avg_clear_time_s']:.1f}s, "
          f"扫描 {rep['n_scan_rounds']} 轮, 检测 {rep['n_measure']} 次")
rec("S2 mock 演练: 清除率>0 且每个被清除源平均时间有限",
    all(r is not None and r > 0 for r in ratios) and all(t and t > 0 for t in
        times),
    f"{cases} 局: 清除率 {[round(r, 3) for r in ratios]}, 平均时间 "
    f"{[round(t, 1) for t in times]} s")

# S3 策略不会超预算 (mock 会计时; 超时会 accepted=false)
arena = MockArena(seed=seed + 7, budget_s=400.0)
rb = P3Robot(arena, P3Config(), log=None)
rep = rb.run()
rec("S3 策略在时间预算内收工 (不触发超时拒动)",
    rep["n_rejected"] == 0 and rep["virtual_time_s"] <= 400.0 + 1e-9,
    f"虚拟时间 {rep['virtual_time_s']:.1f}s / 400s, 拒动 {rep['n_rejected']} 次")

# S4 已清除的频道不再被检测 ("跳过已清理过的点, 节省时间")
arena = MockArena(seed=seed + 11)
rb = P3Robot(arena, P3Config(), log=None)
rep = rb.run()
after = {}
for ch, r in rb.recs.items():
    if r.cleared and r.cleared_at is not None:
        k = sum(1 for (_x, _y, t, _res) in r.meas_log if t > r.cleared_at + 1e-9)
        if k:
            after[ch] = k
rec("S4 已清除频道不再被检测", not after,
    f"已清除 {sum(1 for r in rb.recs.values() if r.cleared)} 个频道; "
    f"清除后仍被检测的频道 {after or '无'}")

# S5 定位区域一定包含真源 (问题 1 的交会算法 + 模拟器误差界的一致性)
bad5 = []
n5 = 0
src_map = {s["channel"]: (s["x_m"], s["y_m"]) for s in (rep["truth"] or
    {}).get("sources", [])}
for ch, r in rb.recs.items():
    if r.n_bearings < 2:
        continue
    br = rb.bounded_region(r)
    if not br or not br.get("poly"):
        continue
    n5 += 1
    xy = src_map.get(ch)
    if xy is None:
        continue
    P = np.asarray(br["poly"], float)
    if not _point_in_polygon(np.asarray(xy, float), P):

```

```

        bad5.append(ch)
rec("S5 定位区域（多边形）包含真源", not bad5,
    f"检查 {n5} 个已定位频道, 越界 {bad5 or '无'}")

# S6 探索兜底: 没有任何方位时选"离已访问点最远"的点
rb6 = P3Robot(MockArena(seed=seed + 3), P3Config(), log=None)
sel6, _f = rb6.choose_scan_point()
rec("S6 无方位时退化为探索点（离已访问点最远）",
    sel6["kind"] == "explore" and math.hypot(*sel6["xy"]) > 1000.0,
    f"探索点 {tuple(round(v) for v in sel6['xy'])}, 距原点 "
    f"{math.hypot(*sel6['xy']):.0f} m")

# S7 动态重规划: 每清一个目标都重规划一次; 静态对照为 0
arena7 = MockArena(seed=seed + 13)
rb7 = P3Robot(arena7, P3Config(), log=None)
rep7 = rb7.run()
arena7s = MockArena(seed=seed + 13)
rb7s = P3Robot(arena7s, P3Config(dynamic_replan=False), log=None)
rep7s = rb7s.run()
rec("S7 动态重规划: 清除途中每步重规划, 静态对照 0 次",
    rep7["n_replans"] >= rep7["cleared"] >= 1 and rep7s["n_replans"] == 0,
    f"动态: 清除 {rep7['cleared']} 个、重规划 {rep7['n_replans']} 次、"
    f"中途新定位插队 {rep7['n_midphase_targets']} 个;"
    f"静态: 重规划 {rep7s['n_replans']} 次、中途插队 {rep7s['n_midphase_targets']} 个")

# S8 动态航路: 默认不裁剪（目标齐全）、顺序与航路排序一致
rb8 = P3Robot(MockArena(seed=seed + 21), P3Config(), log=None)
rb8.arena.enter()
for ch, (gx, gy) in ((1, (300.0, 200.0)), (2, (-500.0, 700.0)), (3, (900.0,
    -400.0))):
    r8 = ChannelRec(ch)
    # 在真源周围取两条正交视线（交会角 90°）⇒ 定位区域小且有界
    for ang, dist in ((0.0, 500.0), (90.0, 500.0)):
        sx = gx + dist * math.cos(math.radians(ang))
        sy = gy + dist * math.sin(math.radians(ang))
        a = math.degrees(math.atan2(gy - sy, gx - sx)) % 360.0
        r8.pts.append((sx, sy))
        r8.svds.append(a)
    rb8.recs[ch] = r8
tg8 = rb8.clear_targets()
tour8 = rb8.plan_clear_tour(tg8)
rec("S8 动态航路: 默认不裁剪、顺序与航路排序一致",
    len(tg8) == 3 and len(tour8) == 3
    and [t[0] for t in tour8] == rb8.order_targets(list(tg8)),
    f"{len(tg8)} 个已定位频道 -> 航路 {[t[0] for t in tour8]}, "
    f"各区域半径 {[round(t[2]['radius']) for t in tour8]} m")

# S9 覆盖式试探**不得被点数上限截断**（截断会丢掉外圈、破坏 20 m 覆盖保证）
rb9 = P3Robot(MockArena(seed=seed + 31), P3Config(), log=None)
n9_uncapped = len(rb9.probe_points(np.zeros(2), 195.0))

```

```

rb9c = P3Robot(MockArena(seed=seed + 31),
                P3Config(probe_unlimited=False, probe_max_points=60), log=None)
n9_capped = len(rb9c.probe_points(np.zeros(2), 195.0))
# 覆盖保证: r*=195 m 的外接圆内任一点到最近候选点必须 <= 20 m
pts9 = np.asarray(rb9c.probe_points(np.zeros(2), 195.0), float)
rng9 = np.random.default_rng(11)
worst9 = 0.0
for _ in range(400):
    a = rng9.uniform(0, 2 * math.pi)
    rad = 195.0 * math.sqrt(rng9.random())
    p = np.array([rad * math.cos(a), rad * math.sin(a)])
    worst9 = max(worst9, float(np.min(np.hypot(pts9[:, 0] - p[0],
                                                pts9[:, 1] - p[1]))))
rec("S9 覆盖式试探不被截断 (大 r* 时仍满足 20 m 覆盖保证)",
    worst9 <= R_CLEAR + 1e-6 and n9_uncapped > n9_capped,
    f"r*=195 m: {n9_uncapped} 个候选点 (封顶对照 {n9_capped} 个), "
    f"最大覆盖距离 {worst9:.2f} m")

# S10 预算不足时**不得**把"未尝试"记成"清除失败" (失败归因三分)
arena10 = MockArena(seed=seed + 41)
rb10 = P3Robot(arena10, P3Config(), log=None)
rb10.arena.enter()
rec10 = ChannelRec(4)
for (sx, sy) in ((0.0, 0.0), (0.0, 300.0)): # 两条正交视线 -> 小而紧的区域
    a = math.degrees(math.atan2(150.0 - sy, 100.0 - sx)) % 360.0
    rec10.pts.append((sx, sy))
    rec10.svds.append(a)
rb10.recs[4] = rec10
br10 = rb10.bounded_region(rec10)
arena10.vtime = arena10.budget - 0.5 # 只剩 0.5 s: 连一次 /clear 都付不起
f10 = rb10.remaining()
st10 = rb10.sweep_clear(4, rec10, np.asarray(br10["center"], float),
                        float(br10["radius"]), poly=br10.get("poly"))
rec("S10 预算不足时不误记 clear_fail / 不发 /clear",
    st10 == SWEEP_BUDGET and rec10.clear_fail == 0
    and rb10.stats["n_probe_issued"] == 0
    and rb10.stats["n_sweep_aborted"] == 1,
    f"剩余预算 {f10:.1f} s (付不起一次 /clear) => 返回 {st10!r}, "
    f"clear_fail={rec10.clear_fail}, 发出 /clear {rb10.stats['n_probe_issued']}
    次, "
    f"n_sweep_aborted={rb10.stats['n_sweep_aborted']}")

# S11 区域中心在"排序决策 -> 执行"之间**不发生漂移** (否则排序依据会过期)
arena11 = MockArena(seed=seed + 51)
rb11 = P3Robot(arena11, P3Config(), log=None)
orig_plan11 = rb11.plan_clear_tour
orig_clear11 = rb11.clear_target
snap11 = {}
drift11 = []

def plan11(targets, remaining_s=None):

```

```

    for t in (targets or []):
        snap11[t[0]] = tuple(t[2]["center"])
    return orig_plan11(targets, remaining_s=remaining_s)

def clear11(ch, rec):
    br = rb11.bounded_region(rec)
    if br is not None and ch in snap11:
        drift11.append(math.dist(snap11[ch], tuple(br["center"])))
    return orig_clear11(ch, rec)

rb11.plan_clear_tour = plan11
rb11.clear_target = clear11
rb11.run()
rec("S11 区域中心在'排序决策→执行'之间不漂移",
    bool(drift11) and max(drift11) == 0.0,
    f"检查 {len(drift11)} 次执行, 最大漂移 {max(drift11) if drift11 else
        float('nan'):.1f} m"
    f" (新方位只在 /measure 后产生, 逼近途中不产生)")

# S12 航路排序的三种模式: 第一步必为最近邻 (nn / nn2opt_keepfirst),
# 或允许 2-opt 改写第一步但总路程不更差 (nn2opt)
rb12 = P3Robot(MockArena(seed=seed + 61), P3Config(), log=None)
rb12.arena.enter()
pts12 = [(300.0, 200.0), (-500.0, 700.0), (900.0, -400.0),
          (-1200.0, -300.0), (200.0, -1500.0)]
for i, (gx, gy) in enumerate(pts12, start=1):
    r12 = ChannelRec(i)
    for ang in (0.0, 90.0):
        sx = gx + 500.0 * math.cos(math.radians(ang))
        sy = gy + 500.0 * math.sin(math.radians(ang))
        r12.pts.append((sx, sy))
        r12.svds.append(math.degrees(math.atan2(gy - sy, gx - sx)) % 360.0)
    rb12.recs[i] = r12
tg12 = rb12.clear_targets()
start12 = tuple(rb12.pos)
nearest12 = min(tg12, key=lambda t: math.dist(start12, t[2]["center"]))[0]

def plen12(order):
    seq = [next(t for t in tg12 if t[0] == ch) for ch in order]
    d = math.dist(start12, seq[0][2]["center"])
    for a, b in zip(seq, seq[1:]):
        d += math.dist(a[2]["center"], b[2]["center"])
    return d

rb12.cfg.tour = "nn"
o_nn = rb12.order_targets(list(tg12))
rb12.cfg.tour = "nn2opt_keepfirst"
o_kf = rb12.order_targets(list(tg12))
rb12.cfg.tour = "nn2opt"
o_2o = rb12.order_targets(list(tg12))
rec("S12 航路排序: nn 与 keepfirst 第一步为最近邻; 2-opt 不劣于 nn",

```

```

o_nn[0] == nearest12 and o_kf[0] == nearest12
and plen12(o_2o) <= plen12(o_nn) + 1e-9,
f"最近目标=频道{nearest12}; nn 第一步={o_nn[0]}, keepfirst={o_kf[0]}, "
f"2-opt={o_2o[0]}; 路程 nn={plen12(o_nn):.0f} m, 2-opt={plen12(o_2o):.0f} m")

if verbose:
    print(f"\n自检总结: {'全部通过' if ok_all else '存在失败项'}")
return {"ok": ok_all, "checks": out}

def main():
    ap = argparse.ArgumentParser(description="问题3 机器狗策略（离线自检 / 单局演练）")
    ap.add_argument("--selfcheck", action="store_true")
    ap.add_argument("--mock", action="store_true", help="跑一局离线演练并打印明细")
    ap.add_argument("--seed", type=int, default=20260913)
    ap.add_argument("--dist-weight", type=float, default=1.5)
    ap.add_argument("--grid-step", type=float, default=100.0)
    ap.add_argument("--quiet", action="store_true")
    args = ap.parse_args()
    if args.selfcheck:
        res = selfcheck_mock(verbose=True)
        raise SystemExit(0 if res["ok"] else 1)
    cfg = P3Config(dist_weight=args.dist_weight, grid_step=args.grid_step)
    arena = MockArena(seed=args.seed)
    rb = P3Robot(arena, cfg, log=None if args.quiet else print)
    rep = rb.run()
    print(json.dumps({k: v for k, v in rep.items()
                      if k not in ("truth", "channels", "mock_stats")},
                     ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```

0.6 Q4/src/p4_arena_ext.py

"""问题 4 的场景生成（定向 / 全向混合案例）。

单独成一个模块，是为了让 `p4\_run.py`（演练/正式测试入口）、`p4\_selftest.py`（自检）与 `p4\_diag.py`（诊断）用**同一套**案例定义 —— 否则"自检通过但演练漏源"这类问题会被掩盖。

题目口径（附件 2 第 2.2 节 + 附录 1）：

- * 定向源的**有效覆盖角度范围是 180°** $\Rightarrow$ 覆盖半角 ``cone\_half = 90°``;
- * 定向方向未知，可以是任意角度;
- * 有效接收半径在 1000-1500 m 之间;
- * 干扰源总数 10-16 个（本题按 10-16 均匀取）。

"""

```
from __future__ import annotations
```

```
import math
```

```

import numpy as np

from p3_arena import MockArena, MockSource, R_ARENA, R_RECV_MAX, R_RECV_MIN

SCENARIOS = ("directed", "directed_only", "directed_edge", "omni", "uniform",
             "mixed_uniform", "minrange")

def directed_case(seed=0, n=None, spec=None, directed_frac=0.5,
                 r_recv=(R_RECV_MIN, R_RECV_MAX), budget=360000.0,
                 outward_frac=0.5):
    """造一个案例；返回 `` (arena, sources)``。

    * ``spec`` 给定 ``[(channel, x, y, r_recv, dir_deg), ...]`` 时直接照做（自检用）；
    * ``n`` 不给就随机取 10-16 个源；
    * ``directed_frac``：定向源比例；``outward_frac``：其中"朝向背离原点"的比例
      （这是最难的一类：靶区里的点几乎都落在它背后）。
    """
    if spec is not None:
        srcls = [MockSource(int(c), float(x), float(y), float(rr),
                             cone_half=90.0, dir_deg=float(dd))
                  for (c, x, y, rr, dd) in spec]
        return MockArena(seed=int(seed), budget_s=float(budget), sources=srcls, srcls

    rng = np.random.default_rng(int(seed))
    k = int(n) if n else int(rng.integers(10, 17))
    k = max(1, min(k, 20))
    chans = rng.choice(np.arange(1, 21), size=k, replace=False)
    srcls = []
    for ch in chans:
        r = R_ARENA * math.sqrt(float(rng.random()))
        a = float(rng.uniform(0.0, 2.0 * math.pi))
        x, y = r * math.cos(a), r * math.sin(a)
        if float(rng.random()) < float(directed_frac):
            d = (math.degrees(a) % 360.0) if float(rng.random()) <
                float(outward_frac) \
            else float(rng.uniform(0.0, 360.0))
            srcls.append(MockSource(int(ch), x, y, float(rng.uniform(*r_recv)),
                                    cone_half=90.0, dir_deg=d))
        else:
            srcls.append(MockSource(int(ch), x, y, float(rng.uniform(*r_recv))))
    return MockArena(seed=int(seed), budget_s=float(budget), sources=srcls, srcls

def make_arena(scenario, seed, n_sources=None, budget=360000.0,
              directed_frac=0.5, r_recv=(R_RECV_MIN, R_RECV_MAX)):
    """按场景名造案例（`p4_run.py` 的入口用）。

    ``directed``：全向/定向混合；``directed_only``：全部定向；
    ``directed_edge``：源偏外圈  $r \in [1500, 1800]$ （专打"贴边朝外"的弱点）；

```

```

``omni``: 全向（作为对照，检验策略没有为定向源牺牲全向性能）。
"""
if scenario in ('uniform', 'mixed_uniform', 'minrange'):
    return directed_case(seed, n=n_sources, directed_frac=.5 if
        scenario=='mixed_uniform' else 1.,
        outward_frac=0., budget=budget,
        r_rcv=(1000., 1000.) if scenario=='minrange' else r_rcv)[0]
if scenario == "omni":
    arena, _ = directed_case(seed, n=n_sources, directed_frac=0.0,
        budget=budget, r_rcv=r_rcv)
    return arena
if scenario == "directed_only":
    arena, _ = directed_case(seed, n=n_sources, directed_frac=1.0,
        budget=budget, r_rcv=r_rcv)
    return arena
if scenario == "directed_edge":
    rng = np.random.default_rng(int(seed) + 991)
    k = int(n_sources) if n_sources else int(rng.integers(10, 17))
    k = max(1, min(k, 20))
    chans = rng.choice(np.arange(1, 21), size=k, replace=False)
    srcs = []
    for ch in chans:
        r = math.sqrt(rng.uniform(1500.0 ** 2, R_ARENA ** 2))
        a = float(rng.uniform(0.0, 2.0 * math.pi))
        x, y = r * math.cos(a), r * math.sin(a)
        if float(rng.random()) < float(directed_frac):
            d = (math.degrees(a) % 360.0) if float(rng.random()) < 0.5 \
                else float(rng.uniform(0.0, 360.0))
            srcs.append(MockSource(int(ch), x, y, float(rng.uniform(*r_rcv)),
                cone_half=90.0, dir_deg=d))
        else:
            srcs.append(MockSource(int(ch), x, y, float(rng.uniform(*r_rcv))))
    return MockArena(seed=int(seed), budget_s=float(budget), sources=srcs)
arena, _ = directed_case(seed, n=n_sources, directed_frac=directed_frac,
    budget=budget, r_rcv=r_rcv)
return arena

```

0.7 Q4/src/p4_certificate.py

"""Conservative continuous position/orientation coverage for problem 4.

A source at p is heard at q if $|q-p| \leq 1000$ and $(q-p).d \geq 0$ for some unknown unit direction d . Reversing the protocol's direction convention just replaces d by $-d$; this certificate includes all directions.

The disk is covered by square cells and the unit circle by angle intervals. For a cell centre c , radius ρ and orientation half width h , a measurement q certifies an entire position/orientation cell when

```

|q-c| + rho <= 1000,
angle(q-c, d_center) + h <= acos(rho / |q-c|).

```

The first inequality is the triangle inequality. The second gives $(q-c).d \geq \rho$ for every direction in the interval, hence $(q-p).d \geq 0$ for every p in the square. Thus there is no interpolation assumption. Only actual no_signal replies may exclude states of an uncleared channel.

Partial coverage is a lower bound on area-times-angle coverage under a uniform position and orientation reference measure, NOT a clearance-rate guarantee or an inferred source probability. Boundary squares are assigned zero area in this lower bound, unless every state has been certified.

```

"""
from __future__ import annotations

from dataclasses import dataclass
import math

import numpy as np

@dataclass(frozen=True)
class CertificateResult:
    complete: bool
    excluded_fraction_lower_bound: float
    n_uncovered_states: int
    n_cells: int
    n_orientations: int

class DirectionalCertificate:
    """Reusable state discretisation with strict spatial and angular margins.

    ``station_mask(q)`` returns a boolean (n_cells, n_dir) array. Union masks
    of no_signal measurements to certify an absent channel; union masks of
    intended stations to certify a static exploration layout. Use
    ``from_log(rec.meas_log)`` for an actual ChannelRec measurement log.
    Coordinates are never read from the simulator's hidden source state.
    """

    def __init__(self, step_m=60.0, n_dir=32, arena_radius_m=1800.0,
                  reception_radius_m=1000.0):
        if step_m <= 0 or n_dir < 4 or arena_radius_m <= 0:
            raise ValueError("positive geometry and at least 4 orientations required")
        self.step_m = float(step_m)
        self.n_dir = int(n_dir)
        self.arena_radius_m = float(arena_radius_m)
        self.reception_radius_m = float(reception_radius_m)
        # An integer number of exact squares covers  $[-R,R]^2$ . Keep every
        # square intersecting the disk, including cells whose centres lie out.
        n_side = int(math.ceil(2 * self.arena_radius_m / self.step_m))
        side = 2 * self.arena_radius_m / n_side
        self.cell_side_m = side

```

```

ax = -self.arena_radius_m + (np.arange(n_side) + 0.5) * side
xx, yy = np.meshgrid(ax, ax)
pts = np.column_stack((xx.ravel(), yy.ravel()))
closest = np.maximum(np.abs(pts) - side / 2, 0)
keep = np.sum(closest * closest, axis=1) <= self.arena_radius_m ** 2
self.centers = pts[keep]
self.rho = side / math.sqrt(2.0)
self.angles = (np.arange(self.n_dir) + 0.5) * (2 * math.pi / self.n_dir)
self.angle_half_width = math.pi / self.n_dir
farthest = np.abs(self.centers) + side / 2
self.interior = np.sum(farthest * farthest, axis=1) <= self.arena_radius_m
    ** 2
self._weight = side ** 2 / (math.pi * self.arena_radius_m ** 2 * self.n_dir)
self._cache = {}

@property
def shape(self):
    return (len(self.centers), self.n_dir)

def empty(self):
    return np.zeros(self.shape, dtype=bool)

def station_mask(self, xy, *, cache=True):
    xy = (float(xy[0]), float(xy[1]))
    if not all(math.isfinite(v) for v in xy):
        raise ValueError("station coordinates must be finite")
    if cache and xy in self._cache:
        return self._cache[xy]
    delta = np.asarray(xy) - self.centers
    dist = np.hypot(delta[:, 0], delta[:, 1])
    valid = (dist + self.rho <= self.reception_radius_m - 1e-8) & (dist >
        self.rho)
    out = self.empty()
    if np.any(valid):
        a = np.arctan2(delta[valid, 1], delta[valid, 0])
        diff = np.abs((a[:, None] - self.angles + math.pi) % (2 * math.pi) -
            math.pi)
        half = np.arccos(self.rho / dist[valid]) - self.angle_half_width
        out[valid] = diff <= half[:, None] - 1e-12
    if cache:
        if len(self._cache) >= 256:
            self._cache.clear()
        out.flags.writeable = False
        self._cache[xy] = out
    return out

def mask_for_points(self, points):
    out = self.empty()
    for xy in points:
        out |= self.station_mask(xy)
    return out

```

```

def all_orientation_cells(self, points):
    """Certify spatial cells by exact circular-interval union.

    This stronger final check has the same strict spatial margin, but no
    angular discretisation loss. A cell is True only when the station
    arcs cover all of  $[0, 2\pi]$ . A False value means unproved, not a known
    blind spot. Useful for final certificates with step_m=20 or 30.
    """
    points = np.asarray(list(points), dtype=float).reshape(-1, 2)
    if not len(points):
        return np.zeros(len(self.centers), dtype=bool)
    if not np.isfinite(points).all():
        raise ValueError("station coordinates must be finite")
    delta = points[None, :, :] - self.centers[:, None, :]
    dist = np.linalg.norm(delta, axis=2)
    valid = (dist + self.rho <= self.reception_radius_m - 1e-8) & (dist >
        self.rho)
    half = np.arccos(np.minimum(self.rho / np.maximum(dist, self.rho), 1.0))
    angle = np.arctan2(delta[:, :, 1], delta[:, :, 0])
    start = (angle - half) % (2 * math.pi)
    end = start + 2 * half
    starts = np.concatenate((start, start - 2 * math.pi), axis=1)
    ends = np.concatenate((end, end - 2 * math.pi), axis=1)
    valid2 = np.concatenate((valid, valid), axis=1)
    # Clip wrapping arcs to the interval we certify; invalid/empty arcs
    # become a zero-width interval at  $2\pi$ .
    live = valid2 & (ends > 0) & (starts < 2 * math.pi)
    starts = np.where(live, np.maximum(starts, 0), 2 * math.pi)
    ends = np.where(live, np.minimum(ends, 2 * math.pi), 0.0)
    order = np.argsort(starts, axis=1)
    starts = np.take_along_axis(starts, order, axis=1)
    ends = np.take_along_axis(ends, order, axis=1)
    reach = np.maximum.accumulate(ends, axis=1)
    # Use a small negative tolerance: tiny numerical gaps never establish
    # coverage. Initial/final equality is clipping at exact 0 and  $2\pi$ .
    # Placeholders at  $2\pi$  should not count as a required join; their end
    # is zero so they cannot extend the actual covered interval.
    joined = np.all((starts[:, 1:] == 2 * math.pi)
        | (starts[:, 1:] <= reach[:, :-1] - 1e-12), axis=1)
    return (starts[:, 0] == 0) & joined & (reach[:, -1] == 2 * math.pi)

def continuous_complete(self, points):
    return bool(np.all(self.all_orientation_cells(points)))

def from_log(self, meas_log):
    """Accept ChannelRec entries (x, y, time, result) or action dicts."""
    out = self.empty()
    for rec in meas_log:
        if isinstance(rec, dict):
            result = rec.get("measure_result", rec.get("result"))

```

```

        xy = (rec.get("x", rec.get("x_m")), rec.get("y", rec.get("y_m")))
    else:
        xy, result = rec[:2], rec[3]
        if result == "no_signal":
            out |= self.station_mask(xy)
    return out

def summarize(self, mask):
    mask = np.asarray(mask, dtype=bool)
    if mask.shape != self.shape:
        raise ValueError(f"expected coverage shape {self.shape}, got {mask.shape}")
    left = int(mask.size - np.count_nonzero(mask))
    frac = 1.0 if left == 0 else float(np.count_nonzero(mask[self.interior]) *
        self._weight)
    return CertificateResult(left == 0, frac, left, len(self.centers),
        self.n_dir)

def gap_witnesses(self, mask, limit=20):
    """Representative uncertain (x,y,d_deg), useful for gap planning.

    These are centres of unproved cells, not asserted physical blind spots.
    In particular a centre can lie just outside the arena on boundary cells.
    """
    ii, jj = np.nonzero(~np.asarray(mask, dtype=bool))
    if len(ii) > int(limit):
        take = np.linspace(0, len(ii) - 1, int(limit), dtype=int)
        ii, jj = ii[take], jj[take]
    return [(float(self.centers[i, 0]), float(self.centers[i, 1]),
        math.degrees(float(self.angles[j]))) for i, j in zip(ii, jj)]

def gap_candidates(self, mask, limit=32, offset_m=650.0):
    """Stations which individually certify a representative missing state."""
    offset = float(offset_m)
    if not (self.rho / math.cos(self.angle_half_width) < offset
        < self.reception_radius_m - self.rho):
        raise ValueError("offset cannot certify a whole position/orientation
            cell")
    return [(x + offset * math.cos(math.radians(a)),
        y + offset * math.sin(math.radians(a)))
        for x, y, a in self.gap_witnesses(mask, limit)]

def ring_layout(rings, include_origin=True):
    """Generate (radius,count,phase_deg) rings for static comparisons."""
    out = [(0.0, 0.0)] if include_origin else []
    for ring in rings:
        radius, count = float(ring[0]), int(ring[1])
        phase = math.radians(float(ring[2])) if len(ring) > 2 else 0.0
        out.extend((radius * math.cos(phase + 2 * math.pi * k / count),
            radius * math.sin(phase + 2 * math.pi * k / count))

```

```

        for k in range(count))
    return out

def strict_layout():
    """25 stations incl. origin; certified at step=20, n_dir=96.

    Origin + 950 m x 6 + 950*sqrt(3) m x 6 (30 degree phase) + 1900 m x 12.
    An open nearest-neighbour/2-opt tour from the origin is about 19.49 km.
    This is a usable certified candidate layout, not a station-count optimum.
    """
    return ring_layout(((950.0, 6, 0.0), (950.0 * math.sqrt(3.0), 6, 30.0),
                        (1900.0, 12, 0.0)))

def certify_channels(records, *, certificate=None):
    """Report each uncleared channel; a heard/near pending source prevents exit."""
    cert = certificate or DirectionalCertificate()
    out = {}
    for ch, rec in records.items():
        if rec.cleared:
            out[int(ch)] = {"complete": True, "reason": "cleared"}
            continue
        positive = any(entry[3] in ("direction", "near") for entry in rec.meas_log)
        result = cert.summarize(cert.from_log(rec.meas_log))
        out[int(ch)] = {"complete": bool(result.complete and not positive),
                        "reason": "heard_pending" if positive else
                        "no_signal_certificate",
                        "excluded_fraction_lower_bound":
                        result.excluded_fraction_lower_bound,
                        "n_uncovered_states": result.n_uncovered_states}
    return out

```

0.8 Q4/src/p4_grid.py

"""问题 4 的扫描几何：三角形格网 (triangular lattice) 覆盖半径与"必被听到"判据。

用户的设想

> 如果取一个合适大小的三角形，它的三个顶点里**至少有一个**能接收到落在它内部的
 > 干扰源信号；要保证一定能发现，边长取 1000 m 左右比较合适。于是用三角形网格
 > 覆盖整个区域，会形成内外两个圈；先从原点出发到第一个圈上的点扫描，再到第二个
 > 圈上的顶点扫描，就基本能覆盖所有干扰源。

本模块把这个直觉写成**可计算的判据**，并给出网格参数。

三个定义（都被后面的数值报告使用）

-
1. **覆盖半径** `cov(p)` = p 到最近停靠点的距离。源的接收半径 ≥ 1000 m，
 所以 `cov(p) > 1000` $\Rightarrow$ p 处若有源，任何停靠点都听不到它。
 2. **定向覆盖半径** `dir_cov(p)` = p 到最近**可被听到**停靠点的距离：

q 能听到 p 处的源 $\iff |q-p| \leq 1000$ 且 $(q-p) \cdot d \geq 0$

其中 d 是源的定向方向 (180° 覆盖角)。后一个条件等价于 " q 落在以 p 为界、朝向 d 的那个闭半平面里"。

3. **顶点三向覆盖 (用户的直觉)**: 若 p 落在三角形 $v_1v_2v_3$ 内部, 则 $p = \sum \lambda_i v_i$ ($\lambda_i \geq 0$), 于是对任意方向 d 有 $\sum \lambda_i (v_i - p) \cdot d = 0$
 $\Rightarrow$ 必有某个 i 使 $(v_i - p) \cdot d \geq 0$ 。换句话说: 三角形的三个顶点方向正向张成整个平面, 任何定向方向都至少被一个顶点"看到"。
所以只要再保证 $\max_i |v_i - p| \leq 1000$ (等边三角形内的上界为边长 a), 这个三角形内的任何源 (不论朝向) 都必被某个顶点听到。

推论: 完整三角剖分且边长 $a \leq 1000$ m 是充分条件; 还必须覆盖靶区边界。
 $a/\sqrt{3}$ 仅表示最近格点距离, 不能代替最远顶点距离。下方离散评估只作诊断;
连续位置与任意朝向的完成证书见 `p4_certificate.py`。

```
"""
from __future__ import annotations

import math

import numpy as np

from p3_arena import R_ARENA, R_RECV_MIN

# 三角形格网的基向量 (边长为 a 的等边三角形): u = (a, 0), v = (a/2, a*sqrt(3)/2)
SQRT3 = math.sqrt(3.0)

# -----
# 格网生成
# -----
def lattice_index_range(a, r_cover):
    """列出"到原点距离 ≤ r_cover"的格点所需的格坐标范围。"""
    k = int(math.ceil(r_cover / (a * SQRT3 / 2.0))) + 1
    return range(-k, k + 1)

def triangular_lattice(a, r_cover=R_ARENA, origin=(0.0, 0.0), dedup=True):
    """边长为 `a` 的三角形格网中, 到 `origin` 距离 ≤ `r_cover` 的全部顶点。

    这是"覆盖整个靶区"所需的**一对一点集** (每个点在格网里的坐标唯一)。
    """
    ux, uy = float(a), 0.0
    vx, vy = float(a) / 2.0, float(a) * SQRT3 / 2.0
    ox, oy = float(origin[0]), float(origin[1])
    out = []
    seen = set()
    for i in lattice_index_range(a, r_cover):
        for j in lattice_index_range(a, r_cover):
            x = i * ux + j * vx
            y = i * uy + j * vy
```

```

        if math.hypot(x - ox, y - oy) <= r_cover + 1e-9:
            key = (round(x, 6), round(y, 6))
            if dedup and key in seen:
                continue
            seen.add(key)
            out.append((x, y))
    out.sort(key=lambda p: (math.hypot(p[0] - ox, p[1] - oy),
                             math.atan2(p[1] - oy, p[0] - ox)))
    return out

def hex_ring_lattice(a, rings=2, origin=(0.0, 0.0)):
    """同心六边形环"形式的格点: 第 k 环半径 = k*a, 含 6k 个点。

    与 :func:`triangular_lattice` 是同一个格网, 只是**按环分组**——
    这正是用户说的"内外两个圈": 环 0 = 原点, 环 1 = 一圈 6 个点,
    环 2 = 一圈 12 个点。按环序走就是"先内圈、再外圈"的扫描顺序。
    """
    ox, oy = float(origin[0]), float(origin[1])
    u = np.array([float(a), 0.0])
    v = np.array([float(a) / 2.0, float(a) * SQRT3 / 2.0])
    out = [[] for _ in range(int(rings) + 1)]
    seen = set()
    for i in range(-int(rings), int(rings) + 1):
        for j in range(-int(rings), int(rings) + 1):
            p = i * u + j * v
            r = float(np.hypot(*p))
            k = int(round(r / float(a)))
            if k > int(rings):
                continue
            key = (round(float(p[0]), 6), round(float(p[1]), 6))
            if key in seen:
                continue
            seen.add(key)
            out[k].append((ox + float(p[0]), oy + float(p[1])))
    for k in range(len(out)):
        out[k].sort(key=lambda q: math.atan2(q[1] - oy, q[0] - ox) % (2 * math.pi))
    return out

# -----
# 覆盖判据 (数值)
# -----

def eval_grid(step=20.0, r_arena=R_ARENA):
    """用于评估的细网格 (默认 20 m, 靶区内约 2.5 万个点)。"""
    xs = np.arange(-r_arena, r_arena + 1e-9, step)
    GX, GY = np.meshgrid(xs, xs)
    rr = np.hypot(GX, GY)
    m = rr <= r_arena + 1e-9
    return GX[m].copy(), GY[m].copy()

```

```

def cov_stats(pts, gx=None, gy=None, r_cover=R_RECV_MIN):
    """``(最大未覆盖距离, 未覆盖点占比)``: 到最近停靠点的距离分布。

    完备性判据: ``最大未覆盖距离 <= 1000 m``  $\Rightarrow$  任一点 (不论朝向) 都会被某个
    停靠点听到 — 对**全向**源严格成立; 对定向源还需要半平面条件, 见
    :func:`dir_cov_stats`.
    """
    if gx is None:
        gx, gy = eval_grid()
    P = np.asarray(pts, float).reshape(-1, 2)
    d2 = ((gx[:, None] - P[None, :, 0]) ** 2 + (gy[:, None] - P[None, :, 1]) ** 2)
    dmin = np.sqrt(np.min(d2, axis=1))
    return float(dmin.max()), float((dmin > float(r_cover)).mean())

def dir_cov_stats(pts, gx=None, gy=None, r_cover=R_RECV_MIN, r_arena=R_ARENA,
                  n_dir=16, origin=(0.0, 0.0), chunk=4000):
    """定向源的"必被听到"判据: 对每个评估点、每个方向求最近**可听**停靠点距离。

    ``dir_cov(p)`` = min over d of max over admissible q ... 不对, 正确口径是:
    对给定方向 d, "能被听到"要求存在 q 同时满足距离与半平面条件, 所以

    
$$\text{dir\_cov}(p, d) = \min\{ |q-p| : |q-p| \leq r\_cover, (q-p) \cdot d \geq 0 \}$$


    我们报告 ``max over d of dir_cov(p, d)`` (最坏朝向) 与"无任何可听停靠点"的占比。
    返回 ``(最坏朝向距离, 无覆盖占比, 半平面不可覆盖占比)``:
    第三项 = 连"忽略距离上限"的半平面里都没有停靠点的比例 (这是几何硬伤,
    加多少圈都救不回来, 只能把停靠点铺到靶区边界之外)。
    """
    if gx is None:
        gx, gy = eval_grid()
    P = np.asarray(pts, float).reshape(-1, 2)
    angs = np.arange(int(n_dir)) * (2.0 * math.pi / int(n_dir))
    worst = np.zeros(gx.size)
    unc = np.zeros(gx.size, bool)
    hard = np.zeros(gx.size, bool)
    r2 = float(r_cover) ** 2
    for i0 in range(0, gx.size, int(chunk)):
        sl = slice(i0, min(i0 + int(chunk), gx.size))
        px = gx[sl][:, None] - P[None, :, 0]
        py = gy[sl][:, None] - P[None, :, 1]
        d2 = px ** 2 + py ** 2
        d = np.sqrt(d2)
        # 每个停靠点方向 (用于半平面判定)
        for a in angs:
            cos, sin = math.cos(a), math.sin(a)
            par = px * cos + py * sin # (q-p)·d
            ok = (par >= -1e-9) & (d <= float(r_cover) + 1e-9)
            any_ok = ok.any(axis=1)
            best = np.where(any_ok, np.min(np.where(ok, d, np.inf), axis=1),

```

```

        np.inf)
    worst[sl] = np.maximum(worst[sl], np.where(np.isfinite(best), best, 0.0))
    unc[sl] |= (~any_ok)
    any_par = (par >= -1e-9).any(axis=1)
    hard[sl] |= (~any_par)
    return (float(worst.max()), float(unc.mean()), float(hard.mean()))

def hex_cover_radius(a):
    """无限三角形格网的覆盖半径 =  $a/\sqrt{3}$  (外接圆半径)。"""
    return float(a) / SQRT3

def ring_set(rings):
    """`[(r, n), ...]` -> 停靠点列表 (环间半格错开, 避免放射状缝隙)。"""
    out = []
    for k, (r, n) in enumerate(rings):
        off = (math.pi / float(n)) if (k % 2 == 1) else 0.0
        for i in range(int(n)):
            ang = off + 2.0 * math.pi * i / float(n)
            out.append((float(r) * math.cos(ang), float(r) * math.sin(ang)))
    return out

def tour_len(pts, start=(0.0, 0.0), two_opt=True):
    """最近邻 + 2-opt 的航路长度 (用于比较"走完这些停靠点要多少米")。"""
    P = np.asarray(pts, float).reshape(-1, 2)
    if P.size == 0:
        return 0.0, []
    cur = np.asarray(start, float)
    left = list(range(P.shape[0]))
    seq = []
    while left:
        j = min(left, key=lambda i: float(np.hypot(*(P[i] - cur))))
        seq.append(j)
        left.remove(j)
        cur = P[j]

    def total(order):
        d = float(np.hypot(*(P[order[0]] - np.asarray(start, float))))
        for x, y in zip(order, order[1:]):
            d += float(np.hypot(*(P[y] - P[x])))
        return d

    best = total(seq)
    if two_opt:
        improved = True
        while improved:
            improved = False
            for i in range(len(seq) - 1):
                for j in range(i + 1, len(seq)):

```

```

        cand = seq[:i] + seq[i:j + 1][::-1] + seq[j + 1:]
        d = total(cand)
        if d < best - 1e-9:
            seq, best, improved = cand, d, True
    return best, seq

```

0.9 Q4/src/p4_layout_v2.py

```

"""Adaptive continuous directional layout certificate and candidate layouts.

Every refined square is certified by the same exact angular-interval union
and strict spatial margin as DirectionalCertificate. Refinement changes the
proof's resolution; it does not replace the continuous square by samples.
"""
from __future__ import annotations
import math
import numpy as np
from p4_certificate import DirectionalCertificate, ring_layout

def adaptive_complete(points, initial_step_m=40., min_step_m=.625, *,
    detail=False):
    cert=DirectionalCertificate(initial_step_m,48)
    counts=[]
    while True:
        covered=cert.all_orientation_cells(points)
        left=cert.centers[~covered]
        counts.append((cert.cell_side_m,len(cert.centers),len(left)))
        if not len(left):
            return (True,counts) if detail else True
        # Find an actual unsupported interior point before refining large
        # blind regions. This is only an early rejection, never acceptance.
        centers,rho=cert.centers,cert.rho
        cert.centers=left[np.sum(left*left,axis=1)<cert.arena_radius_m**2]
        cert.rho=0.
        point_gap=not np.all(cert.all_orientation_cells(points))
        cert.centers,cert.rho=centers,rho
        if point_gap:
            return (False,counts) if detail else False
        if cert.cell_side_m/2 < min_step_m-1e-12:
            return (False,counts) if detail else False
        side=cert.cell_side_m/2
        offsets=np.array([[-1.,-1.],[-1.,1.],[1.,-1.],[1.,1.]])*side/2
        centers=(left[:,None,:]+offsets[None,:,:]).reshape(-1,2)
        closest=np.maximum(np.abs(centers)-side/2,0)
        cert.centers=centers[np.sum(closest*closest,axis=1)<=cert.arena_radius_m**2]
        cert.cell_side_m=side
        cert.rho=side/math.sqrt(2)

def compact_layout():

```

```

    """22 points; 17.70 km static open tour; certified by 5 m refinement."""
    return ring_layout(((980.,7,0.), (1856.,14,0.)))

def compact21_layout():
    """21 points; 17.91 km tour; certificate refines to 2.5 m."""
    return ring_layout(((998.,8,0.), (1867.,12,15.)))

if __name__=='__main__':
    from p4_grid import tour_len
    p=compact_layout()
    print(adaptive_complete(p,detail=True))
    print(tour_len(p)[0])

```

0.10 Q4/src/p4_posterior.py

```

"""Use negative replies for a source estimate, retaining an outer proof region.

The quadrature is a planning prior only. It never excludes geometric source
possibilities, stops search, or replaces the final continuous certificate.
"""

import math
import numpy as np

class P4PosteriorMixin:
    def home_region(self,rec):
        br=super().home_region(rec)
        if br is None or rec.n_bearings!=1 or not br.get('poly') or
            rec.near_hits:return br
        if not getattr(self.cfg,'posterior_estimate',False):return br
        key=(rec.channel,len(rec.meas_log),len(self._p4_home_caps.get(rec.channel,[])))
        cache=getattr(self,'_posterior_cache',None)
        if cache is None:self._posterior_cache={};cache=self._posterior_cache
        if key in cache:return cache[key]
        station=np.array(rec.pts[0]);angle=math.radians(rec.svds[0])
        u=np.array([math.cos(angle),math.sin(angle)])
        polygon=np.array(br['poly']);along=(polygon-station)@u
        ts=np.linspace(max(5.,along.min()),min(1500.,along.max()),150)
        candidates=station+ts[:,None]*u
        radii=np.linspace(1000,1500,9)
        ds=angle+np.linspace(-math.pi/2+math.pi/72,math.pi/2-math.pi/72,36)
        directions=np.stack([np.cos(ds),np.sin(ds)],axis=1)
        valid=(ts[:,None,None]<=radii[None,:,None])*np.ones((1,1,len(ds)),bool)
        omni=(ts[:,None]<=radii[None,:])
        for x,y,_,res in rec.meas_log:
            if res!='no_signal':continue
            v=candidates-(x,y);d=np.linalg.norm(v,axis=1)
            within=d[:,None]<=radii[None,:]
            bearing=(v@directions.T)>=0

```

```

        valid &= ~(within[:, :, None] & bearing[:, None, :])
        omni &= ~within
        prior=getattr(self.cfg, 'posterior_omni_prior', .5)
        weights=ts*(np.linalg.norm(candidates,axis=1)<=1800)
        weights*=prior*omni.mean(axis=1)+(1-prior)*valid.mean(axis=(1,2))
        if weights.sum()>0:
            center=np.average(candidates,axis=0,weights=weights)
            radius=float(np.max(np.linalg.norm(polygon-center,axis=1)))
            br=dict(br,center=tuple(center),radius=radius)
        cache[key]=br
    return br

```

0.11 Q4/src/p4_robot.py

```

"""Q4 legacy grid/spiral policy, retained for paired comparisons.

Use p4_search.P4SearchRobot for the optimized strategy and strict completion.
The radial half-plane masks and their flipped second pass below are legacy
heuristics. They do not certify arbitrary source orientations, and removing
unreachable cells does not prove absence. Historical comments and benchmark
claims below describe those older experiments, not a continuous guarantee.
"""
from __future__ import annotations

import math
import os
import sys
import time
from dataclasses import dataclass, field as dc_field

import numpy as np

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)

from p3_arena import ( # noqa: E402
    ArenaError, CHANNELS, R_ARENA, R_CLEAR, R_RECV_MIN, T_CLEAR_FAIL, T_MEASURE,
    T_SWITCH, V_ROBOT,
)
from p3_robot import SWEEP_OK, ChannelRec, P3Config, P3Robot # noqa: E402
from p4_grid import SQRT3, triangular_lattice # noqa: E402


# -----
# 配置
# -----
@dataclass
class P4Config(P3Config):
    """在 `P3Config` 之上追加"三角形格网 + 定向源"相关参数。"""

```

```
# ---- 格网 ----
grid_a: float = 1000.0 # 三角形格网边长 (m):  $a/\sqrt{3} = 577$  m 覆盖半径
grid_r_max: float = R_ARENA # 格点铺到多大半径 (靶区半径 1800 m)
outer_rings: tuple = ((1850.0, 12),)
"""外侧补漏环 `` (半径 m, 点数) ``: 救"贴边朝内发射"的定向源。
```

【!】先说清这个环到底在救什么 (实测修正了我最初的直觉):
mock 的定向判据是"站→源 的视线落在 ``dir\_deg`` 的 $\pm 90^\circ$ 内",
诊断 100 局漏源 (src/p4_miss_where.py) 显示漏掉的全是
半径 1622-1794 m、朝向与径向夹角 117° - 178° (几乎正对圆心) 的定向源。
也就是说: **"朝内发射"的源必须从它外侧的站位才听得到** — 外环就是干这个的。

靶区边界附近 ($r \gtrsim 1000$) 的源若朝内发射, 覆盖半平面里没有任何"域内"站位,
所以只能把站位铺到它外侧。数值枚举 (src/p4_ring_shape.py):

外环	站数	覆盖半径	定向盲区
r1850x12 (默认)	25	575 m	0.00%
r1900x10	24	615 m	0.66%
r1900x6 ($\approx$ "格网整体铺到 $r=2000$ ")	19	575 m	1.91%
r1850x9	22	639 m	2.59%
无外环 (只铺域内)	13	800 m	38.66%

允许降覆盖时可换 ``((1900.0, 6),)``: 实测 100 局清除率 1.0000 (全向/定向混合
与全定向两批都满分)、指标基本不变, 站数少 6 个; 亦可换成"格网直接铺到 2000"
(``outer\_rings=(), grid\_r\_max=2000``), 两者等价。
"""

```
# ---- 探索 ----
spiral_radius_w: float = 0.35 # 选下一站时对"半径大"的加罚, 形成由内向外
skeleton_radius_w: float = 0.25 # 骨架排序的半径偏置 (由内向外; 太大会横穿靶区)
radius_sort_w: float = 0.0 # 试过"显式由内向外排序", 实测无正向收益 (见
    scan_skeleton), 默认关闭
route_mode: str = "spiral" # 航路排法: "spiral" (问题 3 式绕圈, 默认) |
    "tsp" (几何最短)
ring_band_frac: float = 0.5 # 螺旋航路的"环"分档宽度 = grid_a × 该比例 (m)。
    # 0.5 → r=1732 与 1850 合成一层 (实测最好, 航程 26.3 km);
    # 0.35/0.25/0.18 → 分成 3 层, 反而变差 (28.0 km、长跳 +1)
replan_drift_m: float = 700.0 # 保留字段 (当前实现不再按当前位置重排航线)
station_min_gain: float = 90.0 # 某站的边际覆盖低于它就跳过 (单位: 25 m 网格点数;
    # 90 格  $\approx$  一个 140 m 见方的小片)。
    # 100 局实测 (混合场景, 中位平均定位清除 / 清除率):
    # gain=90 : 472 s / 1.0000 (100 局全清) ← 默认
    # gain=180: 与 90 接近, 但全定向会掉源
    # gain=250: 569 s / 0.9977 (慢 97 s 且漏 3 个源)
    # 结论: **别为了省站抬高这个门限** — 省下的站次
    # 远抵不上"漏源 + 听到更晚"的代价。
detour_follow_m: float = 1400.0 # 顺路清除后离队首超过它就"就近切入"航线
    # (正常站间距 850-1732 m, 取 1400 只在真正被带偏时触发)
cover_reduce: float = 0.0 # **降覆盖档**: >0 时主动放弃这么大比例的可听格点,
    # 只保留"代价最小、覆盖面最大"的若干站位 (见 measure_set):
```

```

# 0.0 (默认) = 全覆盖; 0.3 / 0.5 / 0.7 = 逐档变快、逐档漏源
# 实测见 `src/p4_cover_reduce.py` 与 review $3.12
plan_tour: bool = False # False (默认): **逐站贪心, 把覆盖判据说"还有用"的站
# 全部走完** — 这是模型下"必被发现"的完整保证
# (实测 20 局 268 个源 100% 清除, 代价是 27 站/局);
# True: 先算"最小覆盖集 + 半径偏置 TSP", 只走 14-16 站,
# 航程少 ~4 km、虚拟时间少 ~10%, 代价是有 ~0.4% 的
# 漏源风险(边界处"朝外但偏离径向"的定向源)。
replan_every: int = 0 # >0: 每走这么多站就用"当前位置 + 最新覆盖集"重排航线
cover_slack_m: float = 0.0 # 半平面判据的保守余量 (m)
halfplane_eps: float = 1.0 # 半平面判据的绝对余量 (m2): 避免"正侧向"的边界抖动
cover_grid_step_m: float = 25.0 # 覆盖评估网格步长 (m): 太粗会漏判边界附近的源
# (实测 100 m 会真的漏源; 50 m 仍漏 1/268;
# 25 m 只剩"格点边界"量级的漏判风险)
two_pass: bool = True # 扫两遍(两种半平面约定) ⇒ 对协议约定不敏感
max_station_visits: int = 0 # 0 = 不限制(按覆盖判据自然收工)

# ---- 定位(定向源闭环: 沿视线二分 + 侧移交会) ----
loc_walk_m: float = 520.0 # 沿视线走位的单步初值 (m); 走位用 /clear (3 s)
loc_step_growth: float = 1.4 # 每确认一次"有信号"就把步长乘这个系数(指数跳出界)
loc_max_moves: int = 4 # 走位次数上限(走出无信号边界就停)
loc_bisect_steps: int = 5 # 二分步数上限
loc_bisect_tol_m: float = 40.0 # 二分区间的收敛门限 (m)
loc_cross_radius_m: float = 420.0 # 侧移交会的离源距离 (m): 交会角 ≈45°
loc_clear_radius_m: float = 90.0 # 第一轮试探半径 (m): 由定位精度决定
loc_probe_radius_max_m: float = 160.0 # 区域半径超过它就先做定位而不是硬试
max_solve_attempts: int = 3 # 同一频道最多进入 solve 几次(防止清除阶段空转)
clear_detour_max_m: float = 300.0 #
    **阶段式**(`unified=False`)下"到站顺手清"的绕行上限。
    # 【!】统一航路**不用**这个值(用 interleave_detour_max_m,
    # 默认 ∞)。曾经误把 300 m 套到统一航路上, 结果把大多数
    # 清理延后、积压到航程末尾: 平均定位清除 648 → 1141 s、
    # 行程 34.9 → 64.1 km。两套策略的参数不能混用。
enroute_max_n: int = 3 # 每个停点最多顺路清几个
# ---- 统一航路(边扫描边清理, 见 run_unified) ----
unified: bool = True # True: 一条滚动航路走完两圈, 点也清完(默认)
interleave_detour_max_m: float = 1e9
# **实测最优是不设门限** (听到就清): 扫参见
# `pick_unified_action` 的文档与 `p4_tune_unified.py`。
# 保留这个参数是为了复现"攒着清"的对照。
interleave_loc_radius_m: float = 110.0
# 目标定到"区域半径 ≤ 它"就算定住了, 可以清
interleave_probe_m: float = 80.0 # 统一航路里的小范围试探半径 (m)
interleave_clear_cost_s: float = 90.0
# 估算一次"就地清理"的动作耗时(定位 + 试探)

# ---- 清除(复用问题 3 的覆盖式试探, 保证成功) ----
clear_approach_max: int = 3
clear_est_probe_m: float = 40.0 # 估计点已在这么近时先直接试一次 /clear

```

```

# -----
# 机器狗
# -----
class P4GridRobot(P3Robot):
    """三角形格网扫描 + 定向源闭环定位清除。"""

    def __init__(self, arena, cfg: P4Config | None = None, base=None, log=None,
                  log_prefix=""):
        super().__init__(arena, cfg or P4Config(), base=None, log=log,
                         log_prefix=log_prefix)
        self.stations = self._build_stations()
        self.station_hits = [0] * len(self.stations)
        self.visited_stations: set[int] = set()
        self.loc_pts: dict[int, list] = {} # 定位过程中新增的检测点（画图/复盘用）
        self._near_pt: dict[int, tuple] = {} # "距离 ≤ 5 m" 的检测点：到那儿直接清
        self._tried_ray: set[int] = set() # 已经做过"沿视线落到二分点"的频道
        self.coverage_flip = False # 当前扫描使用的半平面约定（见 coverable_mask）
        self.unified_second_pass = False # 统一航路：是否处于"第二遍（必须清）"
        self.first_pass_heard: set | None = None # 第一遍结束时就已听到的频道
        self.phase = "scan"
        self._cov_cache: dict = {}
        self._covered_cache: dict = {}

    # ----- 格网 -----
    def _build_stations(self):
        cfg = self.cfg
        pts = list(triangular_lattice(cfg.grid_a, r_cover=cfg.grid_r_max))
        for (r, n) in (cfg.outer_rings or ()):
            for i in range(int(n)):
                a = 2.0 * math.pi * i / float(n)
                pts.append((float(r) * math.cos(a), float(r) * math.sin(a)))
        # 去掉重复点（外环可能与格点重合）与原点（原点单独作为第 0 站）
        out, seen = [], set()
        for (x, y) in pts:
            if math.hypot(x, y) < 1.0:
                continue
            key = (round(x, 1), round(y, 1))
            if key in seen:
                continue
            seen.add(key)
            out.append((float(x), float(y)))
        out.sort(key=lambda p: (math.hypot(*p), math.atan2(p[1], p[0]) % (2 *
            math.pi)))
        return out

    def station_xy(self, i):
        return self.stations[int(i)]

    # ----- 选哪些站、按什么顺序走 -----
    def covering_set(self):
        """**最小覆盖站集**：贪心集合覆盖，保证"每个可听格点至少被一个站覆盖"。

```

为什么需要它：覆盖判据（:meth:`coverable\_mask`）是"站点 × 格点"的集合覆盖问题，30 个站里往往只有十几个真正贡献新覆盖。把"必须去的站"算出来，航线才不会为了几个格点多跑几公里。

贪心在这里是**安全**的：它只决定"访问顺序"，不改变覆盖判据 —— 每站到了之后仍然按 :meth:`station\_gain` 复核，收益为 0 的站会被跳过。

```
"""
got = getattr(self, "_covering", None)
if got is not None:
    return got
gx = self._grid_pts()[0]
left = ~self.unreachable_mask()
masks = [self.coverable_mask(i) & ~self.unreachable_mask()
          for i in range(len(self.stations))]
chosen = []
while left.any():
    best, bi, bg = None, -1, 0
    for i, m in enumerate(masks):
        if i in chosen:
            continue
        g = int(np.count_nonzero(m & left))
        if g > bg:
            best, bi, bg = i, i, g
    if bi < 0 or bg <= 0:
        break
    chosen.append(bi)
    left = left & ~masks[bi]
self._covering = chosen
return chosen
```

----- 降覆盖档：把"测哪些站"显式做成一档 -----

```
def measure_set(self):
    """本局实际做检测的站位集合（`cover_reduce > 0` 时会**主动缩水**）。
```

为什么这是一个独立的省时杠杆：时间 $\approx \sum_{\text{站}} (\text{移动} + \text{该站要测的频道数} \times 6 \text{ s})$ 。
 "访问哪些站"由覆盖判据决定（全测一遍才敢说"没有残留源"），
 但如果允许**放弃一部分区域**，就可以只保留一个"代价最小、覆盖大部分"的站集：

$\text{cover_reduce} = r \Rightarrow$ 用贪心集合覆盖，一直取到"剩余未覆盖格点只剩全部格点的 r 倍"为止 —— 于是**故意留 r 比例的面积不测**。

这是"覆盖 $\rightarrow$ 用时"最直接的旋钮： $r=0$ 就是原来的全覆盖行为。

```
"""
r = float(self.cfg.cover_reduce)
if r <= 0.0:
    return None # None = 不做缩减（全覆盖）
got = getattr(self, "_mset", None)
if got is not None:
    return got
gx = self._grid_pts()[0]
```

```

unreach = self.unreachable_mask()
left = ~unreach
total = int(left.sum())
masks = [self.coverable_mask(i, flip=False) & ~unreach
          for i in range(len(self.stations))]
chosen: list[int] = []
target = total * float(r)
while True:
    best, bi, bg = None, -1, 0
    for i, m in enumerate(masks):
        if i in chosen:
            continue
        g = int(np.count_nonzero(m & left))
        if g > bg:
            best, bi, bg = i, i, g
    if bi < 0 or bg <= 0:
        break
    nxt = int(np.count_nonzero(left & ~masks[bi]))
    if nxt <= target:
        break # 已经放弃够了：剩下的区域不测
    chosen.append(bi)
    left = left & ~masks[bi]
self._mset = set(chosen)
self.stats["n_measure_set"] = len(chosen)
self.stats["cover_abandoned"] = float(
    int(np.count_nonzero(left)) / max(total, 1))
self.log(f"降覆盖档 cover_reduce={r:.2f}: 只测 {len(chosen)} 个站位, "
        f"主动放弃 {self.stats['cover_abandoned']:.1%} 的可听格点")
return self._mset

def plan_station_tour(self, start=None):
    """把"必去的站"排成一条航线：最近邻 + 2-opt, **代价里带半径惩罚**。

    为什么不是纯最短航路：平均定位清除时间 = 定位清除总时刻 / 清除个数，
    ***早点听到源"比"少走路"更值钱** — 源一旦被听到就会立刻进入清理队列
    （实测：纯最短航路让某一局的 14 个源晚了 2000 s 才被听到，指标掉 2 个百分点）。
    所以在距离代价上加 ``spiral_radius_w · 半径``：航路自然由内向外展开，
    同时 2-opt 仍能消掉最明显的绕远。

    ``start`` 给定时以它为起点重排（滚动重规划用）；默认从原点出发。
    排序只花几十毫秒，却能把扫描段从 24 km 压到 16-18 km。
    """
    idx = list(self.covering_set())
    if not idx:
        return []
    w = float(self.cfg.spiral_radius_w)
    P = {i: np.asarray(self.stations[i], float) for i in idx}
    origin = np.asarray(self.pos if start is None else start, float)

    def cost(order):
        """总代价 = 航程 + w ·  $\Sigma$  半径（由内向外偏置）。"""

```

```

        if not order:
            return 0.0
        d = float(np.hypot(*(P[order[0]] - origin)))
        for a_, b_ in zip(order, order[1:]):
            d += float(np.hypot(*(P[b_] - P[a_])))
        d += w * sum(float(np.hypot(*P[i])) for i in order)
        return d

    cur = origin
    left = set(idx)
    seq = []
    while left:
        j = min(left, key=lambda i: float(np.hypot(*(P[i] - cur)))
                + w * float(np.hypot(*P[i])))
        seq.append(j)
        left.discard(j)
        cur = P[j]
    best = cost(seq)
    improved = True
    while improved:
        improved = False
        for i in range(len(seq) - 1):
            for j in range(i + 1, len(seq)):
                cand = seq[:i] + seq[i:j + 1][::-1] + seq[j + 1:]
                c = cost(cand)
                if c < best - 1e-9:
                    seq, best, improved = cand, c, True
    self._tour_len_m = cost(seq) - w * sum(float(np.hypot(*P[i])) for i in seq)
    return seq

def greedy_station_order(self):
    """逐步贪心版航线（`plan_tour=False`时的对照）：每步按
    "收益 / 距离 + 半径偏置" 选下一站，直到没有站还能补新覆盖。

    与 :meth:`plan_station_tour` 的差别：贪心每步都看最新覆盖状态（响应性好），
    但没有全局航路结构（实测航程多 5-8 km）；规划版反过来。
    """
    cur = np.asarray(self.pos, float)
    left = set(range(len(self.stations)))
    seq = []
    w = float(self.cfg.spiral_radius_w)
    for _ in range(len(self.stations)):
        cover = self.unheard_mask()
        if not cover.any():
            break
        best, bs = None, None
        for si in left:
            sx, sy = self.stations[si]
            d = math.hypot(sx - cur[0], sy - cur[1])
            if d < self.cfg.min_move:
                continue

```

```

        gain = self.station_gain(si, cover)
        if gain <= self.cfg.station_min_gain:
            continue
        s = float(gain) / (1.0 + d / 500.0) - w * math.hypot(sx, sy) / 100.0
        if bs is None or s > bs:
            best, bs = si, s
    if best is None:
        break
    seq.append(best)
    left.discard(best)
    cur = np.asarray(self.stations[best], float)
    self._tour_len_m = 0.0
    return seq

# ----- 覆盖判据 -----
def _grid_pts(self):
    """覆盖评估网格 (50 m)：用来算"这一站还能补多少覆盖"。

    分辨率不能太粗：覆盖判据是"格点到站点的距离 ≤ 有效接收半径下界 1000 m"，
    而三角格网的覆盖半径已经接近这个量级 (577 m)，用 100 m 粗网格会让
    边界附近的源被漏判（实测有源因为格点间隙而永远没被听到 — 这是**真的漏源**，
    不是启发式误差）。50 m 网格在 30 个站上的额外计算量可以忽略。
    """
    c = getattr(self, "_cov_grid", None)
    if c is None:
        step = float(self.cfg.cover_grid_step_m)
        R = R_ARENA
        xs = np.arange(-R, R + 1e-9, step)
        GX, GY = np.meshgrid(xs, xs)
        rr = np.hypot(GX, GY)
        m = rr <= R + 1e-9
        c = (GX[m].copy(), GY[m].copy())
        self._cov_grid = c
    return c

def coverable_mask(self, si, flip=None):
    """站位 ``si`` 能"听到"的格点掩码（定向源的**保守**判据）。

    半平面条件（以"源在 g、记发射方向为 d"为准）：站位 ``q`` 能听到  $\iff$ 
     $|q-g| \leq 1000$  且  $(q-g) \cdot d \geq 0$ （视线方向落在  $\pm 90^\circ$  覆盖角内）。

    本题只给了"定向方向未知、覆盖角  $180^\circ$ "，**没有**规定 d 的指向约定
    （是"朝 d 发射"还是"d 的反方向发射"）。两种约定给出的可听半平面正好相反：

    * ``flip=False``：d = 径向向外 ``g``  $\implies$  要求在**外侧**（ $(q-g) \cdot g \geq 0$ ）；
    * ``flip=True``：d = 径向向内 ``-g``  $\implies$  要求在**内侧**（ $(q-g) \cdot g \leq 0$ ）。

    为了让策略对协议约定**不敏感**，扫描阶段做**两遍**：第一遍按 ``flip=False``
    覆盖，若有频道仍然没被听到，第二遍按 ``flip=True`` 把另一半半平面也扫掉。
    两遍合起来  $\implies$  无论真实约定是哪一种（甚至两向都发），源都必被听到
    （自检 G3/G4 分别对两种约定核验盲区为 0）。
    """

```

```

"""
if flip is None:
    flip = self.coverage_flip
key = (int(si), bool(flip))
a = self._cov_cache.get(key)
if a is not None:
    return a
gx, gy = self._grid_pts()
qx, qy = self.stations[int(si)]
d2 = (gx - qx) ** 2 + (gy - qy) ** 2
dot = (qx - gx) * gx + (qy - gy) * gy # (q-g)·g: q 在源的外侧
if flip:
    dot = -dot
m = (d2 <= (R_RECV_MIN + self.cfg.cover_slack_m) ** 2) \
    & (dot >= -self.cfg.halfplane_eps)
self._cov_cache[key] = m
return m

def unreachable_mask(self, flip=None):
    """**任何**站位都听不到的格点（在给定半平面约定下）。

    这些点不参与"要不要再跑一站"的判据 —— 加再多站也覆盖不到
    （唯一的办法是换一个半平面约定重扫，见 :meth:`coverable_mask`）。
    """
    flip = self.coverage_flip if flip is None else bool(flip)
    key = ("unreach", flip)
    got = self._cov_cache.get(key)
    if got is not None:
        return got
    m = np.ones(self._grid_pts()[0].size, bool)
    for si in range(len(self.stations)):
        m &= ~self.coverable_mask(si, flip=flip)
    self._cov_cache[key] = m
    return m

def _covered_of(self, ch):
    """频道 ``ch`` 已经被排除的格点掩码（"在那里测过而且能听到"）。

    只在**测过的站位**上算：那一站能听到的格点范围（距离 + 半平面）里若真有源，
    当时就会被听到；没听到 ⇒ 那里没有该频道的源。
    """
    rec = self.recs[int(ch)]
    key = (int(ch), len(rec.meas_stations))
    cache = self._covered_cache
    got = cache.get(key)
    if got is not None:
        return got
    out = np.zeros(self._grid_pts()[0].size, bool)
    for si in rec.meas_stations:
        out |= self.coverable_mask(si)
    if len(cache) > 80:

```

```

        cache.clear()
        cache[key] = out
        return out

def pending_cover(self, ch):
    """频道 ``ch`` 还剩多少"可能有源但我们还没听过"的格点。"""
    return int(np.count_nonzero(~self._covered_of(ch)))

# ----- 站位的边际覆盖 -----
def unheard_mask(self):
    """还没听到过的频道"的联合未覆盖格点掩码（一次算好，供所有站位复用）。

    已经排除 :meth:`unreachable_mask`：那些点任何站位都听不到（定向源朝外、
    外侧又没有站位），不该继续驱动航线。
    """
    gx, _gy = self._grid_pts()
    m = np.zeros(gx.size, bool)
    for ch in self.unheard():
        m |= ~self._covered_of(ch)
    return m & ~self.unreachable_mask()

def station_gain(self, si, unheard_cover=None):
    """这一站对"还没听到过的频道"能补多少未覆盖格点（越少越不值得去）。"""
    m = self.coverable_mask(si)
    if unheard_cover is None:
        unheard_cover = self.unheard_mask()
    return int(np.count_nonzero(m & unheard_cover))

def unheard(self):
    """还没听到过、也没被清除的频道（探索阶段要覆盖的目标）。"""
    return [int(ch) for ch, rec in self.recs.items()
            if not rec.cleared and rec.n_bearings == 0]

def heard_pending(self):
    """听到了但还没清除的频道（清理阶段的目标）。"""
    return [int(ch) for ch, rec in self.recs.items()
            if not rec.cleared and rec.n_bearings > 0]

def unheard_cover_left(self):
    """所有未听频道剩余未覆盖格点的总量（0  $\Rightarrow$  可以断定靶区内没有别的源了）。"""
    return int(np.count_nonzero(self.unheard_mask()))

# ----- 选下一站（由内向外螺旋） -----
def pick_station(self):
    """选下一站：**覆盖收益 vs 行驶代价**，并带一个"由内向外"的螺旋偏置。

    ``score = 新增覆盖格点数 / (1 + 行驶距离/500) - w·半径/100``

    * 分子是"这一站还能补多少我们没听过的地方"（用覆盖判据算，见
      :meth:`coverable_mask`）；分子为 0 的站直接淘汰 —— 这是"该收工就收工"
      的依据，也是"不必把 30 个站全跑一遍"的依据；

```

```

* 分母是行驶代价: **实测过: 只用收益选点会得到 43 km 的锯齿航路**
    (站与站之间来回横穿靶区), 加上距离项后回到最近邻量级;
* 半径偏置让航线自然形成"由内向外"的螺旋(用户说的"先第一个圈、再第二个圈"),
    既不会在内外圈之间乱跳, 也符合"越早听到源越好"的直觉。
"""
if not self.unheard():
    return None
px, py = self.pos
left = self.unheard_mask()
if not left.any():
    return None
best, best_score, best_info = None, None, None
w = float(self.cfg.spiral_radius_w)
for si, (sx, sy) in enumerate(self.stations):
    if si in self.visited_stations:
        continue
    d = math.hypot(sx - px, sy - py)
    if d < self.cfg.min_move:
        continue
    gain = self.station_gain(si, left)
    if gain <= self.cfg.station_min_gain:
        continue
    score = (float(gain) / (1.0 + d / 500.0)
              - w * math.hypot(sx, sy) / 100.0)
    if best_score is None or score > best_score:
        best, best_score, best_info = si, score, {"gain": gain, "dist": d}
return None if best is None else (best, best_info)

def station_channels(self, si, pos=None):
    """在这一站值得测的频道: **还没听到过**的、且这一站能补到新覆盖。

    已经听到过的频道一律不再在这里测 — 定位由清理阶段的专用测点负责
    (实测教训: 在扫描途中反复测同一个已发现频道, 会把它带离定位几何、
    也把航程和虚拟时间白白放大一倍以上)。

    ``cover_reduce > 0`` (降覆盖档) 时还要再过滤一层: 这一站必须**在
    :meth:`measure_set` 里**才测 — 不在集合里的站, 走到也不测 (省下 6 s/频道)。
    """
    keep = self.measure_set()
    if keep is not None and int(si) not in keep:
        return []
    m = self.coverable_mask(si) & ~self.unreachable_mask()
    p = np.asarray(pos if pos is not None else self.pos, float)
    out = []
    for ch in self.unheard():
        rec = self.recs[int(ch)]
        if rec.meas_pts:
            d = min(math.hypot(p[0] - q[0], p[1] - q[1]) for q in rec.meas_pts)
            if d < self.cfg.repeat_gap_m:
                continue
            if not np.any(m & ~self._covered_of(ch)):

```

```

        continue
    out.append(int(ch))
return out

# ----- 探站 -----
def visit_station(self, si):
    """到站位 ``si``: 先清除"就在跟前"的目标, 再测该测的频道。"""
    x, y = self.stations[int(si)]
    self.scan_points.append((float(x), float(y)))
    self.visited_stations.add(int(si))
    self.note_meas((x, y))
    chans = self.station_channels(si, pos=(x, y))
    travel = math.hypot(x - self.pos[0], y - self.pos[1]) / V_ROBOT
    avail = self.remaining() - self.cfg.time_reserve_s - travel - 4.0
    kmax = max(1, int(avail // (T_MEASURE + T_SWITCH)))
    if len(chans) > kmax:
        self.log(f" 剩余时间只够测 {kmax}/{len(chans)} 个频道")
        chans = chans[:kmax]
    if not chans:
        self.log(f"站位 {si} ({x:.0f},{y:.0f}): 没有值得测的频道, 跳过")
        return 0
    self.log(f"站位 {si}/{len(self.stations)} ({x:.0f},{y:.0f})"
            f" (r={math.hypot(x, y):.0f} m, 移动 {travel * V_ROBOT:.0f} m)"
            f"| 待测 {len(chans)} 个 {chans}")
    n = self.scan_at(x, y, chans, station=int(si))
    self.note_meas((float(self.arena.pos[0]), float(self.arena.pos[1])))
    return n

def scan_at(self, x, y, chans, station=None):
    """在 (x, y) 依次检测这些频道 (第一次调用会带来移动耗时)。

    与问题 3 的差别只有一个: 把**站位编号**记进台账, 覆盖记账才成立
    (``_covered_of`` 只认"在能听到的站位上测过"这个事实)。
    """
    n = 0
    for ch in chans:
        if self.aborted or not self.can_afford(T_MEASURE + T_SWITCH + 3.0):
            break
        resp = self.measure(x, y, ch, station=station)
        n += 1
        if resp.get("measure_result") == "near":
            self.do_clear(x, y, ch)
    return n

def note_meas(self, p):
    """记录一次"真实做过的检测"位置 (覆盖度评估口径, 与问题 3 一致)。"""
    p = (float(p[0]), float(p[1]))
    if not self.meas_pts or math.dist(self.meas_pts[-1], p) > 1.0:
        self.meas_pts.append(p)

def measure(self, x, y, ch, station=None):

```

```

"""检测一次；同时登记"这一站测过这个频道"以便覆盖记账。"""
resp = super().measure(x, y, ch)
if resp.get("accepted") is True:
    rec = self.recs[int(ch)]
    if station is not None and int(station) not in rec.meas_stations:
        rec.meas_stations.append(int(station))
    if resp.get("measure_result") == "near":
        self._near_pt[int(ch)] = (float(x), float(y))
return resp

# ----- 清理阶段：定向源闭环定位 -----
def estimate(self, ch):
    """频道的当前位置估计：优先用有界交会区域的中心，否则用当前方位外推。

    返回 `` (est, radius, br) ``。只有一条方位时半径记为 ``inf``：视线外推点
    在视线方向上完全不可信（误差就是整条视线的长度），不能当"区域半径 400 m"
    用 —— 那会让清除阶段去试探一个根本不对的圆。
    """
    rec = self.recs[int(ch)]
    br = self.bounded_region(rec)
    if br is not None:
        return np.asarray(br["center"], float), float(br["radius"]), br
    g = self.cross_estimate(rec)
    if g is not None:
        return g, float(self.cfg.loc_clear_radius_m), None
    est = self.ray_estimate(rec)
    return (est, float("inf"), None)

def ray_estimate(self, rec):
    """只有一条方位时的粗略位置：视线与"半径 R_RECV_MAX 圆"的交点。"""
    if not rec.pts:
        return None
    (sx, sy), a = rec.pts[-1], rec.svds[-1]
    u = np.array([math.cos(math.radians(a)), math.sin(math.radians(a))])
    S = np.array([float(sx), float(sy)])
    b = float(S @ u)
    c = float(S @ S) - R_ARENA * R_ARENA
    disc = b * b - c
    t = (-b + math.sqrt(disc)) if disc > 0 else 1000.0
    t = min(max(t, 100.0), 1500.0)
    return S + t * u

def cross_estimate(self, rec):
    """**最小二乘交会点**：把所有方位线两两求交再取中位数，稳健于个别粗差。

    只有一条方位时返回 ``None``（一条视线定不了点）。视线近似平行时返回 ``None``
    —— 那时交会区域无界，必须换检测点（问题 1 的结论）。
    """
    if rec.n_bearings < 2:
        return None
    pts = []

```

```

n = rec.n_bearings
for i in range(n):
    for j in range(i + 1, n):
        (x1, y1), a1 = rec.pts[i], rec.svds[i]
        (x2, y2), a2 = rec.pts[j], rec.svds[j]
        if math.hypot(x1 - x2, y1 - y2) < 50.0:
            continue # 基线太短, 交会无意义
        u1 = np.array([math.cos(math.radians(a1)), math.sin(math.radians(a1))])
        u2 = np.array([math.cos(math.radians(a2)), math.sin(math.radians(a2))])
        den = float(u1[0] * u2[1] - u1[1] * u2[0])
        if abs(den) < 0.05: # 交会角 < 3°: 病态
            continue
        d = np.array([x2 - x1, y2 - y1], float)
        t1 = float(d[0] * u2[1] - d[1] * u2[0]) / den
        if t1 <= 0.0:
            continue
        pts.append(np.array([x1, y1], float) + t1 * u1)
if not pts:
    return None
return np.median(np.asarray(pts, float), axis=0)

def locate_on_ray(self, ch):
    """沿已测方位做**两侧双分查找**, 直接量出"源在这条视线有多远"。

    物理依据 (关键, 且与朝向无关):

    * 方位是从检测点 ``s`` 指向源 ``g`` 的**射线**, 源一定在这条射线上;
    * ``s`` 自己**一定在源的有效覆盖范围内** (那条方位就是在这里测到的)
      ⇒ 视线上 t=0 处必有信号;
    * 沿射线往外走, 信号迟早消失: 要么是距离超过接收半径, 要么是走出了
      ±90° 覆盖角 — 两种情况都意味着"源在这条视线上的位置就在消失点附近",
      而且**消失点是单调的** (定向源的覆盖范围是一个凸集, 射线与凸集的交
      是一段区间);
    * 于是"有信号/无信号"的分界点就能把源夹住, 二分几步即可收到 40 m 以内。

    走位一律用 ``/clear``: 未命中只要 3 s (``/measure`` 要 6 s)、不改频道、
    不污染方位台账, 而且万一进到 20 m 内会直接把源清掉。

    返回 ``(est, radius, br)``; ``radius = half_width`` 是二分区间的一半。
    """
    rec = self.recs[int(ch)]
    if not rec.pts:
        return None, float("inf"), None
    # 已经有界且够小: 根本不必再走, 直接交给覆盖式试探
    br0 = self.bounded_region(rec)
    if br0 is not None and float(br0["radius"]) * 2.0 <= R_CLEAR:
        return np.asarray(br0["center"], float), float(br0["radius"]), br0
    (sx, sy), a = rec.pts[-1], rec.svds[-1]
    u = np.array([math.cos(math.radians(a)), math.sin(math.radians(a))])
    S = np.array([float(sx), float(sy)])

```

```

def at(t):
    return S + u * float(t)

def far_t():
    """射线出靶区的距离（再远就没有意义了）。"""
    b = float(S @ u)
    c = float(S @ S) - (R_ARENA - 5.0) ** 2
    disc = b * b - c
    return (math.sqrt(disc) - b) if disc > 0 else 600.0

def go_and_clear(p):
    """纯走位：用 /clear 移动过去，命中就返回 True。"""
    if not self.can_afford(math.hypot(*(p - np.asarray(self.pos, float)))
                          / V_ROBOT + 5.0):
        return False, None
    if self.do_clear(float(p[0]), float(p[1]), int(ch)):
        return True, "success"
    if self.aborted:
        return False, None
    self.note_meas((float(p[0]), float(p[1])))
    return False, "moved"

# ---- (1) 向外**指数跳步**，尽快建立上界 ----
# t=0 就是当初测到方位的地方，那里必有信号（省掉一次 6 s 的测量）；
# 向外用 1.4 倍递增的步长跳（520 → 730 → 1020 ...），边界通常在 2-3 步内被夹住。
# 为什么要跳大步：成本的大头是**走路**（5 m/s），每多一次"走 500 m 再测一次"
# 就是 100 s + 6 s；指数跳步把 5 次试探压到 2 次。
t_lo, t_hi = 0.0, None
t_cur = max(self.cfg.loc_walk_m,
            min(float((np.asarray(self.pos, float) - S) @ u) +
                self.cfg.loc_walk_m,
                far_t()))
step = self.cfg.loc_walk_m
for _ in range(self.cfg.loc_max_moves):
    hit, _st = go_and_clear(at(t_cur))
    if hit:
        return at(t_cur), 0.0, None
    if self.aborted:
        break
    r = self.measure(float(at(t_cur)[0]), float(at(t_cur)[1]), int(ch))
    self.note_meas((float(at(t_cur)[0]), float(at(t_cur)[1])))
    res = r.get("measure_result")
    self.stats["n_locate"] = self.stats.get("n_locate", 0) + 1
    self.log(f" 定位频道{ch}: 沿视线 t={t_cur:.0f} m -> {res}")
    if res == "near":
        self._near_pt[int(ch)] = (float(at(t_cur)[0]), float(at(t_cur)[1]))
        return at(t_cur), 0.0, None
    if res == "direction":
        t_lo = t_cur
        step *= self.cfg.loc_step_growth
        nxt = t_cur + step

```

```

        if nxt > far_t():
            nxt = far_t()
        if nxt <= t_cur + 1.0:
            break
        t_cur = nxt
    else:
        t_hi = t_cur
        break

# ---- (2) 二分夹逼 ----
n_bi = 0
while (t_hi is not None and n_bi < self.cfg.loc_bisect_steps
       and (t_hi - t_lo) > self.cfg.loc_bisect_tol_m):
    tm = 0.5 * (t_lo + t_hi)
    p = at(tm)
    if not self.can_afford(math.hypot(*(p - np.asarray(self.pos, float)))
                           / V_ROBOT + T_MEASURE + T_SWITCH + 4.0):
        break
    r = self.measure(float(p[0]), float(p[1]), int(ch))
    self.note_meas((float(p[0]), float(p[1])))
    n_bi += 1
    self.stats["n_locate"] = self.stats.get("n_locate", 0) + 1
    res = r.get("measure_result")
    self.log(f" 定位频道{ch}: 二分 t={tm:.0f} m -> {res}")
    if res == "near":
        self._near_pt[int(ch)] = (float(p[0]), float(p[1]))
        return p, 0.0, None
    if res == "direction":
        t_lo = tm
    else:
        t_hi = tm
if t_hi is None:
    t_est, half = t_lo, float(self.cfg.loc_walk_m)
else:
    t_est, half = 0.5 * (t_lo + t_hi), 0.5 * (t_hi - t_lo)
return at(t_est), half, None

# -----
# 定位二: 拿第二条方位做交会 (把区域压到 20 m 量级)
# -----
def admissible_near(self, q, est, r_est=None, margin=40.0, flip=None):
    """估计站位 ``q`` 能否听到"位于 ``est``、朝向径向"的定向源。

    判据与 :meth:`coverable_mask` 一致 (距离 + 半平面), 只是作用在单点上,
    用于排候选 ("这个侧移点还在源的覆盖角里吗")。 ``flip`` 见 :meth:`coverable_mask`。
    """
    if flip is None:
        flip = self.coverage_flip
    q = np.asarray(q, float)
    e = np.asarray(est, float)
    v = q - e
    # 源 → 站位

```

```

d = float(np.hypot(*v))
lim = (R_RECV_MIN if r_est is None else float(r_est)) - float(margin)
if d > lim:
    return False
dot = float(v[0] * e[0] + v[1] * e[1])
return dot <= 0.0 if flip else dot >= 0.0

def locate_by_cross(self, ch):
    """在"旁侧"再取一条方位：与已有视线交会，把位置压到几十米量级。

    侧移点选在**与最后一条视线垂直**的方向上、离估计位置
    ``loc_cross_radius_m`` 处 — 交会角  $\approx 45^\circ$ 、基线  $\approx 400$  m
     $\Rightarrow$  区域半径  $\approx R \cdot \tan 1^\circ / \sin 45^\circ \approx 10$  m，覆盖式试探一次就中。

    侧移点还要满足"估计位置在自己朝原点一侧" (:meth:`admissible_near`)：
    这是定向源"朝外发射"时唯一还听得到的半平面。返回是否确实拿到了方位。
    """
    rec = self.recs[int(ch)]
    if not rec.pts:
        return False
    (sx, sy), a = rec.pts[-1], rec.svds[-1]
    u = np.array([math.cos(math.radians(a)), math.sin(math.radians(a))])
    S = np.array([float(sx), float(sy)])
    t_p = float((np.asarray(self.pos, float) - S) @ u)
    t_hat = min(max(t_p, 220.0), 1900.0)
    G = S + u * t_hat
    perp = np.array([-u[1], u[0]])
    best = None
    for sgn in (+1.0, -1.0):
        for frac in (1.0, 0.6, 0.35): # 先试全侧移，再退而求其次
            q = G + perp * sgn * self.cfg.loc_cross_radius_m * frac
            if float(np.hypot(*q)) > R_ARENA - 1.0:
                continue
            if not self.admissible_near(q, G):
                continue
            key = float(np.hypot(*(q - np.asarray(self.pos, float))))
            if best is None or key < best[0]:
                best = (key, q, frac)
            break
    if best is None:
        # 覆盖角筛不出候选：直接取"离原点更近"的那一侧（一定在朝外的覆盖半平面里）
        cand_s = [G + perp * sgn * self.cfg.loc_cross_radius_m * 0.5
                    for sgn in (+1.0, -1.0)]
        cand_s = [c for c in cand_s if float(np.hypot(*c)) <= R_ARENA - 1.0]
        if not cand_s:
            return False
        q = min(cand_s, key=lambda c: float(np.hypot(*c)))
        best = (float(np.hypot(*(q - np.asarray(self.pos, float)))), q, 0.5)
    _d, q, frac = best
    trav = float(np.hypot(*(q - np.asarray(self.pos, float))))
    if not self.can_afford(trav / V_ROBOT + T_MEASURE + T_SWITCH + 6.0):

```

```

        return False
    self.log(f" 定位频道{ch}: 侧移到 ({q[0]:.0f},{q[1]:.0f})"
            f" (移动 {trav:.0f} m, 离估计位置 {self.cfg.loc_cross_radius_m *
            frac:.0f} m, "
            f"交会角 $\approx 45^\circ$ ) ")
    r = self.measure(float(q[0]), float(q[1]), int(ch))
    self.note_meas((float(q[0]), float(q[1])))
    self.stats["n_locate"] = self.stats.get("n_locate", 0) + 1
    if r.get("measure_result") == "near":
        self._near_pt[int(ch)] = (float(q[0]), float(q[1]))
        return True
    return r.get("measure_result") == "direction"

def localize(self, ch):
    """闭环定位: **先侧移交会(准、贵)→ 交会不成再沿视线二分(便宜、糙)**。

    为什么这个顺序: 误差的**横向分量**才是清除的瓶颈(沿视线的距离误差只会让
    试探点沿视线前后偏一点, 而横向偏 200 m 就完全试不到)。侧移交会同时压掉
    横向与纵向误差(基线 400 m、交会角  $45^\circ \Rightarrow$  区域半径  $\approx 10$  m),
    所以它是"值得付 180 s"的一步; 只有连交会都拿不到方位时,
    才退回"沿视线二分"(对贴边朝外的源, 它的横向误差  $\approx d \cdot \tan 1^\circ$ )。

    返回 ``(est, radius, br)``。
    """
    rec = self.recs[int(ch)]
    br = self.bounded_region(rec)
    if br is not None and float(br["radius"]) <= self.cfg.loc_probe_radius_max_m:
        return np.asarray(br["center"], float), float(br["radius"]), br
    # 1) 侧移交会(最多两轮: 第二轮是在"位置估计更准"之后再做一次)
    for _ in range(2):
        if not self.locate_by_cross(ch):
            break
        br = self.bounded_region(rec)
        if br is not None and float(br["radius"]) <=
            self.cfg.loc_probe_radius_max_m:
            return np.asarray(br["center"], float), float(br["radius"]), br
        if not self.can_afford(self.cfg.min_action_budget_s + 40.0):
            break
    g = self.cross_estimate(rec)
    if g is not None:
        return g, float(self.cfg.loc_clear_radius_m), None
    # 2) 交会不成: 沿视线二分量距离
    est, rad, _br = self.locate_on_ray(ch)
    if est is None:
        return None, float("inf"), None
    return est, float(rad), None

def sweep_ok(self, ch, rec, c, r, poly=None):
    """覆盖式试探的**布尔**结论。

    ``P3Robot.sweep_clear`` 现在返回三态字符串

```

```

    (``SWEEP_OK`` / ``SWEEP_MISS`` / ``SWEEP_BUDGET``, 见 ``p3_robot`` 的
    "失败归因三分"); 三者在布尔上下文里**都非空**, 直接拿来做 ``if`` 判定会把
    "预算不足、一个 /clear 都没发"误判成成功。本模块只需要"清掉了没有",
    因此统一折算回布尔。
    """
    return self.sweep_clear(ch, rec, c, r, poly=poly) == SWEEP_OK

def solve_channel(self, ch):
    """把一个已发现频道定位并清除（定向源版本）。"""
    rec = self.recs[int(ch)]
    if rec.cleared:
        return True
    # (0a) 定位过程中出现过 near (距离 ≤ 5 m): 回那个点直接清, 别再做任何试探
    near_p = self._near_pt.get(int(ch))
    if near_p is not None:
        d = float(np.hypot(*(np.asarray(self.pos, float) - np.asarray(near_p,
            float))))
        if self.can_afford(d / V_ROBOT + 6.0):
            self.log(f" 频道{ch} 曾在 ({near_p[0]:.0f},{near_p[1]:.0f}) 测到 near"
                f" (距离 ≤ 5 m) -> 回到该点直接清除")
            if self.do_clear(float(near_p[0]), float(near_p[1]), int(ch)):
                return True
            self._near_pt.pop(int(ch), None)
    est, rad, _br = self.estimate(ch)
    # (0b) 估计点就在跟前: 直接试一次 /clear (3 s 的事)
    if est is not None and float(np.hypot(*(np.asarray(self.pos, float) - est)))
        \
        <= self.cfg.clear_est_probe_m:
        if self.do_clear(float(self.pos[0]), float(self.pos[1]), int(ch)):
            return True
    # (1) 定位: 区域够小就直接用; 否则沿视线二分把"距离"量准
    br = self.bounded_region(rec)
    if not (br is not None
        and float(br["radius"]) <= self.cfg.loc_probe_radius_max_m):
        est, rad, br = self.localize(ch)
        if est is None:
            rec.clear_fail += 1
            self.log(f" [x] 频道{ch} 无法定位 (方位数据不足)")
            return False
    # (2) 逼近定位区域中心 (顺带补一条方位, 常常直接 near)
    for _ in range(self.cfg.clear_approach_max):
        br = self.bounded_region(rec)
        if br is None:
            break
        c = np.asarray(br["center"], float)
        rr = float(br["radius"])
        if rr > self.cfg.loc_probe_radius_max_m:
            break
        d = math.hypot(*(np.asarray(self.pos, float) - c))
        if d <= max(rr, self.cfg.arrive_radius_m):
            break

```

```

    if not self.can_afford(d / V_ROBOT + T_MEASURE + T_SWITCH + 6.0):
        break
    resp = self.measure(float(c[0]), float(c[1]), int(ch))
    self.note_meas((float(c[0]), float(c[1])))
    if resp.get("measure_result") == "near":
        return self.do_clear(float(c[0]), float(c[1]), int(ch))
    if resp.get("accepted") is not True:
        return False
# (3) 第一轮试探: 有界区域(且不大)就按**交会多边形**试 — 交会区域常是细长
# 四边形, 按多边形只在"到区域 ≤ 20 m"的格点上试, 比外接圆省一半以上的点,
# 而且"区域必含真源"是问题 1 的结论, 所以这一轮命中率最高。
self.phase = "clear"
br = self.bounded_region(rec)
if br is not None and float(br["radius"]) <= self.cfg.loc_probe_radius_max_m:
    if self.sweep_ok(int(ch), rec, np.asarray(br["center"], float),
                    float(br["radius"]), poly=br.get("poly")):
        return True
# (4) 第二轮: 以**估计点**为中心的小圆(半径由定位精度决定)
# 定向源的清除与朝向无关 ⇒ 只要"估计点到真源 ≤ 20 m"就够了;
# 沿视线二分的距离精度约 ±20 m, 横向误差约 0.0175·d ⇒ 半径 90 m 足够。
if est is not None:
    if self.sweep_ok(int(ch), rec, np.asarray(est, float),
                    self.cfg.loc_clear_radius_m, poly=None):
        return True
# (5) 第三轮: 估计点也不可信时, 用"最后一条视线"上的二分区间中点再试一次
if rec.n_bearings >= 1:
    br = self.bounded_region(rec)
    if br is None:
        est2, _r2, _b2 = self.locate_on_ray(ch)
        if est2 is not None and self.sweep_ok(
            int(ch), rec, np.asarray(est2, float),
            self.cfg.loc_clear_radius_m, poly=None):
            return True
# (5) 第四轮: **沿视线二分给出的是"与协议约定无关"的距离信息** —
# 只要二分区间收得够窄(≤ 230 m, 说明测点确实落在源的覆盖范围内),
# 就以该位置为中心做一次小圆试探。这一轮专门对付"几条视线近似平行
# (交会角小)、有界区域又细又长"的目标 — 那类目标的交会区域虽然把源
# 夹住了, 但细长区域用外接圆试探会大量空跑。
if (rec.n_bearings >= 1 and self.can_afford(self.cfg.min_action_budget_s +
    30.0)
    and int(ch) not in self._tried_ray):
    self._tried_ray.add(int(ch))
    est3, rad3, _b3 = self.locate_on_ray(ch)
    if est3 is not None and float(rad3) <= 230.0:
        d3 = float(np.hypot(*(np.asarray(self.pos, float)
            - np.asarray(est3, float))))
        if self.can_afford(d3 / V_ROBOT + T_MEASURE + T_SWITCH + 8.0):
            self.log(f" 沿视线落到 ({est3[0]:.0f},{est3[1]:.0f})"
                f" (区间 ±{float(rad3):.0f} m) 再取一条方位")
            r3 = self.measure(float(est3[0]), float(est3[1]), int(ch))
            self.note_meas((float(est3[0]), float(est3[1])))

```

```

        if r3.get("measure_result") == "near":
            self._near_pt[int(ch)] = (float(est3[0]), float(est3[1]))
            if self.do_clear(float(est3[0]), float(est3[1]), int(ch)):
                return True
        br = self.bounded_region(rec)
        if br is not None \
            and float(br["radius"]) <= self.cfg.loc_probe_radius_max_m:
            if self.sweep_ok(int(ch), rec,
                            np.asarray(br["center"], float),
                            float(br["radius"]), poly=br.get("poly")):
                return True
            if self.sweep_ok(int(ch), rec, np.asarray(est3, float),
                            self.cfg.loc_clear_radius_m, poly=None):
                return True
        self.phase = "locate"
        rec.clear_fail += 1
        self.log(f" [x] 频道{ch} 本轮未清除成功")
        return False

# ----- 主循环 -----
def sweep_pass(self, flip=False):
    """走一遍扫描航线：按"必去站位"顺序移动，每站只测还没听到过的频道。

    返回本遍实际访问的站数。三个提前收工判据：

    * 该半平面约定下"还没有被排除的格点"清零 ⇒ 这一遍已经证明没有漏源；
    * 航线走完；
    * 时间不够。

    每站之后还会做两件"顺路"的事：把估计位置就在附近的目标清掉
    (:meth:`_opportunistic_clear`)，以及按需重排航线(``replan_every``)。
    """
    self.coverage_flip = bool(flip)
    tour = (self.plan_station_tour() if self.cfg.plan_tour
            else self.greedy_station_order())
    self.stats["n_covering_stations"] = len(tour)
    self.stats["tour_len_m"] = float(getattr(self, "_tour_len_m", 0.0))
    self.log(f"扫描航线: {len(tour)} 个必去站位"
            f" ({'覆盖集 + 半径偏置 TSP' if self.cfg.plan_tour else '逐步贪心'}) , "
            f"预计长度 {getattr(self, '_tour_len_m', 0.0):.0f} m")
    visited = 0
    step_i = 0
    while step_i < len(tour):
        if self.aborted or self.remaining() <= self.cfg.time_reserve_s:
            break
        if self.unheard_cover_left() <= 0: # 已经没有"可能有源"的地方了
            self.log(" 所有未听频道的可能位置都被排除 -> 提前结束扫描")
            break
        if self.cfg.max_station_visits and visited >= self.cfg.max_station_visits:
            break
        si = tour[step_i]

```

```

        step_i += 1
        # 复核: 这一站对"还没听到过的频道"还有没有新覆盖 (有的站会被别的站代劳)
        if self.station_gain(si, self.unheard_mask()) <=
            self.cfg.station_min_gain:
            self.log(f" 站位 {si} 的覆盖职责已被其他站代劳 -> 跳过")
            continue
        x, y = self.stations[int(si)]
        dist = math.hypot(x - self.pos[0], y - self.pos[1])
        if not self.can_afford(dist / V_ROBOT + T_MEASURE + T_SWITCH + 4.0):
            self.log("剩余时间不足以再跑一站 -> 结束扫描")
            break
        self.visit_station(si)
        visited += 1
        # 顺路: 刚听到的目标若绕不了多远就当场清掉 ("早清一个源"直接改善指标)
        self._opportunistic_clear(max_n=self.cfg.enroute_max_n)
        # 滚动重规划 (可选)
        if (self.cfg.replan_every and visited % self.cfg.replan_every == 0
            and self.remaining() > self.cfg.time_reserve_s
            and not self.aborted):
            tour = self.plan_station_tour()
            step_i = 0
            self.stats["n_replans"] = self.stats.get("n_replans", 0) + 1
            self.log(f" 滚动重规划: 剩余必去站位 {len(tour)} 个")
    return visited

def run(self):
    t0 = time.time()
    resp = self.arena.enter()
    if resp.get("accepted") is not True:
        raise ArenaError(f"/enter 未被接受: {resp}")
    base = self.cfg
    self.log(f"进入靶区: 预算 {self.remaining():.0f}
        s ({self.arena.budget_kind()}); "
        f"策略=三角形格网扫描 (a={base.grid_a:.0f} m, "
        f"{len(self.stations)} 个站位 + 原点)")
    try:
        # ----- 阶段 0: 原点 (频道 1 起步, 依次测全部频道) -----
        self.phase = "scan"
        self.scan_points.append(self.pos)
        chans0 = self.channels_to_measure()
        self.log(f"第 0 站 (原点): 待测 {len(chans0)} 个频道 {chans0}")
        self.note_meas(self.pos)
        self.scan_at(self.pos[0], self.pos[1], chans0)
        self.note_meas((float(self.arena.pos[0]), float(self.arena.pos[1])))
        self.log(f" 原点检测完成: 听到 {len(self.heard_pending())} 个频道, "
            f"剩余未覆盖格点 {self.unheard_cover_left()} 个"
            f"(定向覆盖模型: 这些格点还可能藏着没听到过的源)")

        # ----- 阶段 1: 扫描 + 清理 -----
        if self.cfg.unified:
            # **边扫描边清理**: 一条滚动重规划的航路走完两圈, 点也清完 (默认)

```

```

        self.run_unified()
    else:
        # 阶段式对照：先扫两遍，再统一清理
        self._staged_scan_then_clear()
finally:
    try:
        self.arena.exit()
    except ArenaError as e:
        self.log(f"/exit 异常: {e}")
self.stats["program_runtime_s"] = time.time() - t0
return self.report()

# ----- 统一航路：边扫描边清理 -----
def ring_index(self):
    """把站位按半径分"环"：返回 ``{站号: 环号}`` (环号 = 半径分档序号)。

    分档宽度取 ``grid_a/2`` (默认 500 m)：于是 r=0 / r=1000 / r=1732 / r=1850
    落在 0 / 2 / 3 / 3 档 — 内环、中环、外环分得开，
    而"1732 与 1850"这种只差 118 m 的两层会被合到同一环（它们本来就该连着走）。
    """
    got = getattr(self, "_ringidx", None)
    if got is not None:
        return got
    band = max(150.0, float(self.cfg.grid_a) * 0.5)
    self._ringidx = {i: int(round(math.hypot(x, y) / band))
                     for i, (x, y) in enumerate(self.stations)}
    return self._ringidx

def scan_skeleton(self, start=None):
    """扫描站位的骨架顺序：**全局最近邻 + 2-opt，代价里带"由内向外"偏置**。


这里修正了旧实现的一个真实缺陷（`src/p4_route_audit.py` 量化）：  

    旧版按"半径分档 + 档内极角"排成**固定**顺序 — 进环永远从固定角度切进去，  

    于是"切换环"时经常要横穿靶区。实测骨架直线航程 18.7 km，  

    但跳站之后实际扫描段走了 **26.2 km**（多出来的就是跨环长跳）。



规则（对应用户的要求：**切换环时就近进入**）：



1. **全局最近邻**（不分环）：下一个站取"离我最近的那个" —  

        "进环点"自然就是最近的站，而不是某个固定角度；
2. 代价里加 ``w · 半径`` 的小偏置：让航线**整体**由内向外推进  

        （内环先扫 ⇒ 中心附近的源先被听到，对指标有利），  

        但偏置足够小，不会为了"先内后外"去横穿靶区；
3. **2-opt** 消掉自交与绕远。



【!】 ``start`` 只用于"排一条**基准航线**"，不要拿"顺路清除之后的当前位置"来排  

    — 从偏离航线很远的点做最近邻会得到一条"以该点为中心的星形路线"，  

    把原来的环形结构彻底打散（实测这是"跨过中间节点"长跳的真正来源）。  

    真正的处理是"绕行清掉目标后沿基准航线继续"，见 :meth:`build_queue`。


    """
    key = None if start is None else (round(float(start[0]), 1),

```

```

        round(float(start[1]), 1))
cache = getattr(self, "_skeleton_cache", None)
if cache is None:
    cache = self._skeleton_cache = {}
if key is not None and key in cache:
    return cache[key]
if key is None and getattr(self, "_skeleton", None) is not None:
    return self._skeleton
w = float(self.cfg.skeleton_radius_w)
si_all = list(range(len(self.stations)))
P = {si: np.asarray(self.stations[si], float) for si in si_all}
s0 = np.asarray((0.0, 0.0) if start is None else start, float)

def cost(order):
    cur, d = s0, 0.0
    for si in order:
        d += float(np.hypot(*(P[si] - cur))) + w * float(np.hypot(*P[si]))
        cur = P[si]
    return d

def geo_len(order):
    cur, d = s0, 0.0
    for si in order:
        d += float(np.hypot(*(P[si] - cur)))
        cur = P[si]
    return d

def two_opt(order, rounds=40):
    best = cost(order)
    improved, guard = True, 0
    while improved and guard < rounds:
        improved = False
        guard += 1
        for i in range(len(order) - 1):
            for j in range(i + 1, len(order)):
                cand = order[:i] + order[i:j + 1][::-1] + order[j + 1:]
                c = cost(cand)
                if c < best - 1e-9:
                    order, best, improved = cand, c, True
    return order

# 起点构造: 最近邻 / 角向扫描 (内外分环、正反两向)
starts = []
left, cur, seq = set(si_all), s0, []
while left:
    j = min(left, key=lambda si: float(np.hypot(*(P[si] - cur)))
            + w * float(np.hypot(*P[si])))
    seq.append(j)
    left.discard(j)
    cur = P[j]
starts.append(seq)

```

```

c0 = P[seq[0]]
a0 = math.atan2(c0[1] - s0[1], c0[0] - s0[0])
for by_ring in (True, False):
    starts.append(sorted(
        si_all, key=lambda si: ((round(math.hypot(*P[si]) / 900.0)
                                if by_ring else 0),
                                (math.atan2(P[si][1], P[si][0]) - a0)
                                % (2 * math.pi))))
for sgn in (+1, -1):
    starts.append(sorted(si_all,
        key=lambda si: (math.atan2(P[si][1], P[si][0]) - a0)
        * sgn % (2 * math.pi)))
# **cheapest insertion** (圆盘布局上比"最近邻 + 2-opt"更能避免跨环长跳:
# 它总把新站插到"绕行代价最小"的位置, 而不是从当前位置一路贪心往外走)
order = [si_all[0]]
rest = set(si_all[1:])
while rest:
    bo, bd = None, None
    for si in rest:
        p = P[si]
        for k in range(len(order)):
            a_ = s0 if k == 0 else P[order[k - 1]]
            b_ = P[order[k]]
            delta = (math.hypot(*(p - a_)) + math.hypot(*(b_ - p))
                    - math.hypot(*(b_ - a_)))
            if bd is None or delta < bd:
                bo, bd = (si, k), delta
    order.insert(bo[1], bo[0])
    rest.discard(bo[0])
starts.append(order)
best_seq, best_geo = None, float("inf")
for cand in starts:
    cand = two_opt(list(cand))
    g = geo_len(cand)
    if g < best_geo - 1e-9:
        best_seq, best_geo = cand, g
seq = best_seq

# 试过但**否掉**的一个改动: 显式"由内向外"排序(按半径 sort 再 2-opt)。
# 动机是"先扫内环 ⇒ 中心附近的源早被听到 ⇒ 指标更好",
# 但实测(20局): gain=180 时从 620 s 变 625 s(更差);
# gain=250 时从 602 s 变 578 s(更快)**但清除率 1.0000 → 0.9925** (掉 2 个源)。
# 既然"清除率是硬指标、优化不达标就回滚", 这个改动没有保留
# (相关字段 `radius_sort_w` 仍留在配置里, 默认 0 = 关闭, 便于复现)。
if key is not None:
    cache[key] = seq
else:
    self._skeleton = seq
    self._skeleton_len_m = best_geo
return seq

```

```
def spiral_tour(self, start=None):
    """**问题 3 式的"绕圈"航线**：按环由内向外，每环**角向单调推进、不回头**。
```

这是把问题 3 的覆盖扫圈（`src/p3\_sweep.py`：`pick\_stop` 只允许"沿当前绕圈方向单调前进"，`adv ≥ 0.35` 格）搬到问题 4 的格网上：

1. 环 = 半径分档（`grid\_a/2`，于是 $r=1000 / 1732 / 1850$ 各成一层）；
2. **由内向外**逐环扫（内环先扫 $\Rightarrow$ 中心附近的源先被听到，指标对"听到时刻"敏感）；
3. 每环内**角向单调**：方向在第一环的第一段确定（取"往哪边转更近"），之后只沿这个方向走——绝对不会在同一环上回头；
4. 环间**就近衔接**：下一环的起点取"离当前所在站位最近的那个站"，方向**反过来**（下一环沿反方向扫），这样绕完一圈正好回到起点附近，衔接距离最短；
5. 最后用 2-opt 在"保持环序"的前提下微调（只允许环内重排）。

与"几何最短 TSP"的差别：TSP 排法会在环之间来回穿插（半径序列 1000, 1850, 1732, 1000, ...），几何上更短，但**角向来回横跳**；绕圈排法一圈只走一个方向，路径形态与问题 3 的八边形/十二边形一致。

```
"""
band = max(150.0, float(self.cfg.grid_a) * float(self.cfg.ring_band_frac))
rings: dict[int, list[int]] = {}
for si, (x, y) in enumerate(self.stations):
    rings.setdefault(int(round(math.hypot(x, y) / band)), []).append(si)
cur = np.asarray((0.0, 0.0) if start is None else start, float)
seq: list[int] = []
direction = 0.0          # 0 = 未定; +1 = 逆时针; -1 = 顺时针
for k in sorted(rings):
    grp = rings[k]
    if not grp:
        continue
    # 进环点: 离当前位置最近的站 ("切换环时就近进入")
    entry = min(grp, key=lambda si: float(np.hypot(
        *(np.asarray(self.stations[si], float) - cur)))
    a0 = math.atan2(self.stations[entry][1], self.stations[entry][0])
    if direction == 0.0:
        # 定方向: 比较"往两个方向扫完这一环"的角向总路程, 取短的
        direction = 1.0 if len(grp) else 1.0
        # 用"下一站"的角向来判断 (避免固定方向导致第一段就横穿)
        others = [si for si in grp if si != entry]
        if others:
            nxt = min(others, key=lambda si: abs(
                (math.atan2(self.stations[si][1], self.stations[si][0]) - a0
                 + math.pi) % (2 * math.pi) - math.pi))
            da = (math.atan2(self.stations[nxt][1], self.stations[nxt][0])
                  - a0 + math.pi) % (2 * math.pi) - math.pi
            direction = 1.0 if da >= 0 else -1.0
    order = [entry]
    left = [si for si in grp if si != entry]
    cursor = a0
    while left:
        def adv(si, _c=cursor):
```

```

        a = math.atan2(self.stations[si][1], self.stations[si][0])
        d = (a - _c) * direction % (2 * math.pi)
        return d if d > 1e-9 else d + 2 * math.pi
    nxt = min(left, key=adv)
    order.append(nxt)
    left.remove(nxt)
    cursor = math.atan2(self.stations[nxt][1], self.stations[nxt][0])
    seq += order
    cur = np.asarray(self.stations[order[-1]], float)
    direction = -direction    # 下一环反向: 绕回起点附近, 衔接最短
return seq

def build_queue(self, start=None):
    """建立本遍的扫描航线: **一次排好, 全程不再按当前位置重排**。

    为什么不作"动态最近邻/滚动重排"(这是被实测否掉的两版):

    * 旧版 A: 每一步在"还没去过且此刻有用的站"里取最近 ——
      会跳过"此刻收益为 0"的几何中间节点, 出现 2 500 m 的长跳, 之后还要回头补;
    * 旧版 B: 顺路清除后按当前位置重排剩余站位 ——
      从偏离航线 1 200 m 的位置做最近邻, 会把环形航线打散成"以该点为中心的星形",
      实测航程反而从 17.9 km 涨到 26.9 km。

    排法由 ``route_mode`` 决定(对照用):

    * ``"tsp"`` (默认): :meth:`scan_skeleton` (全局 NN + 2-opt + cheapest-insertion
      多起点取优) —— 几何最短, 但会在环之间交替穿插;
    * ``"spiral"``: :meth:`spiral_tour` —— 问题 3 式"由内向外绕圈 + 角向单调不回头"。

    顺路清除只是"绕出去一趟再回来", 跳站只发生在"队首此刻确实没有新覆盖"时
    (:meth:`next_scan_station`)。
    """
    if self.cfg.route_mode == "spiral":
        base = self.spiral_tour(start=start)
        self._queue = [si for si in base if si not in self.visited_stations]
        return self._queue    # 绕圈排法**不能**再做 2-opt (会把角向单调性打乱)
    base = self.scan_skeleton(start=start)
    self._queue = [si for si in base if si not in self.visited_stations]
    self._queue, _ = self._two_opt(self._queue,
                                   start=np.asarray(self.pos, float))
    return self._queue

def _two_opt(self, order, start):
    """对 ``order`` (站号列表) 做 2-opt, 返回 ``(新顺序, 长度)``。"""
    P = {si: np.asarray(self.stations[si], float) for si in order}
    s0 = np.asarray(start, float)

    def cost(o):
        cur, d = s0, 0.0
        for si in o:
            d += float(np.hypot(*(P[si] - cur)))

```

```

        cur = P[si]
    return d

    if len(order) < 4:
        return list(order), cost(order)
    best = cost(order)
    improved = True
    while improved:
        improved = False
        for i in range(len(order) - 1):
            for j in range(i + 1, len(order)):
                cand = order[:i] + order[i:j + 1][::-1] + order[j + 1:]
                c = cost(cand)
                if c < best - 1e-9:
                    order, best, improved = cand, c, True
    return list(order), best

def next_scan_station(self, flip=None):
    """下一个要去的扫描站。

    **两步走**（这是把"就近选站"和"不跨环乱跳"同时做到的办法）：

    1. **正常情况**：走基准航线的队首。基准航线是全局 NN + 2-opt 排出来的，
        几何上连贯（实测单跳最大 1 104 m、无 >1 200 m 的长跳）；
    2. **被顺路清除带偏之后**（当前位置到队首的距离 > ``detour_follow_m``）：
        不再硬走回队首，改为**就近切入**——在基准航线里挑"离我最近的那一站"
        作为新的起点，然后**顺着基准航线的顺序**往下走（不是重新做最近邻！）。

    为什么"顺着基准航线"而不是"重新最近邻"：重新最近邻会从偏离点长出一颗
    **星形路线**，把环形结构彻底打散（实测航程 17.9 km → 26.9 km），
    而且会跳过几何上的中间站、之后还要回头补。
    """
    if not self.unheard():
        return None
    keep = self.measure_set() # 降覆盖档：只在这些站里选（None = 全覆盖）
    # 当前队首是否还值得去
    def usable(si):
        return (si not in self.visited_stations
                and (keep is None or int(si) in keep)
                and self.station_gain(si, self.unheard_mask())
                > self.cfg.station_min_gain)

    while self._queue and not usable(self._queue[0]):
        self._queue.pop(0)
    if not self._queue:
        return None
    pos = np.asarray(self.pos, float)
    d0 = float(np.hypot(*(np.asarray(self.stations[self._queue[0]], float) -
                                pos)))
    if d0 <= self.cfg.detour_follow_m:
        return self._queue[0]

```

```

# 被带偏了：就近切入基准航线（取“离我最近且还有用”的那一站作为新起点）
best, bd = None, None
for k, si in enumerate(self._queue):
    if not usable(si):
        continue
    d = float(np.hypot(*(np.asarray(self.stations[si], float) - pos)))
    if bd is None or d < bd:
        best, bd = k, d
if best is None:
    return None
if best > 0:
    self._queue = self._queue[best:] + self._queue[:best]
    self.stats["n_reroutes"] = self.stats.get("n_reroutes", 0) + 1
    self.log(f" 就近切入航线：跳过 {best} 站，从站 {self._queue[0]} 继续"
            f"（距当前 {bd:.0f} m，原队首 {d0:.0f} m）")
return self._queue[0]

def replan_queue(self, force=False):
    """覆盖职责变化后用**基准航线**重排剩余站位（不再按当前位置重排）。

    旧实现按"当前位置"重排，实测会被顺路清除带偏、把环形航线打散成星形
    （见 :meth:`build_queue` 的说明）。现在只在"有站因为收益为 0 被弹出"或
    显式 ``force`` 时，回到**基准航线**的顺序重排剩余站位。
    """
    self._queue = [si for si in self._queue if si not in self.visited_stations]
    if not self._queue or not force:
        return
    base = self.scan_skeleton()
    rank = {si: k for k, si in enumerate(base)}
    self._queue.sort(key=lambda si: rank.get(si, 10 ** 6))
    self.stats["n_replans"] = self.stats.get("n_replans", 0) + 1

def localize_fast(self, ch):
    """**小幅度移动**把目标定到"能清"的精度，返回 ``(est, radius)``。

    与全量 :meth:`localize` 的差别：这里要的是"够用"而不是"最优"，
    因为每多走 100 m 就是 20 s 的虚拟时间：

    1. 交会区域已经  $\leq$  ``interleave_loc_radius_m``  $\Rightarrow$  直接用（0 次额外测量）；
    2. 侧移交会一次（420 m、交会角  $\approx 45^\circ$ ）—— 一步就把区域压到 10-40 m，
       这就是用户说的"较小幅度的移动来确定大致方位"；
    3. 只做一次；不成再退回"沿视线二分"（与协议约定无关的那条路）。
    """
    rec = self.recs[int(ch)]
    br = self.bounded_region(rec)
    lim = float(self.cfg.interleave_loc_radius_m)
    if br is not None and float(br["radius"]) <= lim:
        return np.asarray(br["center"], float), float(br["radius"]), br
    if rec.n_bearings == 1 and self.can_afford(60.0):
        self.locate_by_cross(ch)
        br = self.bounded_region(rec)

```

```

        if br is not None and float(br["radius"]) <= lim:
            return np.asarray(br["center"], float), float(br["radius"]), br
g = self.cross_estimate(rec)
if g is not None:
    r = float(br["radius"]) if br is not None else lim
    return g, r, br
est, rad, _br = self.locate_on_ray(ch)
if est is None:
    return None, float("inf"), None
return est, float(rad), None

def clear_here(self, ch):
    """在当前附近把一个已听到的目标清掉：小幅度定位 → 小范围覆盖式试探。

    与全量 :meth:`solve_channel` 的差别：不做"逼近中心 + 多边形试探"那一串
    （那是阶段式清理才划算的），只保留"定位一次 + 以估计点为中心的小圆试探"。
    """
    rec = self.recs[int(ch)]
    if rec.cleared:
        return True
    near_p = self._near_pt.get(int(ch))
    if near_p is not None:
        if self.do_clear(float(near_p[0]), float(near_p[1]), int(ch)):
            return True
    est, rad, br = self.localize_fast(ch)
    if est is None:
        return False
    # 顺带：人已经在估计点上/附近，直接试一次 /clear (3-5 s 的事)
    d = float(np.hypot(*(np.asarray(self.pos, float) - np.asarray(est, float))))
    if d <= self.cfg.clear_est_probe_m and \
        self.do_clear(float(self.pos[0]), float(self.pos[1]), int(ch)):
        return True
    self.phase = "probe"
    ok = self.sweep_ok(int(ch), rec, np.asarray(est, float),
                      self.cfg.interleave_probe_m, poly=None)
    if not ok and br is not None and float(br["radius"]) <=
        self.cfg.probe_max_radius_m:
        ok = self.sweep_ok(int(ch), rec, np.asarray(br["center"], float),
                          float(br["radius"]), poly=br.get("poly"))
    self.phase = "scan"
    if not ok:
        rec.clear_fail += 1
    return ok

def pick_unified_action(self, attempts):
    """统一航路的下一步：在"继续扫描"和"就地清掉已发现目标"之间选。

    结论（**实测出来的**，不是先验）：**听到就清**永远优于"攒着以后清"。
    `src/p4_tune_unified.py` 的扫参（20 局）把"绕去清理的距离门限"从 0 扫到 ∞：

    | 门限 | 清除率 | 平均定位清除 | 虚拟时间 | 行程 |

```

```
|---|---|---|---|---|
| 0 (只在第二遍清) | 0.9963 | 1 019 s | 13 607 s | 56.6 km |
| 700 m | 0.9963 | 976 s | 13 027 s | 54.1 km |
| 1 100 m | 0.9963 | 884 s | 11 804 s | 48.7 km |
| **∞ (听到就清)** | **1.0000** | **833 s** | **11 162 s** | **45.9 km** |
```

门限越松越好、一直放到"不设门限"最优 —— 原因是**流水线**：

"绕去清" 的动作与"扫描"的动作在时间上是重叠的（清完就地从当前站继续扫，不用回到原航点），而"攒着"会把所有清除压到航程末尾，既拖长总时间、又让后面的目标等更久。所以这里不设距离门限，只按"最近的目标优先"排。

保留"选哪个目标"的排序（最近优先）与"第二遍必清"的兜底：

第二遍开始时若还有没清的目标（例如上一遍预算不够），它们的优先级翻倍。

```
"""
pos = np.asarray(self.pos, float)
best = None
for ch in self.heard_pending():
    if attempts.get(ch, 0) >= self.cfg.max_solve_attempts:
        continue
    est = self.estimate(ch)[0]
    if est is None:
        continue
    d = float(np.hypot(*(est - pos)))
    # 绕行上限用 ``interleave_detour_max_m`` (默认 ∞ = 不设限)。
    # 【!】 别用 ``clear_detour_max_m``：那是阶段式时代留给"到站顺手清"的 300 m，
    # 套到统一航路上会把大多数清理都延后 ⇒ 积压到航程末尾、来回折返
    # （实测 300 m 上限：648 s → 1141 s、行程 34.9 → 64.1 km）。
    if d > self.cfg.interleave_detour_max_m and not self.unified_second_pass:
        continue
    cost = d / V_ROBOT + self.cfg.interleave_clear_cost_s
    if self.unified_second_pass:
        cost *= 0.5 # 第二遍残留的目标优先
    if best is None or cost < best[0]:
        best = (cost, ch, d)
if best is not None:
    return ("clear", best[1])
si = self.next_scan_station()
return None if si is None else ("station", si)
```

```
def run_unified(self):
    """**边扫描边清理**：一条滚动重规划的航路走完两圈，点也清完。
```

结构（对应用户的要求）：

```
...
```

队列 = 扫描站位（由内向外螺旋）

while 队列非空 或 还有没清的目标：

 每走一站/清一个目标：重读台账、用最新位置重排剩余队列

 下一步 = 已发现目标里最近的一个（就地清），没有就取队首站

 第二遍（换半平面约定）时残留目标优先级翻倍

```
...
```

```

* **听到就地清** (实测最优, 见 :meth:`pick_unified_action`): 扫描与清除在同一条
航路上交错, 清完直接从当前站位继续扫, 不需要回到原航点;
* **第二遍** : 换另一个半平面约定重扫 (兜住"定向方向"的协议约定不确定性),
这一遍里残留的目标优先清掉, 所以两圈走完时点也清完;
* 收工判据不变: 两遍都走完且所有目标已清 => 靶区内没有未发现的源。
"""
n_clear = 0
attempts: dict[int, int] = {}
total_visited = 0
passes = (False, True) if self.cfg.two_pass else (False,)
for pi, flip in enumerate(passes):
    self.unified_second_pass = bool(pi > 0)
    self.coverage_flip = bool(flip)
    if pi > 0:
        self.first_pass_heard = {int(ch) for ch in self.heard_pending()}
    if pi > 0 and not self.unheard() and not self.heard_pending():
        break
    self.log(f"— 第 {pi + 1} 遍 (半平面约定 "
            f"{'朝内' if flip else '朝外'}): "
            f"未听频道 {len(self.unheard())}, 待清目标 "
            f"{len(self.heard_pending())} —")
    # 本遍的航线: 几何一次排好 (全局最近邻 + 2-opt), 全程不再按当前位置重排
    self.build_queue()
    guard = 0
    while not self.aborted:
        guard += 1
        if guard > 3 * (len(self.stations) + len(self.recs)) + 20:
            self.log(" 航路步数超限 -> 结束本遍")
            break
        if self.remaining() <= self.cfg.time_reserve_s:
            self.log(" 时间不够 -> 结束本遍")
            break
        # 先看"能不能顺手清掉", 没有可清的就取队首扫描站
        act = self.pick_unified_action(attempts)
        if act is None:
            break
        kind, target = act
        if kind == "clear":
            ch = int(target)
            attempts[ch] = attempts.get(ch, 0) + 1
            est = self.estimate(ch)[0]
            d = (float("inf") if est is None
                 else float(np.hypot(*(est - np.asarray(self.pos, float)))))
            tag = "(第二遍优先)" if self.unified_second_pass else ""
            self.log(f" 统一航路{tag}: 转去清频道{ch}"
                    f" (估计距离 {d:.0f} m, 第 {attempts[ch]} 次尝试)")
            self.phase = "enroute"
            if self.clear_here(ch):
                n_clear += 1
            self.phase = "scan"

```

```

        # 顺路清除只是"绕出去一趟再回来": **不重排航线**
        # (按当前位置重排会把环形航线打散成星形, 实测航程 +50%)
        continue
    si = int(target)
    x, y = self.stations[si]
    dist = math.hypot(x - self.pos[0], y - self.pos[1])
    if not self.can_afford(dist / V_ROBOT + T_MEASURE + T_SWITCH + 4.0):
        self.log(" 剩余时间不足以再跑一站 -> 结束本遍")
        break
    self.visit_station(si)
    total_visited += 1
    # 到站后先顺手清掉"就在跟前"的目标, 再继续
    if self.cfg.clear_detour_max_m > 0 and not self.aborted:
        self._opportunistic_clear(max_n=self.cfg.enroute_max_n)
    if self.aborted:
        break
self.stats["n_visited_stations"] = len(self.visited_stations)
self.stats["vtime_after_scan"] = float(self.arena.virtual_time_s)
left = self.heard_pending()
self.log(f"统一航路结束: 访问 {len(self.visited_stations)} 站"
        f" (共 {total_visited} 站次), 清除 {n_clear} 个, "
        f"仍未清 {left} | 虚拟 {self.arena.virtual_time_s:.0f} s")
# 收尾: 还有目标没清就用全量定位-清除流程兜底
if left and not self.aborted and self.remaining() > self.cfg.time_reserve_s:
    self.log(f"收尾阶段: 用完整流程清剩余 {left}")
    self.phase = "clear"
    n_clear += self.clear_all_heard()
self.stats["n_cleared"] = n_clear
return n_clear

def _staged_scan_then_clear(self):
    """阶段式(对照): 先按覆盖判据扫两遍, 再统一清理。

    ``P4Config(unified=False)`` 时使用 — 保留它是为了做"边扫边清 vs
    先扫后清"的 A/B (消融表要用实测数字说话)。
    """
    total_visited = 0
    for flip in ((False, True) if self.cfg.two_pass else (False,)):
        self.coverage_flip = bool(flip)
        if flip and not self.unheard():
            break
        left0 = self.unheard()
        if not left0:
            break
        self.log(f"扫描第 {1 + int(flip)} 遍 (半平面约定 "
                f"{'朝内' if flip else '朝外'}) : "
                f"还有 {len(left0)} 个频道没听到过 {left0}")
        total_visited += self.sweep_pass(flip=bool(flip))
        if self.aborted:
            break
    self.stats["n_visited_stations"] = len(self.visited_stations)

```

```

self.stats["vtime_after_scan"] = float(self.arena.virtual_time_s)
left = self.unheard_cover_left()
self.log(f"扫描阶段结束: 访问 {len(self.visited_stations)} 站"
        f" (共 {total_visited} 站次), "
        f"听到 {len(self.heard_pending())} 个频道, "
        f"未覆盖格点 {left} 个"
        f"' (=0, 可断定靶区内已无未发现的源)' if left == 0 else ''"
        f"| 虚拟 {self.arena.virtual_time_s:.0f} s")
self.phase = "clear"
self.stats["n_cleared"] = self.clear_all_heard()

def _opportunistic_clear(self, radius_m=None, max_n=2):
    """扫到哪儿就清哪儿: 刚听到的目标, 只要**绕不了多远**就当场清掉。

    为什么重要 (路径图直接暴露的问题): 平均定位清除时间 = 定位清除总时刻 ÷ 清除个数,
    **"什么时候听到"决定了指标的下限**。第一遍扫描要跑 ~30 站、花掉一半以上的虚拟
    时间; 如果一个源在原地就被听到了却要等整圈扫完才清, 那它的"定位清除时间"
    就是整圈的时间 (实测"中心密集"场景平均 1286 s)。听到就清, 等于把这个数
    压到"走到它跟前"的量级。

    代价与约束:

    * 绕行 = 从当前位置到目标再折回航线的距离; 只有 ``dist ≤ clear_detour_max_m``
      才值得绕 (否则等航线自然路过更省);
    * 估计位置来自"沿视线二分" (与协议约定无关), 精度 ~20-40 m,
      配 90 m 小圆覆盖式试探足够保证清除成功。
    """
    lim = float(self.cfg.clear_detour_max_m if radius_m is None else radius_m)
    n = 0
    for ch in self.heard_pending():
        if n >= max_n or self.aborted:
            break
        if not self.can_afford(self.cfg.min_action_budget_s + 40.0):
            break
        est, rad, _br = self.estimate(ch)
        if est is None:
            continue
        d = float(np.hypot(*(np.asarray(self.pos, float) - np.asarray(est,
            float))))
        if d > lim:
            continue
        self.log(f" 顺路清除频道{ch} (估计位置距当前 {d:.0f} m, 绕行上限 {lim:.0f} m)")
        self.phase = "enroute"
        if self.solve_channel(ch):
            n += 1
        self.phase = "scan"
    return n

def clear_all_heard(self):
    """清理阶段: 滚动时域——每清一个就按最新位置重排剩余目标。

```

三条防呆（都是实测出来的教训）：

```
* 同一频道最多进 :meth:`solve_channel` ``max_solve_attempts`` 次
（否则"定位→试探→失败→再定位"会在同一个点上无限循环，实测烧掉 30 000 s）；
* 剩余时间不够就退出，不做无谓动作；
* 优先清"估计位置离当前位置近"的目标（更可能顺手清掉）。
"""
n = 0
attempts: dict[int, int] = {}
while not self.aborted:
    todo = [ch for ch in self.heard_pending()
             if attempts.get(ch, 0) < self.cfg.max_solve_attempts]
    if not todo:
        break
    if not self.can_afford(self.cfg.min_action_budget_s + 20.0):
        self.log("剩余时间不足 -> 结束清理阶段")
        break
    pos = np.asarray(self.pos, float)
    order = []
    for ch in todo:
        est = self.estimate(ch)[0]
        d = float("inf") if est is None else float(np.hypot(*(est - pos)))
        order.append((d, ch))
    order.sort()
    ch = order[0][1]
    attempts[ch] = attempts.get(ch, 0) + 1
    self.log(f"清理阶段: {len(todo)} 个已发现频道待清 {sorted(todo)}"
            f" -> 先清频道{ch} (第 {attempts[ch]} 次尝试)")
    if self.solve_channel(ch):
        n += 1
left = self.heard_pending()
if left:
    self.log(f"清理阶段结束: 仍未清除 {left} (方位不足或时间不够)")
return n
```

0.12 Q4/src/p4_homing.py

```
"""Reply-only homing for Q4, including sources with arbitrary 180 degree cones.

``P4HomingMixin`` can precede any P4GridRobot subclass in its MRO. It replaces
estimate/region and both local clearance entry points. Negative measurements
never exclude a distance disk: loss of signal can be caused by the cone.
"""
from __future__ import annotations

from dataclasses import dataclass
import math

import numpy as np

from p1_intersection import clip_halfplane, wedge_halfplanes, min_enclosing_circle
```

```

from p3_arena import R_ARENA, R_RECV_MAX, R_CLEAR, V_ROBOT
from p3_robot import _dist_point_polygon
from p4_robot import P4Config, P4GridRobot

@dataclass
class P4HomingConfig(P4Config):
    home_fraction: float = 0.60
    home_lateral_m: float = 100.0
    home_max_steps: int = 8
    home_sweep_radius_m: float = 75.0
    home_clear_spacing_m: float = 28.0

class P4HomingMixin:
    """Finite positive-bearing polygon, short baseline, then adaptive approach."""

    def __init__(self, *args, **kwargs):
        self._p4_home_regions = {}
        self._p4_home_clear_points = {}
        self._p4_home_caps = {}
        super().__init__(*args, **kwargs)

    def home_region(self, rec):
        if rec.near_hits:
            row = next(row for row in reversed(rec.meas_log) if row[3] == 'near')
            return dict(center=tuple(row[:2]), radius=5.0, poly=None,
                        status='bounded', n_bearings=rec.n_bearings)

        if not rec.pts:
            return None

        caps = self._p4_home_caps.get(rec.channel, [])
        key = (rec.channel, len(rec.pts), len(caps))
        if key in self._p4_home_regions:
            return self._p4_home_regions[key]

        poly = [np.array(p, float) for p in
                [(-R_ARENA, -R_ARENA), (R_ARENA, -R_ARENA),
                 (R_ARENA, R_ARENA), (-R_ARENA, R_ARENA)]]

        # Circumscribed disk approximations retain every physically possible
        # source. The extra 0.005 degrees covers bearing serialization rounding.
        for station, radius in [(np.zeros(2), R_ARENA)] + [
            (np.array(p), R_RECV_MAX) for p in rec.pts]:
            for angle in np.arange(0, 2 * math.pi, math.pi / 16):
                u = np.array([math.cos(angle), math.sin(angle)])
                poly = clip_halfplane(poly, u, -float(u @ station) - radius)

        for station, bearing in zip(rec.pts, rec.svds):
            for normal, constant in wedge_halfplanes(station, bearing, 1.005 + 1e-8):
                poly = clip_halfplane(poly, normal, constant)

        for normal, constant in caps:
            poly = clip_halfplane(poly, normal, constant)

        if not poly:
            return None

```

```

arr = np.asarray(poly)
origin = arr.mean(axis=0)
circle = min_enclosing_circle(arr - origin)
center = np.asarray(circle[0]) + origin if circle else origin
radius = float(np.max(np.linalg.norm(arr - center, axis=1)))
result = dict(center=tuple(center), radius=radius, poly=arr.tolist(),
              status='bounded', n_bearings=rec.n_bearings)
self._p4_home_regions[key] = result
return result

def region(self, rec):
    br = self.home_region(rec)
    if br is None:
        return dict(status='none', n_bearings=rec.n_bearings)
    return dict(status='bounded', n_bearings=rec.n_bearings,
                min_enclosing_center=br['center'],
                min_enclosing_radius_m=br['radius'], vertices=br['poly'] or [])

def bounded_region(self, rec):
    return self.home_region(rec)

def estimate(self, ch):
    br = self.home_region(self.recs[int(ch)])
    if br is None:
        return None, float('inf'), None
    return np.asarray(br['center']), br['radius'], br

def _home_clear(self, ch, point):
    point = tuple(map(float, point))
    tried = self._p4_home_clear_points.setdefault(int(ch), [])
    if any(math.dist(point, prev) < 0.2 for prev in tried):
        return False
    if not self.can_afford(math.dist(self.pos, point) / V_ROBOT + 6):
        return False
    if self.do_clear(*point, int(ch)):
        return True
    if not self.aborted:
        tried.append(point)
    return False

def _home_measure(self, ch, point):
    point = tuple(map(float, point))
    if not self.can_afford(math.dist(self.pos, point) / V_ROBOT + 12):
        return None
    reply = self.measure(*point, int(ch))
    self.note_meas(point)
    self.stats['n_locate'] = self.stats.get('n_locate', 0) + 1
    if reply.get('measure_result') == 'near':
        self._home_clear(ch, point)
    return reply.get('measure_result')

```

```

def _home_cross(self, ch, br, scale=1.0):
    """Try both sides of a short baseline, retreating after two negatives.

    All candidate coordinates come from an actual positive bearing. A
    lone negative does not change the feasible source polygon. Trying opposite
    sides avoids depending on a guessed radial emission orientation.
    """
    rec = self.recs[int(ch)]
    source_station = np.asarray(rec.pts[-1], float)
    angle = math.radians(rec.svds[-1])
    u = np.array([math.cos(angle), math.sin(angle)])
    normal = np.array([-u[1], u[0]])
    center = np.asarray(br['center'])
    along = max(0.0, float((center - source_station) @ u))
    forward = getattr(self.cfg, 'home_fraction', 0.60) * along * scale
    lateral = min(getattr(self.cfg, 'home_lateral_m', 100.0),
                  max(22.0, 0.3 * max(along, 1.0))) * scale
    midpoint = source_station + forward * u
    candidates = [midpoint + sign * lateral * normal for sign in (-1, 1)]
    candidates.sort(key=lambda p: math.dist(p, self.pos))
    old_n = rec.n_bearings
    results = []
    for point in candidates:
        result = self._home_measure(ch, point)
        results.append(result)
        if rec.cleared or result is None:
            return rec.cleared
        if rec.n_bearings > old_n:
            return True
    # Paired negatives have more information than one negative. Let S be
    # the positive station and  $Q+/- = S + t*u +/- l*n$ . If a feasible source
    # G lies beyond that cross-section, its bearing ray crosses segment
    # Q-Q+. At least one endpoint is in the same closed emission halfplane
    # as S. Moreover  $t \geq l*(\sec(e)+\tan(e))$  makes BOTH endpoints closer to
    # every such G than S, hence inside its actual reception radius. Thus
    # two negatives rule out the entire far part of this bearing wedge.
    # This uses neither a radial orientation assumption nor a negative
    # 1000 m exclusion disk; a lone negative never adds a constraint.
    err = math.radians(1.005 + 1e-8)
    if (results == ['no_signal', 'no_signal']
        and forward * math.tan(err) <= lateral - 1e-5
        and forward >= lateral * (1.0 / math.cos(err) + math.tan(err)) + 1e-5):
        constant = -float(u @ source_station) - forward - 1e-6
        self._p4_home_caps.setdefault(int(ch), []).append((u, constant))
        self.stats['n_home_pair_caps'] = self.stats.get('n_home_pair_caps', 0) + 1
    return False

def _home_cover_polygon(self, ch, br):
    """Finite geometric fallback, independent of signal reception.

    A square lattice with spacing  $\leq 20 \sqrt{2}$  covers its entire plane

```

```

with clear disks. Keep every lattice point whose disk meets the outer
polygon. This also handles a source exactly on its cone boundary.
"""
if not br['poly']:
    return self._home_clear(ch, br['center'])
poly = np.asarray(br['poly'], float)
step = min(getattr(self.cfg, 'home_clear_spacing_m', 28.0), 28.0)
# Rotate to the polygon principal direction: a one-bearing wedge is
# narrow, so this avoids a large rectangular grid of irrelevant cells.
c = np.asarray(br['center'])
cov = (poly - c).T @ (poly - c)
_values, basis = np.linalg.eigh(cov)
local = (poly - c) @ basis
low = np.floor(local.min(axis=0) / step).astype(int) - 1
high = np.ceil(local.max(axis=0) / step).astype(int) + 1
candidates = []
for i in range(low[0], high[0] + 1):
    for j in range(low[1], high[1] + 1):
        point = c + (np.array([i, j]) * step) @ basis.T
        if _dist_point_polygon(point, poly) <= R_CLEAR + 1e-7:
            candidates.append(tuple(point))
self.stats['n_home_fallback'] = self.stats.get('n_home_fallback', 0) + 1
while candidates and not self.aborted:
    index = min(range(len(candidates)),
                 key=lambda k: math.dist(candidates[k], self.pos))
    point = candidates.pop(index)
    if self._home_clear(ch, point):
        return True
    if not self.can_afford(12):
        break
return False

def clear_here(self, ch):
    rec = self.recs[int(ch)]
    if rec.cleared:
        return True
    previous_phase = self.phase
    self.phase = 'home'
    for iteration in range(getattr(self.cfg, 'home_max_steps', 8)):
        if self.aborted or not self.can_afford(15):
            break
        br = self.home_region(rec)
        if br is None:
            break
        if rec.n_bearings >= 2 or br['radius'] <= R_CLEAR:
            if self._home_clear(ch, br['center']):
                self.phase = previous_phase
                return True
        if br['radius'] <= getattr(self.cfg, 'home_sweep_radius_m', 75.0):
            result = self._home_measure(ch, br['center'])
            if rec.cleared:

```

```

        self.phase = previous_phase
        return True
    updated = self.home_region(rec)
    if updated and updated['radius'] < br['radius'] * 0.8:
        continue
    ok = self._home_cover_polygon(ch, updated or br)
    self.phase = previous_phase
    return ok
result = self._home_measure(ch, br['center'])
if rec.cleared:
    self.phase = previous_phase
    return True
if result in ('direction', 'near'):
    continue
# Retrying with a smaller forward step covers both an overshoot and
# a cone-boundary rejection, without inferring which caused it.
br = self.home_region(rec)
if br is None:
    break
before = rec.n_bearings
for scale in (1.0, 0.5, 0.25):
    if self._home_cross(ch, br, scale) or self.aborted:
        break
if rec.cleared:
    self.phase = previous_phase
    return True
if rec.n_bearings == before:
    break
br = self.home_region(rec)
ok = bool(br and self._home_cover_polygon(ch, br))
self.phase = previous_phase
if not ok:
    rec.clear_fail += 1
return ok

def solve_channel(self, ch):
    return self.clear_here(ch)

class P4HomingRobot(P4HomingMixin, P4GridRobot):
    def __init__(self, arena, cfg=None, **kwargs):
        super().__init__(arena, cfg or P4HomingConfig(), **kwargs)

```

0.13 Q4/src/p4_search.py

```

"""Q4 joint routing, directional coverage, and reply-only homing.

```

```

The legacy radial half-plane coverage model is deliberately not used here.
All scheduled stations are actually measured once on every unheard channel.
"""

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass
import math
import time
import numpy as np
from p3_arena import ArenaError
from p4_homing import P4HomingMixin, P4HomingConfig
from p4_robot import P4GridRobot
from p4_certificate import DirectionalCertificate, strict_layout

@dataclass
class P4SearchConfig(P4HomingConfig):
    tier: str = 'strict'
    search_rings: tuple = ((900.,6),(1900.,12))
    search_layout: str = 'certified'
    joint_route: bool = True
    after_clear_scan: bool = False
    scan_heard: bool = True
    route_polish: bool = True
    immediate_service: bool = False
    certify: bool = True
    certificate_step_m: float = 40.
    certificate_n_dir: int = 48
    prune_stations: bool = False
    endpoint_scan_min_saved: float = 0.0
    min_move: float = 0.
    opening_baseline_m: float = 0.
    jointly_scan_targets: bool = False

def tier_config(name='strict'):
    configs={
        'strict':dict(search_layout='certified',certify=True,prune_stations=False),
        'compact':dict(search_rings=((900.,6),(1900.,12))),
        'dense':dict(search_layout='legacy'),
        'fast99':dict(search_rings=((1000.,6),(1850.,9))),
        'fast98':dict(search_rings=((1000.,4),(1750.,8))),
        'fast95':dict(search_rings=((1100.,4),(1650.,8))),
        'fast90':dict(search_rings=((1450.,8),)),
        'inner':dict(search_rings=((1100.,6),)),
    }
    if name not in configs:raise ValueError(name)
    options=dict(search_layout='rings',certify=False,prune_stations=True)
    options.update(configs[name])
    return P4SearchConfig(tier=name,**options)

class P4SearchRobot(P4HomingMixin, P4GridRobot):
    def __init__(self,arena,cfg=None,base=None,log=None):
        super().__init__(arena,cfg or P4SearchConfig(),base=base,log=log)
        self._search_visits=0

```

```

self.certificate=DirectionalCertificate(self.cfg.certificate_step_m,self.cfg.certificate_n_
self._station_bits=[np.packbits(self.certificate.station_mask(p)) for p in
    self.stations]
self._covered_bits=np.packbits(self.certificate.empty())
self._desired_bits=self._covered_bits.copy()
self._desired_bits |= np.packbits(self.certificate.station_mask((0.,0.)))
for bits in self._station_bits:self._desired_bits |= bits

def _build_stations(self):
    if self.cfg.search_layout=='certified':return strict_layout()[1:]
    if self.cfg.search_layout=='legacy':return super()._build_stations()
    points=[]
    for k,(r,n) in enumerate(self.cfg.search_rings):
        offset=0. if k%2==0 else math.pi/n
        points.extend((r*math.cos(offset+2*math.pi*i/n),
            r*math.sin(offset+2*math.pi*i/n)) for i in range(n))
    return points

@staticmethod
def _length(seq,start):
    return sum(math.dist(a,b['xy']) for a,b in zip([start]+[w['xy'] for w in
        seq],seq))

def _ordered(self,items):
    if not items:return []
    pts=np.array([self.pos]+[w['xy'] for w in
        items]);d=np.linalg.norm(pts[:,None]-pts[None,:],axis=2)
    left=list(range(1,len(pts)));seq=[0]
    while left:
        k=min(left,key=lambda j:d[seq[-1],j]);seq.append(k);left.remove(k)
    if self.cfg.route_polish:
        for _ in range(8):
            changed=False
            for i in range(1,len(seq)-1):
                for j in range(i+1,len(seq)):
                    delta=d[seq[i-1],seq[j]]-d[seq[i-1],seq[i]]
                    if j+1<len(seq):delta+=d[seq[i],seq[j+1]]-d[seq[j],seq[j+1]]
                    if delta < -1e-7:seq[i:j+1]=seq[i:j+1][::-1];changed=True
            if not changed:break
    return [items[i-1] for i in seq[1:]]

def _route(self):
    scans=[dict(kind='station',si=i,xy=p) for i,p in enumerate(self.stations)
        if i not in self.visited_stations] if self.unheard() else []
    targets=[]
    for ch in self.heard_pending():
        c,__=self.estimate(ch)
        if c is not None:targets.append(dict(kind='clear',ch=ch,xy=tuple(c)))
    if self.cfg.jointly_scan_targets and scans:
        scans,targets=self._joint_duties(scans,targets)
    elif self.cfg.prune_stations and scans:

```

```

        scans=self._prune(scans,targets)
    if self.cfg.immediate_service and targets:
        return self._ordered(targets)
    if self.cfg.joint_route:return self._ordered(scans+targets)
    return self._ordered(scans) or self._ordered(targets)

def _joint_duties(self,scans,targets):
    items=[dict(w) for w in scans+targets]
    for w in items:
        w['duty']=True
        w['bits']=self._station_bits[w['si']] if w['kind']=='station' else
            np.packbits(self.certificate.station_mask(w['xy']))
    route=self._ordered(items)
    costs={id(w):0. for w in items}
    for i,w in enumerate(route):
        if w['kind']!='station':continue
        a=self.pos if i==0 else route[i-1]['xy']
        savings=math.dist(a,w['xy'])
        if
            i+1<len(route):savings+=math.dist(w['xy'],route[i+1]['xy'])-math.dist(a,route[i+1]['xy'])
        costs[id(w)]=savings
    for w in sorted(items,key=lambda w:costs[id(w)],reverse=True):
        covered=self._covered_bits.copy()
        for v in items:
            if v is not w and v['duty']:covered |= v['bits']
            if not np.any(self._desired_bits & ~covered):w['duty']=False
    for w in items:del w['bits']
    return ([w for w in items if w['kind']=='station' and w['duty']],
            [w for w in items if w['kind']=='clear'])

def _prune(self,scans,targets,covered=None):
    covered=self._covered_bits if covered is None else covered
    scans=list(scans)
    # Prefer deleting the stations with the largest travel/scan cost.
    route=self._ordered(scans+targets)
    score={}
    for i,w in enumerate(route):
        if w['kind']!='station':continue
        prev=self.pos if i==0 else route[i-1]['xy']
        saving=math.dist(prev,w['xy'])
        if
            i+1<len(route):saving+=math.dist(w['xy'],route[i+1]['xy'])-math.dist(prev,route[i+1]['xy'])
        score[w['si']]=saving
    for w in sorted(scans,key=lambda w:score.get(w['si'],0),reverse=True):
        all_bits=covered.copy()
        for v in scans:
            if v is not w:all_bits |= self._station_bits[v['si']]
            if not np.any(self._desired_bits & ~all_bits):scans.remove(w)
    return scans

def _scan_actual(self,p,extra=True):

```

```

channels=list(self.unheard())
if self.cfg.scan_heard and extra:
    for ch in self.heard_pending():
        rec=self.recs[ch]
        if rec.n_bearings>1:continue
        if math.dist(rec.pts[-1],p)<80.:continue
        c,_,_=self.estimate(ch)
        if c is not None and math.dist(c,p)<=1450:channels.append(ch)
self._last_scan_complete=True
if channels:
    self.scan_points.append(tuple(p));n=self.scan_at(*p,channels)
    self.note_meas(self.pos)
    self._last_scan_complete=n==len(channels) and not self.aborted
    if self._last_scan_complete:
        self._covered_bits |= np.packbits(self.certificate.station_mask(p))
return len(channels)

def scan_at(self,x,y,chans,station=None):
    n=0
    for ch in chans:
        if self.aborted or not
            self.can_afford(math.dist(self.pos,(x,y))/5+12):break
        reply=self.measure(x,y,ch,station=station)
        if reply.get('accepted') is not True:break
        n+=1
        if reply.get('measure_result')=='near':self.do_clear(x,y,ch)
    return n

def _certify_remaining(self):
    cert=DirectionalCertificate(20,48)
    cache={}
    for rec in self.recs.values():
        if rec.cleared:continue
        if rec.n_bearings or rec.near_hits:return False
        points=tuple(sorted(set(tuple(row[:2]) for row in rec.meas_log if
            row[3]=='no_signal'))
        if points not in cache:cache[points]=cert.continuous_complete(points)
        if not cache[points]:return False
    return True

def _endpoint_worth_scan(self):
    if not self.cfg.endpoint_scan_min_saved:return True
    if not self.unheard():return False
    scans=[dict(kind='station',si=i,xy=p) for i,p in enumerate(self.stations)
        if i not in self.visited_stations]
    before=self._prune(scans,[])
    covered=self._covered_bits |
        np.packbits(self.certificate.station_mask(self.pos))
    after=self._prune(scans,[],covered)
    n_saved=len(before)-len(after)
    time_saved=(self._length(self._ordered(before),self.pos)-self._length(self._ordered(after),

```

```

time_saved+=(n_saved-1)*6*len(self.unheard())
return time_saved>=self.cfg.endpoint_scan_min_saved

def _opening_baseline(self):
    channels=self.heard_pending()
    if len(channels)<2 or self.cfg.opening_baseline_m<=0:return
    route=self._route()
    dest=np.array(route[0]['xy']) if route else np.zeros(2)
    best=None
    for a in np.arange(24)*2*math.pi/24:
        p=self.cfg.opening_baseline_m*np.array([math.cos(a),math.sin(a)])
        error=0.
        for ch in channels:
            c,_,_=self.estimate(ch);u=c/np.linalg.norm(c)
            v=c-p;cross=abs(float(u[0]*v[1]-u[1]*v[0]))
            error+=min(500.,math.radians(1.005)*np.linalg.norm(v)*np.linalg.norm(c)/max(cross,1.)
        cost=error+0.25*np.linalg.norm(dest-p)
        if best is None or cost<best[0]:best=(cost,p)
    self.scan_at(*best[1],channels)

def run(self):
    t0=time.perf_counter()
    reply=self.arena.enter()
    if reply.get('accepted') is not True:raise ArenaError(str(reply))
    complete=False
    try:
        self._scan_actual(self.pos,extra=False)
        self._opening_baseline()
        for _ in range(400):
            if self.aborted or not self.can_afford(15):break
            route=self._route()
            if not route:break
            w=route[0]
            if w['kind']=='station':
                self._scan_actual(w['xy'])
                if self._last_scan_complete:
                    self.visited_stations.add(w['si']);self._search_visits+=1
                else:break
            else:
                ch=w['ch']
                before=(self.stats['n_measure'],self.stats['n_clear'])
                self.clear_here(ch)
                if self.recs[ch].cleared and (w.get('duty') or
                    (self.cfg.after_clear_scan and self._endpoint_worth_scan())):
                    self._scan_actual(self.pos)
                    if before==(self.stats['n_measure'],self.stats['n_clear']):break
                self.stats['search_stations']=self._search_visits
                if self.cfg.certify:complete=self._certify_remaining()
        finally:
            self.arena.exit()
    self.stats['program_runtime_s']=time.perf_counter()-t0

```

```

rep=self.report();rep['search_stations']=self._search_visits
mask=np.unpackbits(self._covered_bits,count=int(np.prod(self.certificate.shape))).reshape(s
rep['completion_certified']=bool(self.certificate.summarize(mask).complete
    and not self.heard_pending())
if self.cfg.certify:
    rep['completion_certified']=complete
rep['policy']=self.cfg.tier
rep['last_clear_time_s']=self.stats['vtime_at_last_clear']
rep['termination']='certified_complete' if complete else
    ('budget_or_incomplete' if self.cfg.certify else
    'configured_search_complete')
return rep

```

0.14 Q4/src/p4_route_v2.py

```

"""Optional stronger open-route planning for Q4; no hidden-state access.

This mixin changes only the proposed ordering. P4SearchRobot still owns all
measurements, localization, termination and continuous completion checks.
"""
from __future__ import annotations

import math
from dataclasses import dataclass
import numpy as np
from p4_search import P4SearchConfig, P4SearchRobot

@dataclass
class P4RouteV2Config(P4SearchConfig):
    route_v2_rounds: int = 4
    route_v2_starts: int = 5
    route_service_entry: bool = False
    route_fixed_stations: bool = False
    route_information: bool = False

class P4RouteV2Mixin:
    def _route_information(self, items):
        """Reference-distribution scan latency, used only to rank full tours."""
        if not hasattr(self, '_route_info_points'):
            # Fixed quadrature nodes are independent of mock scenarios/seeds.
            rng=np.random.default_rng(170120)
            n=1024
            angle=rng.uniform(0,2*math.pi,n);radius=1800*np.sqrt(rng.uniform(0,1,n))
            self._route_info_points=np.column_stack([radius*np.cos(angle),radius*np.sin(angle)])
            direct=rng.uniform(0,2*math.pi,n)
            self._route_info_direct=np.column_stack([np.cos(direct),np.sin(direct)])
            self._route_info_range=rng.uniform(1000,1500,n)
            self._route_info_cache={}
        def bits(p):

```

```

        p=tuple(p)
        if p not in self._route_info_cache:
            delta=np.asarray(p)-self._route_info_points
            mask=(np.linalg.norm(delta,axis=1)<=self._route_info_range)&(np.sum(delta*self._route
            self._route_info_cache[p]=int.from_bytes(np.packbits(mask).tobytes(),'big')
        return self._route_info_cache[p]
    covered=0
    for p in self.scan_points:covered |= bits(p)
    masks=[None]+[bits(w['xy']) if w['kind']=='station' else None for w in items]
    n=len(self._route_info_points);base=0.35+0.65*(1-covered.bit_count()/n)
    factor=30*len(self.unheard())/base
    return masks,covered,n,factor

def _fixed_ordered(self, items):
    if not hasattr(self, '_fixed_station_order'):
        scans=[w for w in items if w['kind']=='station']
        self._fixed_station_order=[w['si'] for w in self._strong_ordered(scans)]
    by_station={w['si']:w for w in items if w['kind']=='station'}
    seq=[by_station[i] for i in self._fixed_station_order if i in by_station]
    for w in (w for w in items if w['kind']=='clear'):
        def cost_at(k):
            a=self.pos if k==0 else seq[k-1]['xy']
            b=seq[k]['xy'] if k<len(seq) else None
            return math.dist(a,w['xy'])+(math.dist(w['xy'],b)-math.dist(a,b) if b
                is not None else 0.)
        k=min(range(len(seq)+1),key=cost_at);seq.insert(k,w)
    # Move targets between gaps while preserving the scan sequence.
    for _ in range(3):
        changed=False
        for w in list(seq):
            if w['kind']!='clear':continue
            old=seq.index(w);seq.remove(w)
            k=min(range(len(seq)+1),key=lambda
                k:self._length(seq[:k]+[w]+seq[k:],self.pos))
            seq.insert(k,w);changed |= old!=k
        if not changed:break
    return seq

def _route_key(self, item):
    return (item['kind'], item.get('si', item.get('ch'))))

def _entry_points(self, item):
    c = np.asarray(item['xy'], float)
    if not getattr(self.cfg, 'route_service_entry', False) or item['kind'] !=
        'clear':
        return c.reshape(1, 2)
    rec = self.recs[item['ch']]
    if rec.n_bearings != 1:
        return c.reshape(1, 2)
    s = np.asarray(rec.pts[-1], float)
    a = math.radians(rec.svds[-1]);u = np.array([math.cos(a), math.sin(a)])

```

```

normal = np.array([-u[1], u[0]])
along = max(0., float((c-s) @ u))
forward = self.cfg.home_fraction * along
lateral = min(self.cfg.home_lateral_m, max(22., .3 * max(along, 1.)))
midpoint = s + forward * u
return np.asarray([midpoint-lateral*normal, midpoint+lateral*normal])

def _ordered(self, items):
    if getattr(self.cfg, 'route_fixed_stations', False):
        return self._fixed_ordered(items)
    return self._strong_ordered(items)

def _strong_ordered(self, items):
    if len(items) <= 1:
        return list(items)
    n = len(items)
    pts = np.asarray([self.pos] + [w['xy'] for w in items], float)
    entries = [pts[0].reshape(1, 2)] + [self._entry_points(w) for w in items]
    d = np.zeros((n+1,n+1))
    for j,entry in enumerate(entries):
        d[:,j] = np.min(np.linalg.norm(pts[:,None,:]-entry[None,:,:],axis=2)
                        + np.linalg.norm(entry-pts[j],axis=1)[None,:],axis=1)
    np.fill_diagonal(d, 0.)
    info=self._route_information(items) if
        getattr(self.cfg, 'route_information', False) else None
    symmetric=not getattr(self.cfg, 'route_service_entry', False)

    def cost(seq):
        total=float(np.sum(d[np.asarray([0]+seq[:-1]),seq]))
        if info:
            masks,covered,ns,factor=info
            for j in seq:
                if masks[j] is not None:
                    total+=factor*(0.35+0.65*(1-covered.bit_count()/ns))
                    covered |= masks[j]
            return total

    def improve(seq):
        seq = list(seq); value = cost(seq)
        for iteration in range(getattr(self.cfg, 'route_v2_rounds', 4)):
            best, best_value = None, value
            # Full objective also works for directed service-entry costs.
            for i in range(n-1):
                for j in range(i+1,n):
                    trial = seq[:i] + seq[i:j+1][::-1] + seq[j+1:]
                    if symmetric and info is None:
                        a=0 if i==0 else seq[i-1]
                        delta=d[a,seq[j]]-d[a,seq[i]]
                        if j+1<n:delta+=d[seq[i],seq[j+1]]-d[seq[j],seq[j+1]]
                        val=value+delta
                    else:val = cost(trial)

```

```

        if val < best_value-1e-7:
            best,best_value = trial,val
    for i in range(n):
        node = seq[i];rest=seq[:i]+seq[i+1:]
        a=0 if i==0 else seq[i-1]
        removal=-d[a,node]
        if i+1<n:removal+=d[a,seq[i+1]]-d[node,seq[i+1]]
        for j in range(n):
            if i==j:continue
            trial=rest[:j]+[node]+rest[j:]
            if info is None:
                prev=0 if j==0 else rest[j-1]
                delta=removal+d[prev,node]
                if j<len(rest):delta+=d[node,rest[j]]-d[prev,rest[j]]
                val=value+delta
            else:val=cost(trial)
            if val<best_value-1e-7:
                best,best_value=trial,val
    if best is None:break
    seq,value=best,cost(best)
    return seq,value

# A warm start avoids discarding the global path after a small move.
id_to_i = {self._route_key(w):i+1 for i,w in enumerate(items)}
warm = [id_to_i[k] for k in getattr(self,'_route_v2_previous',[]) if k in
        id_to_i]
for j in range(1,n+1):
    if j in warm:continue
    best = min(range(len(warm)+1), key=lambda k:cost(warm[:k]+[j]+warm[k:]))
    warm.insert(best,j)
starts=[warm]
first_candidates=np.argsort(d[0,1:])[ :getattr(self.cfg,'route_v2_starts',5)]+1
for first in first_candidates:
    seq=[int(first)];left=set(range(1,n+1))-set(seq)
    while left:
        j=min(left,key=lambda j:d[seq[-1],j]);seq.append(j);left.remove(j)
    starts.append(seq)
seq,value=min((improve(seq) for seq in starts),key=lambda x:x[1])
self._route_v2_previous=[self._route_key(items[i-1]) for i in seq]
return [items[i-1] for i in seq]

class P4RouteV2Robot(P4RouteV2Mixin, P4SearchRobot):
    pass

```

0.15 Q4/src/p4_compact.py

```

"""Compact directional survey with adaptively refined continuous proof."""
from dataclasses import dataclass, asdict
import math
import numpy as np

```

```

from p4_search import P4SearchRobot,P4SearchConfig,tier_config
from p4_certificate import ring_layout
from p4_layout_v2 import adaptive_complete
from p4_route_v2 import P4RouteV2Mixin
from p4_posterior import P4PosteriorMixin

@dataclass
class P4CompactConfig(P4SearchConfig):
    tier: str = 'strict'
    compact_rings: tuple = ((998.,8,0.), (1867.,12,15.))
    stronger_route: bool = True
    route_service_entry: bool = False
    route_v2_rounds: int = 4
    route_v2_starts: int = 5
    route_information: bool = True
    extra_bearings: bool = False
    opportunistic_baseline_m: float = 300.
    defer_single: bool = False
    defer_distance_m: float = 1100.
    defer_angle_deg: float = 70.
    posterior_estimate: bool = True
    posterior_omni_prior: float = .5
    rotate_layout: bool = False
    rotation_candidates: int = 14

class P4CompactRobot(P4PosteriorMixin,P4RouteV2Mixin,P4SearchRobot):
    def __init__(self,arena,cfg=None,**kwargs):
        cfg=cfg or P4CompactConfig()
        if cfg.certify and not adaptive_complete(ring_layout(cfg.compact_rings)):
            raise ValueError('Compact layout lacks a continuous certificate')
        super().__init__(arena,cfg,**kwargs)

    def _build_stations(self):
        return ring_layout(self.cfg.compact_rings,False)

    def _opening_baseline(self):
        if self.cfg.rotate_layout and self.heard_pending():
            base=np.array(self.stations)
            targets=[dict(kind='clear',ch=ch,xy=tuple(self.estimate(ch)[0])) for ch
                      in self.heard_pending()]
            best=None
            # The certified arena is a disk, so a rigid rotation preserves
            # its geometric validity. Pick phase using observed targets only.
            symmetry=math.gcd(*(int(n) for _,n,_ in self.cfg.compact_rings))
            for phi in
                np.arange(self.cfg.rotation_candidates)*2*math.pi/(symmetry*self.cfg.rotation_candidates):
                mat=np.array([[math.cos(phi),-math.sin(phi)],[math.sin(phi),math.cos(phi)]]))
                pts=base@mat.T
                items=[dict(kind='station',si=i,xy=tuple(p)) for i,p in

```

```

        enumerate(pts)]+targets
        self._route_v2_previous=[]
        seq=self._ordered(items)
        cost=self._length(seq,self.pos)
        if best is None or cost<best[0]:best=(cost,pts)
        self.stations=[tuple(p) for p in best[1]]
        self._station_bits=[np.packbits(self.certificate.station_mask(p)) for p
            in self.stations]
        self._route_v2_previous=[]
        return super()._opening_baseline()

def _ordered(self,items):
    if self.cfg.stronger_route:return P4RouteV2Mixin._ordered(self,items)
    return P4SearchRobot._ordered(self,items)

def _route(self):
    seq=super()._route()
    if not self.cfg.defer_single:return seq
    scans=[w for w in seq if w['kind']=='station']
    if not scans:return seq
    keep=[]
    for w in seq:
        if w['kind']!='clear' or self.recs[w['ch']].n_bearings!=1:
            keep.append(w);continue
        rec=self.recs[w['ch]];c=np.array(w['xy']);old=np.array(rec.pts[-1])-c
        can_defer=False
        for scan in scans:
            q=np.array(scan['xy']);v=q-c;dist=np.linalg.norm(v)
            if dist>self.cfg.defer_distance_m or
                math.dist(q,rec.pts[-1])<300:continue
            angle=math.degrees(math.acos(float(np.clip((old@v)/max(1e-9,np.linalg.norm(old)*dist))
            if angle<=self.cfg.defer_angle_deg:can_defer=True;break
        if not can_defer:keep.append(w)
    return self._ordered(keep)

def _certify_remaining(self):
    cache={}
    for rec in self.recs.values():
        if rec.cleared:continue
        if rec.n_bearings or rec.near_hits:return False
        points=tuple(sorted(set(tuple(row[:2]) for row in rec.meas_log if
            row[3]=='no_signal'))))
        if points not in cache:cache[points]=adaptive_complete(points)
        if not cache[points]:return False
    return True

def _scan_actual(self,p,extra=True):
    n=super()._scan_actual(p,extra)
    if self.cfg.extra_bearings and extra:
        for ch in self.heard_pending():
            rec=self.recs[ch]

```

```

        if rec.n_bearings<2:continue
        c,rad,_=self.estimate(ch)
        if rad<20 or c is None or math.dist(c,p)>1000:continue
        if math.dist(rec.pts[-1],p)<self.cfg.opportunistic_baseline_m:continue
        self.scan_at(*p,[ch])
    return n

def compact_config(tier='strict'):
    """Second-round presets; fast tier labels are empirical targets only."""
    base=tier_config(tier)
    options=asdict(base)
    options.update(stronger_route=True,posterior_estimate=True,route_information=True)
    if tier=='strict':
        options['compact_rings']=((998.,8,0.),(1867.,12,15.))
    else:
        options['compact_rings']=tuple((r,n,0. if k%2==0 else 180./n)
                                       for k,(r,n) in enumerate(base.search_rings))
    return P4CompactConfig(**options)

```

0.16 Q4/src/p4_run.py

```

"""B 题问题 4：运行入口（离线演练 / 真实模拟器 / 几何自检）。

# 1) 几何自检：三角形格网的覆盖与"定向盲区"（不跑模拟器）
python src/p4_run.py --geometry

# 2) 离线演练（定向 + 全向混合；正式测试只有 3 次机会，调参必须在这里做）
python src/p4_run.py --mode mock --cases 10 --scenario directed --verbose
python src/p4_run.py --mode mock --cases 30 --scenario omni # 全向对照
python src/p4_run.py --mode mock --cases 10 --scenario directed --out out/p4
--figure

# 3) 真实模拟器（先在模拟器里登录、开始"问题4 演练测试"，等接口就绪）
python src/p4_run.py --mode live --robot-id <参赛队号> --url http://127.0.0.1:2026

```

统计口径与题目表 1 一致：

- * 被清除干扰源个数的比例 = 被清除个数 / 干扰源总数；
- * 平均定位清除时间 = 定位清除总时间 / 被清除干扰源个数
（总时间含移动、频道切换、检测、光学精确定位与清除，取最后一个源被清除的时刻）；
- * 完整退出每源时间 = /exit 时的虚拟时间 / 被清除源数（本轮优化主指标）；
- * 程序运行时间 = 程序自身跑完的墙钟时间。

```

"""
from __future__ import annotations

import argparse
import csv
import json
import math
import os
import statistics as st

```

```

import sys
import time

_HERE = os.path.dirname(os.path.abspath(__file__))
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)
_ROOT = os.path.normpath(os.path.join(_HERE, ".."))

for _s in (sys.stdout, sys.stderr):
    try:
        _s.reconfigure(encoding="utf-8", errors="replace")
    except Exception:
        pass

import numpy as np # noqa: E402

from p3_arena import ( # noqa: E402
    ArenaError, HttpArena, MockArena, MockSource, R_ARENA, R_CLEAR, R_RECV_MAX,
    R_RECV_MIN, V_ROBOT,
)
from p3_robot import P3Robot # noqa: E402
from p4_arena_ext import SCENARIOS, directed_case, make_arena # noqa: E402
from p4_robot import P4Config, P4GridRobot # noqa: E402

OUT_DEFAULT = os.path.normpath(os.path.join(_ROOT, "out", "p4"))
DEFAULT_VIRTUAL_BUDGET = 360000.0 # 与真实模拟器一致（虚拟世界限时 100 小时）
LIVE_REAL_BUDGET = 1200.0 # 现实时间预算（/enter 返回）

# 当前最优策略与档位: compact = 21 站紧凑布局 + 后验估计 + 信息排路;
# strict = 100% 档（离线配对: 混合源 5933 s、全定向 6491 s, 360 局 4668 源全部清除并认证）。
# 速度档 fast99/98/95/90 以牺牲清除率为代价换时间（贴边朝外场景 fast98 及以下清除率 0%），
# 正式测试**不要**当默认用; 见 review/p4-compact-design.md。
DEFAULT_POLICY = 'compact'
DEFAULT_TIER = 'strict'

# -----
# 场景（定义搬到了 p4_arena_ext, 自检/诊断共用同一套）
# -----
def make_robot(arena, cfg=None, log=None):
    from p4_search import P4SearchConfig, P4SearchRobot
    from p4_compact import P4CompactConfig, P4CompactRobot, compact_config
    if cfg is None:
        cfg=compact_config('strict')
    if isinstance(cfg, P4CompactConfig):
        return P4CompactRobot(arena, cfg, log=log)
    if isinstance(cfg, P4SearchConfig):
        return P4SearchRobot(arena, cfg, log=log)
    return P4GridRobot(arena, cfg or P4Config(), base=None, log=log)

```

```

# -----
def write_json(path, obj):
    os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        json.dump(obj, f, ensure_ascii=False, indent=2, default=_json_default)
    return path

def _json_default(o):
    if isinstance(o, (np.float64, np.integer)):
        return o.item()
    if isinstance(o, np.ndarray):
        return o.tolist()
    return str(o)

def run_mock_case(seed, cfg, scenario="directed", outdir=None, verbose=False,
                  figure=False, budget_s=None, n_sources=None, directed_frac=.5):
    arena = make_arena(scenario, seed, n_sources=n_sources,
                       budget=(DEFAULT_VIRTUAL_BUDGET if budget_s is None
                               else budget_s), directed_frac=directed_frac)
    rb = make_robot(arena, cfg, log=(lambda m: print(" " + m)) if verbose else None)
    rep = rb.run()
    rep['exit_per_cleared_s']=(rep['virtual_time_s']/rep['cleared'] if
                              rep['cleared'] else None)
    rep["seed"] = seed
    rep["scenario"] = scenario
    rep["n_directed"] = sum(1 for s in arena.sources if s.cone_half < 180.0)
    rep["stations_total"] = len(rb.stations)
    rep["stations_visited"] = len(rb.visited_stations)
    rep["stations"] = [[round(x, 1), round(y, 1)] for (x, y) in rb.stations]
    if outdir:
        d = os.path.join(outdir, f"case_{scenario}_{seed}")
        write_json(os.path.join(d, "summary.json"), rep)
        with open(os.path.join(d, "actions.jsonl"), "w", encoding="utf-8") as f:
            for a in arena.actions:
                f.write(json.dumps(a, ensure_ascii=False, default=_json_default) +
                        "\n")
        write_json(os.path.join(d, "robot_events.json"), rb.events)
        if figure:
            try:
                rep["figure"] = plot_case(arena, rb, rep, os.path.join(d,
                                                                       "p4_case.png"))
            except Exception as e:
                rep["figure_error"] = repr(e)
    return rep

def plot_case(arena, robot, rep, path):
    import matplotlib
    matplotlib.use("Agg")

```

```

import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8.6, 8.6))
th = np.linspace(0, 2 * math.pi, 720)
ax.plot(R_ARENA * np.cos(th), R_ARENA * np.sin(th), "k--", lw=1.0,
        label="target area R=1800 m")
for s in rep.get("truth", {}).get("sources", []):
    col = "tab:green" if s["cleared"] else "tab:red"
    ax.plot([s["x_m"]], [s["y_m"]], "o", color=col, ms=8, zorder=6)
    ax.annotate(f"ch{s['channel']}", (s["x_m"], s["y_m"]), fontsize=8,
               xytext=(4, 4), textcoords="offset points")
# 格网站位
st = np.asarray(rep.get("stations", []), float).reshape(-1, 2)
if st.size:
    ax.plot(st[:, 0], st[:, 1], "^", color="tab:orange", ms=6, alpha=0.75,
            label="triangle lattice stations")
pts = [(a["x"], a["y"]) for a in getattr(arena, "actions", [])
        if a.get("kind") in ("measure", "clear")]
if pts:
    P = np.array([[0.0, 0.0]] + pts)
    ax.plot(P[:, 0], P[:, 1], "-", color="tab:blue", lw=0.9, alpha=0.75,
            label="robot path")
for ch, rec in robot.recs.items():
    for (x, y), a in zip(rec.pts, rec.svds):
        L = 900.0
        ax.plot([x, x + L * math.cos(math.radians(a))],
                [y, y + L * math.sin(math.radians(a))],
                color="tab:purple", lw=0.5, alpha=0.4)
ax.plot([0], [0], "k*", ms=13, label="start (0,0)")
ax.set_aspect("equal")
ax.set_xlim(-1950, 1950)
ax.set_ylim(-1950, 1950)
ax.grid(alpha=0.25)
ax.legend(loc="upper right", fontsize=8)
ax.set_title(f"Q4 drill seed={rep.get('seed')} scenario={rep.get('scenario')}\n"
             f"cleared {rep['cleared']}/{rep['n_sources']} "
             f"vtime={rep['virtual_time_s']:.0f}s "
             f"avg clear={rep['avg_clear_time_s']:.1f}s "
             f"travel={rep['travel_m']:.0f} m", fontsize=9)
fig.tight_layout()
os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
fig.savefig(path, dpi=150)
plt.close(fig)
return path

# -----
def aggregate(reps):
    cl = [r["cleared"] for r in reps]
    tot = [r["n_sources"] for r in reps]
    tms = [r["avg_clear_time_s"] for r in reps if r["avg_clear_time_s"]]

```

```

return {
    "cases": len(reps),
    "n_sources_total": int(sum(tot)),
    "n_cleared_total": int(sum(cl)),
    "clear_ratio_total": (sum(cl) / sum(tot)) if sum(tot) else None,
    "full_clear_cases": sum(1 for r in reps if r["cleared"] == r["n_sources"]),
    "certified_complete_cases": sum(r.get('completion_certified') is True for r
        in reps),
    "cleared_mean": st.mean(cl) if cl else None,
    "avg_clear_time_mean_s": st.mean(tms) if tms else None,
    "avg_clear_time_median_s": st.median(tms) if tms else None,
    "virtual_time_mean_s": st.mean(r["virtual_time_s"] for r in reps),
    "exit_per_cleared_mean_s": st.mean(r['virtual_time_s']/r['cleared'] for r in
        reps if r['cleared']) if any(cl) else None,
    "runtime_mean_s": st.mean(r["program_runtime_s"] for r in reps),
    "travel_mean_m": st.mean(r["travel_m"] for r in reps),
    "n_measure_mean": st.mean(r["n_measure"] for r in reps),
    "stations_mean": st.mean(r["stations_visited"] for r in reps),
    "n_rejected": int(sum(r["n_rejected"] for r in reps)),
    "n_clear_fail": int(sum(r["n_clear_fail"] for r in reps)),
}

def print_table(rows, cols, title=None):
    if title:
        print(f"\n=== {title} ===")
    w = {c: max([len(c)] + [len(f"{r.get(c, '')}") for r in rows]) for c in cols}
    print(" ".join(c.rjust(w[c]) for c in cols))
    print(" ".join("-" * w[c] for c in cols))
    for r in rows:
        cells = []
        for c in cols:
            v = r.get(c)
            if v is None:
                s = ""
            elif isinstance(v, float):
                s = f"{v:.4f}" if abs(v) < 10 else f"{v:.1f}"
            else:
                s = str(v)
            cells.append(s.rjust(w[c]))
        print(" ".join(cells))

# -----
def geometry_report(args):
    """几何自检: 三角形网格的覆盖半径 + 定向盲区 (不跑模拟器)。"""
    if getattr(args, 'policy', 'compact') == 'compact':
        from p4_compact import compact_config
        from p4_layout_v2 import adaptive_complete
        rb = make_robot(_DummyArena(), compact_config(args.tier))
        points = [(0., 0.)] + rb.stations

```

```

        complete, refinement=adaptive_complete(points, detail=True)
        result={'policy':'compact', 'tier':args.tier, 'stations':len(points),
                'continuous_arbitrary_orientation_coverage':complete,
                'refinement':refinement}
        print(json.dumps(result, ensure_ascii=False, indent=2))
        return result
    if getattr(args, 'policy', 'grid')=='joint':
        from p4_search import P4SearchRobot, tier_config
        from p4_certificate import DirectionalCertificate
        cfg=tier_config(args.tier)
        rb=P4SearchRobot(_DummyArena(), cfg)
        cert=DirectionalCertificate(20, 48)
        points=[(0., 0.)]+rb.stations
        complete=cert.continuous_complete(points)
        result={'policy':'joint', 'tier':args.tier, 'stations':len(points),
                'continuous_arbitrary_orientation_coverage':complete}
        print(json.dumps(result, ensure_ascii=False, indent=2))
        return result
    from p4_grid import cov_stats, dir_cov_stats, eval_grid
    cfg = P4Config()
    rb = P4GridRobot(_DummyArena(), cfg)
    stations = [(0.0, 0.0)] + list(rb.stations)
    gx, gy = eval_grid(20.0)
    c, cu = cov_stats(stations, gx, gy)
    dw, db, hd = dir_cov_stats(stations, gx, gy, n_dir=16)
    print(f"三角形网格 a={cfg.grid_a:.0f} m + 外环 "
          f"[{f'r{r:.0f}x{n}' for r, n in (cfg.outer_rings or ())}]"
          f": 站位 {len(stations)} 个 (含原点)")
    print(f" 覆盖半径 (到最近站位) 最大 {c:.0f} m, 未覆盖占比 {cu:.2%}"
          f" [判据  $\leq \{R\_RECV\_MIN:.0f\} m \Rightarrow$  全向源必被听到]")
    print(f" 最坏朝向下最近可听站位 {dw:.0f} m, 定向盲区 {db:.2%}, "
          f"半平面硬伤 {hd:.2%} [离散位置/方向采样, 不是连续覆盖证明]")
    print("\n站位分布 (按半径): ")
    ring = {}
    for (x, y) in stations:
        r = math.hypot(x, y)
        k = 0 if r < 1 else int(round(r / cfg.grid_a))
        ring.setdefault(k, []).append(r)
    for k in sorted(ring):
        rs = ring[k]
        print(f" 第 {k} 层: {len(rs):>3} 个, 半径 {min(rs):>7.0f}-{max(rs):>7.0f} m"
              + (" (原点)" if k == 0 else ""))
    return {"n_stations": len(stations), "cov_max_m": c, "cov_unc": cu,
            "dir_worst_m": dw, "dir_blind": db, "halfplane_hard": hd}

class _DummyArena:
    """只为构造站位列表用的假 arena (不调用任何接口)。"""
    pos = (0.0, 0.0)
    channel = 1
    virtual_time_s = 0.0

```

```

def remaining_budget_s(self):
    return 1e9

def budget_kind(self):
    return "virtual"

def main():
    ap = argparse.ArgumentParser(description="问题4 联合航路与清除率配置")
    ap.add_argument('--policy', choices=['compact', 'joint', 'grid'], default=DEFAULT_POLICY,
                    help=f'compact=紧凑布局与信息排路（默认，当前最优）| joint=上一版 | '
                          'grid=旧策略')
    ap.add_argument('--tier', choices=['strict', 'fast99', 'fast98', 'fast95', 'fast90'], default=DEFAULT_TIER,
                    help='strict=全清档（默认，360 局全部清除并认证）；fast* 用清除率换时间， '
                          '速度档名称是离线标定目标，非逐局清除率保证，正式测试勿用')
    ap.add_argument("--mode", choices=["mock", "live"], default="mock")
    ap.add_argument("--cases", type=int, default=10)
    ap.add_argument("--seed", type=int, default=20260914)
    ap.add_argument("--scenario", default="directed", choices=list(SCENARIOS))
    ap.add_argument("--directed-fraction", type=float, default=0.5)
    ap.add_argument("--n-sources", type=int, default=0, help="0 = 随机 10-16 个")
    ap.add_argument("--out", default=OUT_DEFAULT)
    ap.add_argument("--figure", action="store_true")
    ap.add_argument("--verbose", action="store_true")
    ap.add_argument("--budget", type=float, default=DEFAULT_VIRTUAL_BUDGET)
    ap.add_argument("--grid-a", type=float, default=P4Config.grid_a)
    ap.add_argument("--outer-rings", default=None,
                    help="外侧补漏环，格式 r1:n1,r2:n2；默认取 P4Config 的值， "
                          "留空字符串表示不加外环")
    ap.add_argument("--spiral-w", type=float, default=0.35)
    ap.add_argument("--plan-tour", default="0",
                    help="仅 grid: 0 = 逐站贪心走完旧模型判定的有用站位； "
                          "1 = 最小覆盖集 + 半径偏置 TSP（快 ~10%%，有 ~0.4%% 漏源风险）")
    ap.add_argument("--replan-every", type=int, default=0,
                    help="每走这么多站重排一次航线（0 = 不重排）")
    ap.add_argument("--geometry", action="store_true", help="只做几何自检")
    ap.add_argument("--url", default="http://127.0.0.1:2026")
    ap.add_argument("--robot-id", default="")
    ap.add_argument("--timeout", type=float, default=5.0)
    args = ap.parse_args()

    if args.outer_rings is None:
        rings = tuple(P4Config.outer_rings)
    else:
        rings = []
        for spec in (args.outer_rings or "").split(","):
            spec = spec.strip()
            if spec:
                r, n = spec.split(":")
                rings.append((float(r), int(n)))

```

```

rings = tuple(rings)
cfg = P4Config(grid_a=args.grid_a, outer_rings=rings,
               spiral_radius_w=args.spiral_w,
               plan_tour=bool(int(args.plan_tour)),
               replan_every=int(args.replan_every))
if args.policy=='joint':
    from p4_search import tier_config
    cfg=tier_config(args.tier)
elif args.policy=='compact':
    from p4_compact import compact_config
    cfg=compact_config(args.tier)

if args.geometry:
    geometry_report(args)
    return

if args.mode == "live":
    if not args.robot_id:
        raise SystemExit("live 模式必须提供 --robot-id (参赛队号)")
    d = os.path.join(args.out, time.strftime("live_%Y%m%d_%H%M%S"))
    arena = HttpArena(base_url=args.url, robot_id=args.robot_id,
                     timeout=args.timeout,
                     log_path=os.path.join(d, "protocol.jsonl"))
    rb = make_robot(arena, cfg, log=print)
    print(f"连接模拟器 {args.url} (队号 {args.robot_id}); "
          f"若接口尚未开放会自动等待重试...")
    try:
        rep = rb.run()
        rep['exit_per_cleared_s']=(rep['virtual_time_s']/rep['cleared'] if
                                   rep['cleared'] else None)
    except ArenaError as e:
        print(f"\n[失败] {e}")
        print("请确认: ① 模拟器已在线登录; ② 已开始『问题4 演练测试/正式测试』; "
              "③ 5 秒倒计时结束、界面提示机器狗接口已就绪; ④ 端口一致。")
        write_json(os.path.join(d, "error.json"), {"error": repr(e), "url":
            args.url})
        raise SystemExit(2)
    rep["url"] = args.url
    rep["scenario"] = "live"
    write_json(os.path.join(d, "summary.json"), rep)
    print(json.dumps({k: v for k, v in rep.items()
                      if k not in ("truth", "mock_stats", "channels", "stations")},
                     ensure_ascii=False, indent=2, default=_json_default))
    print(f"\n结果已写入 {d}")
    return

seeds = [args.seed + 100 * i for i in range(args.cases)]
reps = []
t0 = time.time()
for s in seeds:
    rep = run_mock_case(s, cfg, scenario=args.scenario, outdir=args.out,

```

```

        verbose=args.verbose, figure=args.figure,
        budget_s=args.budget,
        n_sources=(args.n_sources or
                    None),directed_frac=args.directed_frac)
    reps.append(rep)
    exit_avg=(f"{rep['exit_per_cleared_s']:.1f}" if rep['exit_per_cleared_s'] is
              not None else '-')
    print(f" seed={s} 清除 {rep['cleared']}/{rep['n_sources']}"
          f" (定向 {rep['n_directed']}) | 定位清除 {rep['avg_clear_time_s']:.1f} s"
          f" | 完整退出 {rep['virtual_time_s']:.0f} s"
          f" (每源 {exit_avg} s) | 检测 {rep['n_measure']}"
          f" | 站位 {rep['stations_visited']}/{rep['stations_total']}"
          f" | 行程 {rep['travel_m']:.0f} m", flush=True)
    agg = aggregate(reps)
    agg["wall_s"] = round(time.time() - t0, 2)
    print()
    print(json.dumps(agg, ensure_ascii=False, indent=2, default=_json_default))
    write_json(os.path.join(args.out, f"benchmark_{args.scenario}.json"),
               {"cfg": cfg.__dict__, "seeds": seeds, "scenario": args.scenario,
                "aggregate": agg,
                "cases": [{k: v for k, v in r.items()
                           if k not in ("truth", "channels", "stations")} for r in reps]})
    with open(os.path.join(args.out, f"benchmark_{args.scenario}.csv"), "w",
              newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f)
        cols = ["seed", "n_sources", "n_directed", "cleared", "clear_ratio",
                "avg_clear_time_s", "virtual_time_s", "exit_per_cleared_s",
                "program_runtime_s",
                "n_measure", "stations_visited", "n_locate", "n_clear",
                "n_clear_fail", "travel_m", "n_rejected"]
        w.writerow(cols)
        for r in reps:
            w.writerow([r.get(c) for c in cols])
    print(f"\n结果已写入 {args.out}")

if __name__ == "__main__":
    main()

```

0.17 Q4/scripts/run_p4.py

"""B 题问题 4: 运行入口（转发到 ``src/p4\_run.py``）。

本机 ``scripts/`` 目录存在 ACL 限制（子进程无法读取该目录下的文件），
所以真正可执行的实现放在 ``src/p4\_run.py``；本文件只是保持目录约定的转发入口。

```

python scripts/run_p4.py --mode mock --cases 10
python src/p4_run.py --mode mock --cases 10 # 等价的直接调用
"""
from __future__ import annotations

```

```
import os
import sys

_HERE = os.path.dirname(os.path.abspath(__file__))
_SRC = os.path.normpath(os.path.join(_HERE, "..", "src"))
if _SRC not in sys.path:
    sys.path.insert(0, _SRC)

from p4_run import main # noqa: E402

if __name__ == "__main__":
    main()
```